Principia Softwarica: The Plan 9 C Compiler `5c`

ARM (32 bits) edition

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
Ken Thompson

January 14, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **12**
1.1 Motivations . . . 12
1.2 The ARM Plan 9 C compiler: `5c` . . . 12
1.3 Other compilers . . . 12
1.4 Getting started . . . 13
1.5 Requirements . . . 13
1.6 About this document . . . 13
1.7 Copyright . . . 13
1.8 Acknowledgments . . . 13

2 Overview **14**
2.1 Compiler principles . . . 15
2.1.1 Frontend . . . 15
2.1.2 Scope resolution . . . 15
2.1.3 Use/def checking . . . 15
2.1.4 Type checking . . . 15
2.1.5 Backend . . . 15
2.1.6 Labelization . . . 15
2.1.7 Linearization . . . 15
2.1.8 Frameization . . . 15
2.1.9 Optimizations . . . 15
2.2 `5c` command-line interface . . . 15
2.3 `helloworld.c` . . . 15
2.4 Input C language . . . 15
2.4.1 C innovations . . . 15
2.5 Output object format . . . 15
2.6 The ARM architecture . . . 15
2.7 Code organization . . . 15
2.8 Software architecture . . . 15
2.9 Book structure . . . 15

3 Core Data Structures **16**
3.1 Abstract syntax tree: `Node` . . . 16
3.2 `Type` system . . . 18
3.3 `Storage_class` . . . 20
3.4 Node and type attributes . . . 21
3.5 `Sym`bols and `hash` table . . . 21
3.5.1 `Sym` . . . 21
3.5.2 `hash` . . . 22
3.5.3 `slookup()` and `lookup()` . . . 22

3.5.4 Identifier symbols . . . 23
3.5.5 Tag symbols . . . 24
3.5.6 Other symbols . . . 24
3.6 Declarations, namespaces, and scope: `Decl` . . . 24
3.7 `Token` . . . 28
3.8 Assembly `Instruction` output . . . 28

4 `main()` 30
4.1 Arguments processing . . . 31
4.2 `compile()` . . . 32
4.3 `gclean()` . . . 34
4.4 Initializations . . . 34
4.4.1 `tinit()` . . . 34
4.4.2 `cinit()` . . . 35
4.4.3 `ginit()` . . . 35
4.4.4 `arginit()` . . . 35

5 Input 36
5.1 Files management: `iostack` . . . 36
5.2 Buffer management: `fi` . . . 37
5.3 `GETC()` . . . 38

6 Lexing 40
6.1 `yylex()` . . . 40
6.2 Peek, seek, and look ahead . . . 40
6.3 Comments . . . 41
6.4 Keywords and identifiers . . . 42
6.4.1 Unicode identifiers . . . 43
6.4.2 Typedef trick . . . 44
6.5 Operators . . . 44
6.6 Numbers . . . 46
6.6.1 Decimals . . . 46
6.6.2 `convvtox()` . . . 47
6.6.3 Hexadecimals and octals . . . 48
6.6.4 Unsigned numbers . . . 49
6.6.5 Long numbers . . . 49
6.6.6 Floats . . . 50
6.7 Characters . . . 51
6.7.1 Escaping characters . . . 52
6.7.2 Unicode characters . . . 54
6.8 Strings . . . 54
6.8.1 Unicode strings . . . 55

7 Preprocessing 56
7.1 `mactab` and `domacro()` dispatch . . . 56
7.2 `#include` . . . 59
7.2.1 Include paths, `-I` . . . 59
7.2.2 Tracing origin and `Hist` . . . 59
7.2.3 `#include` . . . 60
7.3 `#define` . . . 63

7.3.1 `-D` . . . 63
7.3.2 `#define` . . . 64
7.3.3 Macro Expansion . . . 68
7.4 `#undef` . . . 72
7.5 `#ifdef` . . . 72
7.6 `#pragma` . . . 74
7.7 `#line` . . . 74

8 Parsing **76**
8.1 Overview . . . 76
8.2 Declarations and definitions, part one . . . 77
8.2.1 No-declarator declarations . . . 77
8.2.2 Declarator-based declarations . . . 78
8.2.3 Entity declarations, the declarator . . . 78
8.2.4 Function definitions . . . 80
8.3 Statements . . . 80
8.3.1 Blocks . . . 81
8.3.2 Conditionals . . . 82
8.3.3 Loops . . . 82
8.3.4 Control flow jumps . . . 83
8.3.5 Labels and goto . . . 83
8.3.6 Switch . . . 84
8.4 Expressions . . . 84
8.4.1 Numeric constants . . . 85
8.4.2 String constants . . . 86
8.4.3 Entity uses . . . 88
8.4.4 Arithmetic expressions . . . 88
8.4.5 Boolean expressions . . . 89
8.4.6 Assignments . . . 89
8.4.7 Pointers . . . 90
8.4.8 Array accesses . . . 90
8.4.9 Field accesses . . . 90
8.4.10 Function calls . . . 90
8.4.11 Cast . . . 91
8.4.12 Ternary expressions . . . 91
8.4.13 Prefix/postfix . . . 91
8.4.14 `sizeof()` . . . 92
8.5 Initializers and designators . . . 92
8.6 Types and storage classes . . . 93
8.6.1 Basic types . . . 96
8.6.2 Storage classes . . . 98
8.6.3 Qualifiers . . . 99
8.6.4 Structures and unions . . . 100
8.6.5 Enums . . . 104
8.6.6 Pointer and array types . . . 106
8.6.7 Function types, parameter types . . . 106
8.6.8 Typedefs . . . 108
8.7 Declarations and definitions, part two . . . 109
8.7.1 Globals, external declarator . . . 109
8.7.2 Function parameters . . . 109

8.7.3 Locals, automatic declarator . . . 110
8.7.4 Types, abstract declarator . . . 112

9 Checking **113**
9.1 Overview . . . 113
9.2 Printing warnings: `5c -w` and `5c -W` . . . 113
9.3 Symbol resolution . . . 113
9.4 Storage resolution . . . 113
9.4.1 xdecl . . . 113
9.4.2 doinit . . . 114
9.4.3 pdecl . . . 114
9.4.4 adecl . . . 114
9.5 Typechecking helpers . . . 115
9.5.1 Overview . . . 115
9.5.2 Type equality: `sametype()` . . . 115
9.5.3 generic `void*` pointer conversions: `5c -V` . . . 117
9.5.4 Type merge: `tmerge()` . . . 117
9.5.5 Set of types constants: `typexxx[]` . . . 118
9.5.6 Type compatibility: `tcompat()` . . . 119
9.5.7 Compatibility policies: `txxx` . . . 120
9.5.8 Typechecker: `tcom()` . . . 121
9.5.9 Array to pointer conversions . . . 122
9.6 Typechecking expressions . . . 123
9.6.1 Numeric constants . . . 123
9.6.2 String constants . . . 123
9.6.3 Entity uses . . . 123
9.6.4 Arithmetic expressions . . . 124
9.6.5 Arithmetic conversions . . . 127
9.6.6 Pointer arithmetic . . . 130
9.6.7 Boolean expressions . . . 131
9.6.8 Assignments . . . 133
9.6.9 Pointers . . . 136
9.6.10 Array accesses . . . 137
9.6.11 Field accesses . . . 137
9.6.12 Function calls . . . 138
9.6.13 Cast . . . 141
9.6.14 Ternary expressions . . . 141
9.6.15 Prefix/postfix . . . 143
9.6.16 `sizeof()` . . . 143
9.7 Typechecking statements . . . 144
9.7.1 `return` . . . 144
9.7.2 `switch` . . . 144
9.7.3 `if` . . . 145
9.8 Typechecking declarations and constant evaluation . . . 145
9.8.1 Enumerations . . . 145
9.8.2 Arrays . . . 146
9.8.3 Functions . . . 146
9.8.4 Typedef expansions . . . 147
9.9 Lint checking . . . 147
9.9.1 Unused variables . . . 147

9.9.2 Used before set . . . 148
9.9.3 False positives silencing: SET()/USED() . . . 148
9.9.4 Redeclared or unused labels . . . 149
9.9.5 Unreachable code . . . 149
9.9.6 Missing return . . . 152
9.9.7 Unused expression . . . 152
9.9.8 Constant if: 5c -c . . . 153
9.9.9 Out of range shifting . . . 153
9.9.10 Useless comparisons . . . 153
9.10 Const checking . . . 153

10 Assembly Generation **155**
10.1 Overview . . . 155
10.2 pc, p, and nextpc() . . . 155
10.3 Functions: codgen() . . . 156
10.3.1 gpseudo() . . . 157
10.3.2 gbranch() . . . 157
10.3.3 patch() . . . 158
10.3.4 Offset . . . 158
10.4 Statements: gen() . . . 158
10.4.1 Blocks, sequences . . . 159
10.4.2 Conditionals . . . 159
10.4.3 Labels and goto . . . 160
10.4.4 Loops . . . 161
10.4.5 Control flow jumps . . . 165
10.5 Register allocation . . . 166
10.5.1 reg . . . 166
10.5.2 OREGISTER . . . 167
10.5.3 nodreg() . . . 167
10.5.4 regfree() . . . 167
10.6 Typechecking, expliciting, annotating, and rewriting: complex() . . . 168
10.7 Addressability and complexity: xcom() . . . 168
10.7.1 Addressable: INDEXED . . . 169
10.7.2 Addressability: addable . . . 169
10.7.3 Complexity: complex . . . 170
10.7.4 FNX . . . 171
10.8 Memory moves . . . 171
10.8.1 gmove() . . . 171
10.8.2 Conversions . . . 174
10.8.3 gins() . . . 176
10.8.4 naddr() . . . 176
10.8.5 regalloc() . . . 178
10.9 Expressions: cgen() . . . 179
10.9.1 Numeric constants . . . 180
10.9.2 Entity uses . . . 180
10.9.3 String constants . . . 181
10.9.4 Arithmetic expressions . . . 183
10.9.5 Boolean expressions . . . 188
10.9.6 Assignments part 1 . . . 188
10.9.7 Pointers . . . 189

10.9.8 Assignments part 2 . . . 191
10.9.9 Assignments and arithmetic . . . 193
10.9.10 Array accesses . . . 194
10.9.11 Field accesses . . . 194
10.9.12 Cast . . . 201
10.9.13 Ternary expressions . . . 202
10.9.14 Prefix/postfix . . . 203
10.9.15 `sizeof()` . . . 204
10.10 Boolean expressions . . . 204
10.11 Function calls . . . 209
10.11.1 `gargs()` . . . 210
10.11.2 Caller-save vs Callee-save . . . 211
10.11.3 Indirect function calls . . . 212
10.11.4 Complex calls . . . 212
10.12 Initializers . . . 213
10.13 Toplevel globals . . . 221
10.13.1 `gextern()` . . . 221
10.14 Complex values . . . 222
10.14.1 Complex types . . . 222
10.14.2 Complex expression . . . 222
10.14.3 Complex argument . . . 222
10.14.4 Complex return value . . . 222
10.14.5 `vlong` constant . . . 223
10.15 Other topics . . . 224
10.15.1 Sizes . . . 224
10.15.2 First argument . . . 224
10.15.3 Alignment . . . 225
10.16 Advanced assembly generation . . . 228
10.16.1 Switch . . . 228

11 Object File Generation **235**
11.1 Object format . . . 235
11.2 Instruction output: `outcode()` . . . 235
11.3 Operand output: `zaddr()` . . . 236
11.4 Symbol table, `zname()`, `h` and `ANAME` . . . 238
11.5 File and line information: `outhist()` . . . 238

12 AST-level Optimizations **240**
12.1 AST simplifications . . . 240
12.1.1 General rewrites: `ccom()` . . . 240
12.1.2 Constant evaluation: `evconst()` . . . 250
12.2 Comma hoisting: `comma()` . . . 254
12.3 Bitshifting opportunities . . . 256
12.4 And opportunities . . . 258
12.5 Immediate operands . . . 258
12.5.1 Substraction . . . 258
12.5.2 Addition, or, etc. . . . 259
12.5.3 Multiplication . . . 259
12.6 Associative-commutative arithmetic optimisations . . . 272
12.6.1 `Term` . . . 272

12.6.2 acom() . . . 272

13 Assembly-level Optimisations **279**
13.1 Nop detection . . . 279
13.2 ARM special instructions . . . 279
13.3 Register passing argument: REGARG . . . 279
13.4 Register allocation optimisations . . . 281
13.4.1 regopt() . . . 281
13.4.2 noretval() . . . 282
13.4.3 Pass 1 . . . 283
13.4.4 Pass 2 . . . 285
13.4.5 Pass 3 . . . 286
13.4.6 Pass 4 . . . 286
13.4.7 pass 5 . . . 287
13.4.8 pass 6 . . . 288
13.4.9 pass 7, peep hole optimizations . . . 289
13.4.10 pass 8 . . . 289
13.4.11 pass 9 . . . 290
13.4.12 Bits . . . 290
13.4.13 Var . . . 292
13.4.14 Reg . . . 292
13.4.15 XXX . . . 293
13.5 Peephole optimizer: peep() . . . 306
13.6 Dominators . . . 330

14 Linking Support **332**
14.1 #pragma lib and automagic linking . . . 332
14.2 Safe linking with type signatures: 5c -T . . . 333
14.2.1 sign() . . . 334
14.2.2 #pragma incomplete . . . 336

15 Debugging Support **338**
15.1 General debugging metadata . . . 338
15.2 Acid debugger metadata: 5c -a . . . 338
15.2.1 Entities . . . 338
15.2.2 Structure/union definitions . . . 340
15.3 Pickle: 5c -Z . . . 344

16 Profiling Support **348**
16.1 Text attributes . . . 348
16.2 #pragma profile . . . 348

17 Advanced Topics TODO **350**
17.1 Advanced C features . . . 350
17.1.1 Function and function pointer automatic conversions . . . 350
17.1.2 Array and pointer automatic conversions . . . 350
17.1.3 Local static . . . 350
17.1.4 Local volatile . . . 351
17.1.5 Bitfields . . . 352
17.1.6 Old style prototypes . . . 356

17.1.7 _Noreturn() . . . 359
17.2 C extensions . . . 359
17.2.1 Typing extensions: typeext() . . . 359
17.2.2 Unnamed structure elements . . . 360
17.2.3 Struct constructors . . . 363
17.2.4 Per-processor storage: extern register . . . 365
17.2.5 Type reflection: typestr() . . . 366
17.2.6 Signature reflection: signof() . . . 374
17.2.7 Format's arguments checking . . . 375
17.3 Floats . . . 382
17.4 64 bits operations . . . 387
17.4.1 com64init() . . . 387
17.4.2 Rewriting: com64() . . . 389
17.4.3 bool64() . . . 396
17.5 Endianness . . . 396
17.6 Using an external preprocessor: /bin/cpp . . . 396
17.7 Processing multiple files . . . 397
17.8 Other optimizations . . . 398
17.8.1 Packing: #pragma pack . . . 398
17.8.2 Minimizing #line history . . . 399

18 Conclusion **400**

A Debugging **401**
A.1 Dumpers . . . 401
A.1.1 cc/ Fmt data structures . . . 401
A.1.2 5c/ Fmt data structures . . . 406
A.1.3 xxxnames[] . . . 411
A.1.4 AST dumper: prtree() and 5c -x . . . 413
A.2 5c -v, verbose mode . . . 415
A.3 Preprocessing debugging . . . 415
A.3.1 Macro debugging: 5c -m . . . 415
A.3.2 File inclusion debugging: 5c -e . . . 415
A.3.3 Line information debugging: 5c -f: . . . 415
A.4 Parsing debugging . . . 415
A.4.1 Printing names: 5c -L . . . 415
A.4.2 Printing declarations: 5c -d . . . 415
A.5 Typing debugging . . . 416
A.5.1 Printing expression type trees: 5c -t . . . 416
A.6 Code generation debugging . . . 417
A.6.1 Printing processed opcodes: 5c -G . . . 417
A.6.2 Printing code generation information: 5c -g . . . 417
A.6.3 Printing generated assembly: 5c -S . . . 417
A.7 Optimization debugging . . . 419
A.7.1 Printing arithmetic trees: 5c -m . . . 419
A.7.2 Printing registerization: 5c -r . . . 419
A.8 Printing initializations: 5c -i . . . 419

B Error Management **421**
B.0.1 `errorexit()` . . . 421
B.0.2 `yyerror()` . . . 421
B.0.3 `diag()` . . . 422
B.0.4 `fatal()` . . . 422
B.0.5 Error location . . . 423

C Libc **425**
C.1 Memory management . . . 425
C.2 String management . . . 427
C.3 Bit operations . . . 427
C.4 Mathematic functions . . . 427
C.5 Key/value data structure . . . 428
C.6 Portability . . . 428

D Extra Code **429**
D.1 `cc/` . . . 429
D.1.1 `cc/cc.h` . . . 429
D.1.2 `cc/globals.c` . . . 436
D.1.3 `cc/omachcap.c` . . . 438
D.1.4 `cc/compat.c` . . . 438
D.1.5 `cc/utils.c` . . . 439
D.1.6 `cc/mac.c` . . . 439
D.1.7 `cc/sub.c` . . . 440
D.1.8 `cc/bits.c` . . . 446
D.1.9 `cc/lex.c` . . . 446
D.1.10 `cc/acid.c` . . . 448
D.1.11 `cc/pickle.c` . . . 449
D.1.12 `cc/com64.c` . . . 449
D.1.13 `cc/dcl.c` . . . 450
D.1.14 `cc/funct.c` . . . 452
D.1.15 `cc/scon.c` . . . 452
D.1.16 `cc/dpchk.c` . . . 453
D.1.17 `cc/com.c` . . . 454
D.2 `cc2/` . . . 455
D.2.1 `cc2/pswt.c` . . . 455
D.2.2 `cc2/pgen.c` . . . 455
D.3 `5c/` . . . 455
D.3.1 `5c/enam.c` . . . 455
D.3.2 `5c/gc.h` . . . 456
D.3.3 `5c/machcap.c` . . . 460
D.3.4 `5c/globals2.c` . . . 460
D.3.5 `5c/list.c` . . . 461
D.3.6 `5c/txt.c` . . . 462
D.3.7 `5c/peep.c` . . . 463
D.3.8 `5c/sgen.c` . . . 464
D.3.9 `5c/swt.c` . . . 464
D.3.10 `5c/mul.c` . . . 465
D.3.11 `5c/reg.c` . . . 466
D.3.12 `5c/cgen.c` . . . 467

Glossary 468

Indexes 469

References 469

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a C compiler.

1.1 Motivations

Why a C compiler? Because I think you are a better programmer if you fully understand how things work under the hood, and understanding how a compiler translates high-level C code in low-level assembly, and ultimately machine code, is essential.

Here are a few questions I hope this book will answer:

- How does a compiler parse a complex programming language?
- How does a compiler manage the scope of variables, structures, enumerations, or typedefs?
- How does a typechecker work?
- How does a compiler translate conditionals, loops, and other control-flow constructs in basic assembly jumps?
- How does a compiler translate a switch?
- How does a compiler convert arbitrary complex expressions into simpler assembly instructions using a small set of registers?
- How does a function pass arguments to another function? Are registers used? Is the stack used? How does a function communicate its return value to the caller?

1.2 The ARM Plan 9 C compiler: `5c`

1.3 Other compilers

Here are a few compilers that I considered for this book, but which I ultimately discarded:

- gcc
- clang
- lcc
- compcert

- bcc
- fbcc and tcc

1.4 Getting started

1.5 Requirements

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the author of `5c`, Ken Thompson, who wrote in some sense most of this book.

Chapter 2

Overview

2.1 Compiler principles

2.1.1 Frontend

2.1.2 Scope resolution

2.1.3 Use/def checking

2.1.4 Type checking

2.1.5 Backend

2.1.6 Labelization

2.1.7 Linearization

2.1.8 Frameization

2.1.9 Optimizations

2.2 `5c` command-line interface

2.3 `helloworld.c`

2.4 Input C language

2.4.1 C innovations

2.5 Output object format

2.6 The ARM architecture

2.7 Code organization

2.8 Software architecture

2.9 Book structure

Chapter 3

Core Data Structures

3.1 Abstract syntax tree: Node

⟨*enum* node_kind 16a⟩≡ (429b)

```
enum Node_kind
{
    OXXX,

    // ----------------------------------------------------------------------
    // Names
    // ----------------------------------------------------------------------
    ONAME, // for uses and declarations

    // ----------------------------------------------------------------------
    // Expressions
    // ----------------------------------------------------------------------
    ⟨expression nodes 84j⟩

    // ----------------------------------------------------------------------
    // Statements
    // ----------------------------------------------------------------------
    ⟨statement nodes 82a⟩

    // ----------------------------------------------------------------------
    // Declarations (parameters, initializers, bit fields)
    // ----------------------------------------------------------------------
    ⟨declaration nodes 92g⟩

    // ----------------------------------------------------------------------
    // Misc
    // ----------------------------------------------------------------------
    ⟨misc nodes 17b⟩

    // ----------------------------------------------------------------------
    // Post parsing nodes
    // ----------------------------------------------------------------------
    ⟨after parsing nodes 167c⟩

    OEND
};
```

⟨*struct* Node 16b⟩≡ (429b)

```
struct Node
{
    // enum<node_kind>
```

```
    char op;

    // option<ref_own<Node>>
    Node* left;
    // option<ref_own<Node>>
    Node* right;

    // after preprocessing, global lineno
    long lineno;

    ⟨Node value fields 17c⟩

    // ----------------------------------------------------------------------
    // Post parsing annotations
    // ----------------------------------------------------------------------
    ⟨Node type and storage fields 19e⟩
    ⟨Node code generation fields 21b⟩
    ⟨Node origin tracking fields 91f⟩

    // ----------------------------------------------------------------------
    // Misc
    // ----------------------------------------------------------------------
    ⟨Node parsing helper fields 106b⟩
};
```

⟨*constant* Z 17a⟩≡ (429b)

```
#define Z ((Node*)nil)
```

⟨*misc nodes* 17b⟩≡ (16a)

```
OLIST, // of stmts/labels/parameters/...  and also for pairs/triples/...
```

⟨Node *value fields* 17c⟩≡ (16b) 85e ▷

```
Sym* sym; // for ONAME, ODOT/OELEM, OXXX of OLABEL/OGOTO
```

⟨*function* new 17d⟩≡ (443b)

```
Node*
new(int t, Node *l, Node *r)
{
    Node *n;

    n = alloc(sizeof(Node));
    n->op = t;
    n->left = l;
    n->right = r;

    ⟨new() set lineno 17e⟩

    return n;
}
```

⟨new() *set lineno* 17e⟩≡ (17d) 399e ▷

```
if(l && t != OGOTO)
    n->lineno = l->lineno;
else if(r)
    n->lineno = r->lineno;
else
    n->lineno = lineno;
```

⟨*global* lineno 17f⟩≡ (436d)

```
long lineno;
```

⟨*global* `nearln` 18a⟩≡ (436d)

```
long nearln;
```

⟨*function* `new1` 18b⟩≡ (443b)

```
Node*
new1(int o, Node *l, Node *r)
{
    Node *n;

    n = new(o, l, r);
    n->lineno = nearln;
    return n;
}
```

⟨*global* `znode` 18c⟩≡ (460b)

```
Node znode;
```

3.2 Type system

⟨*enum* `type_kind` 18d⟩≡ (429b)

```
enum Type_kind
{
    TXXX,

    ⟨Type_kind integer cases 18e⟩
    ⟨Type_kind float cases 18f⟩
    ⟨Type_kind void case 18g⟩
    ⟨Type_kind composite cases 19a⟩
    ⟨Type_kind other cases 108g⟩

    NTYPE,
};
```

⟨`Type_kind` *integer cases* 18e⟩≡ (18d)

```
TCHAR,
TUCHAR,

TSHORT,
TUSHORT,

TINT,
TUINT,

TLONG,
TULONG,

TVLONG,
TUVLONG,
```

⟨`Type_kind` *float cases* 18f⟩≡ (18d)

```
TFLOAT,
TDOUBLE,
```

⟨`Type_kind` *void case* 18g⟩≡ (18d)

```
TVOID,
```

⟨Type_kind *composite cases* 19a⟩≡ (18d)

```
TIND,
TARRAY,

TFUNC,

TSTRUCT,
TUNION,

TENUM,
```

⟨*struct* Type 19b⟩≡ (429b)

```
struct Type
{
    // enum<type_kind>
    char etype;

    // option<ref_own<Type>, e.g. for '*int' have TIND -link-> TINT
    Type* link;
    // option<list<ref_own<Type>>, next = Type.down, for TFUNC and TSTRUCT
    Type* down;

    ⟨Type qualifier fields 20c⟩

    // ----------------------------------------------------------------------
    // Post parsing annotations
    // ----------------------------------------------------------------------
    ⟨Type code generation fields 21d⟩

    ⟨Type other fields 19c⟩
};
```

⟨Type *other fields* 19c⟩≡ (19b) 19d ▷

```
Sym* tag;
```

⟨Type *other fields* 19d⟩+≡ (19b) ◁ 19c 370b ▷

```
Sym* sym; // for fields in structures
```

⟨Node *type and storage fields* 19e⟩≡ (16b) 19f ▷

```
Type* type;
```

⟨Node *type and storage fields* 19f⟩+≡ (16b) ◁ 19e 21a ▷

```
// enum<type_kind>, inline of Node.type->etype?
char etype;
```

⟨*constant* T 19g⟩≡ (429b)

```
#define T ((Type*)nil)
```

⟨*function* typ 19h⟩≡ (443b)

```
Type*
typ(int et, Type *d)
{
    Type *t;

    t = alloc(sizeof(Type));
    t->etype = et;
    t->link = d;

    t->down = T;
    t->garb = GXXX;
```

```
    t->sym = S;

    t->width = ewidth[et];

    t->offset = 0;
    t->shift = 0;
    t->nbits = 0;

    return t;
}
```

⟨*global* types 20a⟩≡ (436d)

```
Type* types[NTYPE];
```

⟨cinit() *types initialization* 20b⟩≡ (35a)

```
types[TXXX] = T;

types[TCHAR]   = typ(TCHAR, T);
types[TUCHAR]  = typ(TUCHAR, T);
types[TSHORT]  = typ(TSHORT, T);
types[TUSHORT] = typ(TUSHORT, T);
types[TINT]    = typ(TINT, T);
types[TUINT]   = typ(TUINT, T);
types[TLONG]   = typ(TLONG, T);
types[TULONG]  = typ(TULONG, T);
types[TVLONG]  = typ(TVLONG, T);
types[TUVLONG] = typ(TUVLONG, T);

types[TFLOAT]  = typ(TFLOAT, T);
types[TDOUBLE] = typ(TDOUBLE, T);

types[TVOID]   = typ(TVOID, T);

types[TENUM] = typ(TENUM, T);
types[TFUNC] = typ(TFUNC, types[TINT]);
types[TIND] = typ(TIND, types[TVOID]);
```

⟨Type *qualifier fields* 20c⟩≡ (19b)

```
// enum<qualifier>
char garb;
```

⟨*enum* qualifier 20d⟩≡ (429b)

```
enum Qualifier
{
    GXXX  = 0, // None

    GCONSTNT = 1<<0,
    GVOLATILE = 1<<1,

    NGTYPES  = 1<<2,
    ⟨Qualifier other cases 337⟩

};
```

3.3 Storage_class

⟨*enum* storage_class 20e⟩≡ (429b)

```
enum Storage_class
```

```
{
    CXXX,

    CAUTO,
    CPARAM,

    CEXTERN,
    CGLOBL,
    CSTATIC,

    ⟨Storage_class cases 108j⟩

    NCTYPES,
};
```

⟨Node *type and storage fields* 21a⟩+≡ (16b) ◁19f

```
// enum<storage_class>
char class;
```

3.4 Node and type attributes

⟨Node *code generation fields* 21b⟩≡ (16b) 21c▷

```
// address-able, used as a bool to mark lvalues, if can assign you can take
// the address of. used by xcom() to assign ``addressibility''.
// (also (ab)used as a bool to mark label uses (true = used in a goto))
char addable;
```

⟨Node *code generation fields* 21c⟩+≡ (16b) ◁21b 158h▷

```
// complexity in number of registers. for register allocation?
// (also (ab)used as FNX special value)
// (also (ab)used as a bool to mark label definitions (true = already defined))
char complex;
```

⟨Type *code generation fields* 21d⟩≡ (19b) 227a▷

```
long width; // ewidth[Type.etype]
```

3.5 Symbols and hash table

3.5.1 Sym

⟨*struct* Sym 21e⟩≡ (429b)

```
struct Sym
{
    // Symbolic names are used for:
    //  - identifiers (locals/params/globals, functions, typedefs
    //    and also enum constants)
    //  - tags (struct/union/enum)
    //  - labels (for the goto)
    // and also during parsing for:
    //  - macros (the #define)
    //  - keywords lexeme (abuse)

    // ----------------------------------------------------------------------
    // The "key"
    // ----------------------------------------------------------------------
    char *name;
```

```
    // ----------------------------------------------------------------------
    // The "value" for the different "namespaces"
    // ----------------------------------------------------------------------
    ⟨Sym identifier value fields 23b⟩
    ⟨Sym enum value fields 105b⟩
    ⟨Sym tag value fields 24b⟩
    ⟨Sym label value fields 83g⟩

    ⟨Sym macro value fields 24d⟩
    ⟨Sym lexeme value fields 24e⟩

    // ----------------------------------------------------------------------
    // Extra
    // ----------------------------------------------------------------------
    ⟨Sym extra fields 22c⟩
};
```

3.5.2 hash

⟨*global* hash 22a⟩≡ (436d)

```
// hash<string, ref_own?<Sym>>, (next in bucket = Sym.link)
Sym* hash[NHASH];
```

⟨*constant* NHASH 22b⟩≡ (429b)

```
#define NHASH  1024
```

⟨Sym *extra fields* 22c⟩≡ (21e)

```
// list<ref<Sym>> (next = Sym.link) bucket of hashtbl 'hash'
Sym* link;
```

⟨*constant* S 22d⟩≡ (429b)

```
#define S ((Sym*)nil)
```

⟨cinit() *hash initialization* 22e⟩≡ (35a)

```
for(i=0; i<NHASH; i++)
    hash[i] = S;
```

3.5.3 slookup() and lookup()

⟨*function* slookup 22f⟩≡ (446b)

```
Sym*
slookup(char *s)
{

    strcpy(symb, s);
    return lookup();
}
```

⟨*global* symb 22g⟩≡ (436d)

```
char symb[NSYMB];
```

⟨*constant* NSYMB 22h⟩≡ (429b)

```
#define NSYMB  1500
```

⟨*function* lookup 23a⟩≡ (446b)

```
Sym*
lookup(void)
{
    Sym *s;
    ulong h;
    char *p;
    int c, n;

    // h = hash(symb)
    h = 0;
    for(p=symb; *p;) {
        h = h * 3;
        h += *p++;
    }
    n = (p - symb) + 1;
    if((long)h < 0)
        h = ~h;
    h %= NHASH;

    c = symb[0];
    for(s = hash[h]; s != S; s = s->link) {
        if(s->name[0] != c)
            continue;
        if(strcmp(s->name, symb) == 0)
            return s;
    }

    // else
    s = alloc(sizeof(Sym));
    s->name = alloc(n);
    memmove(s->name, symb, n);

    strcpy(s->name, symb);

    s->link = hash[h];
    hash[h] = s;

    syminit(s);

    return s;
}
```

3.5.4 Identifier symbols

⟨Sym *identifier value fields* 23b⟩≡ (21e) 24a ▷

```
⟨Sym identifier value, type and storage fields 23c⟩
⟨Sym identifier value, scope fields 23d⟩
⟨Sym identifier value, checking fields 147c⟩
⟨Sym identifier value, code generation fields 158g⟩
```

⟨Sym *identifier value, type and storage fields* 23c⟩≡ (23b)

```
// ref<Type>
Type* type;
// enum<Storage_class>
char class;
```

⟨Sym *identifier value, scope fields* 23d⟩≡ (23b)

```
ushort block;
```

⟨Sym *identifier value fields* 24a⟩+≡ (21e) ◁23b
```
long varlineno;
```

3.5.5 Tag symbols

⟨Sym *tag value fields* 24b⟩≡ (21e) 24c▷
```
Type* suetag;
```

⟨Sym *tag value fields* 24c⟩+≡ (21e) ◁24b
```
ushort sueblock;
```

3.5.6 Other symbols

⟨Sym *macro value fields* 24d⟩≡ (21e)
```
char* macro;
```

⟨Sym *lexeme value fields* 24e⟩≡ (21e)
```
// enum<lexeme>
ushort lexical;
```

⟨*function* syminit 24f⟩≡ (446b)
```
void
syminit(Sym *s)
{
    s->lexical = LNAME;
    s->block = 0;
    s->offset = 0;

    s->type = T;
    s->class = CXXX;
    s->suetag = T;

    s->sig = SIGNONE;
    ⟨syminit() remaining initialisations 147d⟩
}
```

3.6 Declarations, namespaces, and scope: Decl

⟨*enum* dxxx 24g⟩≡ (429b)
```
enum Namespace
{
    DMARK, // special mark to help separate the different scopes

    DAUTO, // locals/parameters/globals/typedefs/functions identifiers
    DSUE,  // struct/union/enum tags
    DLABEL,// labels, goto
};
```

⟨*struct* Decl 24h⟩≡ (429b)
```
struct Decl
{
    Sym* sym;
    // enum<Namespace>
    short val;

    ⟨Decl sym copy fields 26b⟩
```

```
    // Extra fields
    ⟨Decl extra fields 25b⟩
};
```

⟨*global* dclstack 25a⟩≡ (436d)

```
// list<list<ref_own<Decl>>> (next = Decl.link, nested lists via DMARK)
Decl* dclstack;
```

⟨Decl *extra fields* 25b⟩≡ (24h)

```
// list<ref_own<Decl> of dclstack
Decl* link;
```

⟨*constant* D 25c⟩≡ (429b)

```
#define D ((Decl*)nil)
```

⟨cinit() *dclstack initialization* 25d⟩≡ (35a)

```
dclstack = D;
```

⟨*function* push 25e⟩≡ (450)

```
Decl*
push(void)
{
    Decl *d;

    d = alloc(sizeof(Decl));
    // add_stack(d, dclstack)
    d->link = dclstack;
    dclstack = d;

    return d;
}
```

⟨*function* push1 25f⟩≡ (450)

```
Decl*
push1(Sym *s)
{
    Decl *d;

    d = push();
    d->sym = s;
    d->val = DAUTO;

    ⟨push1() save symbol fields in decl 27g⟩

    return d;
}
```

⟨*global* blockno 25g⟩≡ (436d)

```
int blockno;
```

⟨*global* autobn 25h⟩≡ (436d)

```
int autobn;
```

⟨cinit() *namespace globals initialization* 25i⟩≡ (35a) 158d ▷

```
blockno = 0;
autobn = blockno;
```

⟨*function* markdcl 26a⟩≡ (450)

```
//@Scheck: used by cc.y
void markdcl(void)
{
    Decl *d;

    blockno++;

    d = push();
    d->val = DMARK;

    // saving current values
    d->block = autobn;
    d->offset = autoffset;

    autobn = blockno;
}
```

⟨Decl *sym copy fields* 26b⟩≡ (24h) 26c ▷

```
Type* type;  // for Sym.type and Sym.suetag
ushort block; // for Sym.block and Sym.sueblock and autobn
```

⟨Decl *sym copy fields* 26c⟩+≡ (24h) ◁ 26b 27d ▷

```
long offset; // for Sym.offset and autoffset
```

⟨*function* revertdcl 26d⟩≡ (450)

```
//@Scheck: used by cc.y
Node* revertdcl(void)
{
    Decl *d;
    Sym *s;
    Node *n;
    ⟨revertdcl() other locals 351c⟩

    n = Z;
    for(;;) {
        // d = pop(dclstack)
        d = dclstack;
        ⟨revertdcl() sanity check d 26e⟩
        dclstack = d->link;

        s = d->sym;
        switch(d->val) {
        ⟨revertdcl() switch declaration type cases 26f⟩
        }
    }
    return n;
}
```

⟨revertdcl() *sanity check d* 26e⟩≡ (26d)

```
if(d == D) {
    diag(Z, "pop off dcl stack");
    break;
}
```

⟨revertdcl() *switch declaration type cases* 26f⟩≡ (26d) 27a ▷

```
case DMARK:
    // restore info previous block
    autobn = d->block;
    autoffset = d->offset;
    // we popped everything, exit loop and return
    return n;
```

⟨revertdcl() *switch declaration type cases* 27a⟩+≡ (26d) ◁26f 27b▷

```
case DSUE:
    ⟨revertdcl() debug revert DSUE 416d⟩

    // retore info from previous tag
    s->suetag = d->type;
    s->sueblock = d->block;

    break;
```

⟨revertdcl() *switch declaration type cases* 27b⟩+≡ (26d) ◁27a 27c▷

```
case DLABEL:
    ⟨revertdcl() debug revert DLABEL 416e⟩

    ⟨reverdcl() DLABEL case, warn if label not used 149c⟩
    s->label = Z;
    break;
```

⟨revertdcl() *switch declaration type cases* 27c⟩+≡ (26d) ◁27b

```
case DAUTO:
    ⟨revertdcl() debug revert DAUTO 416f⟩

    ⟨reverdcl() DAUTO case, warn if auto declared but not used 148c⟩
    ⟨reverdcl() if volatile symbol 351d⟩
    // restore info previous identifier
    ⟨reverdcl() DAUTO case, restore symbol fields from decl 27f⟩

    break;
```

⟨Decl *sym copy fields* 27d⟩+≡ (24h) ◁26c 27e▷

```
char class;
long varlineno;
```

⟨Decl *sym copy fields* 27e⟩+≡ (24h) ◁27d

```
bool aused;
```

⟨reverdcl() *DAUTO case, restore symbol fields from decl* 27f⟩≡ (27c)

```
s->type = d->type;
s->offset = d->offset;
s->block = d->block;

s->class = d->class;
s->varlineno = d->varlineno;
s->aused = d->aused;
```

⟨push1() *save symbol fields in decl* 27g⟩≡ (25f)

```
d->type   = s->type;
d->offset = s->offset;
d->block  = s->block;

d->class  = s->class;
d->aused  = s->aused;
d->varlineno = s->varlineno;
```

3.7 Token

⟨*union yacc* 28a⟩≡ (76a)

```
%union  {
    vlong   vval;
    double  dval;
    struct
    {
        char*   s;
        long    l;
    } sval;
    Sym*    sym;

   ⟨union yacc other fields 76c⟩
}
```

⟨*token declarations* 28b⟩≡ (76a) 28c ▷

```
%token  <sym>   LNAME LTYPE
%token  <vval>  LCONST LUCONST  LLCONST  LULCONST   LVLCONST LUVLCONST
%token  <dval>  LFCONST LDCONST
%token  <sval>  LSTRING LLSTRING
```

⟨*token declarations* 28c⟩+≡ (76a) ◁ 28b 148d ▷

```
%token  LVOID   LCHAR LSHORT LINT LLONG   LDOUBLE LFLOAT   LSIGNED LUNSIGNED
%token  LSTRUCT LUNION LENUM
%token  LTYPEDEF
%token  LCONSTNT LVOLATILE  LRESTRICT LINLINE
%token  LAUTO LSTATIC LEXTERN LREGISTER
%token  LIF LELSE  LWHILE LDO LFOR  LBREAK LCONTINUE  LRETURN LGOTO
%token  LSWITCH LCASE LDEFAULT
%token  LSIZEOF
```

3.8 Assembly Instruction output

⟨*struct* Prog*(arm)* 28d⟩≡ (456)

```
struct Prog
{
    // enum<Opcode>, from 5.out.h
    char as;

    Adr from;
    Adr to;

    long lineno; // of the corresponding line in the C program!

    ⟨Prog other fields(arm) 28e⟩

    // Extra
    ⟨Prog extra fields(arm) 155f⟩
};
```

⟨Prog *other fields(arm)* 28e⟩≡ (28d) 28f ▷

```
// option<enum<registr>>, None = R_NONE, see 5.out.h
// abused also to represent TEXT or DATA attributes (e.g. NOPROF)
char reg;
```

⟨Prog *other fields(arm)* 28f⟩+≡ (28d) ◁ 28e

```
// bitset<?>
byte scond;
```

⟨*struct* Adr*(arm)* 29a⟩≡ (456)

```
struct Adr
{
    // enum<Operand_kind> (D_NONE by default)
    char type;

    long offset; // generic value, offset or simple constant
    char sval[NSNAME];
    double dval;
    Ieee ieee;

    ⟨Adr other fields(arm) 29c⟩
    ⟨Adr type fields(arm) 178a⟩
};
```

⟨*constant* A 29b⟩≡ (456)

```
#define A ((Adr*)nil)
```

⟨Adr *other fields(arm)* 29c⟩≡ (29a) 29d ▷

```
// option<enum<registr>> None = R_NONE
char reg;
```

⟨Adr *other fields(arm)* 29d⟩+≡ (29a) ◁ 29c

```
Sym* sym;
// enum<Sym_kind>
char symkind;
```

⟨*global* zprog 29e⟩≡ (460b)

```
Prog zprog;
```

⟨ginit() *zprog initialisation* 29f⟩≡ (35d)

```
zprog.link = P;
zprog.as = AGOK;
zprog.reg = R_NONE;
zprog.from.type = D_NONE;
zprog.from.symkind = N_NONE;
zprog.from.reg = R_NONE;
zprog.to = zprog.from;
zprog.scond = COND_ALWAYS;
```

Chapter 4

main()

⟨*global* thechar 30a⟩≡ (436d)

```
int thechar;
```

⟨*global* thestring 30b⟩≡ (436d)

```
char* thestring;
```

⟨*global* outfile 30c⟩≡ (436d)

```
char* outfile = nil;
```

⟨*function* main 30d⟩≡ (446b)

```
//@Scheck: not dead, entry point :)
void main(int argc, char *argv[])
{
    int err;
    ⟨main() locals 30e⟩

    memset(debug, 0, sizeof(debug));
    ⟨main() xxxinit() 31a⟩
    ⟨main() remaining initialisation 31i⟩

    ARGBEGIN {
    ⟨main() command line processing 31c⟩
    } ARGEND

    if(argc < 1 && outfile == nil) {
        print("usage: %cc [-options] files\n", thechar);
        errorexit();
    }

    ⟨main() multiple files handling 397b⟩
    //else

    err = compile((argc == 0) ? "stdin" : argv[0], defs, ndef);

    if(err)
        errorexit();
    exits(nil);
}
```

⟨main() *locals* 30e⟩≡ (30d) 31d ▷

```
// growing_array<string>
char **defs = nil;
int ndef = 0;
```

⟨main() *xxxinit()* 31a⟩≡ (30d)
```
tinit(); // type globals initialisation
cinit(); // C lexing/parsing globals initialisation
ginit(); // arch dependent globals initialisation, 5c/8c/...
arginit(); // printf argument checking initialisation
```

⟨ginit() *thexxx initialisation* 31b⟩≡ (35d)
```
thechar = '5';
thestring = "arm";
```

4.1 Arguments processing

⟨main() *command line processing* 31c⟩≡ (30d) 31e ▷
```
case 'o':
    outfile = ARGF();
    break;
```

⟨main() *locals* 31d⟩+≡ (30d) ◁ 30e 31f ▷
```
char *p;
```

⟨main() *command line processing* 31e⟩+≡ (30d) ◁ 31c 31h ▷
```
case 'D':
    p = ARGF();
    if(p) {
        ⟨main() when -D in command line processing, grow defs array 31g⟩
         defs[ndef++] = p;

         dodefine(p);
    }
    break;
```

⟨main() *locals* 31f⟩+≡ (30d) ◁ 31d 31j ▷
```
char **np;
int maxdef = 0;
```

⟨main() *when -D in command line processing, grow defs array* 31g⟩≡ (31e)
```
 // realloc, growing array
 if(ndef >= maxdef){
     maxdef += 50;
     np = alloc(maxdef * sizeof *np);
     if(defs != nil)
         memmove(np, defs, (maxdef - 50) * sizeof *np);
     defs = np;
 }
```

⟨main() *command line processing* 31h⟩+≡ (30d) ◁ 31e 32a ▷
```
case 'I':
    p = ARGF();
    if(p)
        setinclude(p);
    break;
```

⟨main() *remaining initialisation* 31i⟩≡ (30d) 348b ▷
```
setinclude(".");
```

⟨main() *locals* 31j⟩+≡ (30d) ◁ 31f 397a ▷
```
int c;
```

⟨main() *command line processing* 32a⟩+≡ (30d) ◁31h

```
default:
    c = ARGC();
    if(c >= 0 && c < sizeof(debug))
        debug[c]++;
    break;
```

4.2 compile()

⟨*global* outbuf 32b⟩≡ (436d)

```
Biobuf outbuf;
```

⟨*global* diagbuf 32c⟩≡ (436d)

```
Biobuf diagbuf;
```

⟨compile() *locals* 32d⟩≡ (32e) 33c▷

```
char *p;
fdt ofd;
static bool first = true;
```

⟨*function* compile 32e⟩≡ (446b)

```
/// main -> <> (called multiple times if multifile parallel option)
int
compile(char *infile, char **defs, int ndef)
{
    ⟨compile() locals 32d⟩

    ⟨compile() set p to basename(infile) and adjust include 33d⟩

    if(outfile == nil) {
        ⟨compile() set outfile, using p, to basename(infile).thechar 33e⟩
    }

    ⟨compile() setinclude("/thestring/include") or INCLUDE 33g⟩

    if (first)
        Binit(&diagbuf, STDOUT, OWRITE);
    first = false;

    ⟨compile() if writing acid to standard output 338a⟩
    else {
        ofd = mycreat(outfile, 0664);
        ⟨compile() sanity check ofd 33a⟩
        Binit(&outbuf, ofd, OWRITE);
    }

    ⟨compile() initialize IO 36g⟩

    ⟨compile() if use ANSI preprocessor 396e⟩
    else {
        if(strcmp(infile, "stdin") == 0)
            newfile(infile, STDIN);
        else
            newfile(infile, -1);
    }

    // The big call!
    yyparse();
```

```
    if(!debug['a'] && !debug['Z'])
        // Another important call, calls outcode()
        gclean();
    return nerrors;
}
```

⟨compile() *sanity check ofd* 33a⟩≡ (32e)

```
if(ofd < 0) {
    diag(Z, "cannot open %s - %r", outfile);
    outfile = nil;
    errorexit();
}
```

⟨cinit() *lexing globals initialization* 33b⟩≡ (35a) 36a ▷

```
nerrors = 0;
```

⟨compile() *locals* 33c⟩+≡ (32e) ◁ 32d 33f ▷

```
char ofile[400];
```

⟨compile() *set p to basename(infile) and adjust include* 33d⟩≡ (32e)

```
strcpy(ofile, infile);
p = utfrrune(ofile, pathchar());
if(p) {
    *p++ = '\0';
    if(!debug['.'])
        include[0] = strdup(ofile);
} else
    p = ofile;
```

⟨compile() *set outfile, using p, to basename(infile).thechar* 33e⟩≡ (32e)

```
outfile = p;
if(outfile) {
    if(p = utfrrune(outfile, '.'))
        if(p[1] == 'c' && p[2] == 0)
            p[0] = '\0';
    p = utfrune(outfile, 0);
    ⟨compile() adjust p for outfile if acid option 338b⟩
    ⟨compile() adjust p for outfile if pickle option 344a⟩
    else {
        p[0] = '.';
        p[1] = thechar;
        p[2] = '\0';
    }
} else
    outfile = "/dev/null";
```

⟨compile() *locals* 33f⟩+≡ (32e) ◁ 33c 396d ▷

```
char incfile[20];
```

⟨compile() *setinclude("/thestring/include") or INCLUDE* 33g⟩≡ (32e)

```
if(p = getenv("INCLUDE")) {
    setinclude(p);
} else {
    if(systemtype(Plan9)) {
        sprint(incfile, "/%s/include", thestring);
        setinclude(strdup(incfile));
        setinclude("/sys/include");
    }
}
```

4.3 gclean()

⟨*function* gclean*(arm)* 34a⟩≡ (462)

```
/// main -> compile -> yyparse; <>
void
gclean(void)
{
    ⟨gclean() locals 167a⟩

    ⟨gclean() sanity check reg 167b⟩

    ⟨gclean() adjust symstring width 182b⟩
    ⟨gclean() adjust symrathole width 195f⟩

    ⟨gclean() generate all AGLOBL pseudo opcodes 221b⟩
    ⟨gclean() generate last opcode, AEND 156c⟩

    // generate the whole output file using outbuf global
    outcode();
}
```

4.4 Initializations

4.4.1 tinit()

⟨*function* tinit 34b⟩≡ (443b)

```
/// main -> <>
void
tinit(void)
{
    int *ip;
    Init *p;

    ⟨tinit() initialise thash 34c⟩
    ⟨tinit() initialise xxxnames debugging arrays 412c⟩
    ⟨tinit() initialise typexxx type sets 118g⟩
    ⟨tinit() initialise tcompat arrays 120e⟩
    ⟨tinit() initialise 32 bits defaults type sets 144e⟩
}
```

⟨tinit() *initialise thash* 34c⟩≡ (34b)

```
for(p=thashinit; p->code >= 0; p++) {
    urk("thash", nelem(thash), p->code);
    thash[p->code] = p->value;
}
```

⟨*function* urk 34d⟩≡ (443b)

```
void
urk(char *name, int max, int i)
{
    if(i >= max) {
        fprint(2, "bad tinit: %s %d>=%d\n", name, i, max);
        exits("init");
    }
}
```

4.4.2 cinit()

⟨*function* cinit 35a⟩≡ (446b)

```
/// main -> <>
void
cinit(void)
{
    int i;
    ⟨cinit() locals 42b⟩

    ⟨cinit() lexing globals initialization 33b⟩
    ⟨cinit() memory globals initialization 425e⟩
    ⟨cinit() types initialization 20b⟩
    ⟨cinit() hash initialization 22e⟩
    ⟨cinit() symbol table initialization 43a⟩
    ⟨cinit() namespace globals initialization 25i⟩
    ⟨cinit() dclstack initialization 25d⟩
    ⟨cinit() symstring initialization 181i⟩
    ⟨cinit() nodproto initialization 140f⟩
    ⟨cinit() pathname initialisation from cwd 35c⟩
    ⟨cinit() fmtinstall 401b⟩
}
```

⟨*global* pathname 35b⟩≡ (436d)

```
char* pathname;
```

⟨cinit() *pathname initialisation from cwd* 35c⟩≡ (35a)

```
pathname = allocn(pathname, 0, 100);
if(getwd(pathname, 99) == 0) {
    pathname = allocn(pathname, 100, 900);
    if(getwd(pathname, 999) == 0)
        strcpy(pathname, "/???");
}
```

4.4.3 ginit()

⟨*function* ginit*(arm)* 35d⟩≡ (462)

```
/// main -> <>
void
ginit(void)
{
    ⟨ginit() locals 195d⟩

    ⟨ginit() thexxx initialisation 31b⟩
    ⟨ginit() pc initialisation 155b⟩
    ⟨ginit() misc initialisations 156b⟩
    ⟨ginit() zprog initialisation 29f⟩
    ⟨ginit() special nodes initialisation 157b⟩
    ⟨ginit() com64init initialisation 387c⟩
    ⟨ginit() reg and resvreg initialisation 166e⟩
}
```

4.4.4 arginit()

Chapter 5

Input

⟨cinit() *lexing globals initialization* 36a⟩+≡ (35a) ◁33b 37b▷

```
lineno = 1;
```

5.1 Files management: iostack

⟨*struct* Io 36b⟩≡ (429b)

```
struct Io
{
    // option<fdt> (None = -1)
    short f;
    ⟨Io buffer fields 37g⟩
    // Extra
    ⟨Io extra fields 36d⟩
};
```

⟨*global* iostack 36c⟩≡ (436d)

```
// list<ref_own<Io> (next = Io.link)
Io* iostack;
```

⟨Io *extra fields* 36d⟩≡ (36b)

```
// list<ref_own<Io>> (from = iostack or iofree)
Io* link;
```

⟨*constant* I 36e⟩≡ (429b)

```
#define I ((Io*)nil)
```

⟨*global* ionext 36f⟩≡ (436d)

```
// option<ref<Io>>
Io* ionext;
```

⟨compile() *initialize IO* 36g⟩≡ (32e)

```
// for embedded cpp
newio();
```

⟨*function* newio 36h⟩≡ (446b)

```
void
newio(void)
{
    Io *i;

    ⟨newio() allocate a new Io in i or find a free one 37c⟩
    i->c = 0;
    i->f = -1;
    ionext = i;
}
```

⟨*global* `iofree` 37a⟩≡ (436d)

```
Io* iofree;
```

⟨`cinit()` *lexing globals initialization* 37b⟩+≡ (35a) ◁36a 41a▷

```
iostack = I;
iofree = I;
```

⟨`newio()` *allocate a new Io in* `i` *or find a free one* 37c⟩≡ (36h)

```
static int pushdepth = 0;

i = iofree;
if(i == I) {
    ⟨newio() sanity check depth of macro expansion 37d⟩
    i = alloc(sizeof(*i));
} else
    // pop(iofree)
    iofree = i->link;
```

⟨`newio()` *sanity check depth of macro expansion* 37d⟩≡ (37c)

```
pushdepth++;
if(pushdepth > 1000) {
    yyerror("macro/io expansion too deep");
    errorexit();
}
```

⟨*function* `newfile` 37e⟩≡ (446b)

```
/// main | macinc -> <>
void
newfile(char *s, int f)
{
    Io *i;

    ⟨newfile() debug 415d⟩

    // add_list(ionext, iostack)
    i = ionext;
    i->link = iostack;
    iostack = i;

    i->f = f;
    if(f < 0)
        i->f = open(s, 0);
    if(i->f < 0) {
        yyerror("%cc: %r: %s", thechar, s);
        errorexit();
    }
    fi.c = 0;

    ⟨newfile() call linehist 60e⟩
}
```

5.2 Buffer management: `fi`

⟨*constant* `BUFSIZ` 37f⟩≡ (429b)

```
#define BUFSIZ  8192
```

⟨`Io` *buffer fields* 37g⟩≡ (36b) 39▷

```
char b[BUFSIZ];
```

⟨*struct* Fi 38a⟩≡ (429b)

```
struct Fi
{
    // ref<char> (target = Io.b)
    char* p;
    // remaining characters in Io.b to read
    int c;
};
```

⟨*global* fi 38b⟩≡ (436d)

```
struct Fi fi;
```

5.3 GETC()

⟨*function* GETC 38c⟩≡ (446b)

```
#define GETC() ((--fi.c < 0)? filbuf(): (*fi.p++ & 0xff))
```

⟨*constant* EOF 38d⟩≡ (446b)

```
#define EOF (-1)
```

⟨*function* filbuf 38e⟩≡ (446b)

```
int
filbuf(void)
{
    Io *i;

loop:
    i = iostack;
    if(i == I)
        return EOF;
    if(i->f < 0)
        goto pop;
    fi.c = read(i->f, i->b, BUFSIZ) - 1;
    if(fi.c < 0) {
        close(i->f);
        ⟨filbuf() when close file, call linehist 60f⟩
        goto pop;
    }
    fi.p = i->b + 1;
    return i->b[0] & 0xff;

⟨filbuf() pop 38f⟩
}
```

⟨filbuf() *pop* 38f⟩≡ (38e)

```
pop:
    // pop(iostack)
    iostack = i->link;
    // push(i, iofree)
    i->link = iofree;
    iofree = i;

    // i = top(iostack), the fresh top of the stack input file
    i = iostack;
    if(i == I)
        return EOF;
    // restore file pointers
    fi.p = i->p;
```

```
    fi.c = i->c;
    if(--fi.c < 0)
        goto loop;
    return *fi.p++ & 0xff;
```

⟨Io *buffer fields* 39⟩+≡ (36b) ◁37g

```
  // like Fi, saved pointers in Io.b
  char* p;
  short c;
```

Chapter 6

Lexing

6.1 yylex()

⟨*function* yylex 40a⟩≡ (446b)

```
//@Scheck: not dead, called by yyparse
long yylex(void)
{
    long c; // Rune?
    ⟨yylex() locals 41c⟩

    ⟨yylex() peekc handling, starting part 41b⟩
l0:
    c = GETC();

l1:
    ⟨yylex() if unicode character 43g⟩
    if(isspace(c)) {
        ⟨yylex() if c is newline 40b⟩
        // ignore spaces
        goto l0;
    }
    ⟨yylex() before switch, if isxxx 43d⟩
    switch(c) {
    ⟨yylex() switch c cases 41e⟩
    default:
        return c;
    }
    ⟨yylex() peekc handling, ending part 41d⟩
    return c;

⟨yylex() labels 43f⟩
}
```

⟨yylex() *if c is newline* 40b⟩≡ (40a)

```
if(c == '\n')
    lineno++;
```

6.2 Peek, seek, and look ahead

⟨*global* peekc 40c⟩≡ (436d)

```
int peekc;
```

⟨*constant* IGN 40d⟩≡ (446b)

```
#define IGN (-2)
```

⟨cinit() *lexing globals initialization* 41a⟩+≡ (35a) ◁37b

```
peekc = IGN;
```

⟨yylex() *peekc handling, starting part* 41b⟩≡ (40a)

```
if(peekc != IGN) {
    c = peekc;
    peekc = IGN;
    goto l1;
}
```

⟨yylex() *locals* 41c⟩≡ (40a) 43b▷

```
long c1;
```

⟨yylex() *peekc handling, ending part* 41d⟩≡ (40a)

```
peekc = c1;
```

⟨yylex() *switch c cases* 41e⟩≡ (40a) 41f▷

```
case EOF:
    peekc = EOF;
    return -1; // EOF
```

6.3 Comments

⟨yylex() *switch c cases* 41f⟩+≡ (40a) ◁41e 43c▷

```
case '/':
    c1 = GETC();
    if(c1 == '*') {
        // '/''*' read; skip everything until next '*''/'
        for(;;) {
            c = getr();
            while(c == '*') {
                c = getr();
                if(c == '/')
                    goto l0;
            }
            if(c == EOF) {
                yyerror("eof in comment");
                errorexit();
            }
        }
    }
    if(c1 == '/') {
        // '/''/' read; skip everything until next '\n'
        for(;;) {
            c = getr();
            if(c == '\n')
                goto l0;
            if(c == EOF) {
                yyerror("eof in comment");
                errorexit();
            }
        }
    }
    if(c1 == '=')
        return LDVE;
    break;
```

6.4 Keywords and identifiers

⟨*global* itab 42a⟩≡ (446b)

```
struct
{
    char *name;
    // enum<lexeme>
    ushort lexical;
    // option<enum<Type_kind>>, None = 0 (or TXXX)
    ushort type;
} itab[] =
{
    "auto",  LAUTO,  0,
    "static", LSTATIC, 0,
    "extern", LEXTERN, 0,
    "register", LREGISTER, 0,

    "const", LCONSTNT, 0,
    "volatile", LVOLATILE, 0,

    "inline", LINLINE, 0,
    "restrict", LRESTRICT, 0,

    "void",  LVOID,  TVOID,
    "char",  LCHAR,  TCHAR,
    "int",  LINT,  TINT,
    "short", LSHORT,  TSHORT,
    "long",  LLONG,  TLONG,
    "float", LFLOAT,  TFLOAT,
    "double", LDOUBLE, TDOUBLE,

    "unsigned", LUNSIGNED, 0,
    "signed", LSIGNED, 0,

    "struct", LSTRUCT, 0,
    "union", LUNION,  0,
    "enum",  LENUM,  0,

    "typedef", LTYPEDEF, 0,

    "if",  LIF,  0,
    "else",  LELSE,  0,
    "while", LWHILE,  0,
    "do",  LDO,  0,
    "for",  LFOR,  0,
    "break", LBREAK,  0,
    "continue", LCONTINUE, 0,
    "switch", LSWITCH, 0,
    "case",  LCASE,  0,
    "default", LDEFAULT, 0,
    "return", LRETURN, 0,
    "goto",  LGOTO,  0,

    "sizeof", LSIZEOF, 0,
    ⟨itab entries, kencc extensions 148e⟩
    0
};
```

⟨cinit() *locals* 42b⟩≡ (35a) 181h ▷

```
Sym *s;
```

⟨cinit() *symbol table initialization* 43a⟩≡ (35a)

```
for(i=0; itab[i].name; i++) {
    s = slookup(itab[i].name);
    s->lexical = itab[i].lexical;
    if(itab[i].type != 0)
        s->type = types[itab[i].type];
}
```

⟨yylex() *locals* 43b⟩+≡ (40a) ◁41c 43e▷

```
char *cp;
```

⟨yylex() *switch c cases* 43c⟩+≡ (40a) ◁41f 44c▷

```
case '_':
    cp = symb;
    goto talph;
```

⟨yylex() *before switch, if isxxx* 43d⟩≡ (40a) 46b▷

```
if(isalpha(c)) {
    cp = symb;
    ⟨yylex() if L followed by unicode strings or character 54a⟩
    goto talph;
}
```

⟨yylex() *locals* 43e⟩+≡ (40a) ◁43b 46e▷

```
Sym *s;
```

⟨yylex() *labels* 43f⟩≡ (40a) 46d▷

```
talph:
    /*
     * cp is set to symb and some
     * prefix has been stored
     */
    for(;;) {
        ⟨yylex() in talph case, if unicode character 44a⟩
        if(!isalnum(c) && c != '_')
            break;
        *cp++ = c;
        c = GETC();
    }
    *cp = '\0';
    ⟨yylex() in talph case, debug symbol 415f⟩
    peekc = c;

    // the important call!
    s = lookup();

    ⟨yylex() in talph case, if macro symbol 68c⟩

    yylval.sym = s;
    ⟨yylex() alpha case, return LTYPE if typedef symbol s 44b⟩
    return s->lexical;
```

6.4.1 Unicode identifiers

⟨yylex() *if unicode character* 43g⟩≡ (40a)

```
if(c >= Runeself) {
    /*
     * extension --
     * all multibyte runes are alpha
```

```
     */
    cp = symb;
    goto talph;
}
```

⟨yylex() *in talph case, if unicode character* 44a⟩≡ (43f)

```
if(c >= Runeself) {
    for(c1=0;;) {
        cp[c1++] = c;
        if(fullrune(cp, c1))
            break;
        c = GETC();
    }
    cp += c1;
    c = GETC();
    continue;
}
```

6.4.2 Typedef trick

⟨yylex() *alpha case, return LTYPE if typedef symbol s* 44b⟩≡ (43f)

```
if(s->class == CTYPEDEF || s->class == CTYPESTR)
    return LTYPE;
```

6.5 Operators

⟨yylex() *switch c cases* 44c⟩+≡ (40a) ◁43c 44d▷

```
case '*':
    c1 = GETC();
    if(c1 == '=')
        return LMLE;
    break;
```

⟨yylex() *switch c cases* 44d⟩+≡ (40a) ◁44c 44e▷

```
case '%':
    c1 = GETC();
    if(c1 == '=')
        return LMDE;
    break;
```

⟨yylex() *switch c cases* 44e⟩+≡ (40a) ◁44d 44f▷

```
case '+':
    c1 = GETC();
    if(c1 == '+')
        return LPP;
    if(c1 == '=')
        return LPE;
    break;
```

⟨yylex() *switch c cases* 44f⟩+≡ (40a) ◁44e 45a▷

```
case '-':
    c1 = GETC();
    if(c1 == '-')
        return LMM;
    if(c1 == '=')
        return LME;
    if(c1 == '>')
        return LMG;
    break;
```

⟨yylex() *switch c cases* 45a⟩+≡ (40a) ◁44f 45b▷

```
case '>':
    c1 = GETC();
    if(c1 == '>') {
        c = LRSH;
        c1 = GETC();
        if(c1 == '=')
            return LRSHE;
        break;
    }
    if(c1 == '=')
        return LGE;
    break;
```

⟨yylex() *switch c cases* 45b⟩+≡ (40a) ◁45a 45c▷

```
case '<':
    c1 = GETC();
    if(c1 == '<') {
        c = LLSH;
        c1 = GETC();
        if(c1 == '=')
            return LLSHE;
        break;
    }
    if(c1 == '=')
        return LLE;
    break;
```

⟨yylex() *switch c cases* 45c⟩+≡ (40a) ◁45b 45d▷

```
case '=':
    c1 = GETC();
    if(c1 == '=')
        return LEQ;
    break;
```

⟨yylex() *switch c cases* 45d⟩+≡ (40a) ◁45c 45e▷

```
case '!':
    c1 = GETC();
    if(c1 == '=')
        return LNE;
    break;
```

⟨yylex() *switch c cases* 45e⟩+≡ (40a) ◁45d 45f▷

```
case '&':
    c1 = GETC();
    if(c1 == '&')
        return LANDAND;
    if(c1 == '=')
        return LANDE;
    break;
```

⟨yylex() *switch c cases* 45f⟩+≡ (40a) ◁45e 46a▷

```
case '|':
    c1 = GETC();
    if(c1 == '|')
        return LOROR;
    if(c1 == '=')
        return LORE;
    break;
```

⟨yylex() *switch c cases* 46a⟩+≡ (40a) ◁45f 50d▷

```
case '^':
    c1 = GETC();
    if(c1 == '=')
        return LXORE;
    break;
```

6.6 Numbers

⟨yylex() *before switch, if isxxx* 46b⟩+≡ (40a) ◁43d

```
if(isdigit(c))
    goto tnum;
```

⟨*enum* numxxx 46c⟩≡ (446b)

```
enum Numxxx
{
    Numdec  = 1<<0,
    ⟨Numxxx cases 49a⟩
    Numflt  = 1<<4,
};
```

⟨yylex() *labels* 46d⟩+≡ (40a) ◁43f 47a▷

```
tnum:
    c1 = 0;
    cp = symb;
    if(c != '0') {
        // not an hexa or octal number
        c1 |= Numdec;
        for(;;) {
            *cp++ = c;
            c = GETC();
            if(isdigit(c))
                continue;
            goto dc;
        }
    }
    ⟨yylex() in number case, 0xxx handling 48⟩
⟨yylex() in number case, decimal dc label handling 46f⟩
```

6.6.1 Decimals

⟨yylex() *locals* 46e⟩+≡ (40a) ◁43e 54c▷

```
vlong vv;
// enum<Type_kind>
long t;
```

⟨yylex() *in number case, decimal dc label handling* 46f⟩≡ (46d)

```
dc:
    ⟨yylex() in number case, in decimal case, float handling 50c⟩
    //Fallthrough
⟨yylex() in number case, in decimal case, ncu suffix label handling 49b⟩
⟨yylex() in number case, in decimal case, long suffix label handling 49e⟩
    *cp = '\0';
    peekc = c;

    if(mpatov(symb, &yylval.vval))
        yyerror("overflow in constant");
```

```
    vv = yylval.vval;
    ⟨yylex() in number case, in decimal case, if vlong number 50a⟩
    ⟨yylex() in number case, in decimal case, if long number 50b⟩
    ⟨yylex() in number case, in decimal case, if unsigned number 49c⟩
    c = LCONST;
    t = TINT;
    goto nret;
```

⟨yylex() *labels* 47a⟩+≡ (40a) ◁46d 50e▷

```
nret:
    yylval.vval = convvtox(vv, t);
    if(yylval.vval != vv){
        nearln = lineno;
        warn(Z, "truncated constant: %T %s", types[t], symb);
    }
    return c;
```

6.6.2 convvtox()

⟨*function* convvtox 47b⟩≡ (449b)

```
vlong
convvtox(vlong c, int et)
{
    int n;

    n = 8 * ewidth[et];
    c &= MASK(n);
    if(!typeu[et])
        if(c & SIGN(n))
            c |= ~MASK(n);
    return c;
}
```

⟨*function* mpatov 47c⟩≡ (446b)

```
/// yylex()#ncu -> <>
/*
 * convert a string, s, to vlong in *v
 * return conversion overflow.
 * required syntax is [0[x]]d*
 */
bool
mpatov(char *s, vlong *v)
{
    vlong n, nn;
    int c;

    n = 0;
    c = *s;
    if(c == '0')
        goto oct;
    while(c = *s++) {
        if(c >= '0' && c <= '9')
            nn = n*10 + c-'0';
        else
            goto bad;
        if(n < 0 && nn >= 0)
            goto bad;
        n = nn;
```

```
    }
    goto out;

oct:
    s++;
    c = *s;
    if(c == 'x' || c == 'X')
        goto hex;
    while(c = *s++) {
        if(c >= '0' || c <= '7')
            nn = n*8 + c-'0';
        else
            goto bad;
        if(n < 0 && nn >= 0)
            goto bad;
        n = nn;
    }
    goto out;

hex:
    s++;
    while(c = *s++) {
        if(c >= '0' && c <= '9')
            c += 0-'0';
        else
        if(c >= 'a' && c <= 'f')
            c += 10-'a';
        else
        if(c >= 'A' && c <= 'F')
            c += 10-'A';
        else
            goto bad;
        nn = n*16 + c;
        if(n < 0 && nn >= 0)
            goto bad;
        n = nn;
    }
out:
    *v = n;
    return false;

bad:
    *v = ~0;
    return true;
}
```

6.6.3 Hexadecimals and octals

⟨`yylex()` *in number case, 0xxx handling* 48⟩≡ (46d)

```
// else, octal or hexadecimal number
*cp++ = c;
c = GETC();
if(c == 'x' || c == 'X')
    for(;;) {
        *cp++ = c;
        c = GETC();
        if(isdigit(c))
            continue;
        if(c >= 'a' && c <= 'f')
```

```
            continue;
        if(c >= 'A' && c <= 'F')
            continue;
        if(cp == symb+2)
            yyerror("malformed hex constant");
        goto ncu;
    }
if(c < '0' || c > '7')
    goto dc;
for(;;) {
    if(c >= '0' && c <= '7') {
        *cp++ = c;
        c = GETC();
        continue;
    }
    goto ncu;
}
//Fallthrough
```

6.6.4 Unsigned numbers

⟨Numxxx *cases* 49a⟩≡ (46c) 49d ▷

```
Numuns  = 1<<2,
```

⟨yylex() *in number case, in decimal case, ncu suffix label handling* 49b⟩≡ (46f)

```
ncu:
    if((c == 'U' || c == 'u') && !(c1 & Numuns)) {
        c = GETC();
        c1 |= Numuns;
        goto ncu;
    }
```

⟨yylex() *in number case, in decimal case, if unsigned number* 49c⟩≡ (46f)

```
if((c1 & Numuns) || convvtox(vv, TINT) < 0) {
    c = LUCONST;
    t = TUINT;
    goto nret;
}
```

6.6.5 Long numbers

⟨Numxxx *cases* 49d⟩+≡ (46c) ◁ 49a

```
Numlong  = 1<<1,
Numvlong = 1<<3,
```

⟨yylex() *in number case, in decimal case, long suffix label handling* 49e⟩≡ (46f)

```
if((c == 'L' || c == 'l') && !(c1 & Numvlong)) {
    c = GETC();
    if(c1 & Numlong)
        c1 |= Numvlong;
    c1 |= Numlong;
    goto ncu;
}
```

⟨yylex() *in number case, in decimal case, if vlong number* 50a⟩≡ (46f)

```
if(c1 & Numvlong) {
    if((c1 & Numuns) || convvtox(vv, TVLONG) < 0) {
        c = LUVLCONST;
        t = TUVLONG;
        goto nret;
    }
    c = LVLCONST;
    t = TVLONG;
    goto nret;
}
```

⟨yylex() *in number case, in decimal case, if long number* 50b⟩≡ (46f)

```
if(c1 & Numlong) {
    if((c1 & Numuns) || convvtox(vv, TLONG) < 0) {
        c = LULCONST;
        t = TULONG;
        goto nret;
    }
    c = LLCONST;
    t = TLONG;
    goto nret;
}
```

6.6.6 Floats

⟨yylex() *in number case, in decimal case, float handling* 50c⟩≡ (46f)

```
if(c == '.')
    goto casedot;
if(c == 'e' || c == 'E')
    goto casee;
```

⟨yylex() *switch c cases* 50d⟩+≡ (40a) ◁46a 51▷

```
case '.':
    c1 = GETC();
    if(isdigit(c1)) {
        cp = symb;
        *cp++ = c;
        c = c1;
        c1 = 0;
        goto casedot;
    }
    break;
```

⟨yylex() *labels* 50e⟩+≡ (40a) ◁47a

```
casedot:
    for(;;) {
        *cp++ = c;
        c = GETC();
        if(!isdigit(c))
            break;
    }
    if(c != 'e' && c != 'E')
        goto caseout;
    // Fallthrough

casee:
    *cp++ = 'e';
    c = GETC();
```

```
    if(c == '+' || c == '-') {
        *cp++ = c;
        c = GETC();
    }
    if(!isdigit(c))
        yyerror("malformed fp constant exponent");
    while(isdigit(c)) {
        *cp++ = c;
        c = GETC();
    }
    // Fallthrough
caseout:
    if(c == 'L' || c == 'l') {
        c = GETC();
        c1 |= Numlong; // DEAD? used?
    } else
    if(c == 'F' || c == 'f') {
        c = GETC();
        c1 |= Numflt;
    }
    *cp = 0;
    peekc = c;
    yylval.dval = strtod(symb, nil);
    if(isInf(yylval.dval, 1) || isInf(yylval.dval, -1)) {
        yyerror("overflow in float constant");
        yylval.dval = 0;
    }
    if(c1 & Numflt)
        return LFCONST;
    // else
    return LDCONST;
```

6.7 Characters

⟨yylex() *switch c cases* 51⟩+≡ (40a) ◁50d 54d▷

```
case '\'':
    /* '.' */
    c = escchar('\'', false, false);
    if(c == EOF)
        c = '\'';
    c1 = escchar('\'', false, false);
    if(c1 != EOF) {
        yyerror("missing '");
        peekc = c1;
    }

    vv = c;
    yylval.vval = convvtox(vv, TUCHAR);
    if(yylval.vval != vv)
        yyerror("overflow in character constant: 0x%lx", c);
    else
    if(c & 0x80){
        nearln = lineno;
        warn(Z, "sign-extended character constant");
    }
    yylval.vval = convvtox(vv, TCHAR);
    return LCONST;
```

6.7.1 Escaping characters

⟨*function* escchar 52a⟩≡ (446b)

```
long
escchar(long e, int longflg, bool escflg)
{
    long c, l;
    int i;

loop:
    c = getr();
    if(c == '\n') {
        yyerror("newline in string");
        return EOF;
    }
    if(c != '\\') {
        if(c == e)
            c = EOF;
        return c;
    }
    // else c is '\\'
    c = getr();

    ⟨escchar() if hexadecimal character 53b⟩
    ⟨escchar() if octal character 52b⟩
    switch(c)
    {
    case '\n': goto loop; // escaped newline

    case 'n': return '\n';
    case 't': return '\t';
    case 'b': return '\b';
    case 'r': return '\r';

    case 'f': return '\f';
    case 'a': return '\a';
    case 'v': return '\v';
    }
    return c;
}
```

⟨escchar() *if octal character* 52b⟩≡ (52a)

```
if(c >= '0' && c <= '7') {
    /*
     * note this is not ansi,
     * supposed to only accept 3 oct
     */
    i = 2;
    if(longflg)
        i = 8;
    l = c - '0';
    for(; i>0; i--) {
        c = getc();
        if(c >= '0' && c <= '7') {
            l = l*8 + c-'0';
            continue;
        }
        unget(c);
    }
    if(escflg)
```

```
        l |= ESC;
    return l;
}
```

⟨*function* getr 53a⟩≡ (446b)

```
long
getr(void)
{
    int c, i;
    char str[UTFmax+1];
    Rune rune;


    c = getc();
    if(c < Runeself)
        return c;
    i = 0;
    str[i++] = c;

loop:
    c = getc();
    str[i++] = c;
    if(!fullrune(str, i))
        goto loop;
    c = chartorune(&rune, str);
    if(rune == Runeerror && c == 1) {
        nearln = lineno;
        diag(Z, "illegal rune in string");
        for(c=0; c<i; c++)
            print(" %.2x", *(uchar*)(str+c));
        print("\n");
    }
    return rune;
}
```

⟨escchar() *if hexadecimal character* 53b⟩≡ (52a)

```
if(c == 'x') {
    /*
     * note this is not ansi,
     * supposed to only accept 2 hex
     */
    i = 2;
    if(longflg)
        i = 6;
    l = 0;
    for(; i>0; i--) {
        c = getc();
        if(c >= '0' && c <= '9') {
            l = l*16 + c-'0';
            continue;
        }
        if(c >= 'a' && c <= 'f') {
            l = l*16 + c-'a' + 10;
            continue;
        }
        if(c >= 'A' && c <= 'F') {
            l = l*16 + c-'A' + 10;
            continue;
        }
        unget(c);
```

```
        break;
    }
    if(escflg)
        l |= ESC;
    return l;
}
```

6.7.2 Unicode characters

⟨yylex() *if L followed by unicode strings or character* 54a⟩≡ (43d)

```
if(c != 'L')
    goto talph;

*cp++ = c;
c = GETC();
if(c == '\'') {
    /* L'x' */
    c = escchar('\'', true, false);
    if(c == EOF)
        c = '\'';
    c1 = escchar('\'', true, false);
    if(c1 != EOF) {
        yyerror("missing '");
        peekc = c1;
    }
    yylval.vval = convvtox(c, TRUNE);
    return LUCONST;
}
if(c == '"') {
    goto caselq;
}
```

6.8 Strings

⟨*constant* ESC 54b⟩≡ (446b)

```
#define ESC (Runemask+1)  /* Rune flag: a literal byte */
```

⟨yylex() *locals* 54c⟩+≡ (40a) ◁46e

```
Rune rune;
```

⟨yylex() *switch c cases* 54d⟩+≡ (40a) ◁51 55▷

```
case '"':
    strcpy(symb, "\"<string>\""); // for error reporting
    cp = alloc(0);
    c1 = 0;

    /* "..." */
    for(;;) {
        c = escchar('"', false, true);
        if(c == EOF)
            break;
        if(c & ESC) {
            cp = allocn(cp, c1, 1);
            cp[c1++] = c;
        } else {
            rune = c;
            c = runelen(rune);
```

```
            cp = allocn(cp, c1, c);
            runetochar(cp+c1, &rune);
            c1 += c;
        }
    }
    yylval.sval.l = c1;
    do {
        cp = allocn(cp, c1, 1);
        cp[c1++] = '\0';
    } while(c1 & MAXALIGN);
    yylval.sval.s = cp;
    return LSTRING;
```

6.8.1 Unicode strings

⟨yylex() *switch c cases* 55⟩+≡ (40a) ◁54d 56a▷

```
 caselq:
    /* L"..." */
    strcpy(symb, "\"L<string>\"");
    cp = alloc(0);
    c1 = 0;
    for(;;) {
        c = escchar('"', true, false);
        if(c == EOF)
            break;
        cp = allocn(cp, c1, sizeof(TRune));
        *(TRune*)(cp + c1) = c;
        c1 += sizeof(TRune);
    }
    yylval.sval.l = c1;
    do {
        cp = allocn(cp, c1, sizeof(TRune));
        *(TRune*)(cp + c1) = 0;
        c1 += sizeof(TRune);
    } while(c1 & MAXALIGN);
    yylval.sval.s = cp;
    return LLSTRING;
```

Chapter 7

Preprocessing

7.1 mactab and domacro() dispatch

⟨yylex() *switch c cases* 56a⟩+≡ (40a) ◁55

```
case '#':
    domacro();
    goto l0;
```

⟨*global* mactab 56b⟩≡ (439b)

```
struct
{
    char *macname;
    void (*macf)(void);
} mactab[] =
{
    "ifdef",    nil,    /* macif(0) */
    "ifndef",   nil,    /* macif(1) */
    "else",     nil,    /* macif(2) */
    "endif",    macend,

    "include",  macinc,
    "line",     maclin,
    "define",   macdef,
    "undef",    macund,

    "pragma",   macprag,
    0
};
```

⟨*function* domacro 56c⟩≡ (439b)

```
void
domacro(void)
{
    Sym *s;
    int i;

    s = getsym();
    ⟨domacro() set s to endif symbol if no symbol 57a⟩

    for(i=0; mactab[i].macname; i++)
        if(strcmp(s->name, mactab[i].macname) == 0) {
            if(mactab[i].macf)
                // dispatcher!
                (*mactab[i].macf)();
            else
```

```
                macif(i);
            return;
        }
    // else
    yyerror("unknown #: %s", s->name);
    macend();
}
```

⟨domacro() *set s to endif symbol if no symbol* 57a⟩≡ (56c)

```
if(s == S)
    s = slookup("endif");
```

⟨*function* getsym 57b⟩≡ (439b)

```
Sym*
getsym(void)
{
    int c;
    char *cp;

    c = getnsc();
    if(!isalpha(c) && c != '_' && c < Runeself) {
        unget(c);
        return S;
    }
    // else isalpha() or '_'
    for(cp = symb;;) {
        if(cp <= symb+NSYMB-4)
            *cp++ = c;
        c = getc();
        if(isalnum(c) || c == '_' || c >= Runeself)
            continue;
        unget(c);
        break;
    }
    *cp = '\0';
    if(cp > symb+NSYMB-4)
        yyerror("symbol too large: %s", symb);
    return lookup();
}
```

⟨*function* getnsc 57c⟩≡ (446b)

```
int
getnsc(void)
{
    int c;

    ⟨getnsc() peekc handling 58a⟩
    else
        c = GETC();

    for(;;) {
        if(c >= Runeself || !isspace(c))
            return c;
        if(c == '\n') {
            lineno++;
            return c;
        }
        // else, was a space, so continue
        c = GETC();
    }
```

```
    return -1; // unreachable
}
```

⟨getnsc() *peekc handling* 58a⟩≡ (57c)

```
if(peekc != IGN) {
    c = peekc;
    peekc = IGN;
}
```

⟨*function* unget 58b⟩≡ (446b)

```
void
unget(int c)
{

    peekc = c;
    if(c == '\n')
        lineno--;
}
```

⟨*function* macend 58c⟩≡ (439b)

```
/// (domacro | macinc  -> yyerror; <>) | (macund -> <>)
void
macend(void)
{
    int c;

    for(;;) {
        c = getnsc();
        if(c < 0 || c == '\n')
            return;
    }
}
```

⟨*function* getc 58d⟩≡ (446b)

```
int
getc(void)
{
    int c;

    ⟨getc() peekc handling 58e⟩
    else
        c = GETC();

    if(c == '\n')
        lineno++;
    if(c == EOF) {
        yyerror("End of file");
        errorexit();
    }
    return c;
}
```

⟨getc() *peekc handling* 58e⟩≡ (58d)

```
if(peekc != IGN) {
    c = peekc;
    peekc = IGN;
}
```

7.2 #include

7.2.1 Include paths, -I

⟨*global* include 59a⟩≡ (436d)
```
// growing_array<string>
char** include;
```

⟨*global* ninclude 59b⟩≡ (436d)
```
int ninclude;
```

⟨*global* maxinclude 59c⟩≡ (436d)
```
int maxinclude;
```

⟨*function* setinclude 59d⟩≡ (446b)
```
/// main -> <>(".") | main -> <>
void
setinclude(char *p)
{
    int i;
    char *e, **np;

    while(*p != '\0') {
        e = strchr(p, ' ');
        if(e != nil)
            *e = '\0';

        // already there?
        for(i=0; i < ninclude; i++)
            if(strcmp(p, include[i]) == 0)
                break;
        // else
        if(i >= ninclude){
            ⟨setinclude() grow the array if necessary 59e⟩
            include[ninclude++] = p;
        }

        if(e == nil)
            break;
        p = e+1;
    }
}
```

⟨setinclude() *grow the array if necessary* 59e⟩≡ (59d)
```
// grow the array
if(i >= maxinclude){
    maxinclude += 20;
    np = alloc(maxinclude * sizeof *np);
    if(include != nil)
        memmove(np, include, (maxinclude - 20) * sizeof *np);
    include = np;
}
```

7.2.2 Tracing origin and Hist

⟨*struct* Hist 59f⟩≡ (429b)
```
struct Hist
{
    // option<ref_own<string> (None = nil = a ``pop'')
```

```
    char* name;

    // global line of this Hist
    long line;
    // 0 for #include, +n for #line, -1 for #pragma lib (ugly)
    long offset;

    // Extra
    ⟨Hist extra fields 60b⟩
};
```

⟨*constant* H 60a⟩≡ (429b)

```
#define H ((Hist*)nil)
```

⟨Hist *extra fields* 60b⟩≡ (59f)

```
Hist* link;
```

⟨*global* hist 60c⟩≡ (436d)

```
// list<ref_own<Hist>> (next = Hist.link)
Hist* hist;
```

⟨*global* ehist 60d⟩≡ (436d)

```
Hist* ehist;
```

⟨newfile() *call linehist* 60e⟩≡ (37e)

```
linehist(s, 0);
```

⟨filbuf() *when close file, call linehist* 60f⟩≡ (38e)

```
linehist(0, 0);
```

⟨*function* linehist 60g⟩≡ (439b)

```
void
linehist(char *f, int offset)
{
    Hist *h;

    ⟨linehist() possibly overwrite last line directive 399c⟩
    ⟨linehist() debug 415e⟩

    h = alloc(sizeof(Hist));
    h->name = f;
    h->line = lineno;
    h->offset = offset;

    // add_list(h, hist)
    h->link = H;
    if(ehist == H) {
        hist = h;
        ehist = h;
        return;
    }
    ehist->link = h;
    ehist = h;
}
```

7.2.3 #include

⟨*constant* STRINGSZ 60h⟩≡ (429b)

```
#define STRINGSZ 200
```

⟨*function* macinc 61a⟩≡ (439b)

```
void
macinc(void)
{
    char str[STRINGSZ];
    char *hp = str;
    int i;
    fdt f = -1;
    ⟨macinc() other locals 61b⟩

    // lexing

    ⟨macinc() lexing the included filename 61c⟩

    // action

    ⟨macinc() find and store the included filename full path in symb 62a⟩

    c = strlen(symb) + 1;

    // hp = malloc(c);
    while(c & 3)
        c++;
    while(nhunk < c)
        gethunk();
    hp = hunk;
    nhunk -= c;
    hunk += c;

    memcpy(hp, symb, c);

    newio();
    pushio();
    newfile(hp, f);
    return;

bad:
    unget(c);
    yyerror("syntax in #include");
    macend();
}
```

⟨macinc() *other locals* 61b⟩≡ (61a)

```
int c0, c;
```

⟨macinc() *lexing the included filename* 61c⟩≡ (61a)

```
c0 = getnsc();
if(c0 != '"') {
    c = c0;
    if(c0 != '<')
        goto bad;
    c0 = '>';
}
for(;;) {
    c = getc();
    if(c == c0)
        break;
    if(c == '\n')
        goto bad;
    *hp++ = c;
```

```
}
*hp = '\0';

c = getcom();
if(c != '\n')
    goto bad;
```

⟨macinc() *find and store the included filename full path in* symb 62a⟩≡ (61a)

```
for(i=0; i<ninclude; i++) {
    if(i == 0 && c0 == '>') // do not look in '.' for system headers
        continue;

    strcpy(symb, include[i]);
    strcat(symb, "/");
    if(strcmp(symb, "./") == 0)
        symb[0] = '\0';
    strcat(symb, str);

    f = open(symb, 0);
    if(f >= 0)
        break;

}
if(f < 0)
    strcpy(symb, str);
```

⟨*function* pushio 62b⟩≡ (446b)

```
void
pushio(void)
{
    Io *i;

    i = iostack;
    if(i == I) {
        yyerror("botch in pushio");
        errorexit();
    }
    i->p = fi.p;
    i->c = fi.c;
}
```

⟨*function* getcom 62c⟩≡ (439b)

```
int
getcom(void)
{
    int c;

    for(;;) {
        c = getnsc();
        if(c != '/')
            break;
        // else c == '/'
        c = getc();
        ⟨getcom() if // comment 63a⟩
        if(c != '*')
            break;
        // else c == '*'
        ⟨getcom() when /* comment 63b⟩
        if(c == '\n')
            break;
```

```
    }
    return c;
}
```

⟨getcom() *if* // *comment* 63a⟩≡ (62c)

```
if(c == '/') {
    while(c != '\n')
        c = getc();
    break;
}
```

⟨getcom() *when* /* *comment* 63b⟩≡ (62c)

```
c = getc();
for(;;) {
    if(c == '*') {
        c = getc();
        if(c != '/')
            continue;
        c = getc();
        break;
    }
    if(c == '\n') {
        yyerror("comment across newline");
        break;
    }
    c = getc();
}
```

7.3 #define

7.3.1 -D

⟨*function* dodefine 63c⟩≡ (439b)

```
void
dodefine(char *cp)
{
    Sym *s;
    char *p;
    long l;

    strcpy(symb, cp);
    p = strchr(symb, '=');
    if(p) {
        *p++ = '\0';
        s = lookup();
        l = strlen(p) + 2; /* +1 null, +1 nargs */

        while(l & 3)
            l++;
        while(nhunk < l)
            gethunk();

        *hunk = '\0'; // 0 nargs
        strcpy(hunk+1, p);
        s->macro = hunk;

        hunk += l;
        nhunk -= l;
```

```
    } else {
        s = lookup();
        s->macro = "\0001"; /* \000 is nargs */
    }
    ⟨dodefine() debug 415a⟩
}
```

7.3.2 #define

⟨*constant* NARG 64a⟩≡ (439b)

```
#define NARG 25
```

⟨*function* macdef 64b⟩≡ (439b)

```
void
macdef(void)
{
    Sym *s;
    char *args[NARG]; // parameters
    // option<int> (None = -1), size of parameters
    int n;
    bool dots;
    char *base; // body
    ⟨macdef() other locals 64e⟩

    // lexing
    ⟨macdef lexing the macro name, set s 64d⟩
    ⟨macdef lexing the parameters, set args, n, and dots 65a⟩
    ⟨macdef lexing the body, set base 66a⟩

    // action

    *base = n+1; // nargs (+1 to differentiate #define foo() from #define foo)
    if(dots)
        *base |= VARMAC;
    s->macro = base;
    ⟨macdef() debug 415b⟩
    return;

bad:
    if(s == S)
        yyerror("syntax in #define");
    else
        yyerror("syntax in #define: %s", s->name);
    macend();
}
```

⟨*constant* VARMAC 64c⟩≡ (439b)

```
#define VARMAC 0x80
```

⟨macdef *lexing the macro name, set* s 64d⟩≡ (64b)

```
s = getsym();
if(s == S)
    goto bad;
if(s->macro)
    yyerror("macro redefined: %s", s->name);
```

⟨macdef() *other locals* 64e⟩≡ (64b) 65c ▷

```
int c;
Sym *a;
```

⟨macdef *lexing the parameters, set* args, n, *and* dots 65a⟩≡ (64b)

```
c = getc();
n = -1; // no parameter (which is different than 0 argument)
dots = false;

if(c == '(') {
    n++; // at least 0 parameters now
    c = getnsc();
    if(c != ')') {
        unget(c);
        for(;;) {
            a = getsymdots(&dots);
            if(a == S)
                goto bad;
            if(n >= NARG) {
                yyerror("too many arguments in #define: %s", s->name);
                goto bad;
            }
            args[n++] = a->name;

            c = getnsc();
            if(c == ')')
                break;
            if(c != ',' || dots)
                goto bad;
        }
    }
    c = getc();
}
if(isspace(c))
    if(c != '\n')
        c = getnsc();
```

⟨*function* getsymdots 65b⟩≡ (439b)

```
Sym*
getsymdots(bool *dots)
{
    int c;
    Sym *s;

    s = getsym();
    if(s != S)
        return s;

    c = getnsc();
    if(c != '.'){
        unget(c);
        return S;
    }
    if(getc() != '.' || getc() != '.')
        yyerror("bad dots in macro");
    *dots = true;
    return slookup("__VA_ARGS__");
}
```

⟨macdef() *other locals* 65c⟩+≡ (64b) ◁64e 66b▷

```
//option<char> (None = 0, either ' or ")
int ischr; // quote character
int len;
```

⟨macdef *lexing the body, set* base 66a⟩≡ (64b)

```
base = hunk;
len = 1;
ischr = 0;

for(;;) {
    ⟨macdef() when lexing the body, if identifier 66c⟩
    ⟨macdef() when lexing the body, if string or character or comment 67a⟩
    ⟨macdef() when lexing the body, if antislash outside a string 68a⟩
    if(c == '\n')
        break;
    ⟨macdef() when lexing the body, if sharp character, escape it 68b⟩

    // else, a regular character (space, punctuation, etc)
    base = allocn(base, len, 1);
    base[len++] = c;

    // GETC
    c = ((--fi.c < 0)? filbuf(): (*fi.p++ & 0xff));
    if(c == '\n')
        lineno++;
    if(c == -1) {
        yyerror("eof in a macro: %s", s->name);
        break;
    }
}
do {
    base = allocn(base, len, 1);
    base[len++] = '\0';
} while(len & 3);
```

⟨macdef() *other locals* 66b⟩+≡ (64b) ◁65c

```
char *np;
int i;
```

⟨macdef() *when lexing the body, if identifier* 66c⟩≡ (66a)

```
// identifier
if(isalpha(c) || c == '_') {
    np = symb;
    *np++ = c;
    c = getc();
    while(isalnum(c) || c == '_') {
        *np++ = c;
        c = getc();
    }
    *np = '\0';

    for(i=0; i<n; i++)
        if(strcmp(symb, args[i]) == 0)
            break;
    if(i >= n) {
        i = strlen(symb);
        base = allocn(base, len, i);
        memcpy(base+len, symb, i);
        len += i;
        continue;
    }
    // else
    base = allocn(base, len, 2);
    base[len++] = '#';
```

```
        base[len++] = 'a' + i;
        continue;
    }
```

⟨macdef() *when lexing the body, if string or character or comment* 67a⟩≡ (66a)

```
    if(ischr){
        // inside a string or character
        if(c == '\\'){
            base = allocn(base, len, 1);
            base[len++] = c;
            c = getc();
        }else if(c == ischr)
            ischr = 0;
    }else{
        // string or character
        if(c == '"' || c == '\''){
            base = allocn(base, len, 1);
            base[len++] = c;
            ischr = c;
            c = getc();
            continue;
        }
        ⟨macdef() if comment 67b⟩
    }
```

⟨macdef() *if comment* 67b⟩≡ (67a)

```
    // comment
    if(c == '/') {
        c = getc();
        if(c == '/'){
            // // comment
            c = getc();
            for(;;) {
                if(c == '\n')
                    break;
                c = getc();
            }
            continue;
        }
        if(c == '*'){
            // /* comment
            c = getc();
            for(;;) {
                if(c == '*') {
                    c = getc();
                    if(c != '/')
                        continue;
                    c = getc();
                    break;
                }
                if(c == '\n') {
                    yyerror("comment and newline in define: %s", s->name);
                    break;
                }
                c = getc();
            }
            continue;
        }
        base = allocn(base, len, 1);
        base[len++] = '/';
```

```
        continue;
    }
```

⟨macdef() *when lexing the body, if antislash outside a string* 68a⟩≡ (66a)

```
    // antislash outside a string
    if(c == '\\') {
        c = getc();
        if(c == '\n') {
            c = getc();
            continue;
        }
        // else
        base = allocn(base, len, 1);
        base[len++] = '\\';
        continue;
    }
```

⟨macdef() *when lexing the body, if sharp character, escape it* 68b⟩≡ (66a)

```
    if(c == '#') // escape # by adding an extra #
      if(n > 0) {
        base = allocn(base, len, 1);
        base[len++] = c;
    }
```

7.3.3 Macro Expansion

⟨yylex() *in talph case, if macro symbol* 68c⟩≡ (43f)

```
    if(s->macro) {
        newio();
        cp = ionext->b;
        macexpand(s, cp);

        pushio();
        // push_list(ionext, iostack)
        ionext->link = iostack;
        iostack = ionext;

        fi.p = cp;
        fi.c = strlen(cp);

        ⟨yylex() in talph case, when macro symbol, peekc handling 68d⟩
        goto l0;
    }
```

⟨yylex() *in talph case, when macro symbol, peekc handling* 68d⟩≡ (68c)

```
    if(peekc != IGN) {
        cp[fi.c++] = peekc;
        cp[fi.c] = '\0';
        peekc = IGN;
    }
```

macexpand()

⟨*function* macexpand 68e⟩≡ (439b)

```
    void
    macexpand(Sym *s, char *b)
    {
        int  nargs;
```

```
    bool dots;
    int  c;
    ⟨macexpand() other locals 69a⟩

    // if macro has no parameter
    if(*s->macro == '\0') {
        strcpy(b, s->macro+1);
        ⟨macexpand() debug macro expansion 415c⟩
        return;
    }
    // else, macro has parameters

    nargs = (char)(*s->macro & ~VARMAC) - 1;
    dots = *s->macro & VARMAC;

    c = getnsc();
    if(c != '(')
        goto bad;

    ⟨macexpand() parsing the arguments 69b⟩

    ⟨macexpand() substituting the arguments 71b⟩

    ⟨macexpand() debug macro expansion 415c⟩

    return;

bad:
    yyerror("syntax in macro expansion: %s", s->name);
    *b = '\0';
    return;

toobig:
    yyerror("too much text in macro expansion: %s", s->name);
    *b = '\0';
}
```

Extracting the arguments

⟨macexpand() *other locals* 69a⟩≡ (68e)

```
char buf[2000];
int l; // depth
char *arg[NARG]; // the arguments
int n;
char *cp, *ecp;
char *ob = b;
```

⟨macexpand() *parsing the arguments* 69b⟩≡ (68e)

```
n = 0;
c = getc();
if(c != ')') {
    unget(c);

    l = 0;
    cp = buf;
    ecp = cp + sizeof(buf)-4;
    arg[n++] = cp;

    for(;;) {
```

```
        if(cp >= ecp)
            goto toobig;
        c = getc();

        ⟨macexpand() when parsing the arguments, if string 70a⟩
        ⟨macexpand() when parsing the arguments, if character 70b⟩
        ⟨macexpand() when parsing the arguments, if comment 71a⟩
        if(l == 0) {
            if(c == ',') {
                if(n == nargs && dots) {
                    *cp++ = ',';
                    continue;
                }
                *cp++ = '\0';
                arg[n++] = cp;
                if(n > nargs)
                    break;
                continue;
            }
            if(c == ')')
                break;
        }
        if(c == '\n')
            c = ' ';
        *cp++ = c;
        if(c == '(')
            l++;
        if(c == ')')
            l--;
    }
    *cp = '\0';
}
if(n != nargs) {
    yyerror("argument mismatch expanding: %s", s->name);
    *b = '\0';
    return;
}
```

⟨macexpand() *when parsing the arguments, if string* 70a⟩≡ (69b)

```
// string argument
if(c == '"')
    for(;;) {
        if(cp >= ecp)
            goto toobig;
        *cp++ = c;
        c = getc();
        if(c == '\\') {
            *cp++ = c;
            c = getc();
            continue;
        }
        if(c == '\n')
            goto bad;
        if(c == '"')
            break;
    }
```

⟨macexpand() *when parsing the arguments, if character* 70b⟩≡ (69b)

```
// quote argument
if(c == '\'')
```

```
    for(;;) {
        if(cp >= ecp)
            goto toobig;
        *cp++ = c;
        c = getc();
        if(c == '\\') {
            *cp++ = c;
            c = getc();
            continue;
        }
        if(c == '\n')
            goto bad;
        if(c == '\'')
            break;
    }
```

⟨macexpand() *when parsing the arguments, if comment* 71a⟩≡ (69b)

```
// comment
if(c == '/') {
    c = getc();
    switch(c) {
    case '*':
        for(;;) {
            c = getc();
            if(c == '*') {
                c = getc();
                if(c == '/')
                    break;
            }
        }
        *cp++ = ' '; // ???
        continue;
    case '/':
        while((c = getc()) != '\n')
            ;
        break;
    default:
        unget(c);
        c = '/';
    }
}
```

Substituting the arguments

⟨macexpand() *substituting the arguments* 71b⟩≡ (68e)

```
cp = s->macro+1;
for(;;) {
    c = *cp++;
    if(c == '\n') // can happen?
        c = ' ';
    if(c != '#') {
        *b++ = c;
        if(c == '\0')
            break;
        continue;
    }
    // else, a #
    c = *cp++;
    if(c == '\0')
```

```
        goto bad;
    if(c == '#') { // # are escaped as double ##
        *b++ = c;
        continue;
    }
    // else, a #a to #z
    c -= 'a';
    if(c < 0 || c >= n)
        continue;
    strcpy(b, arg[c]);
    b += strlen(arg[c]);
}
*b = '\0';
```

7.4 #undef

⟨*function* macund 72a⟩≡ (439b)

```
void
macund(void)
{
    Sym *s;

    // lexing
    s = getsym();
    macend();
    if(s == S) {
        yyerror("syntax in #undef");
        return;
    }

    // action
    s->macro = nil;
}
```

7.5 #ifdef

⟨*function* macif 72b⟩≡ (439b)

```
void
macif(int f)
{
    Sym *s;
    ⟨macif() other locals 73b⟩

    if(f == 2) // else
        goto skip;

    // lexing

    ⟨macif() lexing the symbol 73a⟩

    if((s->macro != nil) ^ f)
        return;
    // else

    // action
    ⟨macif() skipping text 73c⟩
```

```
bad:
    yyerror("syntax in #if(n)def");
    macend();
}
```

⟨macif() *lexing the symbol* 73a⟩≡ (72b)

```
s = getsym();
if(s == S)
    goto bad;
if(getcom() != '\n')
    goto bad;
```

⟨macif() *other locals* 73b⟩≡ (72b)

```
int c;
int l; // depth
bool bol; // beginning of line
```

⟨macif() *skipping text* 73c⟩≡ (72b)

```
skip:
    bol = true;
    l = 0;

    for(;;) {
        c = getc();
        if(c != '#') {
            if(!isspace(c))
                bol = false;
            if(c == '\n')
                bol = true;
            continue;
        }
        // else a #xxx
        if(!bol)
            continue;

        s = getsym();
        if(s == S)
            continue;
        if(strcmp(s->name, "endif") == 0) {
            if(l) {
                l--;
                continue;
            }
            // else l == 0
            macend();
            return;
        }
        if(strcmp(s->name, "ifdef") == 0 || strcmp(s->name, "ifndef") == 0) {
            l++;
            continue;
        }
        if(l == 0 && f != 2 && strcmp(s->name, "else") == 0) {
            macend();
            return;
        }
    }
```

7.6 #pragma

⟨*function* macprag 74a⟩≡ (439b)

```
void
macprag(void)
{
    Sym *s;
    ⟨macprag() other locals 332a⟩

    // lexing

    s = getsym();

    ⟨macprag() if pragma xxx 332b⟩
     else {
        while(getnsc() != '\n')
            ;
        return;
    }
}
```

7.7 #line

⟨*function* maclin 74b⟩≡ (439b)

```
void
maclin(void)
{
    char *cp;
    int c;
    long n;

    // lexing

    n = getnsn();
    c = getc();
    if(n < 0)
        goto bad;

    for(;;) {
        if(c == ' ' || c == '\t') {
            c = getc();
            continue;
        }
        if(c == '"')
            break;
        if(c == '\n') {
            strcpy(symb, "<noname>");
            goto nn;
        }
        goto bad;
    }
    cp = symb;
    for(;;) {
        c = getc();
        if(c == '"')
            break;
        *cp++ = c;
    }
```

```
    *cp = '\0';
    c = getcom();
    if(c != '\n')
        goto bad;

    // action

nn:
    c = strlen(symb) + 1;
    while(c & 3)
        c++;

    while(nhunk < c)
        gethunk();
    cp = hunk;
    memcpy(hunk, symb, c);
    nhunk -= c;
    hunk += c;

    linehist(cp, n);
    return;

bad:
    unget(c);
    yyerror("syntax in #line");
    macend();
}
```

⟨*function* getnsn 75⟩≡ (439b)

```
long
getnsn(void)
{
    long n;
    int c;

    c = getnsc();
    if(c < '0' || c > '9')
        return -1;
    n = 0;
    while(c >= '0' && c <= '9') {
        n = n*10 + c-'0';
        c = getc();
    }
    unget(c);
    return n;
}
```

Chapter 8

Parsing

8.1 Overview

⟨*cc/cc.y* 76a⟩≡
```
%{
#include "cc.h"
%}
⟨union yacc 28a⟩
⟨token declarations 28b⟩
⟨priority and associativity declarations 76b⟩
⟨type declarations 76d⟩
%%
⟨grammar 77a⟩
%%
```

⟨*priority and associativity declarations* 76b⟩≡ (76a)
```
%left   ';'
%left   ','
%right  '=' LPE LME LMLE LDVE LMDE LRSHE LLSHE LANDE LXORE LORE
%right  '?' ':'

%left   LOROR
%left   LANDAND
%left   '|'
%left   '^'
%left   '&'
%left   LEQ LNE
%left   '<' '>' LLE LGE
%left   LLSH LRSH
%left   '+' '-'
%left   '*' '/' '%'

%right  LMM LPP LMG '.' '[' '('
```

⟨union yacc *other fields* 76c⟩≡ (28a) 95a ▷
```
Node*   node;
```

⟨*type* declarations 76d⟩≡ (76a) 79c ▷
```
%type   <node>  name tag
%type   <node>  stmnt   block slist ulstmnt labels label forexpr
%type   <node>  expr    cexpr lexpr  xuexpr uexpr pexpr  elist  string lstring
%type   <node>  adecl   qual init ilist  adlist qlist  arglist
%type   <node>  xdecor xdecor2 edecor  abdecor abdecor1 abdecor2 abdecor3
%type   <node>  zexpr zelist zcexpr zarglist
```

⟨*grammar* 77a⟩≡ (76a)

```
prog:
  /* empty */
|   prog xdecl

⟨declarator rules 77b⟩
⟨statements rules 80e⟩
⟨expressions rules 84d⟩
⟨initializers rules 92h⟩
⟨types, storage classes, qualifiers rules 95b⟩
⟨names rules 79a⟩

⟨extra grammar rules 78a⟩
⟨ebnf grammar rules 77g⟩
```

8.2 Declarations and definitions, part one

⟨*declarator rules* 77b⟩≡ (77a)

```
⟨external declarator rules 77c⟩
⟨automatic declarator rules 110e⟩
⟨parameter declarator rules 107a⟩
⟨abstract declarator rules 112a⟩
⟨structure element declarator rules 103b⟩
```

⟨*external declarator rules* 77c⟩≡ (77b)

```
⟨xdecl rule 77d⟩
⟨xdlist rule 78c⟩
⟨xdecor rule 78f⟩
⟨xdecor2 rule 78g⟩
```

8.2.1 No-declarator declarations

⟨*xdecl rule* 77d⟩≡ (77c) 78b ▷

```
/*
 * external declarator
 */
xdecl:
    zctlist ';'          { dodecl(xdecl, lastclass, lasttype, Z); }
```

⟨*global* `lastclass` 77e⟩≡ (436d)

```
// enum<storage_class>
char lastclass;
```

⟨*global* `lasttype` 77f⟩≡ (436d)

```
Type* lasttype;
```

⟨*ebnf grammar rules* 77g⟩≡ (77a) 81e ▷

```
zctlist:
 /* empty */
    {
        lastclass = CXXX;
        lasttype = types[TINT];
    }
|   ctlist
```

⟨*extra grammar rules* 78a⟩≡ (77a)

```
ctlist:
    types
    {
        lastclass = $1.c;
        lasttype = $1.t;
    }
```

8.2.2 Declarator-based declarations

⟨*xdecl rule* 78b⟩+≡ (77c) ◁77d 80c▷

```
|   zctlist xdlist ';'
```

⟨*xdlist rule* 78c⟩≡ (77c) 92f▷

```
xdlist:
    xdecor  { dodecl(xdecl, lastclass, lasttype, $1); }
|   xdlist ',' xdlist
```

⟨*function* dodecl 78d⟩≡ (450)

```
Node*
dodecl(void (*f)(int,Type*,Sym*), int class, Type *t, Node *n)
{
    ⟨dodecl() locals 79e⟩

    nearln = lineno;
    ⟨dodecl() other initializations 352i⟩

loop:
    if(n != Z)
     switch(n->op) {
     // can adjust t using information from node n (type information part2)
     ⟨dodecl() switch node kind cases 79f⟩
     default:
         diag(n, "unknown declarator: %O", n->op);
         break;
     }

    lastdcltype = t;
    return n;
}
```

⟨*global* lastdcl 78e⟩≡ (436d)

```
Type* lastdcltype;
```

8.2.3 Entity declarations, the declarator

⟨*xdecor rule* 78f⟩≡ (77c) 106a▷

```
xdecor:
    xdecor2
```

⟨*xdecor2 rule* 78g⟩≡ (77c) 106d▷

```
xdecor2:
    tag
|   '(' xdecor ')'              { $$ = $2;   }
```

⟨*names rules* 79a⟩≡ (77a) 79d ▷

```
tag:
    ltag
    {
        $$ = new(ONAME, Z, Z);
        $$->sym = $1;
        ⟨tag rule, set other fields to name node 79b⟩
    }
```

⟨*tag rule, set other fields to name node* 79b⟩≡ (79a)

```
$$->type = $1->type;
$$->etype = ($1->type != T) ? $1->type->etype : TVOID;
$$->class = $1->class;

$$->xoffset = $1->offset;
```

⟨*type* declarations 79c⟩+≡ (76a) ◁76d 94b ▷

```
%type   <sym>   ltag
```

⟨*names rules* 79d⟩+≡ (77a) ◁79a 88b ▷

```
ltag:
    LNAME
|   LTYPE
```

⟨dodecl() *locals* 79e⟩≡ (78d) 106e ▷

```
Sym *s;
```

⟨dodecl() *switch node kind cases* 79f⟩≡ (78d) 106c ▷

```
case ONAME:
    ⟨dodecl() in ONAME case, break if empty callback 91h⟩
    s = n->sym;

    // callback! should set s->type, s->class, etc.
    (*f)(class, t, s);

    ⟨dodecl() case ONAME, if local static variable 350f⟩
    ⟨dodecl() case ONAME, other initializations 352e⟩

    // propagate symbol information to node
    n->sym = s; //?? dead?
    n->type = s->type;
    n->etype = (s->type != T) ? s->type->etype : TVOID;
    n->class = s->class;
    n->xoffset = s->offset;

    ⟨dodecl() debug declaration 415g⟩
    ⟨dodecl() case ONAME, debugging support 338c⟩

    s->varlineno = lineno;
    break;
```

⟨*function* xdecl 79g⟩≡ (450)

```
/// dodecl(<>, ..., node n != Z) -> <>
void
xdecl(int class, Type *t, Sym *s)
{
    long o = 0; // offset

    // adjusting class (and possibly o for CEXREG)
    switch(class) {
```

```
    ⟨xdecl() switch class cases 113d⟩
    }
    ⟨xdecl() sanity check class after switch class 114c⟩
    ⟨xdecl() sanity check type after switch class 115c⟩
    ⟨xdecl() merge type declarations 117c⟩

    s->type = t;
    s->class = class;
    s->block = 0;
    s->offset = o;
}
```

8.2.4 Function definitions

⟨*global* `firstdcl` 80a⟩≡ (436d)

```
Decl* firstdcl;
```

⟨*global* `thisfn` 80b⟩≡ (436d)

```
Type* thisfntype;
```

⟨*xdecl rule* 80c⟩+≡ (77c) ◁78b

```
|   zctlist xdecor
    {
        lastdcltype = T; //dead??
        ⟨xdecl rule, initializations before processing a function 225b⟩
        dodecl(xdecl, lastclass, lasttype, $2);
        ⟨xdecl rule, sanity check lastdcltype is a function type 80d⟩
        thisfntype = lastdcltype;
        markdcl();
        firstdcl = dclstack;
        argmark($2, 0);
    }
    pdecl
    {
        argmark($2, 1);
    }
    block
    {
        Node *n;
        n = revertdcl();
        ⟨xdecl rule, adjust block body with possible hidden generated nodes 351e⟩
        ⟨xdecl rule, debug function body 413a⟩
        if(!debug['a'] && !debug['Z'])
            codgen($6, $2); // !!!!!!!!!!!!!!!!!!!!
    }
```

⟨*xdecl rule, sanity check lastdcltype is a function type* 80d⟩≡ (80c)

```
if(lastdcltype == T || lastdcltype->etype != TFUNC) {
    diag($2, "not a function");
    lastdcltype = types[TFUNC];
}
```

8.3 Statements

⟨*statements rules* 80e⟩≡ (77a) 81b▷

```
block:
 '{' slist '}'
```

```
    {
        $$ = invert($2);
        if($$ == Z)
            $$ = new(OLIST, Z, Z);
    }

slist:
  /* empty */      { $$ = Z;    }
|   slist adecl    { $$ = new(OLIST, $1, $2); }
|   slist stmnt    { $$ = new(OLIST, $1, $2); }
```

⟨*function* invert 81a⟩≡ (443b)

```
Node*
invert(Node *n)
{
    Node *i;

    if(n == Z || n->op != OLIST)
        return n;
    i = n;
    for(n = n->left; n != Z; n = n->left) {
        if(n->op != OLIST)
            break;
        i->left = n->right;
        n->right = i;
        i = n;
    }
    i->left = n;
    return i;
}
```

⟨*statements rules* 81b⟩+≡ (77a) ◁80e 81c▷

```
stmnt:
    ulstmnt
|   labels ulstmnt { $$ = new(OLIST, $1, $2); }
|   error ';'      { $$ = Z; }
```

⟨*statements rules* 81c⟩+≡ (77a) ◁81b 82f▷

⟨*ulstmnt rule* 81d⟩
⟨*label rule* 83e⟩

⟨*ulstmnt rule* 81d⟩≡ (81c) 81f▷

```
ulstmnt:
    zcexpr ';'
```

⟨*ebnf grammar rules* 81e⟩+≡ (77a) ◁77g 85c▷

```
labels:
    label
|   labels label  { $$ = new(OLIST, $1, $2); }
```

8.3.1 Blocks

⟨*ulstmnt rule* 81f⟩+≡ (81c) ◁81d 82b▷

```
|   {
        markdcl();
    }
    block
    {
        Node *n;
```

```
        n = revertdcl();
        ⟨ulstmt rule, adjust block body with possible hidden generated nodes 351f⟩
        else
            $$ = $2;
    }
```

8.3.2 Conditionals

⟨*statement nodes* 82a⟩≡ (16a) 82c ▷

```
  OIF,
```

⟨*ulstmnt rule* 82b⟩+≡ (81c) ◁ 81f 82d ▷

```
  |   LIF '(' cexpr ')' stmnt
      {
          $$ = new(OIF, $3, new(OLIST, $5, Z));
          if($5 == Z)
              warn($3, "empty if body");
      }
  |   LIF '(' cexpr ')' stmnt LELSE stmnt
      {
          $$ = new(OIF, $3, new(OLIST, $5, $7));
          if($5 == Z)
              warn($3, "empty if body");
          if($7 == Z)
              warn($3, "empty else body");
      }
```

8.3.3 Loops

⟨*statement nodes* 82c⟩+≡ (16a) ◁ 82a 83a ▷

```
  OWHILE,
  ODWHILE,
  OFOR,
```

⟨*ulstmnt rule* 82d⟩+≡ (81c) ◁ 82b 82e ▷

```
  |   LWHILE '(' cexpr ')' stmnt          { $$ = new(OWHILE, $3, $5); }
  |   LDO stmnt LWHILE '(' cexpr ')' ';'  { $$ = new(ODWHILE, $5, $2); }
```

⟨*ulstmnt rule* 82e⟩+≡ (81c) ◁ 82d 83b ▷

```
  |   {
          markdcl();
      }
      LFOR '(' forexpr ';' zcexpr ';' zcexpr ')' stmnt
      {
          Node *n;
          n = revertdcl();
          ⟨ulstmt rule, adjust forexpr with possible hidden generated nodes 352a⟩
          $$ = new(OFOR, new(OLIST, $6, new(OLIST, $4, $8)), $10);
      }
```

⟨*statements rules* 82f⟩+≡ (77a) ◁ 81c

```
  forexpr:
      zcexpr
  |   ctlist adlist { $$ = $2; }
```

8.3.4 Control flow jumps

⟨*statement nodes* 83a⟩+≡ (16a) ◁82c 83d▷

```
ORETURN,

OBREAK,
OCONTINUE,
```

⟨*ulstmnt rule* 83b⟩+≡ (81c) ◁82e 83c▷

```
|   LRETURN zcexpr ';'
    {
        $$ = new(ORETURN, $2, Z);
        $$->type = thisfntype->link;
    }
```

⟨*ulstmnt rule* 83c⟩+≡ (81c) ◁83b 83f▷

```
|   LBREAK ';'     { $$ = new(OBREAK, Z, Z); }
|   LCONTINUE ';'  { $$ = new(OCONTINUE, Z, Z); }
```

8.3.5 Labels and goto

⟨*statement nodes* 83d⟩+≡ (16a) ◁83a 84a▷

```
OLABEL,
OGOTO,
```

⟨*label rule* 83e⟩≡ (81c) 84c▷

```
|   LNAME ':'       { $$ = new(OLABEL, dcllabel($1, true), Z); }
```

⟨*ulstmnt rule* 83f⟩+≡ (81c) ◁83c 84b▷

```
|   LGOTO ltag ';' { $$ = new(OGOTO, dcllabel($2, false), Z); }
```

⟨Sym *label value fields* 83g⟩≡ (21e)

```
Node* label;
```

⟨*function* dcllabel 83h⟩≡ (450)

```
//@Scheck: used by cc.y
Node* dcllabel(Sym *s, bool defcontext)
{
    Decl *d;
    Decl d1;
    Node *n;

    n = s->label;
    ⟨dcllabel() if n not null, mark node as declared or used 149b⟩
    // else, new label

    d = push();
    d->sym = s;
    d->val = DLABEL;

    //pop_stack(dclstack) (we add the label to the function scope instead)
    dclstack = d->link;
    //add_list(d, firstdcl)
    d1 = *firstdcl;
    *firstdcl = *d;
    *d = d1;
    firstdcl->link = d;
    firstdcl = d;

    n = new(OXXX, Z, Z);
```

```
    n->sym = s;
    ⟨dcllabel() when new label, set def and use fields 149a⟩
    s->label = n;

    ⟨dcllabel() debug declaration 415h⟩
    return n;
}
```

8.3.6 Switch

⟨*statement nodes* 84a⟩+≡ (16a) ◁83d 148f▷

```
OSWITCH,
OCASE, // for default too, in which case Node.left is null
```

⟨*ulstmnt rule* 84b⟩+≡ (81c) ◁83f 148g▷

```
|   LSWITCH '(' cexpr ')' stmnt
    {
        ⟨ulstmt rule, SWITCH case, adjust cexpr node 144b⟩
         $$ = new(OSWITCH, $3, $5);
    }
```

⟨*label rule* 84c⟩+≡ (81c) ◁83e

```
label:
    LCASE expr ':'  { $$ = new(OCASE, $2, Z); }
|   LDEFAULT ':'    { $$ = new(OCASE, Z, Z); }
```

8.4 Expressions

⟨*expressions rules* 84d⟩≡ (77a) 84i▷

```
⟨expr rule 84e⟩
⟨xuexpr rule 84f⟩
⟨uexpr rule 84g⟩
⟨pexpr rule 84h⟩
```

⟨*expr rule* 84e⟩≡ (84d) 88d▷

```
expr:
    xuexpr
```

⟨*xuexpr rule* 84f⟩≡ (84d) 91e▷

```
xuexpr:
    uexpr
```

⟨*uexpr rule* 84g⟩≡ (84d) 88f▷

```
uexpr:
    pexpr
```

⟨*pexpr rule* 84h⟩≡ (84d) 85f▷

```
pexpr:
    '(' cexpr ')' { $$ = $2; }
```

⟨*expressions rules* 84i⟩+≡ (77a) ◁84d 85a▷

```
cexpr:
    expr
|   cexpr ',' cexpr { $$ = new(OCOMMA, $1, $3); }
```

⟨*expression nodes* 84j⟩≡ (16a) 85d▷

```
OCOMMA,
```

⟨*expressions rules* 85a⟩+≡ (77a) ◁84i 85b▷

```
zexpr:
  /* empty */ { $$ = Z; }
|   lexpr
```

⟨*expressions rules* 85b⟩+≡ (77a) ◁85a 86f▷

```
lexpr:
    expr
    {
        $$ = new(OCAST, $1, Z);
        $$->type = types[TLONG];
    }
```

⟨*ebnf grammar rules* 85c⟩+≡ (77a) ◁81e 91a▷

```
zcexpr:
  /* empty */ { $$ = Z; }
|   cexpr
```

8.4.1 Numeric constants

⟨*expression nodes* 85d⟩+≡ (16a) ◁84j 86c▷

```
OCONST,
```

Integers

⟨Node *value fields* 85e⟩+≡ (16b) ◁17c 86a▷

```
vlong vconst; /* non fp const */ // for OCONST
```

⟨*pexpr rule* 85f⟩+≡ (84d) ◁84h 85g▷

```
|   LCONST
    {
        $$ = new(OCONST, Z, Z);
        $$->type = types[TINT];
        $$->vconst = $1;
        $$->cstring = strdup(symb);
    }
|   LLCONST
    {
        $$ = new(OCONST, Z, Z);
        $$->type = types[TLONG];
        $$->vconst = $1;
        $$->cstring = strdup(symb);
    }
|   LVLCONST
    {
        $$ = new(OCONST, Z, Z);
        $$->type = types[TVLONG];
        $$->vconst = $1;
        $$->cstring = strdup(symb);
    }
```

⟨*pexpr rule* 85g⟩+≡ (84d) ◁85f 86b▷

```
|   LUCONST
    {
        $$ = new(OCONST, Z, Z);
        $$->type = types[TUINT];
        $$->vconst = $1;
        $$->cstring = strdup(symb);
```

```
        }
    |   LULCONST
        {
            $$ = new(OCONST, Z, Z);
            $$->type = types[TULONG];
            $$->vconst = $1;
            $$->cstring = strdup(symb);
        }
    |   LUVLCONST
        {
            $$ = new(OCONST, Z, Z);
            $$->type = types[TUVLONG];
            $$->vconst = $1;
            $$->cstring = strdup(symb);
        }
```

Floats

⟨Node *value fields* 86a⟩+≡ (16b) ◁85e 86e▷

```
double fconst;  /* fp constant */ // for OCONST
```

⟨*pexpr rule* 86b⟩+≡ (84d) ◁85g 86d▷

```
    |   LFCONST
        {
            $$ = new(OCONST, Z, Z);
            $$->type = types[TFLOAT];
            $$->fconst = $1;
            $$->cstring = strdup(symb);
        }
    |   LDCONST
        {
            $$ = new(OCONST, Z, Z);
            $$->type = types[TDOUBLE];
            $$->fconst = $1;
            $$->cstring = strdup(symb);
        }
```

8.4.2 String constants

ASCII strings

⟨*expression nodes* 86c⟩+≡ (16a) ◁85d 87a▷

```
OSTRING,
```

⟨*pexpr rule* 86d⟩+≡ (84d) ◁86b 88a▷

```
    |   string
    |   lstring
```

⟨Node *value fields* 86e⟩+≡ (16b) ◁86a 87b▷

```
char* cstring; /* character string */ // for OSTRING (and OCONST)
```

⟨*expressions rules* 86f⟩+≡ (77a) ◁85b 87d▷

```
string:
    LSTRING
    {
        $$ = new(OSTRING, Z, Z);
        $$->type = typ(TARRAY, types[TCHAR]);
        $$->etype = TARRAY;
```

```
        $$->type->width = $1.l + 1;
        $$->cstring = $1.s;
        ⟨string rule, set sym and class for OSTRING 181f⟩
    }
|   string LSTRING
    {
        char *s;
        int n;

        n = $1->type->width - 1;
        s = alloc(n+$2.l+MAXALIGN);

        memcpy(s, $1->cstring, n);
        memcpy(s+n, $2.s, $2.l);
        s[n+$2.l] = '\0';

        $$ = $1;
        $$->type->width += $2.l;
        $$->cstring = s;
    }
```

Unicode strings

⟨*expression nodes* 87a⟩+≡ (16a) ◁86c 88c▷

```
OLSTRING,
```

⟨Node *value fields* 87b⟩+≡ (16b) ◁86e 167d▷

```
TRune* rstring; /* rune string */ // for OLSTRING
```

⟨*constant* TRUNE 87c⟩≡ (93f)

```
/* adapt size of Rune to target system's size */
TRUNE = sizeof(TRune)==4? TUINT: TUSHORT,
```

⟨*expressions rules* 87d⟩+≡ (77a) ◁86f 91b▷

```
lstring:
    LLSTRING
    {
        $$ = new(OLSTRING, Z, Z);
        $$->type = typ(TARRAY, types[TRUNE]);
        $$->etype = TARRAY;
        $$->type->width = $1.l + sizeof(TRune);
        $$->rstring = (TRune*)$1.s;
        ⟨string rule, set sym and class for OSTRING 181f⟩
    }
|   lstring LLSTRING
    {
        char *s;
        int n;

        n = $1->type->width - sizeof(TRune);
        s = alloc(n+$2.l+MAXALIGN);

        memcpy(s, $1->rstring, n);
        memcpy(s+n, $2.s, $2.l);
        *(TRune*)(s+n+$2.l) = 0;

        $$ = $1;
        $$->type->width += $2.l;
        $$->rstring = (TRune*)s;
    }
```

8.4.3 Entity uses

⟨*pexpr rule* 88a⟩+≡ (84d) ◁86d 90d▷

```
  |   name
```

⟨*names rules* 88b⟩+≡ (77a) ◁79d

```
  name:
      LNAME
      {
          $$ = new(ONAME, Z, Z);
          ⟨name rule, if local static variable 350g⟩
          $$->sym = $1;

          // propagate symbol information to node
          $$->type = $1->type;
          $$->etype = ($1->type != T) ? $1->type->etype : TVOID;
          $$->xoffset = $1->offset;
          $$->class = $1->class;
          ⟨name rule, LNAME case, adjust more fields 148b⟩
      }
```

8.4.4 Arithmetic expressions

⟨*expression nodes* 88c⟩+≡ (16a) ◁87a 88e▷

```
  OADD,
  OSUB,

  OMUL,
  ODIV,
  OMOD,
```

⟨*expr rule* 88d⟩+≡ (84d) ◁84e 88h▷

```
  |   expr '+' expr { $$ = new(OADD, $1, $3); }
  |   expr '-' expr { $$ = new(OSUB, $1, $3); }
  |   expr '*' expr { $$ = new(OMUL, $1, $3); }
  |   expr '/' expr { $$ = new(ODIV, $1, $3); }
  |   expr '%' expr { $$ = new(OMOD, $1, $3); }
```

⟨*expression nodes* 88e⟩+≡ (16a) ◁88c 88g▷

```
  OPOS,
  ONEG,
```

⟨*uexpr rule* 88f⟩+≡ (84d) ◁84g 89d▷

```
  |   '+' xuexpr { $$ = new(OPOS, $2, Z); }
  |   '-' xuexpr { $$ = new(ONEG, $2, Z); }
```

⟨*expression nodes* 88g⟩+≡ (16a) ◁88e 89a▷

```
  OAND,
  OOR,
  OXOR,

  OASHL,
  OASHR,
```

⟨*expr rule* 88h⟩+≡ (84d) ◁88d 89b▷

```
  |   expr '&' expr  { $$ = new(OAND, $1, $3); }
  |   expr '^' expr  { $$ = new(OXOR, $1, $3); }
  |   expr '|' expr  { $$ = new(OOR, $1, $3); }

  |   expr LRSH expr { $$ = new(OASHR, $1, $3); }
  |   expr LLSH expr { $$ = new(OASHL, $1, $3); }
```

8.4.5 Boolean expressions

⟨*expression nodes* 89a⟩+≡ (16a) ◁88g 89c▷

```
  OANDAND,
  OOROR,
```

⟨*expr rule* 89b⟩+≡ (84d) ◁88h 89f▷

```
  |   expr LANDAND expr { $$ = new(OANDAND, $1, $3); }
  |   expr LOROR expr   { $$ = new(OOROR, $1, $3); }
```

⟨*expression nodes* 89c⟩+≡ (16a) ◁89a 89e▷

```
  ONOT,
  OCOM,
```

⟨*uexpr rule* 89d⟩+≡ (84d) ◁88f 90c▷

```
  |   '!' xuexpr { $$ = new(ONOT, $2, Z); }
  |   '~' xuexpr { $$ = new(OCOM, $2, Z); }
```

⟨*expression nodes* 89e⟩+≡ (16a) ◁89c 89g▷

```
  OEQ,
  ONE,

  OLT,
  OGT,
  OLE,
  OGE,
```

⟨*expr rule* 89f⟩+≡ (84d) ◁89b 89h▷

```
  |   expr LEQ expr  { $$ = new(OEQ, $1, $3); }
  |   expr LNE expr  { $$ = new(ONE, $1, $3); }

  |   expr '<' expr  { $$ = new(OLT, $1, $3); }
  |   expr '>' expr  { $$ = new(OGT, $1, $3); }
  |   expr LLE expr  { $$ = new(OLE, $1, $3); }
  |   expr LGE expr  { $$ = new(OGE, $1, $3); }
```

8.4.6 Assignments

⟨*expression nodes* 89g⟩+≡ (16a) ◁89e 89i▷

```
  OAS,
```

⟨*expr rule* 89h⟩+≡ (84d) ◁89f 90a▷

```
  |   expr '=' expr  { $$ = new(OAS, $1, $3); }
```

⟨*expression nodes* 89i⟩+≡ (16a) ◁89g 90b▷

```
  OASADD,
  OASSUB,

  OASMUL,
  OASMOD,
  OASDIV,

  OASAND,
  OASOR,
  OASXOR,

  OASASHL,
  OASASHR,
```

⟨*expr rule* 90a⟩+≡ (84d) ◁89h 91j▷

```
|   expr LPE expr  { $$ = new(OASADD, $1, $3); }
|   expr LME expr  { $$ = new(OASSUB, $1, $3); }
|   expr LMLE expr { $$ = new(OASMUL, $1, $3); }
|   expr LDVE expr { $$ = new(OASDIV, $1, $3); }
|   expr LMDE expr { $$ = new(OASMOD, $1, $3); }
|   expr LANDE expr { $$ = new(OASAND, $1, $3); }
|   expr LXORE expr { $$ = new(OASXOR, $1, $3); }
|   expr LORE expr  { $$ = new(OASOR, $1, $3); }
|   expr LLSHE expr { $$ = new(OASASHL, $1, $3); }
|   expr LRSHE expr { $$ = new(OASASHR, $1, $3); }
```

8.4.7 Pointers

⟨*expression nodes* 90b⟩+≡ (16a) ◁89i 90e▷

```
OIND, // for uses (dereference) and also declarations
OADDR,
```

⟨*uexpr rule* 90c⟩+≡ (84d) ◁89d 91l▷

```
|   '*' xuexpr { $$ = new(OIND, $2, Z); }
|   '&' xuexpr { $$ = new(OADDR, $2, Z); }
```

8.4.8 Array accesses

⟨*pexpr rule* 90d⟩+≡ (84d) ◁88a 90f▷

```
|   pexpr '[' cexpr ']' { $$ = new(OIND, new(OADD, $1, $3), Z); }
```

8.4.9 Field accesses

⟨*expression nodes* 90e⟩+≡ (16a) ◁90b 90g▷

```
ODOT,
```

⟨*pexpr rule* 90f⟩+≡ (84d) ◁90d 90h▷

```
|   pexpr '.' ltag
    {
        $$ = new(ODOT, $1, Z);
        $$->sym = $3;
    }
|   pexpr LMG ltag
    {
        $$ = new(ODOT, new(OIND, $1, Z), Z);
        $$->sym = $3;
    }
```

8.4.10 Function calls

⟨*expression nodes* 90g⟩+≡ (16a) ◁90e 91d▷

```
OFUNC, // used for uses (calls) but also defs (and decls)
```

⟨*pexpr rule* 90h⟩+≡ (84d) ◁90f 92b▷

```
|   pexpr '(' zelist ')'
    {
        $$ = new(OFUNC, $1, Z);
        ⟨pexpr rule, implicit declaration of unknown function 91c⟩
        $$->right = invert($3);
    }
```

⟨*ebnf grammar rules* 91a⟩+≡ (77a) ◁85c

```
zelist:
  /* empty */ { $$ = Z; }
|   elist
```

⟨*expressions rules* 91b⟩+≡ (77a) ◁87d

```
elist:
    expr
|   elist ',' elist { $$ = new(OLIST, $1, $3); }
```

⟨*pexpr rule, implicit declaration of unknown function* 91c⟩≡ (90h)

```
if(($1->op == ONAME) && ($1->type == T))
    dodecl(xdecl, CXXX, types[TINT], $$);
```

8.4.11 Cast

⟨*expression nodes* 91d⟩+≡ (16a) ◁90g 91i▷

```
OCAST,
```

⟨*xuexpr rule* 91e⟩+≡ (84d) ◁84f 363b▷

```
|   '(' tlist abdecor ')' xuexpr
    {
        $$ = new(OCAST, $5, Z);
        dodecl(NODECL, CXXX, $2, $3);
        $$->type = lastdcltype;
        $$->xcast = true;
    }
```

⟨`Node` *origin tracking fields* 91f⟩≡ (16b) 251a▷

```
bool  xcast;
```

⟨*constant* `NODECL` 91g⟩≡ (429b)

```
#define NODECL ((void(*)(int, Type*, Sym*)) nil)
```

⟨`dodecl()` *in ONAME case, break if empty callback* 91h⟩≡ (79f)

```
if(f == NODECL)
    break;
```

8.4.12 Ternary expressions

⟨*expression nodes* 91i⟩+≡ (16a) ◁91d 91k▷

```
OCOND,
```

⟨*expr rule* 91j⟩+≡ (84d) ◁90a

```
|   expr '?' cexpr ':' expr { $$ = new(OCOND, $1, new(OLIST, $3, $5)); }
```

8.4.13 Prefix/postfix

⟨*expression nodes* 91k⟩+≡ (16a) ◁91i 92a▷

```
OPREDEC,
OPREINC,
```

⟨*uexpr rule* 91l⟩+≡ (84d) ◁90c 92e▷

```
|   LPP xuexpr { $$ = new(OPREINC, $2, Z); }
|   LMM xuexpr { $$ = new(OPREDEC, $2, Z); }
```

⟨*expression nodes* 92a⟩+≡ (16a) ◁91k 92c▷
```
  OPOSTINC,
  OPOSTDEC,
```

⟨*pexpr rule* 92b⟩+≡ (84d) ◁90h 92d▷
```
  |   pexpr LPP { $$ = new(OPOSTINC, $1, Z); }
  |   pexpr LMM { $$ = new(OPOSTDEC, $1, Z); }
```

8.4.14 sizeof()

⟨*expression nodes* 92c⟩+≡ (16a) ◁92a 93a▷
```
  OSIZE,
```

⟨*pexpr rule* 92d⟩+≡ (84d) ◁92b 374e▷
```
  |   LSIZEOF '(' tlist abdecor ')'
      {
          $$ = new(OSIZE, Z, Z);
          dodecl(NODECL, CXXX, $3, $4);
          $$->type = lastdcltype;
      }
```

⟨*uexpr rule* 92e⟩+≡ (84d) ◁91l 374d▷
```
  |   LSIZEOF uexpr { $$ = new(OSIZE, $2, Z); }
```

8.5 Initializers and designators

⟨*xdlist rule* 92f⟩+≡ (77c) ◁78c
```
  |   xdecor
      {
          $1 = dodecl(xdecl, lastclass, lasttype, $1);
      }
      '=' init
      {
          doinit($1->sym, $1->type, 0L, $4);
      }
```

⟨*declaration nodes* 92g⟩≡ (16a) 92j▷
```
  OINIT,
```

⟨*initializers rules* 92h⟩≡ (77a) 92i▷
```
  init:
      expr
  |   '{' ilist '}' { $$ = new(OINIT, invert($2), Z); }
```

⟨*initializers rules* 92i⟩+≡ (77a) ◁92h 93b▷
```
  ilist:
      qlist
  |   init
  |   qlist init { $$ = new(OLIST, $1, $2); }

  qlist:
      init ','
  |   qlist init ','  { $$ = new(OLIST, $1, $2); }
  |   qual
  |   qlist qual      { $$ = new(OLIST, $1, $2); }
```

⟨*declaration nodes* 92j⟩+≡ (16a) ◁92g 107h▷
```
  OELEM,  // field designator
```

⟨*expression nodes* 93a⟩+≡ (16a) ◁92c 213d▷

```
OARRAY, // used for uses (designator) and declarations
```

⟨*initializers rules* 93b⟩+≡ (77a) ◁92i

```
qual:
    '[' lexpr ']' { $$ = new(OARRAY, $2, Z); }
|   '.' ltag
    {
        $$ = new(OELEM, Z, Z);
        $$->sym = $2;
    }
|   qual '='
```

⟨*global* initlist 93c⟩≡ (436d)

```
Node* initlist;
```

⟨*function* doinit 93d⟩≡ (450)

```
/// yyparse -> xdlist -> <>
Node*
doinit(Sym *s, Type *t, long o, Node *a)
{
    Node *n;

    ⟨doinit() sanity check t 93e⟩
    ⟨doinit() possibly adjust class 114d⟩
    n = initlist;

    if(a->op == OINIT)
        a = a->left;
    initlist = a;

    a = init1(s, t, o, false);
    ⟨doinit() sanity check initlist after init1 213e⟩
    initlist = n;

    return a;
}
```

⟨doinit() *sanity check t* 93e⟩≡ (93d)

```
if(t == T)
    return Z;
```

8.6 Types and storage classes

⟨*enum* type_kind_bis 93f⟩≡ (429b)

```
enum type_kind_bis {
    // ----------------------------------------------------------------------
    // Type (see separate Type_kind)
    // ----------------------------------------------------------------------

    // ----------------------------------------------------------------------
    // Class storage (temporary, see CAUTO/CEXTERN/... for final storage)
    // ----------------------------------------------------------------------
    ⟨Type_kind_bis storage cases 98b⟩

    // ----------------------------------------------------------------------
    // Qualifiers (aka garbage) (temporary, see GCONSTNT/GVOLATILE)
    // ----------------------------------------------------------------------
```

```
    ⟨Type_kind_bis qualifier cases 99c⟩

    // ----------------------------------------------------------------------
    // Signs (temporary, see TUINT/TULONG/... for final types)
    // ----------------------------------------------------------------------
    ⟨Type_kind_bis sign cases 96e⟩

    // ----------------------------------------------------------------------
    // Misc
    // ----------------------------------------------------------------------
    ⟨Type_kind_bis misc cases 358c⟩

    NALLTYPES,

    ⟨constant TRUNE 87c⟩
};
```

⟨*enum* bxxx 94a⟩≡ (429b)

```
enum Bxxx
{
    BCHAR  = 1L<<TCHAR,
    BUCHAR  = 1L<<TUCHAR,
    BSHORT  = 1L<<TSHORT,
    BUSHORT  = 1L<<TUSHORT,
    BINT  = 1L<<TINT,
    BUINT  = 1L<<TUINT,
    BLONG  = 1L<<TLONG,
    BULONG  = 1L<<TULONG,
    BVLONG  = 1L<<TVLONG,
    BUVLONG  = 1L<<TUVLONG,
    BFLOAT  = 1L<<TFLOAT,
    BDOUBLE  = 1L<<TDOUBLE,

    BIND  = 1L<<TIND,
    BFUNC  = 1L<<TFUNC,
    BARRAY  = 1L<<TARRAY,
    BVOID  = 1L<<TVOID,
    BSTRUCT  = 1L<<TSTRUCT,
    BUNION  = 1L<<TUNION,
    BENUM  = 1L<<TENUM,

    BDOT  = 1L<<TDOT,

    BCONSTNT = 1L<<TCONSTNT,
    BVOLATILE = 1L<<TVOLATILE,

    BUNSIGNED = 1L<<TUNSIGNED,
    BSIGNED  = 1L<<TSIGNED,

    BAUTO  = 1L<<TAUTO,
    BEXTERN  = 1L<<TEXTERN,
    BSTATIC  = 1L<<TSTATIC,
    BTYPEDEF = 1L<<TTYPEDEF,
    BREGISTER = 1L<<TREGISTER,
    ⟨Bxxx cases 367a⟩

    ⟨Bxxx constants 97a⟩
};
```

⟨*type* declarations 94b⟩+≡ (76a) ◁79c 95c▷

```
%type   <lval>  tname cname gname
```

⟨union yacc *other fields* 95a⟩+≡ (28a) ◁76c 95e▷

```
long    lval; //bitset<enum<Bxxx> >
```

⟨*types, storage classes, qualifiers rules* 95b⟩≡ (77a) 95d▷

```
⟨tname rule 96d⟩
⟨cname rule 98c⟩
⟨gname rule 99d⟩
```

⟨*type* declarations 95c⟩+≡ (76a) ◁94b 95f▷

```
%type   <lval>   gcname   gctname
```

⟨*types, storage classes, qualifiers rules* 95d⟩+≡ (77a) ◁95b 95g▷

```
gctname:
    tname
|   gname
|   cname

gcname:
    gname
|   cname
```

⟨union yacc *other fields* 95e⟩+≡ (28a) ◁95a 95h▷

```
Type*   type;
```

⟨*type* declarations 95f⟩+≡ (76a) ◁95c 95i▷

```
%type   <type> tlist
```

⟨*types, storage classes, qualifiers rules* 95g⟩+≡ (77a) ◁95d 95k▷

```
tlist:
    types
    {
        $$ = $1.t;
        if($1.c != CXXX)
            diag(Z, "illegal combination of class 4: %s", cnames[$1.c]);
    }
```

⟨union yacc *other fields* 95h⟩+≡ (28a) ◁95e 102h▷

```
struct
{
    Type*   t;
    // option<enum<Storage_class> >, None = CXXX
    byte   c;
} tycl;
```

⟨*type* declarations 95i⟩+≡ (76a) ◁95f 95j▷

```
%type   <tycl>  types
```

⟨*type* declarations 95j⟩+≡ (76a) ◁95i 100d▷

```
%type   <lval>  gctnlist gcnlist zgnlist
```

⟨*types, storage classes, qualifiers rules* 95k⟩+≡ (77a) ◁95g 95l▷

```
gctnlist:
    gctname
|   gctnlist gctname { $$ = typebitor($1, $2); }
```

⟨*types, storage classes, qualifiers rules* 95l⟩+≡ (77a) ◁95k 96b▷

```
gcnlist:
    gcname
|   gcnlist gcname { $$ = typebitor($1, $2); }
```

⟨*function* typebitor 96a⟩≡ (443b)

```
//@Scheck: used by cc.y
long typebitor(long a, long b)
{
    long c;

    c = a | b;
    if(a & b)
        if((a & b) == BLONG)
            c |= BVLONG;  /* long long => vlong */
        else
            warn(Z, "once is enough: %Q", a & b);
    return c;
}
```

8.6.1 Basic types

⟨*types, storage classes, qualifiers rules* 96b⟩+≡ (77a) ◁95l 98a▷

⟨*types rule* 96c⟩

⟨*types rule* 96c⟩≡ (96b) 96g▷

```
types:
   tname
    {
        $$.c = CXXX;
        $$.t = simplet($1);
    }
```

⟨*tname rule* 96d⟩≡ (95b) 96f▷

```
tname:  /* type words */
    LCHAR     { $$ = BCHAR; }
|   LSHORT    { $$ = BSHORT; }
|   LINT      { $$ = BINT; }
|   LLONG     { $$ = BLONG; }

|   LFLOAT    { $$ = BFLOAT; }
|   LDOUBLE   { $$ = BDOUBLE; }

|   LVOID     { $$ = BVOID; }
```

⟨Type_kind_bis *sign cases* 96e⟩≡ (93f)

```
TUNSIGNED,
TSIGNED,
```

⟨*tname rule* 96f⟩+≡ (95b) ◁96d

```
|   LSIGNED   { $$ = BSIGNED; }
|   LUNSIGNED { $$ = BUNSIGNED; }
```

⟨*types rule* 96g⟩+≡ (96b) ◁96c 100b▷

```
|   gcnlist
    {
        $$.t = simplet($1);
        $$.c = simplec($1);
        $$.t = garbt($$.t, $1);
    }
|   tname gctnlist
    {
        $$.t = simplet(typebitor($1, $2));
        $$.c = simplec($2);
```

```
        $$.t = garbt($$.t, $2);
    }
|   gcnlist tname
    {
        $$.t = simplet($2);
        $$.c = simplec($1);
        $$.t = garbt($$.t, $1);
    }
|   gcnlist tname gctnlist
    {
        $$.t = simplet(typebitor($2, $3));
        $$.c = simplec($1|$3);
        $$.t = garbt($$.t, $1|$3);
    }
```

⟨Bxxx *constants* 97a⟩≡ (94a) 97b ▷

```
/* these can be overloaded with complex types */
BCLASS  = BAUTO|BEXTERN|BSTATIC|BTYPEDEF|BTYPESTR|BREGISTER,
```

⟨Bxxx *constants* 97b⟩+≡ (94a) ◁97a 120b ▷

```
BGARB  = BCONSTNT|BVOLATILE,
```

⟨*function* simplet 97c⟩≡ (443b)

```
Type*
simplet(long b)
{

    b &= ~BCLASS & ~BGARB;

    switch(b) {
    case BCHAR:
    case BCHAR|BSIGNED:
        return types[TCHAR];

    case BCHAR|BUNSIGNED:
        return types[TUCHAR];

    case BSHORT:
    case BSHORT|BINT:
    case BSHORT|BSIGNED:
    case BSHORT|BINT|BSIGNED:
        return types[TSHORT];

    case BUNSIGNED|BSHORT:
    case BUNSIGNED|BSHORT|BINT:
        return types[TUSHORT];

    case 0: // default to 'int' when no type indicated at all
    case BINT:
    case BINT|BSIGNED:
    case BSIGNED:
        return types[TINT];

    case BUNSIGNED:
    case BUNSIGNED|BINT:
        return types[TUINT];

    case BLONG:
    case BLONG|BINT:
    case BLONG|BSIGNED:
```

```
    case BLONG|BINT|BSIGNED:
        return types[TLONG];

    case BUNSIGNED|BLONG:
    case BUNSIGNED|BLONG|BINT:
        return types[TULONG];

    case BVLONG|BLONG:
    case BVLONG|BLONG|BINT:
    case BVLONG|BLONG|BSIGNED:
    case BVLONG|BLONG|BINT|BSIGNED:
        return types[TVLONG];

    case BVLONG|BLONG|BUNSIGNED:
    case BVLONG|BLONG|BINT|BUNSIGNED:
        return types[TUVLONG];

    case BFLOAT:
        return types[TFLOAT];

    case BDOUBLE:
    case BDOUBLE|BLONG:
    case BFLOAT|BLONG:
        return types[TDOUBLE];

    case BVOID:
        return types[TVOID];
    }

    diag(Z, "illegal combination of types %Q", b);
    return types[TINT];
}
```

⟨*types, storage classes, qualifiers rules* 98a⟩+≡ (77a) ◁96b 100c▷

```
zgnlist:
 /* empty */       { $$ = 0; }
|   zgnlist gname  { $$ = typebitor($1, $2); }
```

8.6.2 Storage classes

⟨Type_kind_bis *storage cases* 98b⟩≡ (93f) 367b▷

```
TAUTO = NTYPE,
TEXTERN,
TSTATIC,

TTYPEDEF, // ugly, not really a storage class
TREGISTER,
```

⟨*cname rule* 98c⟩≡ (95b) 108h▷

```
cname:  /* class words */
    LAUTO     { $$ = BAUTO; }
|   LSTATIC   { $$ = BSTATIC; }
|   LEXTERN   { $$ = BEXTERN; }
|   LREGISTER { $$ = BREGISTER; }
|   LINLINE   { $$ = 0; }
```

⟨*function* simplec 99a⟩≡ (443b)

```
//@Scheck: used by cc.y
int simplec(long b)
{

    b &= BCLASS;
    switch(b) {
    case 0:
    case BAUTO:
        return CAUTO;
    case BEXTERN:
        return CEXTERN;
    case BSTATIC:
        return CSTATIC;

    ⟨simplec() cases 99b⟩
    }
    diag(Z, "illegal combination of classes %Q", b);
    return CXXX;
}
```

⟨simplec() *cases* 99b⟩≡ (99a) 108i ▷

```
// skipping register
case BREGISTER:
    return CXXX;
case BAUTO|BREGISTER:
    return CAUTO;
```

8.6.3 Qualifiers

⟨Type_kind_bis *qualifier cases* 99c⟩≡ (93f)

```
TCONSTNT,
TVOLATILE,
```

⟨*gname rule* 99d⟩≡ (95b)

```
gname:  /* garbage words */
    LCONSTNT  { $$ = BCONSTNT; }
|   LVOLATILE { $$ = BVOLATILE; }
|   LRESTRICT { $$ = 0; }
```

⟨*function* garbt 99e⟩≡ (443b)

```
//@Scheck: used by cc.y
Type* garbt(Type *t, long b)
{
    Type *t1;

    if(b & BGARB) {
        t1 = copytyp(t);
        t1->garb = simpleg(b);
        return t1;
    }
    return t;
}
```

⟨*function* copytyp 99f⟩≡ (443b)

```
Type*
copytyp(Type *t)
{
    Type *nt;
```

```
    nt = typ(TXXX, T); // could just do alloc(sizeof(Type))
    *nt = *t;
    return nt;
}
```

⟨*function* simpleg 100a⟩≡ (443b)

```
int
simpleg(long b)
{

    b &= BGARB;
    switch(b) {
    case BCONSTNT:           return GCONSTNT;
    case BVOLATILE:          return GVOLATILE;
    case BVOLATILE|BCONSTNT: return GCONSTNT|GVOLATILE;
    }
    return GXXX;
}
```

8.6.4 Structures and unions

Complex types

⟨*types rule* 100b⟩+≡ (96b) ◁96g

```
|  complex
    {
        $$.c = CXXX;
        $$.t = $1;
    }
|  complex gctnlist
    {
        $$.t = $1;
        $$.c = simplec($2);
        $$.t = garbt($$.t, $2);
        if($2 & ~BCLASS & ~BGARB)
            diag(Z, "duplicate types given: %T and %Q", $1, $2);
    }
|   gcnlist complex zgnlist
    {
        $$.t = $2;
        $$.c = simplec($1);
        $$.t = garbt($$.t, $1|$3);
    }
```

⟨*types, storage classes, qualifiers rules* 100c⟩+≡ (77a) ◁98a 103a▷

```
⟨complex rule 100e⟩
```

⟨*type* declarations 100d⟩+≡ (76a) ◁95j 102e▷

```
%type   <type>  complex
```

Structure/union uses

⟨*complex rule* 100e⟩≡ (100c) 101c▷

```
complex:
    LSTRUCT ltag
    {
        dotag($2, TSTRUCT, 0);
```

```
        $$ = $2->suetag;
    }
|   LUNION ltag
    {
        dotag($2, TUNION, 0);
        $$ = $2->suetag;
    }
```

⟨*function* dotag 101a⟩≡ (450)

```
//@Scheck: used by cc.y
Type* dotag(Sym *s, int et, int bn)
{
    ⟨dotag() locals 102a⟩

    ⟨dotag() if bn not zero and bn not sueblock 102b⟩
    // never defined before, return a newly allocated (but incomplete) Type
    if(s->suetag == T) {
        s->suetag = typ(et, T); // link is null for now
        s->sueblock = autobn;
    }
    ⟨dotag() sanity check tag redeclaration 101b⟩
    if(s->suetag->tag == S)
        s->suetag->tag = s;

    return s->suetag;
}
```

⟨dotag() *sanity check tag redeclaration* 101b⟩≡ (101a)

```
if(s->suetag->etype != et)
    diag(Z, "tag used for more than one type: %s", s->name);
```

Structure/union definitions

⟨*complex rule* 101c⟩+≡ (100c) ◁100e 102d▷

```
|   LSTRUCT ltag
    {
        dotag($2, TSTRUCT, autobn);
    }
    sbody
    {
        $$ = $2->suetag;
        if($$->link != T)
            diag(Z, "redeclare tag: %s", $2->name);
        $$->link = $4;
        ⟨complex rule, when parse a structure definition, align the struct 227b⟩
    }
|   LUNION ltag
    {
        dotag($2, TUNION, autobn);
    }
    sbody
    {
        $$ = $2->suetag;
        if($$->link != T)
            diag(Z, "redeclare tag: %s", $2->name);
        $$->link = $4;
        ⟨complex rule, when parse a structure definition, align the struct 227b⟩
    }
```

⟨dotag() *locals* 102a⟩≡ (101a)

```
Decl *d;
```

⟨dotag() *if bn not zero and bn not sueblock* 102b⟩≡ (101a)

```
if(bn != 0 && bn != s->sueblock) {
    d = push();
    d->sym = s;
    d->val = DSUE;
    // save old values
    d->type = s->suetag;
    d->block = s->sueblock;
    // prepare for new value
    s->suetag = T;
}
```

⟨*global* taggen 102c⟩≡ (436d)

```
//@Scheck: used by cc.y
int taggen;
```

⟨*complex rule* 102d⟩+≡ (100c) ◁101c 104b▷

```
|   LSTRUCT sbody
    {
        taggen++;
        sprint(symb, "_%d_", taggen);
        $$ = dotag(lookup(), TSTRUCT, autobn);
        $$->link = $2;
        ⟨complex rule, when parse a structure definition, align the struct 227b⟩
    }
|   LUNION sbody
    {
        taggen++;
        sprint(symb, "_%d_", taggen);
        $$ = dotag(lookup(), TUNION, autobn);
        $$->link = $2;
        ⟨complex rule, when parse a structure definition, align the struct 227b⟩
    }
```

Fields

⟨*type* declarations 102e⟩+≡ (76a) ◁100d

```
%type   <type> sbody
```

⟨*global* strf 102f⟩≡ (436d)

```
// list<ref_own<Type>> (next = Type.down, last = strl)
Type* strf;
```

⟨*global* strl 102g⟩≡ (436d)

```
// ref<Type> (head = strf)
Type* strl;
```

⟨union yacc *other fields* 102h⟩+≡ (28a) ◁95h

```
struct
{
    Type*   t1; // save strf
    Type*   t2; // save strl
    Type*   t3; // save lastype
    uchar   c;  // save lastclass
} tyty;
```

⟨*types, storage classes, qualifiers rules* 103a⟩+≡ (77a) ◁100c 105c▷

```
sbody:
    '{'
    {
        // save
        $<tyty>$.t1 = strf;
        $<tyty>$.t2 = strl;
        $<tyty>$.t3 = lasttype;
        $<tyty>$.c = lastclass;

        strf = T;
        strl = T;
        ⟨sbody rule, initializations before parsing the fields 352f⟩
        lastclass = CXXX;
        lasttype = T;
    }
    edecl
    '}'
    {
        $$ = strf;

        // restore
        strf = $<tyty>2.t1;
        strl = $<tyty>2.t2;
        lasttype = $<tyty>2.t3;
        lastclass = $<tyty>2.c;
    }
```

⟨*structure element declarator rules* 103b⟩≡ (77b) 103c▷

```
/*
 * structure element declarator
 */
edecl:
    tlist
    {
        lasttype = $1;
    }
    zedlist ';'
|   edecl tlist
    {
        lasttype = $2;
    }
    zedlist ';'

⟨zedlist rule 360h⟩

edlist:
    edecor              { dodecl(edecl, CXXX, lasttype, $1); }
|   edlist ',' edlist
```

⟨*structure element declarator rules* 103c⟩+≡ (77b) ◁103b

⟨*edecor rule* 103d⟩

⟨*edecor rule* 103d⟩≡ (103c) 352k▷

```
edecor:
    xdecor
    {
        ⟨edecor rule, set fields after parsed a field declarator 352g⟩
    }
```

⟨*function* edecl 104a⟩≡ (450)

```
/// dodecl(<>) -> <>
//@Scheck: used by cc.y
void edecl(int c, Type *t, Sym *s)
{
    Type *t1;

    ⟨edecl() if unnamed structure element 360i⟩
    else
        if(c != CXXX)
            diag(Z, "structure element cannot have class: %s", s->name);

    t1 = t;
    t = copytyp(t1);
    t->sym = s;
    t->down = T;

    ⟨edecl() if lastfield 353a⟩

    // add_list(t, strf, strl)
    if(strf == T)
        strf = t;
    else
        strl->down = t;
    strl = t;
}
```

8.6.5 Enums

Enumeration type uses and definitions

⟨*complex rule* 104b⟩+≡ (100c) ◁102d 104e▷

```
|   LENUM ltag
    {
        dotag($2, TENUM, 0);
        $$ = $2->suetag;
        if($$->link == T) // default type
            $$->link = types[TINT];
        $$ = $$->link;
    }
```

⟨*struct* En 104c⟩≡ (429b)

```
struct En
{
    Type* tenum;  /* type of entire enum */
    Type* cenum;  /* type of current enum run */
    ⟨En value fields 145c⟩
};
```

⟨*global* en 104d⟩≡ (436d)

```
struct En en;
```

⟨*complex rule* 104e⟩+≡ (100c) ◁104b 108k▷

```
|   LENUM ltag
    {
        dotag($2, TENUM, autobn);
    }
    '{'
    {
```

```
        en.tenum = T;
        en.cenum = T;
    }
    enum
    '}'
    {
        $$ = $2->suetag;
        if($$->link != T)
            diag(Z, "redeclare tag: %s", $2->name);
        ⟨complex rule, after processed an enum, sanity check tenum 105a⟩
        $$->link = en.tenum;
        $$ = en.tenum;
    }
|   LENUM
    '{'
    {
        en.tenum = T;
        en.cenum = T;
    }
    enum
    '}'
    {
        $$ = en.tenum;
    }
```

⟨*complex rule, after processed an enum, sanity check tenum* 105a⟩≡ (104e)

```
if(en.tenum == T) {
    diag(Z, "enum type ambiguous: %s", $2->name);
    en.tenum = types[TINT];
}
```

Enumeration constant definitions

⟨Sym *enum value fields* 105b⟩≡ (21e) 145f▷

```
// ref<Type>
Type* tenum;
```

⟨*types, storage classes, qualifiers rules* 105c⟩+≡ (77a) ◁103a

```
enum:
    LNAME           { doenum($1, Z); }
|   LNAME '=' expr  { doenum($1, $3); }

|   enum ',' enum
|   enum ','
```

⟨*function* doenum 105d⟩≡ (450)

```
//@Scheck: used by cc.y
void doenum(Sym *s, Node *n)
{

    if(n) {
        ⟨doenum() typecheck and evaluate expression 145d⟩
        if(n->op != OCONST) {
            diag(n, "enum not a constant: %s", s->name);
            return;
        }

        en.cenum = n->type; // inferred type
        en.tenum = maxtype(en.cenum, en.tenum);
```

```
        ⟨doenum() save current value of enumeration constant 145e⟩
    }
    if(dclstack)
        push1(s); // will be reverted once out of scope

    // check for redeclaration and set s->type to TENUM
    xdecl(CXXX, types[TENUM], s);

    if(en.cenum == T) {
        en.cenum = types[TINT];
        en.tenum = types[TINT];
        en.lastenum = 0;
    }
    s->tenum = en.cenum; // Sym.tenum, not En.tenum here

    ⟨doenum() set value for symbol 145g⟩

    ⟨doenum() debug declaration 416b⟩
    ⟨doenum() debugging support 338d⟩
}
```

8.6.6 Pointer and array types

⟨*xdecor rule* 106a⟩+≡ (77c) ◁78f

```
|   '*' zgnlist xdecor
    {
        $$ = new(OIND, $3, Z);
        $$->nodegarb = simpleg($2);
    }
```

⟨Node *parsing helper fields* 106b⟩≡ (16b)

```
// enum<qualifier>
char nodegarb;
```

⟨dodecl() *switch node kind cases* 106c⟩+≡ (78d) ◁79f 106f▷

```
case OIND:
    t = typ(TIND, t);
    t->garb = n->nodegarb;
    n = n->left;
    goto loop;
```

⟨*xdecor2 rule* 106d⟩+≡ (77c) ◁78g 106g▷

```
|   xdecor2 '[' zexpr ']'      { $$ = new(OARRAY, $1, $3); }
```

⟨dodecl() *locals* 106e⟩+≡ (78d) ◁79e 146a▷

```
Node *n1;
```

⟨dodecl() *switch node kind cases* 106f⟩+≡ (78d) ◁106c 107b▷

```
case OARRAY:
    t = typ(TARRAY, t);
    n1 = n->right; // possible size
    n = n->left;
    ⟨dodecl() switch node kind cases, case OARRAY, if array has a size 146b⟩
    goto loop;
```

8.6.7 Function types, parameter types

⟨*xdecor2 rule* 106g⟩+≡ (77c) ◁106d

```
|   xdecor2 '(' zarglist ')'   { $$ = new(OFUNC, $1, $3); }
```

⟨*parameter declarator rules* 107a⟩≡ (77b) 107e ▷

```
zarglist:
  /* empty */   { $$ = Z; }
|   arglist     { $$ = invert($1); }
```

⟨dodecl() *switch node kind cases* 107b⟩+≡ (78d) ◁106f 352l ▷

```
case OFUNC:
    t = typ(TFUNC, t);
    t->down = fnproto(n);
    n = n->left;
    goto loop;
```

⟨*function* fnproto 107c⟩≡ (450)

```
Type*
fnproto(Node *n)
{
    ⟨fnproto() old prototype style handling 357f⟩
    return fnproto1(n->right);
}
```

⟨*function* fnproto1 107d⟩≡ (450)

```
Type*
fnproto1(Node *n)
{
    Type *t;

    if(n == Z)
        return T;

    switch(n->op) {
    ⟨fnproto1() switch node kind cases 107g⟩
    }
    diag(n, "unknown op in fnproto");
    return T;
}
```

⟨*parameter declarator rules* 107e⟩+≡ (77b) ◁107a 356b ▷

```
⟨arglist rule 107f⟩
```

⟨*arglist rule* 107f⟩≡ (107e) 107i ▷

```
arglist:
    name
```

⟨fnproto1() *switch node kind cases* 107g⟩≡ (107d) 108a ▷

```
case ONAME:
    diag(n, "incomplete argument prototype");
    return typ(TINT, T);
```

⟨*declaration nodes* 107h⟩+≡ (16a) ◁92j 108d ▷

```
OPROTO,
```

⟨*arglist rule* 107i⟩+≡ (107e) ◁107f 108b ▷

```
|   tlist xdecor
    {
        $$ = new(OPROTO, $2, Z);
        $$->type = $1;
    }
|   tlist abdecor
    {
        $$ = new(OPROTO, $2, Z);
        $$->type = $1;
    }
```

⟨fnproto1() *switch node kind cases* 108a⟩+≡ (107d) ◁107g 108c▷

```
case OPROTO:
    lastdcltype = T; // dead?
    dodecl(NODECL, CXXX, n->type, n->left);
    t = typ(TXXX, T);
    ⟨fnproto1() when OPROTO case, adjust parameter type 146c⟩
    return t;
```

⟨*arglist rule* 108b⟩+≡ (107e) ◁107i 108e▷

```
|   arglist ',' arglist  { $$ = new(OLIST, $1, $3); }
```

⟨fnproto1() *switch node kind cases* 108c⟩+≡ (107d) ◁108a 108f▷

```
case OLIST:
    t = fnproto1(n->left);
    if(t != T)
        t->down = fnproto1(n->right);
    return t;
```

⟨*declaration nodes* 108d⟩+≡ (16a) ◁107h 352j▷

```
ODOTDOT,
```

⟨*arglist rule* 108e⟩+≡ (107e) ◁108b

```
|   '.' '.' '.'          { $$ = new(ODOTDOT, Z, Z); }
```

⟨fnproto1() *switch node kind cases* 108f⟩+≡ (107d) ◁108c

```
case ODOTDOT:
    return typ(TDOT, T);
```

⟨Type_kind *other cases* 108g⟩≡ (18d)

```
TDOT, // ... in function types
```

8.6.8 Typedefs

⟨*cname rule* 108h⟩+≡ (95b) ◁98c 366g▷

```
|   LTYPEDEF  { $$ = BTYPEDEF; }
```

⟨simplec() *cases* 108i⟩+≡ (99a) ◁99b 365c▷

```
case BTYPEDEF:
    return CTYPEDEF;
```

⟨Storage_class *cases* 108j⟩≡ (20e) 350d▷

```
CTYPEDEF,
```

⟨*complex rule* 108k⟩+≡ (100c) ◁104e

```
|   LTYPE { $$ = tcopy($1->type); }
```

8.7 Declarations and definitions, part two

8.7.1 Globals, external declarator

8.7.2 Function parameters

⟨*function* argmark 109a⟩≡ (450)

```
/// yyparse -> xdecl -> <>
//@Scheck: not dead, used by cc.y
void argmark(Node *n, int pass)
{
    Type *t;

    ⟨argmark() initializations 158e⟩

    for(; n->left != Z; n = n->left) {
        if(n->op != OFUNC || n->left->op != ONAME)
            continue;
        // else, OFUNC with ONAME on n->left, so analyze the parameters
        walkparam(n->right, pass);
        ⟨argmark() if old proto style 357b⟩
        break;
    }

    ⟨argmark() finalizations 158f⟩
}
```

⟨*function* walkparam 109b⟩≡ (450)

```
void
walkparam(Node *n, int pass)
{
    Sym *s;
    Node *n1;

    if(n != Z && n->op == OPROTO && n->left == Z && n->type == types[TVOID])
        return;

loop:
    if(n == Z)
        return;
    switch(n->op) {
    case OLIST:
        walkparam(n->left, pass);
        n = n->right;
        goto loop;

    case OPROTO:
        for(n1 = n; n1 != Z; n1=n1->left)
            if(n1->op == ONAME) {
                if(pass == 0) {
                    s = n1->sym;
                    push1(s);
                    s->offset = -1;
                    break;
                }
                dodecl(pdecl, CPARAM, n->type, n->left);
                break;
            }

        if(n1)
```

```
            break;
        // else no name found
        ⟨walkparam() when OPROTO case, if no name found 110b⟩
        break;

    ⟨walkparam() switch op cases 110a⟩
    default:
        diag(n, "argument not a name/prototype: %O", n->op);
        break;

    }
}
```

⟨walkparam() *switch op cases* 110a⟩≡ (109b) 358b ▷

```
case ODOTDOT:
    break;
```

⟨walkparam() *when OPROTO case, if no name found* 110b⟩≡ (109b)

```
if(pass == 0) {
    /*
     * extension:
     * allow no name in argument declaration
    diag(Z, "no name in argument declaration");
     */
    break;
}
// else pass == 1
⟨walkparam() when OPROTO case, if no name found and old style decl 357c⟩
```

⟨*function* pdecl 110c⟩≡ (450)

```
/// dodecl(<>,...) -> <>
void
pdecl(int class, Type *t, Sym *s)
{
    ⟨pdecl() sanity check s 110d⟩
    ⟨pdecl() adjust type 146d⟩
    ⟨pdecl() adjust class 114e⟩
    ⟨pdecl() sanity check type 146f⟩
    adecl(class, t, s);
}
```

⟨pdecl() *sanity check s* 110d⟩≡ (110c)

```
if(s && s->offset != -1) {
    diag(Z, "not a parameter: %s", s->name);
    return;
}
```

8.7.3 Locals, automatic declarator

⟨*automatic declarator rules* 110e⟩≡ (77b) 111a ▷

```
/*
 * automatic declarator
 */
adecl:
    ctlist ';'        { $$ = dodecl(adecl, lastclass, lasttype, Z); }
|   ctlist adlist ';' { $$ = $2; }
```

⟨*automatic declarator rules* 111a⟩+≡ (77b) ◁110e

```
adlist:
    xdecor
    {
        dodecl(adecl, lastclass, lasttype, $1);
        $$ = Z;
    }
|   xdecor
    {
        $1 = dodecl(adecl, lastclass, lasttype, $1);
    }
    '=' init
    {
        long w;

        w = $1->sym->type->width;
        $$ = doinit($1->sym, $1->type, 0L, $4);
        ⟨adlist rule, after doinit 219a⟩
    }
|   adlist ',' adlist
    {
        $$ = $1;
        if($3 != Z) {
            $$ = $3;
            if($1 != Z)
                $$ = new(OLIST, $1, $3);
        }
    }
```

⟨*function* adecl 111b⟩≡ (450)

```
/// pdecl | dodecl(<>) -> <>
void
adecl(int c, Type *t, Sym *s)
{

    ⟨adecl() adjust storage to CLOCAL if static local variable 350e⟩
    ⟨adecl() adjust storage if function type 114f⟩
    ⟨adecl() adjust storage 115a⟩

    if(s) {
        ⟨adecl() adjust storage if symbol 115b⟩

        if(c != CPARAM)
            push1(s); // CPARAM pushed already in walkparam()

        s->block = autobn;
        s->type = t;
        s->class = c;
        s->offset = 0;
        ⟨adecl() initialize symbol 148a⟩
    }

    switch(c) {
    ⟨adecl() switch class cases 180d⟩
    }
}
```

8.7.4 Types, abstract declarator

⟨*abstract declarator rules* 112a⟩≡ (77b) 112b ▷

```
/*
 * abstract declarator
 */
abdecor:
  /* empty */ { $$ = Z; }
|   abdecor1
```

⟨*abstract declarator rules* 112b⟩+≡ (77b) ◁112a 112c ▷

```
abdecor1:
    '*' zgnlist
    {
        $$ = new(OIND, (Z), Z);
        $$->nodegarb = simpleg($2);
    }
|   '*' zgnlist abdecor1
    {
        $$ = new(OIND, $3, Z);
        $$->nodegarb = simpleg($2);
    }
|   abdecor2
```

⟨*abstract declarator rules* 112c⟩+≡ (77b) ◁112b

```
abdecor2:
    abdecor3
|   abdecor2 '(' zarglist ')'   { $$ = new(OFUNC, $1, $3); }
|   abdecor2 '[' zexpr ']'      { $$ = new(OARRAY, $1, $3); }

abdecor3:
    '(' ')'           { $$ = new(OFUNC, Z, Z); }
|   '[' zexpr ']'     { $$ = new(OARRAY, Z, $2); }
|   '(' abdecor1 ')'  { $$ = $2; }
```

Chapter 9

Checking

9.1 Overview

9.2 Printing warnings: 5c -w and 5c -W

⟨*function* warn 113a⟩≡ (443b)

```
void
warn(Node *n, char *fmt, ...)
{
    char buf[STRINGSZ];
    va_list arg;

    if(debug['w'] || debug['W']) {
        va_start(arg, fmt);
        vseprint(buf, buf+sizeof(buf), fmt, arg);
        va_end(arg);
        ⟨warn() if -W 113b⟩
        // else
        Bprint(&diagbuf, "warning: %L %s\n", (n==Z) ? nearln : n->lineno, buf);
        ⟨warn() if -v 113c⟩
    }
}
```

⟨warn() *if -W* 113b⟩≡ (113a)

```
if(debug['W']) {
    diag(n, "%s", buf);
    return;
}
```

⟨warn() *if -v* 113c⟩≡ (113a)

```
if(n != Z)
  if(debug['v'])
    prtree(n, "warning");
```

9.3 Symbol resolution

9.4 Storage resolution

9.4.1 xdecl

⟨xdecl() *switch class cases* 113d⟩≡ (79g) 114a ▷

```
  case CXXX:
      class = CGLOBL;
      if(s->class == CEXTERN)
          s->class = CGLOBL;
      break;
```

⟨xdecl() *switch class cases* 114a⟩+≡ (79g) ◁113d 114b▷

```
  case CEXTERN:
      if(s->class == CGLOBL)
          class = CGLOBL;
      break;
```

⟨xdecl() *switch class cases* 114b⟩+≡ (79g) ◁114a 365d▷

```
  case CAUTO:
      diag(Z, "overspecified class: %s %s %s", s->name, cnames[class], cnames[s->class]);
      class = CEXTERN;
      break;
```

⟨xdecl() *sanity check class after switch class* 114c⟩≡ (79g)

```
  if(s->class == CSTATIC)
      if(class == CEXTERN || class == CGLOBL) {
          warn(Z, "overspecified class: %s %s %s", s->name, cnames[class], cnames[s->class]);
          class = CSTATIC;
      }
```

9.4.2 doinit

⟨doinit() *possibly adjust class* 114d⟩≡ (93d)

```
  if(s->class == CEXTERN) {
      s->class = CGLOBL;
      ⟨doinit() debug declaration 416a⟩
  }
  ⟨doinit() debug initialization 419a⟩
```

9.4.3 pdecl

⟨pdecl() *adjust class* 114e⟩≡ (110c)

```
  if(class == CXXX)
      class = CPARAM;
  if(class != CPARAM) {
      diag(Z, "parameter cannot have class: %s", s->name);
      class = CPARAM;
  }
```

9.4.4 adecl

⟨adecl() *adjust storage if function type* 114f⟩≡ (111b)

```
  if(t->etype == TFUNC) {
      if(c == CXXX)
          c = CEXTERN;
      if(c == CLOCAL)
          c = CSTATIC;
      if(c == CAUTO || c == CEXREG)
          diag(Z, "function cannot be %s %s", cnames[c], s->name);
  }
```

⟨adecl() *adjust storage* 115a⟩≡ (111b)

```
if(c == CXXX)
    c = CAUTO;
```

⟨adecl() *adjust storage if symbol* 115b⟩≡ (111b)

```
if(s->class == CSTATIC)
    if(c == CEXTERN || c == CGLOBL) {
        warn(Z, "just say static: %s", s->name);
        c = CSTATIC;
    }
if(s->class == CAUTO || s->class == CPARAM || s->class == CLOCAL)
if(s->block == autobn)
    diag(Z, "auto redeclaration of: %s", s->name);
```

9.5 Typechecking helpers

9.5.1 Overview

⟨xdecl() *sanity check type after switch class* 115c⟩≡ (79g)

```
if(s->type != T)
    if(s->class != class || !sametype(t, s->type) || t->etype == TENUM) {
        diag(Z, "external redeclaration of: %s", s->name);
        Bprint(&diagbuf, " %s %T %L\n", cnames[class], t, nearln);
        Bprint(&diagbuf, " %s %T %L\n", cnames[s->class], s->type, s->varlineno);
    }
```

9.5.2 Type equality: sametype()

⟨*function* sametype 115d⟩≡ (450)

```
/// xdecl -> <>
bool
sametype(Type *t1, Type *t2)
{

    if(t1 == t2)
        return true;
    return rsametype(t1, t2, 5, true);
}
```

⟨*function* rsametype 115e⟩≡ (450)

```
bool
rsametype(Type *t1, Type *t2, int n, bool f)
{
    int et;

    n--;
    for(;;) {
        if(t1 == t2)
            return true;
        if(t1 == T || t2 == T)
            return false;
        ⟨rsametype() if n less than zero 116a⟩

        et = t1->etype;
        if(et != t2->etype)
            return false;
```

```
        ⟨rsametype() check function type equality 116b⟩
        ⟨rsametype() check array type equality 116c⟩
        ⟨rsametype() check structure type equality 116d⟩

        // recurse
        t1 = t1->link;
        t2 = t2->link;

        ⟨rsametype() if f and not -V, allow void star 117b⟩
    }
}
```

⟨rsametype() *if n less than zero* 116a⟩≡ (115e)

```
if(n <= 0)
    return true;
```

⟨rsametype() *check function type equality* 116b⟩≡ (115e)

```
if(et == TFUNC) {
    if(!rsametype(t1->link, t2->link, n, false))
        return false;
    t1 = t1->down;
    t2 = t2->down;
    while(t1 != T && t2 != T) {

        ⟨rsametype() continue if old style type 358d⟩

        while(t1 != T || t2 != T) {
            if(!rsametype(t1, t2, n, false))
                return false;
            t1 = t1->down;
            t2 = t2->down;
        }
        break;
    }
    return true;
}
```

⟨rsametype() *check array type equality* 116c⟩≡ (115e)

```
if(et == TARRAY)
    if(t1->width != t2->width && t1->width != 0 && t2->width != 0)
        return false;
```

⟨rsametype() *check structure type equality* 116d⟩≡ (115e)

```
if(typesu[et]) {
    if(t1->link == T)
        snap(t1);
    if(t2->link == T)
        snap(t2);

    if(t1 != t2 && t1->link == T && t2->link == T){
        /* structs with missing or different tag names aren't considered equal */
        if(t1->tag == nil || t2->tag == nil ||
           strcmp(t1->tag->name, t2->tag->name) != 0)
            return false;
    }
    // equal if have same types (but do not care about field names)
    t1 = t1->link;
    t2 = t2->link;
    for(;;) {
        if(t1 == t2)
```

```
            return true;
        if(!rsametype(t1, t2, n, false))
            return false;
        t1 = t1->down;
        t2 = t2->down;
    }
} // end su
```

⟨*function* snap 117a⟩≡ (450)

```
void
snap(Type *t)
{
    if(typesu[t->etype])
     if(t->link == T && t->tag && t->tag->suetag) {
        t->link = t->tag->suetag->link;
        t->width = t->tag->suetag->width;
    }
}
```

9.5.3 generic `void*` pointer conversions: `5c -V`

⟨`rsametype()` *if f and not -V, allow void star* 117b⟩≡ (115e)

```
if((f || !debug['V']) && et == TIND) {
    if(t1 != T && t1->etype == TVOID)
        return true;
    if(t2 != T && t2->etype == TVOID)
        return true;
}
```

9.5.4 Type merge: `tmerge()`

⟨`xdecl()` *merge type declarations* 117c⟩≡ (79g)

```
tmerge(t, s);
```

⟨*function* tmerge 117d⟩≡ (450)

```
/// dodectl(xdecl,...) -> xdecl -> <>
void
tmerge(Type *t1, Sym *s)
{
    Type *t2 = s->type;
    Type *ta, *tb;

/*print("merge %T; %T\n", t1, t2);/**/
    for(;;) {
        if(t1 == T || t2 == T || t1 == t2)
            break;
        if(t1->etype != t2->etype)
            break;
        switch(t1->etype) {
        ⟨tmerge() switch etype t1 cases 358e⟩
        case TARRAY:
            /* should we check array size change? */
            if(t2->width > t1->width)
                t1->width = t2->width;
            break;

        case TUNION:
        case TSTRUCT:
```

```
            return;
        }
        // recurse
        t1 = t1->link;
        t2 = t2->link;
    }
}
```

9.5.5 Set of types constants: typexxx[]

⟨*global* typei 118a⟩≡ (443b)

```
// set<Type_kind>
char typei[NTYPE];
```

⟨*global* typeiinit 118b⟩≡ (443b)

```
int typeiinit[] =
{
    TCHAR, TUCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG, TULONG, TVLONG, TUVLONG, -1,
};
```

⟨*global* typesu 118c⟩≡ (443b)

```
char typesu[NTYPE];
```

⟨*global* typesuinit 118d⟩≡ (443b)

```
int typesuinit[] =
{
    TSTRUCT, TUNION, -1,
};
```

⟨*global* typefd 118e⟩≡ (443b)

```
char typefd[NTYPE];
```

⟨*global* typefdinit 118f⟩≡ (443b)

```
int typefdinit[] =
{
    TFLOAT, TDOUBLE, -1,
};
```

⟨tinit() *initialise typexxx type sets* 118g⟩≡ (34b) 443a ▷

```
for(ip=typeiinit; *ip>=0; ip++) {
    urk("typei", nelem(typei), *ip);
    typei[*ip] = 1;
}
for(ip=typeuinit; *ip>=0; ip++) {
    urk("typeu", nelem(typeu), *ip);
    typeu[*ip] = 1;
}
for(ip=typesuvinit; *ip>=0; ip++) {
    urk("typesuv", nelem(typesuv), *ip);
    typesuv[*ip] = true;
}
for(ip=typeilpinit; *ip>=0; ip++) {
    urk("typeilp", nelem(typeilp), *ip);
    typeilp[*ip] = 1;
}
for(ip=typechlinit; *ip>=0; ip++) {
    urk("typechl", nelem(typechl), *ip);
    typechl[*ip] = 1;
    typechlv[*ip] = 1;
```

```
    //typechlvp[*ip] = 1;
}
for(ip=typechlpinit; *ip>=0; ip++) {
    urk("typechlp", nelem(typechlp), *ip);
    typechlp[*ip] = 1;
    //typechlvp[*ip] = 1;
}
for(ip=typechlpfdinit; *ip>=0; ip++) {
    urk("typechlpfd", nelem(typechlpfd), *ip);
    typechlpfd[*ip] = 1;
}
for(ip=typevinit; *ip>=0; ip++) {
    urk("typev", nelem(typev), *ip);
    typev[*ip] = 1;
    typechlv[*ip] = 1;
    //typechlvp[*ip] = 1;
}
for(ip=typefdinit; *ip>=0; ip++) {
    urk("typefd", nelem(typefd), *ip);
    typefd[*ip] = 1;
}
for(ip=typeafinit; *ip>=0; ip++) {
    urk("typeaf", nelem(typeaf), *ip);
    typeaf[*ip] = 1;
}
for(ip=typesuinit; *ip >= 0; ip++) {
    urk("typesu", nelem(typesu), *ip);
    typesu[*ip] = 1;
}
```

9.5.6 Type compatibility: `tcompat()`

⟨*function* `tcompat` 119a⟩≡ (443b)

```
/// (complex -> tcom -> tcomo) | bcomplex -> <>
bool
tcompat(Node *n, Type *t1, Type *t2, long ttab[])
{

    if(stcompat(n, t1, t2, ttab)) {
        ⟨tcompat() if no T1 119b⟩
        else
            diag(n, "incompatible types: \"%T\" and \"%T\" for op \"%O\"",
                t1, t2, n->op);
        return true;
    }
    return false;
}
```

⟨`tcompat()` *if no T1* 119b⟩≡ (119a)

```
if(t1 == T)
    diag(n, "incompatible type: \"%T\" for op \"%O\"",
        t2, n->op);
```

⟨*function* `stcompat` 119c⟩≡ (443b)

```
/// tcompat -> <>
bool
stcompat(Node *n, Type *t1, Type *t2, long ttab[])
{
    int i;
```

```
    //Type_kind (as a bit)
    ulong b;

    i = 0;
    if(t2 != T)
        i = t2->etype;
    b = 1L << i;
    i = 0;
    if(t1 != T)
        i = t1->etype;
    // type t2 part of set of types compatible with type t1
    if(b & ttab[i]) {
        ⟨stcompat() when t1 matches t2, if ttab is tasign 134a⟩
        ⟨stcompat() when t1 matches t2, if pointers and not a cast op 120a⟩
        return false; // true means error, so false means everything fine
    }
    return true;
}
```

⟨stcompat() *when t1 matches t2, if pointers and not a cast op* 120a⟩≡ (119c)

```
if(n->op != OCAST)
  if(b == BIND && i == TIND)
        if(!sametype(t1, t2))
            return true;
```

9.5.7 Compatibility policies: txxx

⟨Bxxx *constants* 120b⟩+≡ (94a) ◁97b

```
BINTEGER = BCHAR|BUCHAR|BSHORT|BUSHORT|BINT|BUINT|BLONG|BULONG|BVLONG|BUVLONG,
BNUMBER  = BINTEGER | BFLOAT|BDOUBLE,
```

⟨*global* tadd 120c⟩≡ (443b)

```
long tadd[NTYPE];
```

⟨*global* taddinit 120d⟩≡ (443b)

```
Init taddinit[] =
{
    TCHAR,  BNUMBER|BIND, 0,
    TUCHAR,  BNUMBER|BIND, 0,
    TSHORT,  BNUMBER|BIND, 0,
    TUSHORT, BNUMBER|BIND, 0,
    TINT,  BNUMBER|BIND, 0,
    TUINT,  BNUMBER|BIND, 0,
    TLONG,  BNUMBER|BIND, 0,
    TULONG,  BNUMBER|BIND, 0,
    TVLONG,  BNUMBER|BIND, 0,
    TUVLONG, BNUMBER|BIND, 0,

    TFLOAT,  BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,
    TIND,  BINTEGER, 0,
    -1,  0,  0,
};
```

⟨tinit() *initialise tcompat arrays* 120e⟩≡ (34b)

```
for(p=tasigninit; p->code >= 0; p++) {
    urk("tasign", nelem(tasign), p->code);
    tasign[p->code] = p->value;
}
```

```
for(p=tasaddinit; p->code >= 0; p++) {
    urk("tasadd", nelem(tasadd), p->code);
    tasadd[p->code] = p->value;
}
for(p=tcastinit; p->code >= 0; p++) {
    urk("tcast", nelem(tcast), p->code);
    tcast[p->code] = p->value;
}
for(p=taddinit; p->code >= 0; p++) {
    urk("tadd", nelem(tadd), p->code);
    tadd[p->code] = p->value;
}
for(p=tsubinit; p->code >= 0; p++) {
    urk("tsub", nelem(tsub), p->code);
    tsub[p->code] = p->value;
}
for(p=tmulinit; p->code >= 0; p++) {
    urk("tmul", nelem(tmul), p->code);
    tmul[p->code] = p->value;
}
for(p=tandinit; p->code >= 0; p++) {
    urk("tand", nelem(tand), p->code);
    tand[p->code] = p->value;
}
for(p=trelinit; p->code >= 0; p++) {
    urk("trel", nelem(trel), p->code);
    trel[p->code] = p->value;
}
```

9.5.8 Typechecker: `tcom()`

⟨*function* `tcom` 121a⟩≡ (454)

```
/// complex -> <> -> tcomo|tcoma|tcomx -> <>
/*
 * evaluate types
 * evaluate lvalues (addable == 1)
 */
bool
tcom(Node *n)
{

    return tcomo(n, ADDROF);
}
```

⟨*enum* `_anon_ (cc/com.c)` 121b⟩≡ (454)

```
enum
{
    ADDROF = 1<<0,
    CASTOF = 1<<1,
    ADDROP = 1<<2,
};
```

⟨*function* `tcomo` 121c⟩≡ (454)

```
bool
tcomo(Node *n, int f)
{
    Node *l, *r;
    Type *t;
    // bool | int
```

```
    int o;
    ⟨tcomo() other locals 182g⟩

    ⟨tcomo() sanity check n 122a⟩
    n->addable = false;
    l = n->left;
    r = n->right;

    switch(n->op) {
    ⟨tcomo() switch node kind cases 123c⟩
    default:
        diag(n, "unknown op in type complex: %O", n->op);
        goto bad;

    }

    t = n->type;
    ⟨tcomo() sanity check t after typing 122b⟩
    ⟨tcomo() sanity check type width 122c⟩
    ⟨tcomo() special adjustments for arrays and functions 123a⟩
    return false; // everything is fine

⟨tcomo() extra labels 123b⟩

bad:
    n->type = T;
    return true;
}
```

⟨tcomo() *sanity check n* 122a⟩≡ (121c)

```
if(n == Z) {
    diag(Z, "Z in tcom");
    errorexit();
}
```

⟨tcomo() *sanity check t after typing* 122b⟩≡ (121c)

```
if(t == T)
    goto bad;
```

⟨tcomo() *sanity check type width* 122c⟩≡ (121c)

```
if(t->width < 0) {
    snap(t);
    if(t->width < 0) {
        if(typesu[t->etype] && t->tag)
            diag(n, "structure not fully declared %s", t->tag->name);
        else
            diag(n, "structure not fully declared");
        goto bad;
    }
}
```

9.5.9 Array to pointer conversions

⟨*global* typeaf 122d⟩≡ (443b)

```
char typeaf[NTYPE];
```

⟨*global* typeafinit 122e⟩≡ (443b)

```
int typeafinit[] =
{
    TFUNC, TARRAY, -1,
};
```

⟨tcomo() *special adjustments for arrays and functions* 123a⟩≡ (121c)

```
if(typeaf[t->etype]) {
    if(f & ADDROF)
        goto addaddr;
    if(f & ADDROP)
        warn(n, "address of array/func ignored");
}
```

⟨tcomo() *extra labels* 123b⟩≡ (121c)

```
addaddr:
    if(tlvalue(n))
        goto bad;
    l = new1(OXXX, Z, Z);
    *l = *n;
    n->op = OADDR;
    if(l->type->etype == TARRAY)
        l->type = l->type->link;
    n->left = l;
    n->right = Z;
    n->addable = false;
    n->type = typ(TIND, l->type);
    n->type->width = types[TIND]->width;
    return false;
```

9.6 Typechecking expressions

⟨tcomo() *switch node kind cases* 123c⟩≡ (121c) 123d ▷

```
case OCOMMA:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    n->type = r->type;
    break;
```

9.6.1 Numeric constants

⟨tcomo() *switch node kind cases* 123d⟩+≡ (121c) ◁ 123c 123e ▷

```
case OCONST:
    break;
```

9.6.2 String constants

⟨tcomo() *switch node kind cases* 123e⟩+≡ (121c) ◁ 123d 123f ▷

```
case OSTRING:
    ⟨tcomo() in OSTRING case, if not an array or chars 182f⟩
    ⟨tcomo() transform OSTRING in ONAME 181e⟩
    break;
```

9.6.3 Entity uses

⟨tcomo() *switch node kind cases* 123f⟩+≡ (121c) ◁ 123e 124c ▷

```
case ONAME:
    ⟨tcomo() in ONAME case, check if name declared 124a⟩
    ⟨tcomo() in ONAME case, if enum constant 124b⟩
    // else
```

```
    n->addable = true;
    ⟨tcomo() in ONAME case, if extern register 366b⟩
    break;
```

⟨tcomo() *in ONAME case, check if name declared* 124a⟩≡ (123f)

```
if(n->type == T) {
    diag(n, "name not declared: %F", n);
    goto bad;
}
```

⟨tcomo() *in ONAME case, if enum constant* 124b⟩≡ (123f)

```
if(n->type->etype == TENUM) {
    n->op = OCONST;
    if(!typefd[n->type->etype])
        n->vconst = n->sym->vconst;
    else
        n->fconst = n->sym->fconst;
    n->type = n->sym->tenum;
    break;
}
```

9.6.4 Arithmetic expressions

⟨tcomo() *switch node kind cases* 124c⟩+≡ (121c) ◁123f 124d▷

```
case OADD:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, l->type, r->type, tadd))
        goto bad;
    arith(n, true);
    break;
```

⟨tcomo() *switch node kind cases* 124d⟩+≡ (121c) ◁124c 125a▷

```
case OSUB:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, l->type, r->type, tsub))
        goto bad;
    arith(n, true);
    break;
```

⟨*global* tsub 124e⟩≡ (443b)

```
long tsub[NTYPE];
```

⟨*global* tsubinit 124f⟩≡ (443b)

```
Init tsubinit[] =
{
    TCHAR,  BNUMBER, 0,
    TUCHAR,  BNUMBER, 0,
    TSHORT,  BNUMBER, 0,
    TUSHORT, BNUMBER, 0,
    TINT,  BNUMBER, 0,
    TUINT,  BNUMBER, 0,
    TLONG,  BNUMBER, 0,
    TULONG,  BNUMBER, 0,
```

```
    TVLONG,  BNUMBER, 0,
    TUVLONG, BNUMBER, 0,

    TFLOAT,  BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,
    TIND,  BINTEGER|BIND, 0,
    -1,  0,  0,
};
```

⟨tcomo() *switch node kind cases* 125a⟩+≡ (121c) ◁124d 125d▷

```
case OMUL:
case ODIV:
case OLMUL:
case OLDIV:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, l->type, r->type, tmul))
        goto bad;
    arith(n, true);
    ⟨tcomo() in OMUL/ODIV case, adjust opcode if unsigned type 186j⟩
    break;
```

⟨*global* tmul 125b⟩≡ (443b)

```
long tmul[NTYPE];
```

⟨*global* tmulinit 125c⟩≡ (443b)

```
Init tmulinit[] =
{
    TCHAR,  BNUMBER, 0,
    TUCHAR,  BNUMBER, 0,
    TSHORT,  BNUMBER, 0,
    TUSHORT, BNUMBER, 0,
    TINT,  BNUMBER, 0,
    TUINT,  BNUMBER, 0,
    TLONG,  BNUMBER, 0,
    TULONG,  BNUMBER, 0,
    TVLONG,  BNUMBER, 0,
    TUVLONG, BNUMBER, 0,

    TFLOAT,  BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,
    -1,  0,  0,
};
```

⟨tcomo() *switch node kind cases* 125d⟩+≡ (121c) ◁125a 126a▷

```
case OMOD:
case OLMOD:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, l->type, r->type, tand))
        goto bad;
    arith(n, true);
    ⟨tcomo() in OMOD case, adjust opcode if unsigned type 187a⟩
    break;
```

⟨tcomo() *switch node kind cases* 126a⟩+≡ (121c) ◁125d 126b▷

```
case OPOS:
    if(tcom(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩

    r = l;
    l = new(OCONST, Z, Z);
    l->vconst = 0;
    l->type = types[TINT];
    n->op = OADD;
    n->right = r;
    n->left = l;

    if(tcom(l))
        goto bad;
    if(tcompat(n, l->type, r->type, tsub))
        goto bad;
    arith(n, true);
    break;
```

⟨tcomo() *switch node kind cases* 126b⟩+≡ (121c) ◁126a 126d▷

```
case ONEG:
    if(tcom(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩

    if(!machcap(n)) {
        r = l;
        l = new(OCONST, Z, Z);
        l->vconst = 0;
        l->type = types[TINT];
        n->op = OSUB;
        n->right = r;
        n->left = l;

        if(tcom(l))
            goto bad;
        if(tcompat(n, l->type, r->type, tsub))
            goto bad;
    }
    arith(n, true);
    break;
```

⟨*function* machcap*(arm)* 126c⟩≡ (460a)

```
/* default, like old cc */
bool
machcap(Node*)
{
    return false;
}
```

⟨tcomo() *switch node kind cases* 126d⟩+≡ (121c) ◁126b 127c▷

```
case OAND:
case OOR:
case OXOR:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
```

```
    if(tcompat(n, l->type, r->type, tand))
        goto bad;
    arith(n, true);
    break;
```

⟨*global* tand 127a⟩≡ (443b)

```
long tand[NTYPE];
```

⟨*global* tandinit 127b⟩≡ (443b)

```
Init tandinit[] =
{
    TCHAR,  BINTEGER, 0,
    TUCHAR,  BINTEGER, 0,
    TSHORT,  BINTEGER, 0,
    TUSHORT, BINTEGER, 0,

    TINT,  BNUMBER, 0,
    TUINT,  BNUMBER, 0,

    TLONG,  BINTEGER, 0,
    TULONG,  BINTEGER, 0,
    TVLONG,  BINTEGER, 0,
    TUVLONG, BINTEGER, 0,
    -1,  0,  0,
};
```

⟨tcomo() *switch node kind cases* 127c⟩+≡ (121c) ◁126d 131a▷

```
case OASHL:
case OASHR:
case OLSHR:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, l->type, r->type, tand))
        goto bad;

    n->right = Z;
    arith(n, true);

    n->right = new1(OCAST, r, Z);
    n->right->type = types[TINT];
    ⟨tcomo() in OASHL/OASHR case, adjust opcode if unsigned type 186h⟩
    break;
```

9.6.5 Arithmetic conversions

⟨*function* arith 127d⟩≡ (443b)

```
/// tcomo -> <>
/*
 * "the usual arithmetic conversions are performed"
 */
void
arith(Node *n, bool f)
{
    Type *t1, *t2;
    // enum<Type_kind>
    int i, j;
    // enum<Type_kind>
```

```
    int k;
    ⟨arith() other locals 129d⟩

    ⟨arith() set t1 and t2, left and right types 128a⟩
    ⟨arith() set i and j, etype of left and right types 128b⟩

    k = tab[i][j];

    ⟨arith() set node type 129a⟩

    ⟨arith() if substract two pointers 130d⟩
    // else
    ⟨arith() if left type not sametype than node type, add OCAST 129e⟩
    ⟨arith() if right type not sametype than node type, add OCAST 129f⟩
    return;
bad:
    diag(n, "pointer addition not fully declared: %T", n->type->link);
}
```

⟨arith() *set t1 and t2, left and right types* 128a⟩≡ (127d)

```
t1 = n->left->type;
if(n->right == Z)
    t2 = t1;
else
    t2 = n->right->type;
```

⟨arith() *set i and j, etype of left and right types* 128b⟩≡ (127d)

```
i = TXXX;
if(t1 != T)
    i = t1->etype;
j = TXXX;
if(t2 != T)
    j = t2->etype;
```

⟨*global* tab 128c⟩≡ (443b)

```
char tab[NTYPE][NTYPE] =
{
  [TXXX] = { 0,
        },

  [TCHAR] = { 0, TCHAR, TUCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG,
            TULONG, TVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
  [TUCHAR] = { 0, TUCHAR, TUCHAR, TUSHORT, TUSHORT, TUINT, TUINT, TULONG,
            TULONG, TUVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
  [TSHORT] = { 0, TSHORT, TUSHORT, TSHORT, TUSHORT, TINT, TUINT, TLONG,
            TULONG, TVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
  [TUSHORT] = { 0, TUSHORT, TUSHORT, TUSHORT, TUSHORT, TUINT, TUINT, TULONG,
            TULONG, TUVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
  [TINT] = { 0, TINT, TUINT, TINT, TUINT, TINT, TUINT, TLONG,
            TULONG, TVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
  [TUINT] = { 0, TUINT, TUINT, TUINT, TUINT, TUINT, TUINT, TULONG,
            TULONG, TUVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
  [TLONG] = { 0, TLONG, TULONG, TLONG, TULONG, TLONG, TULONG, TLONG,
            TULONG, TVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
```

```
        },
    [TULONG] = { 0, TULONG, TULONG, TULONG, TULONG, TULONG, TULONG, TULONG,
                TULONG, TUVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
    [TVLONG] = { 0, TVLONG, TUVLONG, TVLONG, TUVLONG, TVLONG, TUVLONG, TVLONG,
                TUVLONG, TVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
    [TUVLONG] = { 0, TUVLONG, TUVLONG, TUVLONG, TUVLONG, TUVLONG, TUVLONG, TUVLONG,
                TUVLONG, TUVLONG, TUVLONG, TFLOAT, TDOUBLE, [TIND] = TIND,
        },

    [TFLOAT] = { 0, TFLOAT, TFLOAT, TFLOAT, TFLOAT, TFLOAT, TFLOAT, TFLOAT,
                TFLOAT, TFLOAT, TFLOAT, TFLOAT, TDOUBLE, [TIND] = TIND,
        },
    [TDOUBLE] = { 0, TDOUBLE, TDOUBLE, TDOUBLE, TDOUBLE, TDOUBLE, TDOUBLE, TDOUBLE,
                TDOUBLE, TDOUBLE, TDOUBLE, TFLOAT, TDOUBLE, [TIND] = TIND,
        },

    [TIND] = { 0, TIND, TIND, TIND, TIND, TIND, TIND, TIND,
                TIND, TIND, TIND, TIND, TIND, [TIND] = TIND,
        },
  };
```

⟨arith() *set node type* 129a⟩≡ (127d)

```
  ⟨arith() if result type is a pointer 129b⟩
  else {
      ⟨arith() if f, convert up to at least int 129c⟩
      n->type = types[k];
  }
```

⟨arith() *if result type is a pointer* 129b⟩≡ (129a)

```
  if(k == TIND) {
      if(i == TIND)
          n->type = t1;
      else
      if(j == TIND)
          n->type = t2;
  }
```

⟨arith() *if* f, *convert up to at least int* 129c⟩≡ (129a)

```
  /* convert up to at least int */
  if(f)
    while(k < TINT)
      k += 2;
```

⟨arith() *other locals* 129d⟩≡ (127d)

```
  Node *n1;
  long w;
```

⟨arith() *if left type not sametype than node type, add OCAST* 129e⟩≡ (127d)

```
  if(!sametype(n->type, n->left->type)) {
      n->left = new1(OCAST, n->left, Z);
      n->left->type = n->type;
      ⟨arith() pointer arithmetic with left child 130a⟩
  }
```

⟨arith() *if right type not sametype than node type, add OCAST* 129f⟩≡ (127d)

```
  if(n->right != Z)
  if(!sametype(n->type, n->right->type)) {
      n->right = new1(OCAST, n->right, Z);
      n->right->type = n->type;
      ⟨arith() pointer arithmetic with right child 130c⟩
  }
```

9.6.6 Pointer arithmetic

⟨arith() *pointer arithmetic with left child* 130a⟩≡ (129e)

```
if(n->type->etype == TIND) {
    w = n->type->link->width;
    ⟨arith() if incomplete width 130b⟩
    if(w > 1) {
        n1 = new1(OCONST, Z, Z);
        n1->vconst = w;
        n1->type = n->type;
        n->left = new1(OMUL, n->left, n1);
        n->left->type = n->type;
    }
}
```

⟨arith() *if incomplete width* 130b⟩≡ (130)

```
if(w < 1) {
    snap(n->type->link);
    w = n->type->link->width;
    if(w < 1)
        goto bad;
}
```

⟨arith() *pointer arithmetic with right child* 130c⟩≡ (129f)

```
if(n->type->etype == TIND) {
    w = n->type->link->width;
    ⟨arith() if incomplete width 130b⟩
    if(w != 1) {
        n1 = new1(OCONST, Z, Z);
        n1->vconst = w;
        n1->type = n->type;
        n->right = new1(OMUL, n->right, n1);
        n->right->type = n->type;
    }
}
```

⟨arith() *if substract two pointers* 130d⟩≡ (127d)

```
if(n->op == OSUB)
if(i == TIND && j == TIND) {
    w = n->right->type->link->width;
    if(w < 1 || n->left->type->link == T || n->left->type->link->width < 1)
        goto bad;
    n->type = types[ewidth[TIND] <= ewidth[TLONG]? TLONG: TVLONG];
    if(1 && ewidth[TIND] > ewidth[TLONG]){
        n1 = new1(OXXX, Z, Z);
        *n1 = *n;
        n->op = OCAST;
        n->left = n1;
        n->right = Z;
        n->type = types[TLONG];
    }
    if(w > 1) {
        n1 = new1(OXXX, Z, Z);
        *n1 = *n;
        n->op = ODIV;
        n->left = n1;
        n1 = new1(OCONST, Z, Z);
        n1->vconst = w;
        n1->type = n->type;
        n->right = n1;
```

```
        ⟨arith() when substract two pointers, convert DIV to SHR if can 256c⟩
    }
    return;
}
```

9.6.7 Boolean expressions

⟨tcomo() *switch node kind cases* 131a⟩+≡ (121c) ◁127c 131d▷

```
case OEQ:
case ONE:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    ⟨tcomo() when OEQ/ONE, typing extensions 359b⟩
    if(tcompat(n, l->type, r->type, trel))
        goto bad;
    arith(n, false);

    n->type = types[TINT];
    break;
```

⟨*global* trel 131b⟩≡ (443b)

```
long trel[NTYPE];
```

⟨*global* trelinit 131c⟩≡ (443b)

```
Init trelinit[] =
{
    TCHAR,  BNUMBER, 0,
    TUCHAR,  BNUMBER, 0,
    TSHORT,  BNUMBER, 0,
    TUSHORT, BNUMBER, 0,
    TINT,  BNUMBER, 0,
    TUINT,  BNUMBER, 0,
    TLONG,  BNUMBER, 0,
    TULONG,  BNUMBER, 0,
    TVLONG,  BNUMBER, 0,
    TUVLONG, BNUMBER, 0,

    TFLOAT,  BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,

    TIND,  BIND,  0,
    -1,  0,  0,
};
```

⟨tcomo() *switch node kind cases* 131d⟩+≡ (121c) ◁131a 132a▷

```
case OLT:
case OGE:
case OGT:
case OLE:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    ⟨tcomo() when OLT/OGE/OGT/OLE, typing extensions 382c⟩
    if(tcompat(n, l->type, r->type, trel))
        goto bad;
    arith(n, false);
```

```
    ⟨tcomo() when OLT/OGE/OGT/OLE, adjust opcode if unsigned type 187b⟩
    n->type = types[TINT];
    break;
```

⟨tcomo() *switch node kind cases* 132a⟩+≡ (121c) ◁131d 132c▷

```
  case ONOT:
    if(tcom(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, T, l->type, tnot))
        goto bad;
    n->type = types[TINT];
    break;
```

⟨*global* tnot 132b⟩≡ (443b)

```
  long tnot[1] =
  {
    BNUMBER|BIND,
  };
```

⟨tcomo() *switch node kind cases* 132c⟩+≡ (121c) ◁132a 132d▷

```
  case OCOM:
    if(tcom(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩

    if(!machcap(n)) {
        r = l;
        l = new(OCONST, Z, Z);
        l->vconst = -1;
        l->type = types[TINT];
        n->op = OXOR;
        n->right = r;
        n->left = l;

        if(tcom(l))
            goto bad;
        if(tcompat(n, l->type, r->type, tand))
            goto bad;
    }
    arith(n, false);
    break;
```

⟨tcomo() *switch node kind cases* 132d⟩+≡ (121c) ◁132c 133a▷

```
  case OANDAND:
  case OOROR:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tcompat(n, T, l->type, tnot) |
       tcompat(n, T, r->type, tnot))
        goto bad;
    n->type = types[TINT];
    break;
```

9.6.8 Assignments

⟨tcomo() *switch node kind cases* 133a⟩+≡ (121c) ◁132d 134b▷

```
case OAS:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tlvalue(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    ⟨tcomo() when OAS, typing extension 360a⟩
    if(tcompat(n, l->type, r->type, tasign))
        goto bad;

    ⟨tcomo() when OAS, const checking 153d⟩
    if(!sametype(l->type, r->type)) {
        r = new1(OCAST, r, Z);
        r->type = l->type;
        n->right = r;
    }
    n->type = l->type;
    break;
```

⟨*function* tlvalue 133b⟩≡ (454)

```
bool
tlvalue(Node *n)
{

    if(!n->addable) {
        diag(n, "not an l-value");
        return true;
    }
    return false;
}
```

⟨*global* tasign 133c⟩≡ (443b)

```
// map<Type_kind, set<Type_kind> >
long tasign[NTYPE];
```

⟨*global* tasigninit 133d⟩≡ (443b)

```
Init tasigninit[] =
{
    TCHAR,  BNUMBER, 0,
    TUCHAR,  BNUMBER, 0,
    TSHORT,  BNUMBER, 0,
    TUSHORT, BNUMBER, 0,
    TINT,  BNUMBER, 0,
    TUINT,  BNUMBER, 0,
    TLONG,  BNUMBER, 0,
    TULONG,  BNUMBER, 0,
    TVLONG,  BNUMBER, 0,
    TUVLONG, BNUMBER, 0,

    TFLOAT,  BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,

    TIND,  BIND,  0,
    TSTRUCT, BSTRUCT, 0,
    TUNION,  BUNION,  0,
    -1,  0,  0,
};
```

⟨stcompat() *when t1 matches t2, if ttab is tasign* 134a⟩≡ (119c)

```
if(ttab == tasign)
    if(b == BSTRUCT || b == BUNION)
        if(!sametype(t1, t2))
            return true;
```

⟨tcomo() *switch node kind cases* 134b⟩+≡ (121c) ◁133a 134c▷

```
case OASI: /* same as as, but no test for const */
    n->op = OAS; // converted in regular assign!
    o = tcom(l);
    if(o | tcom(r))
        goto bad;

    ⟨tcomo() when OAS, typing extension 360a⟩
    if(tlvalue(l) || tcompat(n, l->type, r->type, tasign))
        goto bad;
    if(!sametype(l->type, r->type)) {
        r = new1(OCAST, r, Z);
        r->type = l->type;
        n->right = r;
    }
    n->type = l->type;
    break;
```

⟨tcomo() *switch node kind cases* 134c⟩+≡ (121c) ◁134b 135c▷

```
case OASADD:
case OASSUB:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tlvalue(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    ⟨tcomo() when OASADD, typing extension 360b⟩
    if(tcompat(n, l->type, r->type, tasadd))
        goto bad;
    ⟨tcomo() when OASxxx, const checking 154a⟩
    t = l->type;
    arith(n, false);
    ⟨tcomo() when OASxxx, adjust node after arith 134d⟩
    break;
```

⟨tcomo() *when OASxxx, adjust node after arith* 134d⟩≡ (136a 135c 134c)

```
⟨tcomo() when OASxxx, remove casts added by arith 134e⟩
if(!sametype(t, n->type) && !mixedasop(t, n->type)) {
    r = new1(OCAST, n->right, Z);
    r->type = t;
    n->right = r;
    n->type = t;
}
```

⟨tcomo() *when OASxxx, remove casts added by arith* 134e⟩≡ (134d)

```
while(n->left->op == OCAST)
    n->left = n->left->left;
```

⟨*function* mixedasop 134f⟩≡ (443b)

```
bool
mixedasop(Type *l, Type *r)
{
    return !typefd[l->etype] && typefd[r->etype];
}
```

⟨*global* tasadd 135a⟩≡ (443b)

```
long tasadd[NTYPE];
```

⟨*global* tasaddinit 135b⟩≡ (443b)

```
Init tasaddinit[] =
{
    TCHAR,  BNUMBER, 0,
    TUCHAR,  BNUMBER, 0,
    TSHORT,  BNUMBER, 0,
    TUSHORT, BNUMBER, 0,
    TINT,  BNUMBER, 0,
    TUINT,  BNUMBER, 0,
    TLONG,  BNUMBER, 0,
    TULONG,  BNUMBER, 0,
    TVLONG,  BNUMBER, 0,
    TUVLONG, BNUMBER, 0,

    TFLOAT,  BNUMBER, 0,
    TDOUBLE, BNUMBER, 0,

    TIND,  BINTEGER, 0,
    -1,  0,  0,
};
```

⟨tcomo() *switch node kind cases* 135c⟩+≡ (121c) ◁134c 135d▷

```
case OASMUL:
case OASDIV:
case OASLMUL:
case OASLDIV:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tlvalue(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    ⟨tcomo() when OASMUL/OASDIV, typing extensions 382b⟩
    if(tcompat(n, l->type, r->type, tmul))
        goto bad;
    ⟨tcomo() when OASxxx, const checking 154a⟩
    t = l->type;
    arith(n, false);
    ⟨tcomo() when OASxxx, adjust node after arith 134d⟩
    ⟨tcomo() when OASMUL/OASDIV, adjust opcode if unsigned type 187g⟩
    break;
```

⟨tcomo() *switch node kind cases* 135d⟩+≡ (121c) ◁135c 136a▷

```
case OASLSHR:
case OASASHR:
case OASASHL:
    o = tcom(l);
    if(o | tcom(r))
        goto bad;
    if(tlvalue(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, l->type, r->type, tand))
        goto bad;
    n->type = l->type;
    n->right = new1(OCAST, r, Z);
    n->right->type = types[TINT];
```

```
    ⟨tcomo() when OASLSHR, adjust opcode if unsigned type 187f⟩
    break;
```

⟨tcomo() *switch node kind cases* 136a⟩+≡ (121c) ◁135d 136b▷

```
  case OASMOD:
  case OASLMOD:
  case OASOR:
  case OASAND:
  case OASXOR:
      o = tcom(l);
      if(o | tcom(r))
          goto bad;
      if(tlvalue(l))
          goto bad;
      ⟨tcomo() break if isfunct 370a⟩
      if(tcompat(n, l->type, r->type, tand))
          goto bad;
      t = l->type;
      arith(n, false);
      ⟨tcomo() when OASxxx, adjust node after arith 134d⟩
      ⟨tcomo() when OASMOD, adjust opcode if unsigned type 187h⟩
      break;
```

9.6.9 Pointers

⟨tcomo() *switch node kind cases* 136b⟩+≡ (121c) ◁136a 136c▷

```
  case OADDR:
      if(tcomo(l, ADDROP))
          goto bad;
      if(tlvalue(l))
          goto bad;

      ⟨tcomo() when OADDR, check if bitfield 353f⟩
      ⟨tcomo() when OADDR, check if register 366d⟩

      n->type = typ(TIND, l->type);
      n->type->width = types[TIND]->width;
      break;
```

⟨tcomo() *switch node kind cases* 136c⟩+≡ (121c) ◁136b 137a▷

```
  case OIND:
      if(tcom(l))
          goto bad;
      if(tcompat(n, T, l->type, tindir))
          goto bad;
      n->type = l->type->link;
      n->addable = true;
      break;
```

⟨*global* tindir 136d⟩≡ (443b)

```
  long tindir[1] =
  {
      BIND,
  };
```

9.6.10 Array accesses

9.6.11 Field accesses

⟨tcomo() *switch node kind cases* 137a⟩+≡ (121c) ◁136c 138b▷

```
case ODOT:
    if(tcom(l))
        goto bad;
    if(tcompat(n, T, l->type, tdot))
        goto bad;
    if(tcomd(n))
        goto bad;
    break;
```

⟨*global* tdot 137b⟩≡ (443b)

```
long tdot[1] =
{
    BSTRUCT|BUNION,
};
```

⟨*function* tcomd 137c⟩≡ (454)

```
bool
tcomd(Node *n)
{
    Type *t;
    long o;

    o = 0;
    t = dotsearch(n->sym, n->left->type->link, n, &o);
    if(t == T) {
        diag(n, "not a member of struct/union: %F", n);
        return true;
    }
    // convert field and set n->type
    ⟨tcomd() convert field access in offset 194a⟩
    return false;
}
```

⟨*function* dotsearch 137d⟩≡ (443b)

```
/// tcomd -> <>
Type*
dotsearch(Sym *s, Type *t, Node *n, long *off)
{
    Type *t1, *xt, *rt;

    xt = T;

    /*
     * look it up by name
     */
    for(t1 = t; t1 != T; t1 = t1->down)
        if(t1->sym == s) {
            if(xt != T)
                goto ambig;
            xt = t1;
        }

    ⟨dotsearch() look by type 138a⟩

    if(xt != T) {
```

```
        *off = xt->offset;
        return xt;
    }

    ⟨dotsearch() if unnamed substructures 361a⟩

    return xt;

ambig:
    diag(n, "ambiguous structure element: %s", s->name);
    return xt;
}
```

⟨dotsearch() *look by type* 138a⟩≡ (137d)

```
/*
 * look it up by type
 */
if(s->class == CTYPEDEF || s->class == CTYPESTR)
    for(t1 = t; t1 != T; t1 = t1->down)
        if(t1->sym == S && typesu[t1->etype])
            if(sametype(s->type, t1)) {
                if(xt != T)
                    goto ambig;
                xt = t1;
            }
```

9.6.12 Function calls

⟨tcomo() *switch node kind cases* 138b⟩+≡ (121c) ◁137a 140d▷

```
case OFUNC:
    o = tcomo(l, 0);
    if(o)
        goto bad;
    ⟨tcomo() when OFUNC case, add OIND if pointer function call 350a⟩
    if(tcompat(n, T, l->type, tfunct))
        goto bad;
    if(o | tcoma(l, r, l->type->down, true))
        goto bad;
    n->type = l->type->link;
    ⟨tcomo() when OFUNC case, warn function args not checked if old proto 359a⟩
    ⟨tcomo() when OFUNC case, format check 379a⟩
    break;
```

⟨*global* tfunct 138c⟩≡ (443b)

```
long tfunct[1] =
{
    BFUNC,
};
```

⟨*function* tcoma 138d⟩≡ (454)

```
bool
tcoma(Node *l, Node *n, Type *t, bool f)
{
    Node *n1;
    int o;

    ⟨tcoma() start of tcoma, adjust t if TDOT 139b⟩
    ⟨tcoma() if no arguments 139c⟩
```

```
    // recurse over arguments and parameters
    if(n->op == OLIST) {
        o = tcoma(l, n->left, t, false);
        if(t != T) {
            t = t->down;
            if(t == T)
                t = types[TVOID];
        }
        return o | tcoma(l, n->right, t, true);
    }

    // process each argument

    if(f && t != T)
        tcoma(l, Z, t->down, false);

    if(tcom(n) || tcompat(n, T, n->type, targ))
        return true;
    ⟨tcoma() check if too many arguments 140a⟩
    if(t != T) {
        ⟨tcoma() typing extension on argument 360d⟩
        if(stcompat(nodproto, t, n->type, tasign)) {
            diag(l, "argument prototype mismatch \"%T\" for \"%T\": %F",
                n->type, t, l);
            return true;
        }
        ⟨tcoma() adjust type if argument with small type 140b⟩
    }
    else
        ⟨tcoma() adjust type when empty type 140c⟩

    if(t != T && !sametype(t, n->type)) {
        n1 = new1(OXXX, Z, Z);
        *n1 = *n;
        n->op = OCAST;
        n->left = n1;
        n->right = Z;
        n->type = t;
        n->addable = false;
    }
    return false; // everything is fine
}
```

⟨*global* targ 139a⟩≡ (443b)

```
long targ[1] =
{
    BNUMBER|BIND|BSTRUCT|BUNION,
};
```

⟨tcoma() *start of tcoma, adjust t if TDOT* 139b⟩≡ (138d)

```
if(t != T)
if(t->etype == TOLD || t->etype == TDOT) /* .../old in prototype */
    t = T;
```

⟨tcoma() *if no arguments* 139c⟩≡ (138d)

```
if(n == Z) {
    if(t != T && !sametype(t, types[TVOID])) {
        diag(n, "not enough function arguments: %F", l);
        return true;
    }
```

```
    return false;
}
```

⟨tcoma() *check if too many arguments* 140a⟩≡ (138d)

```
if(sametype(t, types[TVOID])) {
    diag(n, "too many function arguments: %F", l);
    return true;
}
```

⟨tcoma() *adjust type if argument with small type* 140b⟩≡ (138d)

```
switch(t->etype) {
case TCHAR:
case TSHORT:
    t = types[TINT];
    break;

case TUCHAR:
case TUSHORT:
    t = types[TUINT];
    break;
}
```

⟨tcoma() *adjust type when empty type* 140c⟩≡ (138d)

```
switch(n->type->etype)
{
case TCHAR:
case TSHORT:
    t = types[TINT];
    break;

case TUCHAR:
case TUSHORT:
    t = types[TUINT];
    break;

case TFLOAT:
    t = types[TDOUBLE];
}
```

⟨tcomo() *switch node kind cases* 140d⟩+≡ (121c) ◁138b 141a▷

```
case ODOTDOT:
    /*
     * tcom has already been called on this subtree
     */
    *n = *n->left;
    if(n->type == T)
        goto bad;
    break;
```

⟨*global* nodproto 140e⟩≡ (436d)

```
Node* nodproto;
```

⟨cinit() *nodproto initialization* 140f⟩≡ (35a)

```
nodproto = new(OPROTO, Z, Z);
```

9.6.13 Cast

⟨tcomo() *switch node kind cases* 141a⟩+≡ (121c) ◁140d 141d▷

```
case OCAST:
    if(n->type == T)
        break;
    if(n->type->width == types[TLONG]->width) {
        if(tcomo(l, ADDROF|CASTOF))
            goto bad;
    } else
        if(tcom(l))
            goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, l->type, n->type, tcast))
        goto bad;
    break;
```

⟨*global* tcast 141b⟩≡ (443b)

```
long tcast[NTYPE];
```

⟨*global* tcastinit 141c⟩≡ (443b)

```
Init tcastinit[] =
{
    TCHAR,  BNUMBER|BIND|BVOID, 0,
    TUCHAR,  BNUMBER|BIND|BVOID, 0,
    TSHORT,  BNUMBER|BIND|BVOID, 0,
    TUSHORT, BNUMBER|BIND|BVOID, 0,
    TINT,  BNUMBER|BIND|BVOID, 0,
    TUINT,  BNUMBER|BIND|BVOID, 0,
    TLONG,  BNUMBER|BIND|BVOID, 0,
    TULONG,  BNUMBER|BIND|BVOID, 0,
    TVLONG,  BNUMBER|BIND|BVOID, 0,
    TUVLONG, BNUMBER|BIND|BVOID, 0,

    TFLOAT,  BNUMBER|BVOID,  0,
    TDOUBLE, BNUMBER|BVOID,  0,

    TIND,  BINTEGER|BIND|BVOID, 0,
    TVOID,  BVOID,   0,
    TSTRUCT, BSTRUCT|BVOID,  0,
    TUNION,  BUNION|BVOID,  0,
    -1,  0,   0,
};
```

9.6.14 Ternary expressions

⟨tcomo() *switch node kind cases* 141d⟩+≡ (121c) ◁141a 143a▷

```
case OCOND:
    o = tcom(l);
    o |= tcom(r->left);
    if(o | tcom(r->right))
        goto bad;
    ⟨tcomo() in OCOND, nil handling 142a⟩
    if(sametype(r->right->type, r->left->type)) {
        r->type = r->right->type;
        n->type = r->type;
        break;
    }
    if(tcompat(r, r->left->type, r->right->type, trel))
```

```
        goto bad;
    arith(r, false);
    n->type = r->type;
    break;
```

⟨tcomo() *in OCOND, nil handling* 142a⟩≡ (141d)

```
if(r->right->type->etype == TIND && vconst(r->left) == 0) {
    r->left->type = r->right->type;
    r->left->vconst = 0;
}
if(r->left->type->etype == TIND && vconst(r->right) == 0) {
    r->right->type = r->left->type;
    r->right->vconst = 0;
}
```

⟨*function* vconst 142b⟩≡ (443b)

```
int
vconst(Node *n)
{
    int i;

    if(n == Z)
        goto no;
    if(n->op != OCONST)
        goto no;
    if(n->type == T)
        goto no;

    switch(n->type->etype)
    {
    case TFLOAT:
    case TDOUBLE:
        i = 100;
        if(n->fconst > i || n->fconst < -i)
            goto no;
        i = n->fconst;
        if(i != n->fconst)
            goto no;
        return i;

    case TVLONG:
    case TUVLONG:
        i = n->vconst;
        if(i != n->vconst)
            goto no;
        return i;

    case TCHAR:
    case TUCHAR:
    case TSHORT:
    case TUSHORT:
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:
    case TIND:
        i = n->vconst;
        if(i != n->vconst)
            goto no;
        return i;
```

```
    }
no:
    return -159; /* first uninteresting constant */
}
```

9.6.15 Prefix/postfix

⟨tcomo() *switch node kind cases* 143a⟩+≡ (121c) ◁141d 143b▷

```
case OPREINC:
case OPREDEC:
case OPOSTINC:
case OPOSTDEC:
    if(tcom(l))
        goto bad;
    if(tlvalue(l))
        goto bad;
    ⟨tcomo() break if isfunct 370a⟩
    if(tcompat(n, l->type, types[TINT], tadd))
        goto bad;
    n->type = l->type;

    if(n->type->etype == TIND)
    if(n->type->link->width < 1) {
        snap(n->type->link);
        if(n->type->link->width < 1)
            diag(n, "inc/dec of a void pointer");
    }
    break;
```

9.6.16 sizeof()

⟨tcomo() *switch node kind cases* 143b⟩+≡ (121c) ◁143a 144a▷

```
case OSIZE:
    if(l != Z) {
        if(l->op != OSTRING && l->op != OLSTRING)
            if(tcomo(l, 0))
                goto bad;
        ⟨tcomo() when OSIZE, check if sizeof bitfield 353g⟩
        n->type = l->type;
    }
    ⟨tcomo() when OSIZE, sanity check n 143c⟩
    n->op = OCONST;
    n->left = Z;
    n->right = Z;
    n->vconst = convvtox(n->type->width, TINT);
    n->type = types[TINT];
    break;
```

⟨tcomo() *when OSIZE, sanity check n* 143c⟩≡ (143b)

```
if(n->type == T)
    goto bad;
if(n->type->width <= 0) {
    diag(n, "sizeof undefined type");
    goto bad;
}
if(n->type->etype == TFUNC) {
    diag(n, "sizeof function");
    goto bad;
}
```

9.7 Typechecking statements

9.7.1 return

⟨tcomo() *switch node kind cases* 144a⟩+≡ (121c) ◁143b 183a▷

```
case ORETURN:
    if(l == Z) {
        if(n->type->etype != TVOID)
            warn(n, "null return of a typed function");
        break;
    }
    if(tcom(l))
        goto bad;
    ⟨tcomo() when ORETURN, typing extensions 360c⟩
    if(tcompat(n, n->type, l->type, tasign))
        break;
    ⟨tcomo() when ORETURN, const checking 154c⟩
    if(!sametype(n->type, l->type)) {
        l = new1(OCAST, l, Z);
        l->type = n->type;
        n->left = l;
    }
    break;
```

9.7.2 switch

⟨*ulstmt rule, SWITCH case, adjust cexpr node* 144b⟩≡ (84b)

```
// generate (0:int - (0:int - x))
// which will force the usual arithmetic conversions
// (and will be simplified later by some transformations)

$$ = new(OCONST, Z, Z);
$$->vconst = 0;
$$->type = types[TINT];
$3 = new(OSUB, $$, $3);

$$ = new(OCONST, Z, Z);
$$->vconst = 0;
$$->type = types[TINT];
$3 = new(OSUB, $$, $3);
```

⟨gen() *when OSWITCH, typecheck condition* 144c⟩≡ (228b)

```
if(!typeswitch[l->type->etype]) {
    diag(n, "switch expression must be integer");
    break;
}
```

⟨*global* typeswitch 144d⟩≡ (436d)

```
// set<Type_kind>
char* typeswitch;
```

⟨tinit() *initialise 32 bits defaults type sets* 144e⟩≡ (34b) 222b▷

```
typeswitch = typechl;
```

⟨*global* typechl 144f⟩≡ (443b)

```
char typechl[NTYPE];
```

⟨*global* typechlinit 145a⟩≡ (443b)

```
int typechlinit[] =
{
    TCHAR, TUCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG, TULONG, -1,
};
```

9.7.3 if

⟨bcomplex() *typecheck n* 145b⟩≡ (160a)

```
if(n->type != T)
   if(tcompat(n, T, n->type, tnot))
      n->type = T;
```

9.8 Typechecking declarations and constant evaluation

9.8.1 Enumerations

⟨En *value fields* 145c⟩≡ (104c)

```
vlong lastenum; /* value of current enum */
double floatenum; /* value of current enum */ // for floats enums
```

⟨doenum() *typecheck and evaluate expression* 145d⟩≡ (105d)

```
complex(n); // will call evconst()
```

⟨doenum() *save current value of enumeration constant* 145e⟩≡ (105d)

```
if(typefd[en.cenum->etype])
    en.floatenum = n->fconst;
else
    en.lastenum = n->vconst;
```

⟨Sym *enum value fields* 145f⟩+≡ (21e) ◁105b

```
vlong vconst;
double fconst;
```

⟨doenum() *set value for symbol* 145g⟩≡ (105d)

```
if(typefd[s->tenum->etype]) {
    s->fconst = en.floatenum;
    en.floatenum++;
} else {
    s->vconst = convvtox(en.lastenum, s->tenum->etype);
    en.lastenum++;
}
```

⟨*function* maxtype 145h⟩≡ (450)

```
/*
 * this routine is very suspect.
 * ANSI requires the enum type to
 * be represented as an 'int'
 * this means that 0x81234567
 * would be illegal. this routine
 * makes signed and unsigned go
 * to unsigned.
 */
Type*
maxtype(Type *t1, Type *t2)
{
```

```
    if(t1 == T)
        return t2;
    if(t2 == T)
        return t1;
    if(t1->etype > t2->etype)
        return t1;
    return t2;
}
```

9.8.2 Arrays

⟨dodecl() *locals* 146a⟩+≡ (78d) ◁106e

```
long v;
```

⟨dodecl() *switch node kind cases, case OARRAY, if array has a size* 146b⟩≡ (106f)

```
t->width = 0; // array[], size could be set by doinit (or specified in n1)
if(n1 != Z) {
    complex(n1); // will call evconst()
    v = (n1->op == OCONST) ? n1->vconst : -1;
    if(v <= 0) {
        diag(n, "array size must be a positive constant");
        v = 1;
    }
    t->width = v * t->link->width;
}
```

9.8.3 Functions

⟨fnproto1() *when OPROTO case, adjust parameter type* 146c⟩≡ (108a)

```
if(lastdcltype != T)
    *t = *paramconv(lastdcltype, true);
```

⟨pdecl() *adjust type* 146d⟩≡ (110c)

```
t = paramconv(t, class==CPARAM);
```

⟨*function* paramconv 146e⟩≡ (450)

```
/// fnproto1 | pdecl -> <>
Type*
paramconv(Type *t, bool f)
{

    switch(t->etype) {
    ⟨paramconv() switch etype cases 350b⟩
    ⟨paramconv() switch etype, adjust type when not f cases 147a⟩
    }
    return t;
}
```

⟨pdecl() *sanity check type* 146f⟩≡ (110c)

```
if(typesu[t->etype] && t->width <= 0)
    diag(Z, "incomplete structure: %s", t->tag->name);
```

⟨paramconv() *switch etype, adjust type when not f cases* 147a⟩≡ (146e)

```
case TFLOAT:
    if(!f)
        t = types[TDOUBLE];
    break;

case TCHAR:
case TSHORT:
    if(!f)
        t = types[TINT];
    break;

case TUCHAR:
case TUSHORT:
    if(!f)
        t = types[TUINT];
    break;
```

9.8.4 Typedef expansions

⟨*function* tcopy 147b⟩≡ (450)

```
/*
 * make a copy of a typedef
 * the problem is to split out incomplete
 * arrays so that it is in the variable
 * rather than the typedef.
 */
Type*
tcopy(Type *t)
{
    Type *tl, *tx;
    int et;

    if(t == T)
        return t;
    et = t->etype;
    if(typesu[et])
        return t;

    tl = tcopy(t->link); // go deep

    if((et == TARRAY && t->width == 0) || tl != t->link) {
        tx = copytyp(t);
        tx->link = tl;
        return tx;
    }
    return t;
}
```

9.9 Lint checking

9.9.1 Unused variables

⟨Sym *identifier value, checking fields* 147c⟩≡ (23b)

```
bool aused;
```

⟨symini t() *remaining initialisations* 147d⟩≡ (24f)

```
s->aused = false;
```

⟨adecl() *initialize symbol* 148a⟩≡ (111b)
```
s->aused = false;
```

⟨*name rule, LNAME case, adjust more fields* 148b⟩≡ (88b)
```
$1->aused = true;
```

⟨reverdcl() *DAUTO case, warn if auto declared but not used* 148c⟩≡ (27c)
```
if(!s->aused) {
    nearln = s->varlineno;
    if(s->class == CAUTO)
        warn(Z, "auto declared and not used: %s", s->name);
    if(s->class == CPARAM)
        warn(Z, "param declared and not used: %s", s->name);
}
```

9.9.2 Used before set

9.9.3 False positives silencing: SET()/USED()

⟨*token declarations* 148d⟩+≡ (76a) ◁28c 366f▷
```
%token  LUSED LSET
```

⟨itab *entries, kencc extensions* 148e⟩≡ (42a) 366e▷
```
"SET",  LSET,  0,
"USED",  LUSED,  0,
```

⟨*statement nodes* 148f⟩+≡ (16a) ◁84a
```
OUSED,
OSET,
```

⟨*ulstmnt rule* 148g⟩+≡ (81c) ◁84b
```
|   LUSED '(' zelist ')' ';' { $$ = new(OUSED, $3, Z); }
|   LSET '(' zelist ')' ';'  { $$ = new(OSET, $3, Z); }
```

⟨gen() *switch node kind cases* 148h⟩≡ (158i) 159a▷
```
case OSET:
case OUSED:
    usedset(n->left, o);
    break;
```

⟨*function* usedset 148i⟩≡ (455b)
```
/// gen -> <>
void
usedset(Node *n, int o)
{
    if(n->op == OLIST) {
        usedset(n->left, o);
        usedset(n->right, o);
        return;
    }

    complex(n);

    switch(n->op) {
    case ONAME:
        if(o == OSET)
            gins(ANOP, Z, n);
        else
            gins(ANOP, n, Z);
```

```
        break;
    ⟨usedset() switch node op cases 352b⟩
    }
}
```

9.9.4 Redeclared or unused labels

⟨dcllabel() *when new label, set def and use fields* 149a⟩≡ (83h)

```
n->complex = defcontext;
n->addable = !defcontext;
```

⟨dcllabel() *if n not null, mark node as declared or used* 149b⟩≡ (83h)

```
if(n != Z) {
    if(defcontext) {
        if(n->complex)
            diag(Z, "label reused: %s", s->name);
        n->complex = true; // declared
    } else
        n->addable = true; // used
    return n;
}
```

⟨reverdcl() *DLABEL case, warn if label not used* 149c⟩≡ (27b)

```
if(s->label && !s->label->addable)
    warn(s->label, "label declared and not used \"%s\"", s->name);
```

9.9.5 Unreachable code

⟨*global* canreach 149d⟩≡ (436d)

```
bool canreach;
```

⟨*global* warnreach 149e⟩≡ (436d)

```
bool warnreach;
```

⟨codgen() *initialisation before call to gen* 149f⟩≡ (156d)

```
canreach = true;
warnreach = true;
```

⟨gen() *if not canreach* 149g⟩≡ (158i)

```
if(!canreach) {
    switch(o) {
    case OLABEL:
    case OCASE:
    case OLIST:
    case OCOMMA:
    case OBREAK:
    case OFOR:
    case OWHILE:
    case ODWHILE:
        /* all handled specially - see switch body below */
        break;
    default:
        if(warnreach) {
            warn(n, "unreachable code %O", o);
            warnreach = false;
        }
    }
}
```

If

⟨bcomplex() *check for deadheads* 150a⟩≡ (160a)

```
if(c != Z && n->op == OCONST && deadheads(c))
    return true;
```

⟨*function* deadhead 150b⟩≡ (443b)

```
/// deadheads -> <>
/*
 * return true if it is impossible to jump into the middle of n.
 */
static bool
deadhead(Node *n, bool caseok)
{
loop:
    if(n == Z)
        return true;

    switch(n->op) {

    case OCASE:
        if(!caseok)
            return false;
        goto rloop;
    case OLABEL:
        return false;



    case OLIST:
        if(!deadhead(n->left, caseok))
            return false;
    rloop:
        n = n->right;
        goto loop;


    case OWHILE:
    case ODWHILE:
    case OFOR:
        goto rloop;


    case OIF:
        return deadhead(n->right->left, caseok) &&
               deadhead(n->right->right, caseok);

    case OSWITCH:
        return deadhead(n->right, true);


    case ORETURN:
    case OGOTO:
    case OCONTINUE:
    case OBREAK:
        return true;

    case OSET:
    case OUSED:
        return true;
```

```
    }
    return true;
}
```

⟨*function* deadheads 151a⟩≡ (443b)

```
/// gen | bcomplex -> <>
bool
deadheads(Node *c)
{
    return deadhead(c->left, false) && deadhead(c->right, false);
}
```

⟨gen() *locals* 151b⟩≡ (158i) 159b ▷

```
bool f;
```

⟨gen() *switch node kind cases, OIF case, if bcomplex error* 151c⟩≡ (159c)

```
if(err) {
    if(typefd[l->type->etype])
        f = !l->fconst;
    else
        f = !l->vconst;
    if(debug['c'])
        print("%L const if %s\n", nearln, f ? "false" : "true");

    if(f) {
        canreach = true;
        supgen(n->right->left);
        oldreach = canreach;
        canreach = true;
        gen(n->right->right);
        /*
         * treat constant ifs as regular ifs for
         * reachability warnings.
         */
        if(!canreach && oldreach && debug['w'] < 2)
            warnreach = false;
    } else {
        canreach = true;
        gen(n->right->left);
        oldreach = canreach;
        canreach = true;
        supgen(n->right->right);
        /*
         * treat constant ifs as regular ifs for
         * reachability warnings.
         */
        if(!oldreach && canreach && debug['w'] < 2)
            warnreach = false;
        canreach = oldreach;
    }
}
```

supgen()

⟨*global* suppress 151d⟩≡ (460b)

```
bool suppress;
```

⟨*function* supgen 152a⟩≡ (455b)

```
void
supgen(Node *n)
{
    bool owarn;
    long opc;
    Prog *olastp;

    if(n == Z)
        return;

    suppress++;
    owarn = warnreach;
    opc = pc;
    olastp = lastp;

    warnreach = false;

    gen(n);

    lastp = olastp;
    olastp->link = nil;
    pc = opc;
    warnreach = owarn;
    suppress--;
}
```

9.9.6 Missing return

⟨codgen() *warn for possible missing return after call to gen* 152b⟩≡ (156d)

```
if(canreach && thisfntype->link->etype != TVOID){
    if(debug['B'])
        warn(Z, "no return at end of function: %s", n1->sym->name);
    else
        diag(Z, "no return at end of function: %s", n1->sym->name);
}
```

9.9.7 Unused expression

⟨cgenrel() *nullwarn check if nn is null* 152c⟩≡ (354c 204 201a 190c 189a 183c)

```
if(nn == Z) {
    nullwarn(l, r);
    break;
}
```

⟨*function* nullwarn 152d⟩≡ (455a)

```
void
nullwarn(Node *l, Node *r)
{
    warn(Z, "result of operation not used");
    if(l != Z)
        cgen(l, Z);
    if(r != Z)
        cgen(r, Z);
}
```

⟨cgenrel() *warn if target node is nil* 153a⟩≡ (180c)

```
if(nn == Z) {
    switch(o) {
    case OINDEX:
        nullwarn(l, r);
        break;
    default:
        nullwarn(Z, Z);
        break;
    }
    return;
}
```

9.9.8 Constant if: 5c -c

9.9.9 Out of range shifting

⟨ccom() *when OASHR/OASHL/OLSHR, check for stupid shift* 153b⟩≡ (243)

```
if(r->op == OCONST) {
    t = n->type->width * 8; /* bits per byte */
    if(r->vconst >= t || r->vconst <= -t)
        warn(n, "stupid shift: %lld", r->vconst);
}
```

⟨ccom() *switch node kind cases* 153c⟩≡ (240a) 240b ▷

```
case OAS:
case OASADD:
case OASSUB:
case OASMUL:
case OASLMUL:
case OASDIV:
case OASLDIV:
case OASMOD:
case OASLMOD:
case OASASHR:
case OASASHL:
case OASLSHR:
case OASAND:
case OASOR:
case OASXOR:
    ccom(l);
    ccom(r);
    if(n->op == OASLSHR || n->op == OASASHR || n->op == OASASHL)
      if(r->op == OCONST) {
        t = n->type->width * 8; /* bits per byte */
        if(r->vconst >= t || r->vconst < 0)
            warn(n, "stupid shift: %lld", r->vconst);
    }
    break;
```

9.9.10 Useless comparisons

9.10 Const checking

⟨tcomo() *when OAS, const checking* 153d⟩≡ (133a)

```
constas(n, l->type, r->type);
```

⟨tcomo() *when OASxxx, const checking* 154a⟩≡ (135c 134c)

```
constas(n, l->type, r->type);
```

⟨*function* constas 154b⟩≡ (443b)

```
void
constas(Node *n, Type *il, Type *ir)
{
    Type *l, *r;

    l = il;
    r = ir;

    if(l == T)
        return;
    if(l->garb & GCONSTNT) {
        warn(n, "assignment to a constant type (%T)", il);
        return;
    }
    if(r == T)
        return;
    for(;;) {
        if(l->etype != TIND || r->etype != TIND)
            break;
        l = l->link;
        r = r->link;
        if(l == T || r == T)
            break;
        if(r->garb & GCONSTNT)
            if(!(l->garb & GCONSTNT)) {
                warn(n, "assignment of a constant pointer type (%T)", ir);
                break;
            }
    }
}
```

⟨tcomo() *when ORETURN, const checking* 154c⟩≡ (144a)

```
constas(n, n->type, l->type);
```

Chapter 10

Assembly Generation

10.1 Overview

10.2 pc, p, and nextpc()

⟨*global* pc 155a⟩≡ (460b)
```
long pc;
```

⟨ginit() *pc initialisation* 155b⟩≡ (35d)
```
pc = 0;
```

⟨*function* nextpc 155c⟩≡ (462)
```
/// gins | gclean | ... -> <>
void
nextpc(void)
{

    p = alloc(sizeof(Prog));
    *p = zprog;
    p->lineno = nearln; // for origin tracking in db (from assembly to C)
    pc++;

    // add_tail(p, firstp/lastp)
    if(firstp == P) {
        firstp = p;
        lastp = p;
        return;
    }
    lastp->link = p;
    lastp = p;
}
```

⟨*global* p 155d⟩≡ (460b)
```
Prog* p;
```

⟨*global* firstp 155e⟩≡ (460b)
```
// list<ref_own<Prog> (next = Prog.link, last = lastp)
Prog* firstp;
```

⟨Prog *extra fields(arm)* 155f⟩≡ (28d)
```
// list<ref<Prog>> (head = firstp)
Prog* link;
```

⟨*constant* P 155g⟩≡ (456)
```
#define P ((Prog*)nil)
```

⟨*global* lastp 156a⟩≡ (460b)

```
// ref<Prog> (head = firstp)
Prog* lastp;
```

⟨ginit() *misc initialisations* 156b⟩≡ (35d) 161e ▷

```
firstp = P;
lastp = P;
```

⟨gclean() *generate last opcode, AEND* 156c⟩≡ (34a)

```
nextpc();
p->as = AEND;
```

10.3 Functions: codgen()

⟨*function* codgen 156d⟩≡ (455b)

```
// main -> compile -> yyparse -> xdecl -> <>
//@Scheck: used by cc.y
void codgen(Node *n, Node *nn)
{
    Prog *sp;
    ⟨codgen() locals 156e⟩

    ⟨codgen() initialisation 210d⟩
    ⟨codgen() set n1 node to node in nn where have ONAME 156f⟩
    nearln = nn->lineno;

    gpseudo(ATEXT, n1->sym, nodconst(stkoff));
    sp = p;

    ⟨codgen() if complex return type 223a⟩
    ⟨codgen() if use REGARG 279e⟩

    ⟨codgen() initialisation before call to gen 149f⟩
    // generate the assembly for the statements in the body
    gen(n);

    ⟨codgen() warn for possible missing return after call to gen 152b⟩
    ⟨codgen() before RET 282c⟩
    gbranch(ORETURN);

    ⟨codgen() register optimisation 281a⟩

    sp->to.offset += maxargsafe;
}
```

⟨codgen() *locals* 156e⟩≡ (156d) 279d ▷

```
Node *n1;
```

⟨codgen() *set n1 node to node in nn where have ONAME* 156f⟩≡ (156d)

```
/*
 * isolate name
 */
for(n1 = nn;; n1 = n1->left) {
    if(n1 == Z) {
        diag(nn, "cant find function name");
        return;
    }
    if(n1->op == ONAME)
        break;
}
```

⟨*global* constnode 157a⟩≡ (460b)

```
Node constnode;
```

⟨ginit() *special nodes initialisation* 157b⟩≡ (35d) 167g ▷

```
constnode.op = OCONST;
constnode.class = CXXX;
constnode.complex = 0;
constnode.addable = 20;
constnode.type = types[TLONG];
```

⟨*function* nodconst 157c⟩≡ (462)

```
Node*
nodconst(long v)
{
    constnode.vconst = v;
    return &constnode;
}
```

10.3.1 gpseudo()

⟨*function* gpseudo*(arm)* 157d⟩≡ (462)

```
/// codgen(function generation) | gextern -> <>
void
gpseudo(int a, Sym *s, Node *n)
{

    nextpc();

    p->as = a;
    p->from.type = D_OREG;
    p->from.sym = s;
    p->from.symkind = (s->class == CSTATIC) ? N_INTERN : N_EXTERN;
    ⟨gpseudo() if TEXT, set possible TEXT attributes 348c⟩
    naddr(n, &p->to);
    if(a == ADATA || a == AGLOBL)
        pc--;
}
```

10.3.2 gbranch()

⟨*function* gbranch*(arm)* 157e⟩≡ (462)

```
/// codgen | ... -> <>
void
gbranch(int o)
{
    int a;

    nextpc();

    switch(o) {
    case ORETURN: a = ARET; break;
    case OGOTO:   a = AB;   break;
    default:
        a = AGOK;
        diag(Z, "bad in gbranch %O",  o);
        nextpc();
    }
    p->as = a;
}
```

10.3.3 patch()

⟨*function* patch 158a⟩≡ (462)

```
/// gmove | gen | ... -> <>
void
patch(Prog *op, long pc)
{

    op->to.type = D_BRANCH;
    op->to.offset = pc;
}
```

10.3.4 Offset

⟨*global* stkoff 158b⟩≡ (436d)

```
long stkoff;
```

⟨*global* autoffset 158c⟩≡ (436d)

```
long autoffset;
```

⟨cinit() *namespace globals initialization* 158d⟩+≡ (35a) ◁25i

```
autoffset = 0;
```

⟨argmark() *initializations* 158e⟩≡ (109a)

```
autoffset = align(0, thisfntype->link, Aarg0);
stkoff = 0;
```

⟨argmark() *finalizations* 158f⟩≡ (109a)

```
autoffset = 0;
stkoff = 0;
```

⟨Sym *identifier value, code generation fields* 158g⟩≡ (23b) 235b▷

```
long offset;
```

⟨Node *code generation fields* 158h⟩+≡ (16b) ◁21c 160c▷

```
long xoffset;
```

10.4 Statements: gen()

⟨*function* gen 158i⟩≡ (455b)

```
/// codgen -> <>
void
gen(Node *n)
{
    // enum<Node_kind> of a statement
    int o;
    ⟨gen() locals 151b⟩

loop:
    if(n == Z)
        return;

    nearln = n->lineno;
    o = n->op;
    ⟨gen() debug opcode 417a⟩
    ⟨gen() if not canreach 149g⟩
    switch(o) {
    ⟨gen() switch node kind cases 148h⟩
    }
}
```

10.4.1 Blocks, sequences

⟨gen() *switch node kind cases* 159a⟩+≡ (158i) ◁148h 159c▷

```
case OLIST:
case OCOMMA:
    gen(n->left);

rloop:
    n = n->right;
    goto loop;
```

10.4.2 Conditionals

⟨gen() *locals* 159b⟩+≡ (158i) ◁151b 159d▷

```
Prog *sp;
Node *l;
bool err;
```

⟨gen() *switch node kind cases* 159c⟩+≡ (158i) ◁159a 160e▷

```
case OIF:
    l = n->left;
    err = bcomplex(l, n->right);
    ⟨gen() switch node kind cases, OIF case, if bcomplex error 151c⟩
    else {
        sp = p; // AB instr for 'else' to patch later (created by bcomplex())
        ⟨gen() when OIF, before gen then part, set canreach 159e⟩

        if(n->right->left != Z) // gen then part
            gen(n->right->left);

        ⟨gen() when OIF, before gen else part, save and set canreach 159f⟩

        if(n->right->right != Z) { // gen else part
            gbranch(OGOTO);
            patch(sp, pc);
            sp = p;
            gen(n->right->right);
        }
        patch(sp, pc);

        ⟨gen() when OIF, after gen everything, set canreach 159g⟩
    }
    break;
```

⟨gen() *locals* 159d⟩+≡ (158i) ◁159b 161f▷

```
bool oldreach;
```

⟨gen() *when OIF, before gen then part, set canreach* 159e⟩≡ (159c)

```
canreach = true;
```

⟨gen() *when OIF, before gen else part, save and set canreach* 159f⟩≡ (159c)

```
oldreach = canreach;
canreach = true;
```

⟨gen() *when OIF, after gen everything, set canreach* 159g⟩≡ (159c)

```
canreach = canreach || oldreach;
if(!canreach)
    warnreach = !suppress;
```

⟨*function* bcomplex 160a⟩≡ (455b)

```
bool//TODO error_1
bcomplex(Node *n, Node *c)
{

    complex(n);
    ⟨bcomplex() typecheck n 145b⟩
    ⟨bcomplex() generate fake goto if type error 160b⟩
    ⟨bcomplex() check for deadheads 150a⟩

    ⟨bcomplex() possibly convert node to funcall if unsupported 64 bit op 396a⟩
    boolgen(n, true, Z);

    return false; // no error //TODO: OK_0
}
```

⟨bcomplex() *generate fake goto if type error* 160b⟩≡ (160a)

```
if(n->type == T) {
    gbranch(OGOTO);
    return false;
}
```

10.4.3 Labels and goto

⟨Node *code generation fields* 160c⟩+≡ (16b) ◁158h 160d▷

```
long pc;
```

⟨Node *code generation fields* 160d⟩+≡ (16b) ◁160c

```
// ref<Prog>, but use 'void*' to be archi independent
void* label;
```

Labels

⟨gen() *switch node kind cases* 160e⟩+≡ (158i) ◁159c 160g▷

```
case OLABEL:
    ⟨gen() when LABEL, set canreach 160f⟩
    l = n->left;
    if(l) {
        l->pc = pc;
        if(l->label)
            patch(l->label, pc);
    }
    gbranch(OGOTO); /* prevent self reference in reg */
    patch(p, pc);

    goto rloop;
```

⟨gen() *when LABEL, set canreach* 160f⟩≡ (160e)

```
canreach = true;
```

Gotos

⟨gen() *switch node kind cases* 160g⟩+≡ (158i) ◁160e 161g▷

```
case OGOTO:
    ⟨gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 161a⟩
    n = n->left;
    ⟨gen() when OGOTO, sanity check n 161b⟩
    gbranch(OGOTO);
```

```
    if(n->pc) {
        patch(p, n->pc);
    } else {
        if(n->label)
             patch(n->label, pc-1);
        n->label = p;
    }
    return;
```

⟨gen() *when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach* 161a⟩≡ (165g 164a 163f 160g)

```
canreach = false;
warnreach = !suppress;
```

⟨gen() *when OGOTO, sanity check n* 161b⟩≡ (160g)

```
if(n == Z) // possible?
    return;
if(n->complex == 0) {
    diag(Z, "label undefined: %s", n->sym->name);
    return;
}
if(suppress)
    return;
```

10.4.4 Loops

⟨*global* breakpc 161c⟩≡ (460b)

```
// option<pc>, None = -1
long breakpc;
```

⟨*global* continpc 161d⟩≡ (460b)

```
// option<pc>, None = -1
long continpc;
```

⟨ginit() *misc initialisations* 161e⟩+≡ (35d) ◁156b 182a▷

```
breakpc = -1;
continpc = -1;
```

while/dowhile

⟨gen() *locals* 161f⟩+≡ (158i) ◁159d 164c▷

```
Prog *spc, *spb;
```

⟨gen() *switch node kind cases* 161g⟩+≡ (158i) ◁160g 162▷

```
case OWHILE:
case ODWHILE:
    l = n->left;

    gbranch(OGOTO);  /* entry */
    sp = p;

    ⟨gen() when OWHILE/ODWHILE, set continpc to pc before goto continue 164d⟩
    gbranch(OGOTO); // goto for continue
    spc = p;

    ⟨gen() when OWHILE/ODWHILE, set breakpc to pc before goto break 164e⟩
    gbranch(OGOTO); // goto for break
    spb = p;
```

```
    patch(spc, pc);
    if(n->op == OWHILE)
        patch(sp, pc);

    bcomplex(l, Z);  /* test */
    patch(p, breakpc);
    ⟨gen() when OWHILE/ODWHILE, increment nbreak 164h⟩

    if(n->op == ODWHILE)
        patch(sp, pc);

    gen(n->right);  /* body */

    gbranch(OGOTO);
    patch(p, continpc);

    patch(spb, pc); // exit

    ⟨gen() when OWHILE/ODWHILE, before exit, restore continpc/breakpc 164k⟩
    break;
```

for

⟨gen() *switch node kind cases* 162⟩+≡ (158i) ◁161g 163f▷

```
  case OFOR:
    l = n->left;
    ⟨gen() when OFOR, check canreach before gen anything 163a⟩

    gen(l->right->left); /* init */

    gbranch(OGOTO);  /* entry */
    sp = p;

    ⟨gen() when OFOR, check canreach after gen init 163b⟩

    ⟨gen() when OFOR, set continpc to pc before goto continue 164f⟩
    gbranch(OGOTO); // goto for continue
    spc = p;

    ⟨gen() when OFOR, set breakpc to pc before goto break 164g⟩
    gbranch(OGOTO); // goto for break
    spb = p;

    patch(spc, pc);

    gen(l->right->right); /* inc */

    patch(sp, pc); // entry

    if(l->left != Z) { /* test */
        bcomplex(l->left, Z);
        patch(p, breakpc);
        ⟨gen() when OFOR, increment nbreak 164i⟩
    }

    ⟨gen() when OFOR, set canreach before gen body 163c⟩

    gen(n->right);  /* body */
```

```
    if(canreach){
        gbranch(OGOTO);
        patch(p, continpc);
        ⟨gen() when OFOR, increment ncontin when canreach end of body 164j⟩
    }
    ⟨gen() when OFOR, check can reach for inc 163d⟩

    patch(spb, pc);

    ⟨gen() when OFOR, before exit, restore continpc/breakpc 165b⟩
    break;
```

⟨gen() *when OFOR, check canreach before gen anything* 163a⟩≡ (162)

```
if(!canreach && warnreach && l->right->left) {
    warn(n, "unreachable code FOR");
    warnreach = false;
}
```

⟨gen() *when OFOR, check canreach after gen init* 163b⟩≡ (162)

```
/*
 * if there are no incoming labels in the
 * body and the top's not reachable, warn
 */
if(!canreach && warnreach && deadheads(n)) {
    warn(n, "unreachable code %O", o);
    warnreach = false;
}
```

⟨gen() *when OFOR, set canreach before gen body* 163c⟩≡ (162)

```
canreach = true;
```

⟨gen() *when OFOR, check can reach for inc* 163d⟩≡ (162)

```
if(!ncontin && l->right->right && warnreach) {
    warn(l->right->right, "unreachable FOR inc");
    warnreach = false;
}
```

⟨gen() *when OFOR, set canreach after gen* 163e⟩≡ (165b)

```
canreach = nbreak!=0;
if(!canreach)
    warnreach = !suppress;
```

Continue/break

⟨gen() *switch node kind cases* 163f⟩+≡ (158i) ◁162 164a▷

```
case OCONTINUE:
    ⟨gen() when OCONTINUE, sanity check continpc 163g⟩
    gbranch(OGOTO);
    patch(p, continpc);
    ncontin++;
    ⟨gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 161a⟩
    break;
```

⟨gen() *when OCONTINUE, sanity check continpc* 163g⟩≡ (163f)

```
if(continpc < 0) {
    diag(n, "continue not in a loop");
    break;
}
```

⟨gen() *switch node kind cases* 164a⟩+≡ (158i) ◁163f 165g▷

```
case OBREAK:
    ⟨gen() when OBREAK, sanity check breakpc 164b⟩
    ⟨gen() when OBREAK, check canreach 165e⟩
    gbranch(OGOTO);
    patch(p, breakpc);
    nbreak++;
    ⟨gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 161a⟩
    break;
```

⟨gen() *when OBREAK, sanity check breakpc* 164b⟩≡ (164a)

```
if(breakpc < 0) {
    diag(n, "break not in a loop");
    break;
}
```

⟨gen() *locals* 164c⟩+≡ (158i) ◁161f 165f▷

```
long sbc, scc;
int snbreak, sncontin;
```

⟨gen() *when OWHILE/ODWHILE, set continpc to pc before goto continue* 164d⟩≡ (161g)

```
scc = continpc;
continpc = pc;
```

⟨gen() *when OWHILE/ODWHILE, set breakpc to pc before goto break* 164e⟩≡ (161g)

```
sbc = breakpc;
snbreak = nbreak;
breakpc = pc;
nbreak = 0;
```

⟨gen() *when OFOR, set continpc to pc before goto continue* 164f⟩≡ (162)

```
scc = continpc;
continpc = pc;
```

⟨gen() *when OFOR, set breakpc to pc before goto break* 164g⟩≡ (162)

```
sbc = breakpc;
snbreak = nbreak;
sncontin = ncontin;
breakpc = pc;
nbreak = 0;
ncontin = 0;
```

⟨gen() *when OWHILE/ODWHILE, increment nbreak* 164h⟩≡ (161g)

```
if(l->op != OCONST || vconst(l) == 0)
    nbreak++;
```

⟨gen() *when OFOR, increment nbreak* 164i⟩≡ (162)

```
if(l->left->op != OCONST || vconst(l->left) == 0)
    nbreak++;
```

⟨gen() *when OFOR, increment ncontin when canreach end of body* 164j⟩≡ (162)

```
ncontin++;
```

⟨gen() *when OWHILE/ODWHILE, before exit, restore continpc/breakpc* 164k⟩≡ (161g)

```
// restore
continpc = scc;
breakpc = sbc;
⟨gen() when OWHILE/ODWHILE, set canreach 165a⟩
nbreak = snbreak;
```

⟨gen() *when OWHILE/ODWHILE, set canreach* 165a⟩≡ (164k)

```
canreach = nbreak!=0;
if(!canreach)
     warnreach = !suppress;
```

⟨gen() *when OFOR, before exit, restore continpc/breakpc* 165b⟩≡ (162)

```
// restore
continpc = scc;
breakpc = sbc;
⟨gen() when OFOR, set canreach after gen 163e⟩
nbreak = snbreak;
ncontin = sncontin;
```

⟨*global* ncontin 165c⟩≡ (436d)

```
int ncontin;
```

⟨*global* nbreak 165d⟩≡ (460b)

```
long nbreak;
```

⟨gen() *when OBREAK, check canreach* 165e⟩≡ (164a)

```
/*
 * Don't complain about unreachable break statements.
 * There are breaks hidden in yacc's output and some people
 * write return; break; in their switch statements out of habit.
 * However, don't confuse the analysis by inserting an
 * unreachable reference to breakpc either.
 */
if(!canreach)
    break;
```

10.4.5 Control flow jumps

⟨gen() *locals* 165f⟩+≡ (158i) ◁164c 228a▷

```
Node nod;
```

⟨gen() *switch node kind cases* 165g⟩+≡ (158i) ◁164a 179d▷

```
case ORETURN:
    ⟨gen() when OGOTO/ORETURN/OBREAK/OCONTINUE, set canreach 161a⟩

    complex(n);
    ⟨gen() when ORETURN, after complex, if no type 166a⟩

    l = uncomma(n->left);
    if(l == Z) {
        ⟨gen() case ORETURN with no argument, before RET 282d⟩
        gbranch(ORETURN);
    }
    else
    ⟨gen() case ORETURN, if complex type 223b⟩
    else {
        regret(&nod, n);
        cgen(l, &nod);
        regfree(&nod);
        ⟨gen() case ORETURN with argument, before RET 282e⟩
        gbranch(ORETURN);
    }
    break;
```

⟨gen() *when ORETURN, after complex, if no type* 166a⟩≡ (165g)

```
if(n->type == T)
    break;
```

uncomma()

⟨*function* uncomma 166b⟩≡ (455b)

```
/// gen(case RETURN) -> <>
Node*
uncomma(Node *n)
{
    while(n != Z && n->op == OCOMMA) {
        cgen(n->left, Z);
        n = n->right;
    }
    return n;
}
```

regret()

⟨*function* regret*(arm)* 166c⟩≡ (462)

```
void
regret(Node *n/*OUT*/, Node *nn/*IN*/)
{
    int r;

    r = REGRET;
    ⟨regret() if float expression, adjust return register 383f⟩
    nodreg(n, nn, r);
    reg[r]++;
}
```

10.5 Register allocation

10.5.1 reg

⟨*global* reg*(arm)* 166d⟩≡ (460b)

```
// map<enum<registr>, int>
char reg[NREG+NFREG];
```

⟨ginit() *reg and resvreg initialisation* 166e⟩≡ (35d) 166g ▷

```
memset(reg, 0, sizeof(reg));

/* don't allocate */
reg[REGTMP] = 1;
reg[REGSB] = 1;
reg[REGSP] = 1;
reg[REGLINK] = 1;
reg[REGPC] = 1;
```

⟨*global* resvreg*(arm)* 166f⟩≡ (462)

```
static char resvreg[nelem(reg)];
```

⟨ginit() *reg and resvreg initialisation* 166g⟩+≡ (35d) ◁ 166e 365b ▷

```
memmove(resvreg, reg, sizeof(reg));
```

⟨gclean() *locals* 167a⟩≡ (34a) 221a ▷
```
int i;
```

⟨gclean() *sanity check reg* 167b⟩≡ (34a) 384b ▷
```
for(i=0; i<NREG; i++)
    if(reg[i] && !resvreg[i])
        diag(Z, "reg %d left allocated", i);
```

10.5.2 OREGISTER

⟨*after parsing nodes* 167c⟩≡ (16a) 186g ▷
```
OREGISTER,
```

⟨Node *value fields* 167d⟩+≡ (16b) ◁ 87b
```
// option<enum<registr>>
int  reg; // for OREGISTER
```

10.5.3 nodreg()

⟨*function* nodreg*(arm)* 167e⟩≡ (462)
```
/// regret | ... -> <>
void
nodreg(Node *n/*OUT*/, Node *nn/*IN*/, int reg)
{
    *n = regnode;
    n->reg = reg;
    n->type = nn->type;
    n->lineno = nn->lineno;
}
```

⟨*global* regnode 167f⟩≡ (460b)
```
Node regnode;
```

⟨ginit() *special nodes initialisation* 167g⟩+≡ (35d) ◁ 157b 195e ▷
```
regnode.op = OREGISTER;
regnode.class = CEXREG;
regnode.reg = REGTMP;
regnode.complex = 0;
regnode.addable = 11;
regnode.type = types[TLONG];
```

10.5.4 regfree()

⟨*function* regfree*(arm)* 167h⟩≡ (462)
```
void
regfree(Node *n)
{
    int i;

    ⟨regfree() sanity checks 168a⟩

    reg[n->reg]--;
    return;

err:
    diag(n, "error in regfree: %d", i);
}
```

⟨regfree() *sanity checks* 168a⟩≡ (167h)

```
i = 0;
if(n->op != OREGISTER && n->op != OINDREG)
    goto err;
i = n->reg;
if(i < 0 || i >= sizeof(reg))
    goto err;
if(reg[i] <= 0)
    goto err;
```

10.6 Typechecking, expliciting, annotating, and rewriting: complex()

⟨*function* complex 168b⟩≡ (454)

```
/// dodecl (array size) | doenum (evconst) | codgen | bcomplex -> <>
void
complex(Node *n)
{

    if(n == Z)
        return;

    nearln = n->lineno;

    ⟨complex() debug tree pre complex 416g⟩
    // typechecking, removing some sugar, lvalue annotate, bitfield annot, etc
    if(tcom(n))
        return;

    // comma hoisting
    comma(n);
    ⟨complex() debug tree after tcom and comma 416h⟩
    // general rewrite
    ccom(n);
    ⟨complex() debug tree after ccom 416i⟩
    // arithmetic rewrite
    acom(n);
    ⟨complex() debug tree after acom 416j⟩

    // addressability and complexity (and shl/shr optimizations)
    xcom(n);
    ⟨complex() debug tree after xcom 416k⟩
}
```

10.7 Addressability and complexity: xcom()

⟨*function* xcom*(arm)* 168c⟩≡ (464a)

```
/*
 * calculate addressability as follows
 *  CONST ==> 20  $value
 *  NAME ==> 10  name
 *  REGISTER ==> 11  register
 *  INDREG ==> 12  *[(reg)+offset]
 *
 *  &10 ==> 2  $name
 *  ADD(2, 20) ==> 2 $name+offset
 *  ADD(3, 20) ==> 3 $(reg)+offset
```

```
 *  &12 ==> 3  $(reg)+offset
 *
 *  *11 ==> 11  ??
 *  *2 ==> 10  name
 *  *3 ==> 12  *(reg)+offset
 * calculate complexity (number of registers) //$
 */
void
xcom(Node *n)
{
    Node *l, *r;
    int t;

    if(n == Z)
        return;

    l = n->left;
    r = n->right;
    n->addable = 0;
    n->complex = 0;

    switch(n->op) {
    ⟨xcom() switch op cases to set addable 169b⟩
    default:
        if(l != Z)
            xcom(l);
        if(r != Z)
            xcom(r);
        break;
    }
    if(n->addable >= 10)
        return;
    // else
    ⟨xcom() set complex if addable less than 10 170e⟩
}
```

10.7.1 Addressable: INDEXED

⟨*constant* INDEXED*(arm)* 169a⟩≡ (456)

```
#define INDEXED 9
```

10.7.2 Addressability: addable

⟨xcom() *switch op cases to set addable* 169b⟩≡ (168c) 169c ▷

```
case OCONST:
    n->addable = 20;
    return;
```

⟨xcom() *switch op cases to set addable* 169c⟩+≡ (168c) ◁ 169b 169d ▷

```
case ONAME:
    n->addable = 10;
    return;
```

⟨xcom() *switch op cases to set addable* 169d⟩+≡ (168c) ◁ 169c 170a ▷

```
case OREGISTER:
    n->addable = 11;
    return;
```

⟨xcom() *switch op cases to set addable* 170a⟩+≡ (168c) ◁169d 170b▷

```
case OINDREG:
    n->addable = 12;
    return;
```

⟨xcom() *switch op cases to set addable* 170b⟩+≡ (168c) ◁170a 170c▷

```
case OADDR:
    xcom(l);
    if(l->addable == 10)
        n->addable = 2;

    if(l->addable == 12)
        n->addable = 3;
    break;
```

⟨xcom() *switch op cases to set addable* 170c⟩+≡ (168c) ◁170b 170d▷

```
case OIND:
    xcom(l);
    if(l->addable == 2)
        n->addable = 10;

    if(l->addable == 3)
        n->addable = 12;
    if(l->addable == 11)
        n->addable = 12;
    break;
```

⟨xcom() *switch op cases to set addable* 170d⟩+≡ (168c) ◁170c 257a▷

```
case OADD:
    xcom(l);
    xcom(r);
    if(l->addable == 20) {
        if(r->addable == 2)
            n->addable = 2;

        if(r->addable == 3)
            n->addable = 3;
    }
    if(r->addable == 20) {
        if(l->addable == 2)
            n->addable = 2;

        if(l->addable == 3)
            n->addable = 3;
    }
    break;
```

10.7.3 Complexity: complex

⟨xcom() *set complex if addable less than 10* 170e⟩≡ (168c)

```
if(l != Z)
    n->complex = l->complex;
if(r != Z) {
    if(r->complex == n->complex)
        n->complex = r->complex+1;
    else
       if(r->complex > n->complex)
           n->complex = r->complex;
```

```
}

if(n->complex == 0)
    n->complex++;

⟨xcom() if 64 bits operation, transform and return 389c⟩
// else
switch(n->op) {
⟨xcom() switch node opkind cases to set complexity 171a⟩
}
```

10.7.4 FNX

⟨xcom() *switch node opkind cases to set complexity* 171a⟩≡ (170e) 259b ▷

```
case OFUNC:
    n->complex = FNX;
    break;
```

⟨*constant* FNX*(arm)* 171b⟩≡ (456)

```
#define FNX  100
```

⟨*constant* FNX 171c⟩≡ (449b)

```
/*
 * this is machine dependent, but it is totally
 * common on all of the 64-bit symulating machines.
 */

#define FNX 100 /* botch -- redefinition */
```

10.8 Memory moves

10.8.1 gmove()

⟨*function* gmove*(arm)* 171d⟩≡ (462)

```
/// cgenrel | ... -> <>
void
gmove(Node *f, Node *t)
{
    // enum<Type_kind> // Txxx
    int ft, tt;
    // enum<Opcode_kind> // Axxx
    int a;
    ⟨gmove() locals 172b⟩

    ft = f->type->etype;
    tt = t->type->etype;

    /*
     * a load --
     * put it into a register then
     * worry what to do with it.
     */
    ⟨gmove() if from is an indirect, registerize and return 172c⟩
    /*
     * a store --
     * put it into a register then
     * store it.
```

```
     */
    ⟨gmove() if to is an indirect, registerize and return 173c⟩

    // at this point f and t should be simpler nodes with
    // registers or constants

    /*
     * type x type cross table
     */
    a = AGOK;
    switch(ft) {
    ⟨gmove() switch from type cases 173f⟩
    }
    ⟨gmove() sanity check a 172a⟩
    ⟨gmove() if samaddr return 279a⟩
    // else

    gins(a, f, t);
}
```

⟨gmove() *sanity check a* 172a⟩≡ (171d)

```
if(a == AGOK)
    diag(Z, "bad opcode in gmove %T -> %T", f->type, t->type);
```

A memory load

⟨gmove() *locals* 172b⟩≡ (171d) 173e ▷

```
Node nod;
```

⟨gmove() *if from is an indirect, registerize and return* 172c⟩≡ (171d)

```
if(f->op == ONAME || f->op == OIND || f->op == OINDREG) {
    switch(ft) {
    ⟨gmove() when indirect from, set MOVxx for a depending on type 174a⟩
    default:
        a = AMOVW;
        break;
    }
    ⟨gmove() when indirect from, if chlp ft and ilp tt 172d⟩
    else
        regalloc(&nod, f, t);
    gins(a, f, &nod);
    gmove(&nod, t); // recurse!
    regfree(&nod);
    return;
}
```

⟨gmove() *when indirect from, if chlp ft and ilp tt* 172d⟩≡ (172c)

```
if(typechlp[ft] && typeilp[tt])
    regalloc(&nod, t, t);
```

⟨*global* typeilp 172e⟩≡ (443b)

```
char typeilp[NTYPE];
```

⟨*global* typeilpinit 172f⟩≡ (443b)

```
int typeilpinit[] =
{
    TINT, TUINT, TLONG, TULONG, TIND, -1
};
```

⟨*global* typechlp 173a⟩≡ (443b)
```
char typechlp[NTYPE];
```

⟨*global* typechlpinit 173b⟩≡ (443b)
```
int typechlpinit[] =
{
    TCHAR, TUCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG, TULONG, TIND, -1,
};
```

A memory store

⟨gmove() *if to is an indirect, registerize and return* 173c⟩≡ (171d)
```
if(t->op == ONAME || t->op == OIND || t->op == OINDREG) {
    switch(tt) {
    ⟨gmove() when indirect to, set MOVxx for a depending on type 174b⟩
    default:
        a = AMOVW;
        break;
    }
    ⟨gmove() when indirect to, if ft equal tt 173d⟩
    else
        regalloc(&nod, t, Z);
    gmove(f, &nod); // recurse!
    gins(a, &nod, t);
    regfree(&nod);
    return;
}
```

⟨gmove() *when indirect to, if ft equal tt* 173d⟩≡ (173c)
```
if(ft == tt)
    regalloc(&nod, t, f);
```

A set

⟨gmove() *locals* 173e⟩+≡ (171d) ◁172b
```
Node nod1;
Prog *p1;
```

⟨gmove() *switch from type cases* 173f⟩≡ (171d) 174c▷
```
case TUINT:
case TULONG:
    ⟨gmove() switch from type cases, TUINT/TULONG case, if float target 385⟩
    // fall through
case TINT:
case TLONG:
case TIND:
    switch(tt) {
    ⟨gmove() switch from type cases, TINT/TLONG case, if float target 386a⟩
    case TCHAR:
    case TUCHAR:
    case TSHORT:
    case TUSHORT:
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:

    case TIND:
        a = AMOVW;
```

```
        break;
    }
    break;
```

10.8.2 Conversions

⟨gmove() *when indirect from, set MOVxx for* a *depending on type* 174a⟩≡ (172c)

```
  case TCHAR:
      a = AMOVB;
      break;
  case TUCHAR:
      a = AMOVBU;
      break;
  case TSHORT:
      a = AMOVH;
      break;
  case TUSHORT:
      a = AMOVHU;
      break;
  case TFLOAT:
      a = AMOVF;
      break;
  case TDOUBLE:
      a = AMOVD;
      break;
```

⟨gmove() *when indirect to, set MOVxx for* a *depending on type* 174b⟩≡ (173c)

```
  case TCHAR:
      a = AMOVB;
      break;
  case TUCHAR:
      a = AMOVBU;
      break;
  case TSHORT:
      a = AMOVH;
      break;
  case TUSHORT:
      a = AMOVHU;
      break;
  case TFLOAT:
      a = AMOVF;
      break;
  case TVLONG: // addon?
  case TDOUBLE:
      a = AMOVD;
      break;
```

⟨gmove() *switch from type cases* 174c⟩+≡ (171d) ◁173f 175a▷

```
  case TSHORT:
      switch(tt) {
      ⟨gmove() switch from type cases, TSHORT case, if float target 386b⟩
      case TUINT:
      case TINT:
      case TULONG:
      case TLONG:
      case TIND:
          a = AMOVH;
          break;
      case TCHAR:
```

```
    case TUCHAR:
    case TSHORT:
    case TUSHORT:
        a = AMOVW;
        break;
    }
    break;
```

⟨gmove() *switch from type cases* 175a⟩+≡ (171d) ◁174c 175b▷

```
  case TUSHORT:
    switch(tt) {
    ⟨gmove() switch from type cases, TUSHORT case, if float target 386c⟩
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:
    case TIND:
        a = AMOVHU;
        break;
    case TCHAR:
    case TUCHAR:
    case TSHORT:
    case TUSHORT:
        a = AMOVW;
        break;
    }
    break;
```

⟨gmove() *switch from type cases* 175b⟩+≡ (171d) ◁175a 175c▷

```
  case TCHAR:
    switch(tt) {
    ⟨gmove() switch from type cases, TCHAR case, if float target 386d⟩
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:
    case TIND:
    case TSHORT:
    case TUSHORT:
        a = AMOVB;
        break;
    case TCHAR:
    case TUCHAR:
        a = AMOVW;
        break;
    }
    break;
```

⟨gmove() *switch from type cases* 175c⟩+≡ (171d) ◁175b 384d▷

```
  case TUCHAR:
    switch(tt) {
    ⟨gmove() switch from type cases, TUCHAR case, if float target 387a⟩
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:
    case TIND:
    case TSHORT:
    case TUSHORT:
        a = AMOVBU;
```

```
        break;
    case TCHAR:
    case TUCHAR:
        a = AMOVW;
        break;
    }
    break;
```

10.8.3 gins()

⟨*function* gins*(arm)* 176a⟩≡ (462)

```
/// gmove | ... -> <>
void
gins(int a, Node *f, Node *t)
{

    nextpc();
    p->as = a;

    if(f != Z)
        naddr(f, &p->from);
    if(t != Z)
        naddr(t, &p->to);

    ⟨gins() debug 417d⟩
}
```

10.8.4 naddr()

⟨*function* naddr*(arm)* 176b⟩≡ (462)

```
void
naddr(Node *n/*IN*/, Adr *a/*OUT*/)
{
    ⟨naddr() locals 177e⟩

    a->type = D_NONE;
    if(n == Z)
        return;

    switch(n->op) {
    ⟨naddr() switch node kind cases 176c⟩
    default:
    bad:
        diag(n, "bad in naddr: %O", n->op);
        break;

    }
}
```

⟨naddr() *switch node kind cases* 176c⟩≡ (176b) 177a ▷

```
case OCONST:
    a->sym = S;
    a->reg = R_NONE;
    ⟨naddr() if float type 383d⟩
    else {
        a->type = D_CONST;
        a->offset = n->vconst;
    }
    break;
```

⟨naddr() *switch node kind cases* 177a⟩+≡ (176b) ◁176c 177b▷

```
case OREGISTER:
    a->type = D_REG;
    a->sym = S;
    a->reg = n->reg;
    ⟨naddr() if float register, adjust a 383e⟩
    break;
```

⟨naddr() *switch node kind cases* 177b⟩+≡ (176b) ◁177a 177c▷

```
case ONAME:
    a->type = D_OREG;
    a->offset = n->xoffset;
    a->sym = n->sym;
    ⟨naddr() when ONAME, save etype 177g⟩

    a->symkind = N_INTERN;
    switch(n->class) {
    ⟨naddr() when ONAME, switch class cases 180e⟩
    default:
        goto bad;
    }
    break;
```

⟨naddr() *switch node kind cases* 177c⟩+≡ (176b) ◁177b 177d▷

```
case OIND:
    naddr(n->left, a);
    if(a->type == D_REG || a->type == D_CONST) {
        a->type = D_OREG;
        break;
    }
    goto bad;
```

⟨naddr() *switch node kind cases* 177d⟩+≡ (176b) ◁177c 177f▷

```
case OADDR:
    naddr(n->left, a);
    if(a->type == D_OREG) {
        a->type = D_CONST;
        break;
    }
    // else
    goto bad;
```

⟨naddr() *locals* 177e⟩≡ (176b)

```
long v;
```

⟨naddr() *switch node kind cases* 177f⟩+≡ (176b) ◁177d 191a▷

```
case OADD:
    if(n->left->op == OCONST) {
        naddr(n->left, a);
        v = a->offset;
        naddr(n->right, a);
    } else {
        naddr(n->right, a);
        v = a->offset;
        naddr(n->left, a);
    }
    a->offset += v;
    break;
```

⟨naddr() *when ONAME, save etype* 177g⟩≡ (177b)

```
a->etype = n->etype;
```

⟨Adr *type fields(arm)* 178a⟩≡ (29a)

```
// enum<Type_kind>
char etype;
```

10.8.5 regalloc()

⟨*function* regalloc*(arm)* 178b⟩≡ (462)

```
/// gmove |  -> <>
void
regalloc(Node *n/*OUT*/, Node *tn/*IN*/, Node *o/*option<IN>*/)
{
    int i, j;
    // opti to remember last one allocated
    static int lasti;

    switch(tn->type->etype) {
    case TCHAR:
    case TUCHAR:
    case TSHORT:
    case TUSHORT:
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:

    case TIND:
        ⟨regalloc() when integer type tn and if OREGISTER o 179c⟩

        j = lasti + REGRET+1;
        for(i=REGRET+1; i<NREG; i++) {
            if(j >= NREG)
                j = REGRET+1;
            if(reg[j] == 0 && resvreg[j] == 0) {
                i = j;
                goto out;
            }
            j++;
        }
        ⟨regalloc() if reach here, out of fixed registers 178c⟩

    ⟨regalloc() switch tn type, float or vlong case 387b⟩
    }
    ⟨regalloc() if reach here, unknown type 179a⟩
    ⟨regalloc() error management 179b⟩

out:
    reg[i]++;
    nodreg(n, tn, i);

    lasti++;
    if(lasti >= 5)
        lasti = 0;

}
```

⟨regalloc() *if reach here, out of fixed registers* 178c⟩≡ (178b)

```
diag(tn, "out of fixed registers");
goto err;
```

⟨regalloc() *if reach here, unknown type* 179a⟩≡ (178b)
```
diag(tn, "unknown type in regalloc: %T", tn->type);
// fallthrough
```

⟨regalloc() *error management* 179b⟩≡ (178b)
```
err:
    nodreg(n, tn, 0);
    return;
```

⟨regalloc() *when integer type tn and if OREGISTER o* 179c⟩≡ (178b)
```
if(o != Z && o->op == OREGISTER) {
    i = o->reg;
    if(i >= 0 && i < NREG)
        goto out;
}
```

10.9 Expressions: cgen()

⟨gen() *switch node kind cases* 179d⟩+≡ (158i) ◁165g 228b▷
```
default:
    complex(n);
    cgen(n, Z);
    break;
```

⟨*function* cgen*(arm)* 179e⟩≡ (467)
```
/// codgen -> gen -> <>
void
cgen(Node *n, Node *nn)
{
    cgenrel(n, nn, false);
}
```

⟨*function* cgenrel*(arm)* 179f⟩≡ (467)
```
/// codgen -> gen -> cgen -> <>
void
cgenrel(Node *n, Node *nn, bool inrel)
{
    // enum<node_kind>
    int o;
    Node *l, *r;
    // enum<Type_kind>
    int t;
    long curs; // to save cursafe
    ⟨cgenrel() locals 180b⟩

    ⟨cgenrel() debug 417b⟩
    if(n == Z || n->type == T)
        return;

    ⟨cgenrel() if complex type node 222e⟩
    else {

        l = n->left;
        r = n->right;
        o = n->op;

        ⟨cgenrel() if node addressable 180c⟩
        // else
```

```
        curs = cursafe; // save
        ⟨cgenrel() if all complex fields more than FNX 212c⟩

        switch(o) {
        ⟨cgenrel() switch node kind cases 180a⟩
        default:
            diag(n, "unknown op in cgen: %O", o);
            break;
        }
        cursafe = curs; // restore
    }
    return;
}
```

⟨cgenrel() *switch node kind cases* 180a⟩≡ (179f) 183c ▷

```
case OCOMMA:
    cgen(l, Z);
    cgen(r, nn);
    break;
```

⟨cgenrel() *locals* 180b⟩≡ (179f)

```
Prog *p1;
Node nod, nod1, nod2, nod3, nod4;
long v;
```

10.9.1 Numeric constants

⟨cgenrel() *if node addressable* 180c⟩≡ (179f)

```
if(n->addable >= INDEXED) {
    ⟨cgenrel() warn if target node is nil 153a⟩
    gmove(n, nn);
    return;
}
```

10.9.2 Entity uses

Parameters

⟨adecl() *switch class cases* 180d⟩≡ (111b) 181a ▷

```
case CPARAM:
    ⟨adecl() if first parameter 224j⟩
    autoffset = align(autoffset, t, Aarg1);
    if(s)
        s->offset = autoffset;
    autoffset = align(autoffset, t, Aarg2);
    break;
```

⟨naddr() *when ONAME, switch class cases* 180e⟩≡ (177b) 181b ▷

```
case CPARAM:
    a->symkind = N_PARAM;
    break;
```

Locals

⟨adecl() *switch class cases* 181a⟩+≡ (111b) ◁180d

```
case CAUTO:
    autoffset = align(autoffset, t, Aaut3);
    stkoff = maxround(stkoff, autoffset);
    s->offset = -autoffset;
    break;
```

⟨naddr() *when ONAME, switch class cases* 181b⟩+≡ (177b) ◁180e 181c▷

```
case CAUTO:
    a->symkind = N_LOCAL;
    break;
```

Globals

⟨naddr() *when ONAME, switch class cases* 181c⟩+≡ (177b) ◁181b 181d▷

```
case CEXTERN: case CGLOBL:
    a->symkind = N_EXTERN;
    break;
```

⟨naddr() *when ONAME, switch class cases* 181d⟩+≡ (177b) ◁181c

```
case CSTATIC:
    a->symkind = N_INTERN;
    break;
```

Enumeration constants

10.9.3 String constants

ASCII strings

⟨tcomo() *transform OSTRING in ONAME* 181e⟩≡ (123e)

```
n->op = ONAME;
n->xoffset = outstring(n->cstring, n->type->width);
n->addable = true;
```

⟨*string rule, set sym and class for OSTRING* 181f⟩≡ (87d 86f)

```
$$->sym = symstring;
$$->class = CSTATIC;
```

⟨*global* symstring 181g⟩≡ (436d)

```
Sym* symstring;
```

⟨cinit() *locals* 181h⟩+≡ (35a) ◁42b

```
Type *t;
```

⟨cinit() *symstring initialization* 181i⟩≡ (35a)

```
symstring = slookup(".string");
symstring->class = CSTATIC;
t = typ(TARRAY, types[TCHAR]);
t->width = 0;
symstring->type = t;
```

⟨*global* nstring 181j⟩≡ (460b)

```
long nstring;
```

⟨*global* mnstring 181k⟩≡ (460b)

```
int mnstring;
```

⟨ginit() *misc initialisations* 182a⟩+≡ (35d) ◁161e 195h▷

```
nstring = 0;
mnstring = 0;
```

⟨gclean() *adjust symstring width* 182b⟩≡ (34a)

```
while(mnstring)
    outstring("", 1L);
symstring->type->width = nstring;
```

⟨*global* string 182c⟩≡ (460b)

```
char string[NSNAME];
```

⟨*function* outstring*(arm)* 182d⟩≡ (464b)

```
long
outstring(char *s, long n)
{
    long r;

    ⟨outstring() return if suppress 182e⟩
    r = nstring;
    while(n) {
        string[mnstring] = *s++;
        mnstring++;
        nstring++;
        if(mnstring >= NSNAME) {
            gpseudo(ADATA, symstring, nodconst(0L));
            p->from.offset += nstring - NSNAME;
            p->reg = NSNAME;
            p->to.type = D_SCONST;
            memmove(p->to.sval, string, NSNAME);
            mnstring = 0;
        }
        n--;
    }
    return r;
}
```

⟨outstring() *return if suppress* 182e⟩≡ (182d)

```
if(suppress)
    return nstring;
```

⟨tcomo() *in OSTRING case, if not an array or chars* 182f⟩≡ (123e)

```
if(n->type->link != types[TCHAR]) {
    o = outstring(0, 0);
    while(o & 3) {
        outstring("", 1);
        o = outstring(0, 0);
    }
}
```

Unicode strings

⟨tcomo() *other locals* 182g⟩≡ (121c)

```
static TRune zer;
```

⟨tcomo() *switch node kind cases* 183a⟩+≡ (121c) ◁144a 363c▷

```
case OLSTRING:
    if(n->type->link != types[TRUNE]) {
        o = outstring(0, 0);
        while(o & 3) {
            outlstring(&zer, sizeof(TRune));
            o = outlstring(0, 0);
        }
    }
    n->op = ONAME;
    n->xoffset = outlstring(n->rstring, n->type->width);
    n->addable = 1;
    break;
```

⟨*function* outlstring 183b⟩≡ (455a)

```
long
outlstring(TRune *s, long n)
{
    char buf[sizeof(TRune)];
    uint c;
    int i;
    long r;

    if(suppress)
        return nstring;
    while(nstring & (sizeof(TRune)-1))
        outstring("", 1);
    r = nstring;
    while(n > 0) {
        c = *s++;
        if(align(0, types[TCHAR], Aarg1)) {
            for(i = 0; i < sizeof(TRune); i++)
                buf[i] = c>>(8*(sizeof(TRune) - i - 1));
        } else {
            for(i = 0; i < sizeof(TRune); i++)
                buf[i] = c>>(8*i);
        }
        outstring(buf, sizeof(TRune));
        n -= sizeof(TRune);
    }
    return r;
}
```

10.9.4 Arithmetic expressions

⟨cgenrel() *switch node kind cases* 183c⟩+≡ (179f) ◁180a 184a▷

```
case OSUB:
    ⟨cgenrel() when OSUB, if l is OCONST 258d⟩
    // else, Fallthrough
case OADD:
case OAND:
case OOR:
case OXOR:
case OASHL:
case OASHR:
case OLSHR:
    ⟨cgenrel() when OADD/OAND/etc, if r is OCONST 259a⟩
    // Fallthrough
case OMUL:
```

```
case OLMUL:
case OLDIV:
case OLMOD:
muldiv:
    ⟨cgenrel() nullwarn check if nn is null 152c⟩
    ⟨cgenrel() when OMUL/OLMUL, mulcon opportunity 259c⟩
    // else, the general case!
    ⟨cgenrel() when OADD/OAND/..., if l more complex than r 184b⟩
    else {
        regalloc(&nod, r, nn);
        cgen(r, &nod);
        regalloc(&nod1, l, Z);
        cgen(l, &nod1);
        gopcode(o, &nod, &nod1, &nod);
    }
    gopcode(OAS, &nod, Z, nn);
    regfree(&nod);
    regfree(&nod1);
    break;
```

⟨cgenrel() *switch node kind cases* 184a⟩+≡ (179f) ◁183c 188e▷

```
case ODIV:
case OMOD:
    ⟨cgenrel() when ODIV/OMOD, convert to SHR if possible 258a⟩
    // else
    goto muldiv;
```

Complexity and register allocation

⟨cgenrel() *when OADD/OAND/..., if l more complex than r* 184b⟩≡ (183c)

```
if(l->complex >= r->complex) {
    regalloc(&nod, l, nn);
    cgen(l, &nod);
    regalloc(&nod1, r, Z);
    cgen(r, &nod1);
    gopcode(o, &nod1, Z, &nod);
}
```

gopcode()

⟨*function* gopcode*(arm)* 184c⟩≡ (462)

```
/// cgenrel -> <>
void
gopcode(int o, Node *f1, Node *f2, Node *t)
{
    // enum<opcode_kind> AXXX
    int a;
    // enum<type_kind> TXXX
    int et;
    Adr ta;
    ⟨gopcode() locals 205b⟩

    et = (f1 != Z && f1->type != T)? f1->type->etype : TLONG;
    ⟨gopcode() initialisation 205c⟩

    a = AGOK;
    switch(o) {
    ⟨gopcode() switch opcode cases 185b⟩
```

```
    }
    ⟨gopcode() sanity check a 185a⟩

    nextpc();
    p->as = a;

    if(f1 != Z)
        naddr(f1, &p->from);
    if(f2 != Z) {
        naddr(f2, &ta);
        p->reg = ta.reg;
    }
    if(t != Z)
        naddr(t, &p->to);

    ⟨gopcode() debug 417e⟩
}
```

⟨gopcode() *sanity check a* 185a⟩≡ (184c)

```
if(a == AGOK)
    diag(Z, "bad in gopcode %O", o);
```

⟨gopcode() *switch opcode cases* 185b⟩≡ (184c) 185c ▷

```
case OADD:
case OASADD:
    a = AADD;
    ⟨gopcode() when OADD, adjust a if float type 382f⟩
    break;
```

⟨gopcode() *switch opcode cases* 185c⟩+≡ (184c) ◁ 185b 185d ▷

```
case OSUB:
case OASSUB:
    a = ASUB;
    ⟨gopcode() when OSUB, ARSB opportunity 279c⟩
    ⟨gopcode() when OSUB, adjust a if float type 383a⟩
    break;
```

⟨gopcode() *switch opcode cases* 185d⟩+≡ (184c) ◁ 185c 185e ▷

```
case OOR:
case OASOR:
    a = AORR;
    break;
```

⟨gopcode() *switch opcode cases* 185e⟩+≡ (184c) ◁ 185d 185f ▷

```
case OAND:
case OASAND:
    a = AAND;
    break;
```

⟨gopcode() *switch opcode cases* 185f⟩+≡ (184c) ◁ 185e 185g ▷

```
case OXOR:
case OASXOR:
    a = AEOR;
    break;
```

⟨gopcode() *switch opcode cases* 185g⟩+≡ (184c) ◁ 185f 186a ▷

```
case OASHL:
case OASASHL:
    a = ASLL;
    break;
```

⟨gopcode() *switch opcode cases* 186a⟩+≡ (184c) ◁185g 186b▷

```
case OASHR:
case OASASHR:
    a = ASRA;
    break;
```

⟨gopcode() *switch opcode cases* 186b⟩+≡ (184c) ◁186a 186c▷

```
case OMUL:
case OASMUL:
    a = AMUL;
    ⟨gopcode() when OMUL, adjust a if float type 383b⟩
    break;
```

⟨gopcode() *switch opcode cases* 186c⟩+≡ (184c) ◁186b 186d▷

```
case ODIV:
case OASDIV:
    a = ADIV;
    ⟨gopcode() when ODIV, adjust a if float type 383c⟩
    break;
```

⟨gopcode() *switch opcode cases* 186d⟩+≡ (184c) ◁186c 188a▷

```
case OMOD:
case OASMOD:
    a = AMOD;
    break;
```

Unsigned operations

⟨*global* typeu 186e⟩≡ (443b)

```
char typeu[NTYPE];
```

⟨*global* typeuinit 186f⟩≡ (443b)

```
int typeuinit[] =
{
    TUCHAR, TUSHORT, TUINT, TULONG, TUVLONG, TIND, -1,
};
```

⟨*after parsing nodes* 186g⟩+≡ (16a) ◁167c 186i▷

```
OLSHR,
```

⟨tcomo() *in OASHL/OASHR case, adjust opcode if unsigned type* 186h⟩≡ (127c)

```
if(typeu[n->type->etype])
    if(n->op == OASHR)
        n->op = OLSHR;
```

⟨*after parsing nodes* 186i⟩+≡ (16a) ◁186g 186k▷

```
OLMUL,
OLDIV,
```

⟨tcomo() *in OMUL/ODIV case, adjust opcode if unsigned type* 186j⟩≡ (125a)

```
if(typeu[n->type->etype]) {
    if(n->op == OMUL)
        n->op = OLMUL;
    if(n->op == ODIV)
        n->op = OLDIV;
}
```

⟨*after parsing nodes* 186k⟩+≡ (16a) ◁186i 187e▷

```
OLMOD,
```

⟨tcomo() *in OMOD case, adjust opcode if unsigned type* 187a⟩≡ (125d)

```
if(typeu[n->type->etype])
    n->op = OLMOD;
```

⟨tcomo() *when OLT/OGE/OGT/OLE, adjust opcode if unsigned type* 187b⟩≡ (131d)

```
if(typeu[n->type->etype])
    n->op = logrel[relindex(n->op)];
```

⟨*global* logrel 187c⟩≡ (443b)

```
char logrel[12] =
{
    OEQ, ONE, OLS, OLS, OLO, OLO, OHS, OHS, OHI, OHI,
};
```

⟨*function* relindex 187d⟩≡ (443b)

```
int
relindex(int o)
{

    switch(o) {
    case OEQ: return 0;
    case ONE: return 1;
    case OLE: return 2;
    case OLS: return 3;
    case OLT: return 4;
    case OLO: return 5;
    case OGE: return 6;
    case OHS: return 7;
    case OGT: return 8;
    case OHI: return 9;
    default:
        diag(Z, "bad in relindex: %O", o);
    }
    return -1; // unreachable
}
```

⟨*after parsing nodes* 187e⟩+≡ (16a) ◁186k 190f▷

```
OASLSHR,
OASLMUL,
OASLDIV,
OASLMOD,
```

⟨tcomo() *when OASLSHR, adjust opcode if unsigned type* 187f⟩≡ (135d)

```
if(typeu[n->type->etype]) {
    if(n->op == OASASHR)
        n->op = OASLSHR;
}
```

⟨tcomo() *when OASMUL/OASDIV, adjust opcode if unsigned type* 187g⟩≡ (135c)

```
if(typeu[n->type->etype]) {
    if(n->op == OASDIV)
        n->op = OASLDIV;
    if(n->op == OASMUL)
        n->op = OASLMUL;
}
```

⟨tcomo() *when OASMOD, adjust opcode if unsigned type* 187h⟩≡ (136a)

```
if(typeu[n->type->etype]) {
    if(n->op == OASMOD)
        n->op = OASLMOD;
}
```

⟨gopcode() *switch opcode cases* 188a⟩+≡ (184c) ◁186d 188b▷

```
case OLSHR:
case OASLSHR:
    a = ASRL;
    break;
```

⟨gopcode() *switch opcode cases* 188b⟩+≡ (184c) ◁188a 188c▷

```
case OLMUL:
case OASLMUL:
    a = AMULU;
    break;
```

⟨gopcode() *switch opcode cases* 188c⟩+≡ (184c) ◁188b 188d▷

```
case OLMOD:
case OASLMOD:
    a = AMODU;
    break;
```

⟨gopcode() *switch opcode cases* 188d⟩+≡ (184c) ◁188c 193b▷

```
case OLDIV:
case OASLDIV:
    a = ADIVU;
    break;
```

10.9.5 Boolean expressions

10.9.6 Assignments part 1

⟨cgenrel() *switch node kind cases* 188e⟩+≡ (179f) ◁184a 189a▷

```
case OAS:
    ⟨cgenrel() when OAS, if bitfield assignment 354d⟩
    if(l->addable >= INDEXED && l->complex < FNX) {
        ⟨cgenrel() when OAS, if right node not addressable or nn 188f⟩
        else
            // both left and right are addressable and nn == Z
            gmove(r, l);
        break;
    }
    // else
    ⟨cgenrel() when OAS, when left node not addressable or too complex 191c⟩
```

⟨cgenrel() *when OAS, if right node not addressable or nn* 188f⟩≡ (188e)

```
if(nn != Z || r->addable < INDEXED) {
    ⟨cgenrel() when OAS, if right node contain a call 188g⟩
    else
        regalloc(&nod, r, nn);
    cgen(r, &nod);
    gmove(&nod, l);
    if(nn != Z)
        gmove(&nod, nn);
    regfree(&nod);
}
```

⟨cgenrel() *when OAS, if right node contain a call* 188g⟩≡ (188f)

```
if(r->complex >= FNX && nn == Z)
    regret(&nod, r);
```

10.9.7 Pointers

Getting the address, lcgen()

⟨cgenrel() *switch node kind cases* 189a⟩+≡ (179f) ◁188e 190c▷

```
case OADDR:
    ⟨cgenrel() nullwarn check if nn is null 152c⟩
    lcgen(l, nn);
    break;
```

⟨*function* lcgen*(arm)* 189b⟩≡ (467)

```
void
lcgen(Node *n, Node *nn)
{
    Node nod;
    ⟨lcgen() other locals 189g⟩

    ⟨lcgen() debug 417f⟩
    ⟨lcgen() sanity check n 190b⟩
    ⟨lcgen() if nn nil 189d⟩
    switch(n->op) {
    ⟨lcgen() switch node cases 189e⟩
    default:
        ⟨lcgen() sanity check n in default case 189c⟩
        nod = *n;
        nod.op = OADDR;
        nod.left = n;
        nod.right = Z;
        nod.type = types[TIND];
        gopcode(OAS, &nod, Z, nn);
        break;

    }
}
```

⟨lcgen() *sanity check n in default case* 189c⟩≡ (189b)

```
if(n->addable < INDEXED) {
    diag(n, "unknown op in lcgen: %O", n->op);
    break;
}
```

⟨lcgen() *if nn nil* 189d⟩≡ (189b)

```
if(nn == Z) {
    nn = &nod;
    regalloc(&nod, n, Z);
}
```

⟨lcgen() *switch node cases* 189e⟩≡ (189b) 189f▷

```
case OIND:
    cgen(n->left, nn);
    break;
```

⟨lcgen() *switch node cases* 189f⟩+≡ (189b) ◁189e 190a▷

```
case OCOMMA:
    cgen(n->left, n->left);
    lcgen(n->right, nn);
    break;
```

⟨lcgen() *other locals* 189g⟩≡ (189b)

```
Prog *p1;
```

⟨lcgen() *switch node cases* 190a⟩+≡ (189b) ◁189f

```
case OCOND:
    bcgen(n->left, 1);
    p1 = p;
    lcgen(n->right->left, nn);
    gbranch(OGOTO);
    patch(p1, pc);
    p1 = p;
    lcgen(n->right->right, nn);
    patch(p1, pc);
    break;
```

⟨lcgen() *sanity check n* 190b⟩≡ (189b)

```
if(n == Z || n->type == T)
    return;
```

Dereferencing the address

⟨cgenrel() *switch node kind cases* 190c⟩+≡ (179f) ◁189a 193a▷

```
case OIND:
    ⟨cgenrel() nullwarn check if nn is null 152c⟩
    regialloc(&nod, n, nn);
    ⟨cgenrel() when OIND, if indirect of OADD constants 191b⟩
    else
        cgen(l, &nod);
    regind(&nod, n);
    gopcode(OAS, &nod, Z, nn);
    regfree(&nod);
    break;
```

⟨*function* regialloc 190d⟩≡ (462)

```
void
regialloc(Node *n, Node *tn, Node *o)
{
    Node nod;

    nod = *tn;
    nod.type = types[TIND];
    regalloc(n, &nod, o);
}
```

⟨*function* regind 190e⟩≡ (462)

```
void
regind(Node *n, Node *nn)
{

    if(n->op != OREGISTER) {
        diag(n, "regind not OREGISTER");
        return;
    }
    n->op = OINDREG;
    n->type = nn->type;
}
```

⟨*after parsing nodes* 190f⟩+≡ (16a) ◁187e 247c▷

```
OINDREG,
```

⟨naddr() *switch node kind cases* 191a⟩+≡ (176b) ◁177f

```
case OINDREG:
    a->type = D_OREG;
    a->sym = S;
    a->offset = n->xoffset;
    a->reg = n->reg;
    break;
```

⟨cgenrel() *when OIND, if indirect of OADD constants* 191b⟩≡ (190c)

```
r = l;
while(r->op == OADD)
    r = r->right;
if(sconst(r) && (v = r->vconst + nod.xoffset) > -4096 && v < 4096) {
    v = r->vconst;
    r->vconst = 0;
    cgen(l, &nod);
    nod.xoffset += v;
    r->vconst = v;
}
```

10.9.8 Assignments part 2

⟨cgenrel() *when OAS, when left node not addressable or too complex* 191c⟩≡ (188e)

```
⟨cgenrel() when OAS, when l not addressable, if l more complex than r 191d⟩
else {
    regalloc(&nod, r, nn);
    cgen(r, &nod);
    reglcgen(&nod1, l, Z);
}
gmove(&nod, &nod1);
regfree(&nod);
regfree(&nod1);
break;
```

⟨cgenrel() *when OAS, when l not addressable, if l more complex than r* 191d⟩≡ (191c)

```
if(l->complex >= r->complex) {
    reglcgen(&nod1, l, Z);
    if(r->addable >= INDEXED) {
        gmove(r, &nod1);
        if(nn != Z)
            gmove(r, nn);
        regfree(&nod1);
        break;
    }
    // else
    regalloc(&nod, r, nn);
    cgen(r, &nod);
}
```

reglcgen()

⟨*function* reglcgen*(arm)* 191e⟩≡ (467)

```
void
reglcgen(Node *t, Node *n, Node *nn)
{
    Node *r;
    long v;
```

```
    regialloc(t, n, nn);
    ⟨reglcgen() if OIND of OADD of constants 192a⟩
    ⟨reglcgen() if OINDREG with small offsets 192c⟩
    // else
    lcgen(n, t);
    regind(t, n);
}
```

⟨reglcgen() *if OIND of OADD of constants* 192a⟩≡ (191e)

```
if(n->op == OIND) {
    r = n->left;
    while(r->op == OADD)
        r = r->right;
    if(sconst(r) && (v = r->vconst+t->xoffset) > -4096 && v < 4096) {
        v = r->vconst;
        r->vconst = 0;
        lcgen(n, t);
        t->xoffset += v;
        r->vconst = v;
        regind(t, n);
        return;
    }
}
```

⟨*function* sconst*(arm)* 192b⟩≡ (462)

```
bool
sconst(Node *n)
{
    vlong vv;

    if(n->op == OCONST) {
        if(!typefd[n->type->etype]) {
            vv = n->vconst;
            if(vv >= (vlong)(-32766) && vv < (vlong)32766)
                return true;
            /*
             * should be specialised for constant values which will
             * fit in different instructionsl; for now, let 5l
             * sort it out
             */
            return true;
        }
    }
    return false;
}
```

⟨reglcgen() *if OINDREG with small offsets* 192c⟩≡ (191e)

```
else if(n->op == OINDREG) {
    if((v = n->xoffset) > -4096 && v < 4096) {
        n->op = OREGISTER;
        cgen(n, t);
        t->xoffset += v;
        n->op = OINDREG;
        regind(t, n);
        return;
    }
}
```

10.9.9 Assignments and arithmetic

⟨cgenrel() *switch node kind cases* 193a⟩+≡ (179f) ◁190c 195a▷

```
case OASADD:
case OASSUB:
case OASAND:
case OASXOR:
case OASOR:
case OASASHL:
case OASASHR:
case OASLSHR:
    ⟨cgenrel() when OASADD/OASSUB/etc, if bitfield assignment 354f⟩
    ⟨cgenrel() when OASADD/OASSUB/etc, if r is OCONST 193d⟩
    // Fallthrough
case OASMUL:
case OASDIV:
case OASMOD:
case OASLMUL:
case OASLDIV:
case OASLMOD:
    ⟨cgenrel() when OASMUL/OASDIV/etc, if bitfield assignment 354e⟩
    ⟨cgenrel() when OASADD/OASUB/etc, if l more complex than r 193c⟩
     else {
        regalloc(&nod1, r, Z);
        cgen(r, &nod1);
        if(l->addable < INDEXED)
            reglcgen(&nod2, l, Z);
        else
            nod2 = *l;
    }

    regalloc(&nod, n, nn);
    gmove(&nod2, &nod);
    gopcode(o, &nod1, Z, &nod);
    gmove(&nod, &nod2);
    if(nn != Z)
        gopcode(OAS, &nod, Z, nn);
    regfree(&nod);
    regfree(&nod1);
    if(l->addable < INDEXED)
        regfree(&nod2);
    break;
```

⟨gopcode() *switch opcode cases* 193b⟩+≡ (184c) ◁188d 208c▷

```
case OAS:
    gmove(f1, t);
    return;
```

⟨cgenrel() *when OASADD/OASUB/etc, if l more complex than r* 193c⟩≡ (193a)

```
if(l->complex >= r->complex) {
    if(l->addable < INDEXED)
        reglcgen(&nod2, l, Z);
    else
        nod2 = *l;
    regalloc(&nod1, r, Z);
    cgen(r, &nod1);
}
```

⟨cgenrel() *when OASADD/OASSUB/etc, if r is OCONST* 193d⟩≡ (193a)

```
if(r->op == OCONST)
```

```
 if(!typefd[r->type->etype])
  if(!typefd[n->type->etype]) {
    if(l->addable < INDEXED)
        reglcgen(&nod2, l, Z);
    else
        nod2 = *l;
    regalloc(&nod, r, nn);
    gopcode(OAS, &nod2, Z, &nod);
    gopcode(o, r, Z, &nod);
    gopcode(OAS, &nod, Z, &nod2);

    regfree(&nod);
    if(l->addable < INDEXED)
        regfree(&nod2);
    break;
}
```

10.9.10 Array accesses

10.9.11 Field accesses

⟨tcomd() *convert field access in offset* 194a⟩≡ (137c)

```
makedot(n, t, o);
```

⟨*function* makedot 194b⟩≡ (443b)

```
/// tcomd -> <>
void
makedot(Node *n, Type *t, long o)
{
    Node *n1, *n2;

    ⟨makedot() if bitfields 354b⟩
    n->addable = n->left->addable;
    if(!n->addable) {
        n1 = new1(OCONST, Z, Z);
        n1->vconst = o;
        n1->type = types[TLONG];
        n->right = n1;
        n->type = t;
        return;
    }
    // else
    n->left->type = t;
    if(o == 0) {
        *n = *n->left;
        return;
    }
    n->type = t;
    n1 = new1(OCONST, Z, Z);
    n1->vconst = o;
    t = typ(TIND, t);
    t->width = types[TIND]->width;
    n1->type = t;

    n2 = new1(OADDR, n->left, Z);
    n2->type = t;

    n1 = new1(OADD, n1, n2);
    n1->type = t;
```

```
    n->op = OIND;
    n->left = n1;
    n->right = Z;
}
```

⟨cgenrel() *switch node kind cases* 195a⟩+≡ (179f) ◁193a 201a▷

```
case ODOT:
    sugen(l, nodrat, l->type->width);
    if(nn != Z) {
        warn(n, "non-interruptable temporary");
        nod = *nodrat;
        if(!r || r->op != OCONST) {
            diag(n, "DOT and no offset");
            break;
        }
        nod.xoffset += (long)r->vconst;
        nod.type = n->type;
        cgen(&nod, nn);
    }
    break;
```

⟨*global* nodrat 195b⟩≡ (460b)

```
Node* nodrat;
```

⟨*global* symrathole 195c⟩≡ (460b)

```
Sym* symrathole;
```

⟨ginit() *locals* 195d⟩≡ (35d)

```
Type *t;
```

⟨ginit() *special nodes initialisation* 195e⟩+≡ (35d) ◁167g 211c▷

```
t = typ(TARRAY, types[TCHAR]);
symrathole = slookup(".rathole");
symrathole->class = CGLOBL;
symrathole->type = t;

nodrat = new(ONAME, Z, Z);
nodrat->sym = symrathole;
nodrat->type = types[TIND];
nodrat->etype = TVOID; // not TIND?
nodrat->class = CGLOBL;
complex(nodrat);
nodrat->type = t;
```

⟨gclean() *adjust symrathole width* 195f⟩≡ (34a)

```
symrathole->type->width = nrathole;
```

⟨*global* nrathole 195g⟩≡ (460b)

```
long nrathole;
```

⟨ginit() *misc initialisations* 195h⟩+≡ (35d) ◁182a 230d▷

```
nrathole = 0;
```

⟨*function* sugen*(arm)* 196⟩≡ (467)

```
void
sugen(Node *n, Node *nn, long w)
{
    Prog *p1;
    Node nod0, nod1, nod2, nod3, nod4, *l, *r;
    Type *t;
    long pc1;
    int i, m, c;

    if(n == Z || n->type == T)
        return;

    ⟨sugen() debug 417g⟩
    if(nn == nodrat)
        if(w > nrathole)
            nrathole = w;

    switch(n->op) {
    case OIND:
        if(nn == Z) {
            nullwarn(n->left, Z);
            break;
        }
    // Fallthrough

    default:
        goto copy;

    case OCONST:
        if(n->type && typev[n->type->etype]) {
            if(nn == Z) {
                nullwarn(n->left, Z);
                break;
            }

            t = nn->type;
            nn->type = types[TLONG];
            reglcgen(&nod1, nn, Z);
            nn->type = t;

            if(align(0, types[TCHAR], Aarg1)) /* isbigendian */
                gopcode(OAS, nod32const(n->vconst>>32), Z, &nod1);
            else
                gopcode(OAS, nod32const(n->vconst), Z, &nod1);
            nod1.xoffset += SZ_LONG;
            if(align(0, types[TCHAR], Aarg1)) /* isbigendian */
                gopcode(OAS, nod32const(n->vconst), Z, &nod1);
            else
                gopcode(OAS, nod32const(n->vconst>>32), Z, &nod1);

            regfree(&nod1);
            break;
        }
        goto copy;

    case ODOT:
        l = n->left;
        sugen(l, nodrat, l->type->width);
        if(nn != Z) {
```

```
        warn(n, "non-interruptable temporary");
        nod1 = *nodrat;
        r = n->right;
        if(!r || r->op != OCONST) {
            diag(n, "DOT and no offset");
            break;
        }
        nod1.xoffset += (long)r->vconst;
        nod1.type = n->type;
        sugen(&nod1, nn, w);
    }
    break;
```

⟨sugen() *switch op cases* 364⟩

```
case OAS:
    if(nn == Z) {
        if(n->addable < INDEXED)
            sugen(n->right, n->left, w);
        break;
    }
    sugen(n->right, nodrat, w);
    warn(n, "non-interruptable temporary");
    sugen(nodrat, n->left, w);
    sugen(nodrat, nn, w);
    break;

case OFUNC:
    if(nn == Z) {
        sugen(n, nodrat, w);
        break;
    }
    if(nn->op != OIND) {
        nn = new1(OADDR, nn, Z);
        nn->type = types[TIND];
        nn->addable = 0;
    } else
        nn = nn->left;
    n = new(OFUNC, n->left, new(OLIST, nn, n->right));
    n->type = types[TVOID];
    n->left->type = types[TVOID];
    cgen(n, Z);
    break;

//TODO: lpsplit and move with other OCOND stuff later
case OCOND:
    bcgen(n->left, 1);
    p1 = p;
    sugen(n->right->left, nn, w);
    gbranch(OGOTO);
    patch(p1, pc);
    p1 = p;
    sugen(n->right->right, nn, w);
    patch(p1, pc);
    break;

case OCOMMA:
    cgen(n->left, Z);
    sugen(n->right, nn, w);
    break;
}
```

```
    return;

copy:
    if(nn == Z)
        return;
    if(n->complex >= FNX && nn->complex >= FNX) {
        t = nn->type;
        nn->type = types[TLONG];
        regialloc(&nod1, nn, Z);
        lcgen(nn, &nod1);
        regsalloc(&nod2, nn);
        nn->type = t;

        gopcode(OAS, &nod1, Z, &nod2);
        regfree(&nod1);

        nod2.type = typ(TIND, t);

        nod1 = nod2;
        nod1.op = OIND;
        nod1.left = &nod2;
        nod1.right = Z;
        nod1.complex = 1;
        nod1.type = t;

        sugen(n, &nod1, w);
        return;
    }

    w /= SZ_LONG;
    if(w <= 2) {
        if(n->complex > nn->complex) {
            reglpcgen(&nod1, n, 1);
            reglpcgen(&nod2, nn, 1);
        } else {
            reglpcgen(&nod2, nn, 1);
            reglpcgen(&nod1, n, 1);
        }
        regalloc(&nod3, &regnode, Z);
        regalloc(&nod4, &regnode, Z);
        if(nod3.reg > nod4.reg){
            /* code below assumes nod3 loaded first */
            Node t = nod3; nod3 = nod4; nod4 = t;
        }
        nod0 = *nodconst((1<<nod3.reg)|(1<<nod4.reg));
        if(w == 2 && nod1.xoffset == 0)
            gmovm(&nod1, &nod0, 0);
        else {
            gmove(&nod1, &nod3);
            if(w == 2) {
                nod1.xoffset += SZ_LONG;
                gmove(&nod1, &nod4);
            }
        }
        if(w == 2 && nod2.xoffset == 0)
            gmovm(&nod0, &nod2, 0);
        else {
            gmove(&nod3, &nod2);
            if(w == 2) {
                nod2.xoffset += SZ_LONG;
```

```
                gmove(&nod4, &nod2);
            }
        }
        regfree(&nod1);
        regfree(&nod2);
        regfree(&nod3);
        regfree(&nod4);
        return;
    }

    if(n->complex > nn->complex) {
        reglpcgen(&nod1, n, 0);
        reglpcgen(&nod2, nn, 0);
    } else {
        reglpcgen(&nod2, nn, 0);
        reglpcgen(&nod1, n, 0);
    }

    m = 0;
    for(c = 0; c < w && c < 4; c++) {
        i = tmpreg();
        if (i == 0)
            break;
        reg[i]++;
        m |= 1<<i;
    }
    nod4 = *(nodconst(m));
    if(w < 3*c) {
        for (; w>c; w-=c) {
            gmovm(&nod1, &nod4, 1);
            gmovm(&nod4, &nod2, 1);
        }
        goto out;
    }

    regalloc(&nod3, &regnode, Z);
    gopcode(OAS, nodconst(w/c), Z, &nod3);
    w %= c;

    pc1 = pc;
    gmovm(&nod1, &nod4, 1);
    gmovm(&nod4, &nod2, 1);

    gopcode(OSUB, nodconst(1), Z, &nod3);
    gopcode(OEQ, nodconst(0), &nod3, Z);
    p->as = ABGT;
    patch(p, pc1);
    regfree(&nod3);

out:
    if (w) {
        i = 0;
        while (c>w) {
            while ((m&(1<<i)) == 0)
                i++;
            m &= ~(1<<i);
            reg[i] = 0;
            c--;
            i++;
        }
```

```
        nod4.vconst = m;
        gmovm(&nod1, &nod4, 0);
        gmovm(&nod4, &nod2, 0);
    }
    i = 0;
    do {
        while ((m&(1<<i)) == 0)
            i++;
        reg[i] = 0;
        c--;
        i++;
    } while (c>0);
    regfree(&nod1);
    regfree(&nod2);
}
```

⟨*function* gmovm*(arm)* 200a⟩≡ (462)

```
void
gmovm(Node *f, Node *t, int w)
{
    gins(AMOVM, f, t);
    p->scond |= C_UBIT;
    if(w)
        p->scond |= C_WBIT;
}
```

⟨*function* tmpreg*(arm)* 200b⟩≡ (462)

```
int
tmpreg(void)
{
    int i;

    for(i=REGRET+1; i<NREG; i++)
        if(reg[i] == 0)
            return i;
    diag(Z, "out of fixed registers");
    return 0;
}
```

⟨*function* reglpcgen*(arm)* 200c⟩≡ (467)

```
void
reglpcgen(Node *n, Node *nn, int f)
{
    Type *t;

    t = nn->type;
    nn->type = types[TLONG];
    if(f)
        reglcgen(n, nn, Z);
    else {
        regialloc(n, nn, Z);
        lcgen(nn, n);
        regind(n, nn);
    }
    nn->type = t;
}
```

10.9.12 Cast

⟨cgenrel() *switch node kind cases* 201a⟩+≡ (179f) ◁195a 202c▷

```
case OCAST:
    ⟨cgenrel() nullwarn check if nn is null 152c⟩
    /*
     * convert from types l->n->nn
     */
    ⟨cgenrel() if same types 201c⟩
    // else
    regalloc(&nod, l, nn);
    cgen(l, &nod);
    regalloc(&nod1, n, &nod);
    if(inrel)
        gmover(&nod, &nod1);
    else
        gopcode(OAS, &nod, Z, &nod1);
    gopcode(OAS, &nod1, Z, nn);
    regfree(&nod1);
    regfree(&nod);
    break;
```

⟨*function* gmover*(arm)* 201b⟩≡ (462)

```
void
gmover(Node *f, Node *t)
{
    int ft, tt, a;

    ft = f->type->etype;
    tt = t->type->etype;
    a = AGOK;
    if(typechlp[ft] && typechlp[tt] && ewidth[ft] >= ewidth[tt]){
        switch(tt){
        case TSHORT:
            a = AMOVH;
            break;
        case TUSHORT:
            a = AMOVHU;
            break;
        case TCHAR:
            a = AMOVB;
            break;
        case TUCHAR:
            a = AMOVBU;
            break;
        }
    }
    if(a == AGOK)
        gmove(f, t);
    else
        gins(a, f, t);
}
```

⟨cgenrel() *if same types* 201c⟩≡ (201a)

```
if(nocast(l->type, n->type)) {
    if(nocast(n->type, nn->type)) {
        cgen(l, nn);
        break;
    }
}
```

⟨*function* nocast 202a⟩≡ (443b)

```
/// cgen -> cgenrel -> <>
/*
 * a cast that generates no code
 * (same size move)
 */
bool
nocast(Type *t1, Type *t2)
{
    int i, b;

    if(t1->nbits)
        return false;
    i = 0;
    if(t2 != T)
        i = t2->etype;
    b = 1<<i;
    i = 0;
    if(t1 != T)
        i = t1->etype;
    if(b & ncast[i])
        return true;
    return false;
}
```

⟨*global* ncast 202b⟩≡ (462)

```
long ncast[NTYPE] =
{
    [TXXX] = 0,
    [TCHAR] = BCHAR|BUCHAR,
    [TUCHAR] = BCHAR|BUCHAR,
    [TSHORT] = BSHORT|BUSHORT,
    [TUSHORT] = BSHORT|BUSHORT,
    [TINT] = BINT|BUINT|BLONG|BULONG|BIND,
    [TUINT] = BINT|BUINT|BLONG|BULONG|BIND,
    [TLONG] = BINT|BUINT|BLONG|BULONG|BIND,
    [TULONG] = BINT|BUINT|BLONG|BULONG|BIND,
    [TVLONG] = BVLONG|BUVLONG,
    [TUVLONG] = BVLONG|BUVLONG,
    [TFLOAT] = BFLOAT,
    [TDOUBLE] = BDOUBLE,
    [TIND] = BLONG|BULONG|BIND,
    [TFUNC] = 0,
    [TARRAY] = 0,
    [TVOID] = 0,
    [TSTRUCT] = BSTRUCT,
    [TUNION] = BUNION,
    [TENUM] = 0,
};
```

10.9.13 Ternary expressions

⟨cgenrel() *switch node kind cases* 202c⟩+≡ (179f) ◁201a 203a▷

```
case OCOND:
    bcgen(l, true);
    p1 = p;
    cgen(r->left, nn);
    gbranch(OGOTO);
    patch(p1, pc);
```

```
    p1 = p;
    cgen(r->right, nn);
    patch(p1, pc);
    break;
```

10.9.14 Prefix/postfix

⟨cgenrel() *switch node kind cases* 203a⟩+≡ (179f) ◁202c 203b▷

```
  case OPOSTINC:
  case OPOSTDEC:
      v = 1;
      if(l->type->etype == TIND)
          v = l->type->link->width;
      if(o == OPOSTDEC)
          v = -v;
      ⟨cgenrel() when OPOSTINC/OPOSTDEC, if bitfield 354h⟩
      if(nn == Z)
          goto pre;

      if(l->addable < INDEXED)
          reglcgen(&nod2, l, Z);
      else
          nod2 = *l;

      regalloc(&nod, l, nn);
      gopcode(OAS, &nod2, Z, &nod);
      regalloc(&nod1, l, Z);
      if(typefd[l->type->etype]) {
          regalloc(&nod3, l, Z);
          if(v < 0) {
              gopcode(OAS, nodfconst(-v), Z, &nod3);
              gopcode(OSUB, &nod3, &nod, &nod1);
          } else {
              gopcode(OAS, nodfconst(v), Z, &nod3);
              gopcode(OADD, &nod3, &nod, &nod1);
          }
          regfree(&nod3);
      } else
          gopcode(OADD, nodconst(v), &nod, &nod1);
      gopcode(OAS, &nod1, Z, &nod2);

      regfree(&nod);
      regfree(&nod1);
      if(l->addable < INDEXED)
          regfree(&nod2);
      break;
```

⟨cgenrel() *switch node kind cases* 203b⟩+≡ (179f) ◁203a 204a▷

```
  case OPREINC:
  case OPREDEC:
      v = 1;
      if(l->type->etype == TIND)
          v = l->type->link->width;
      if(o == OPREDEC)
          v = -v;
      ⟨cgenrel() when OPREINC/OPREDEC, if bitfield 355a⟩

  pre:
      if(l->addable < INDEXED)
```

```
        reglcgen(&nod2, l, Z);
    else
        nod2 = *l;

    regalloc(&nod, l, nn);
    gopcode(OAS, &nod2, Z, &nod);
    if(typefd[l->type->etype]) {
        regalloc(&nod3, l, Z);
        if(v < 0) {
            gopcode(OAS, nodfconst(-v), Z, &nod3);
            gopcode(OSUB, &nod3, Z, &nod);
        } else {
            gopcode(OAS, nodfconst(v), Z, &nod3);
            gopcode(OADD, &nod3, Z, &nod);
        }
        regfree(&nod3);
    } else
        gopcode(OADD, nodconst(v), Z, &nod);
    gopcode(OAS, &nod, Z, &nod2);
    if(nn && l->op == ONAME) /* in x=++i, emit USED(i) */
        gins(ANOP, l, Z);

    regfree(&nod);
    if(l->addable < INDEXED)
        regfree(&nod2);
    break;
```

10.9.15 sizeof()

10.10 Boolean expressions

⟨cgenrel() *switch node kind cases* 204a⟩+≡ (179f) ◁203b 204b▷

```
  case OANDAND:
  case OOROR:
      boolgen(n, true, nn);
      if(nn == Z)
          patch(p, pc);
      break;
```

⟨cgenrel() *switch node kind cases* 204b⟩+≡ (179f) ◁204a 204c▷

```
  case ONOT:
      ⟨cgenrel() nullwarn check if nn is null 152c⟩
      boolgen(n, true, nn);
      break;
```

⟨cgenrel() *switch node kind cases* 204c⟩+≡ (179f) ◁204b 209▷

```
  case OEQ:
  case ONE:
  case OLE:
  case OLT:
  case OGE:
  case OGT:
  case OLO:
  case OLS:
  case OHI:
  case OHS:
      ⟨cgenrel() nullwarn check if nn is null 152c⟩
      boolgen(n, true, nn);
      break;
```

⟨*constant* BTRUE*(arm)* 205a⟩≡ (456)

```
#define BTRUE  0x1000
```

⟨gopcode() *locals* 205b⟩≡ (184c)

```
bool btrue;
```

⟨gopcode() *initialisation* 205c⟩≡ (184c)

```
btrue = o & BTRUE;
o &= ~BTRUE;
```

⟨*function* boolgen*(arm)* 205d⟩≡ (467)

```
/// cgenrel | bcomplex -> <>
void
boolgen(Node *n, int btrue, Node *nn)
{
    int o;
    Prog *p1, *p2;
    Node *l, *r, nod, nod1;
    long curs; // to save cursafe

    ⟨boolgen() debug 417c⟩
    curs = cursafe; // save

    l = n->left;
    r = n->right;
    switch(n->op) {

    case OCONST:
        o = vconst(n);
        if(!btrue)
            o = !o;
        gbranch(OGOTO);
        if(o) {
            p1 = p;
            gbranch(OGOTO);
            patch(p1, pc);
        }
        goto com;

    case OCOMMA:
        cgen(l, Z);
        boolgen(r, btrue, nn);
        break;

    case ONOT:
        boolgen(l, !btrue, nn);
        break;

    case OCOND:
        bcgen(l, 1);
        p1 = p;
        bcgen(r->left, btrue);
        p2 = p;
        gbranch(OGOTO);
        patch(p1, pc);
        p1 = p;
        bcgen(r->right, !btrue);
        patch(p2, pc);
        p2 = p;
        gbranch(OGOTO);
```

```
    patch(p1, pc);
    patch(p2, pc);
    goto com;

case OANDAND:
    if(!btrue)
        goto caseor;

caseand:
    bcgen(l, btrue);
    p1 = p;
    bcgen(r, !btrue);
    p2 = p;
    patch(p1, pc);
    gbranch(OGOTO);
    patch(p2, pc);
    goto com;

case OOROR:
    if(!btrue)
        goto caseand;

caseor:
    bcgen(l, !btrue);
    p1 = p;
    bcgen(r, !btrue);
    p2 = p;
    gbranch(OGOTO);
    patch(p1, pc);
    patch(p2, pc);
    goto com;

case OEQ:
case ONE:
case OLE:
case OLT:
case OGE:
case OGT:
case OHI:
case OHS:
case OLO:
case OLS:
    o = n->op;
    if(btrue)
        o = comrel[relindex(o)];
    if(l->complex >= FNX && r->complex >= FNX) {
        regret(&nod, r);
        cgenrel(r, &nod, btrue);
        regsalloc(&nod1, r);
        gopcode(OAS, &nod, Z, &nod1);
        regfree(&nod);
        nod = *n;
        nod.right = &nod1;
        boolgen(&nod, btrue, nn);
        break;
    }
    if(sconst(l)) {
        regalloc(&nod, r, nn);
        cgenrel(r, &nod, btrue);
        o = invrel[relindex(o)];
```

```
            gopcode(btrue ? o | BTRUE : o, l, &nod, Z);
            regfree(&nod);
            goto com;
        }
        if(sconst(r)) {
            regalloc(&nod, l, nn);
            cgenrel(l, &nod, btrue);
            gopcode(btrue ? o | BTRUE : o, r, &nod, Z);
            regfree(&nod);
            goto com;
        }

        if(l->complex >= r->complex) {
            regalloc(&nod1, l, nn);
            cgenrel(l, &nod1, btrue);
            regalloc(&nod, r, Z);
            cgenrel(r, &nod, btrue);
        } else {
            regalloc(&nod, r, nn);
            cgenrel(r, &nod, btrue);
            regalloc(&nod1, l, Z);
            cgenrel(l, &nod1, btrue);
        }
        gopcode(btrue ? o | BTRUE : o, &nod, &nod1, Z);
        regfree(&nod);
        regfree(&nod1);

    com:
        if(nn != Z) {
            p1 = p;
            gopcode(OAS, nodconst(1), Z, nn);
            gbranch(OGOTO);
            p2 = p;
            patch(p1, pc);
            gopcode(OAS, nodconst(0), Z, nn);
            patch(p2, pc);
        }
        break;

    default:
        regalloc(&nod, n, nn);
        cgen(n, &nod);
        o = ONE;
        if(btrue)
            o = comrel[relindex(o)];
        if(typefd[n->type->etype]) {
            gopcode(btrue ? o | BTRUE : o, nodfconst(0), &nod, Z);
        } else
            gopcode(o, nodconst(0), &nod, Z);
        regfree(&nod);
        goto com;

    }
    cursafe = curs; // restore
}
```

⟨*function* bcgen 207⟩≡ (467)

```
void
bcgen(Node *n, bool btrue)
{
```

```
    if(n->type == T)
        gbranch(OGOTO);
    else
        boolgen(n, btrue, Z);
}
```

⟨*global* comrel 208a⟩≡ (443b)

```
/* OEQ, ONE, OLE, OLS, OLT, OLO, OGE, OHS, OGT, OHI */
char comrel[12] =
{
    ONE, OEQ, OGT, OHI, OGE, OHS, OLT, OLO, OLE, OLS,
};
```

⟨*global* invrel 208b⟩≡ (443b)

```
char invrel[12] =
{
    OEQ, ONE, OGE, OHS, OGT, OHI, OLE, OLS, OLT, OLO,
};
```

⟨gopcode() *switch opcode cases* 208c⟩+≡ (184c) ◁193b 210a▷

```
case OCASE:
case OEQ:
case ONE:
case OLT:
case OLE:
case OGE:
case OGT:
case OLO:
case OLS:
case OHS:
case OHI:
    a = ACMP;
    if(et == TFLOAT)
        a = ACMPF;
    else
    if(et == TDOUBLE || et == TVLONG)
        a = ACMPD;
    nextpc();
    p->as = a;
    naddr(f1, &p->from);
    if(a == ACMP && f1->op == OCONST && p->from.offset < 0 &&
        p->from.offset != 0x80000000) {
        p->as = ACMN;
        p->from.offset = -p->from.offset;
    }
    raddr(f2, p);
    switch(o) {
    case OEQ:
        a = ABEQ;
        break;
    case ONE:
        a = ABNE;
        break;
    case OLT:
        a = ABLT;
        /* ensure NaN comparison is always false */
        if(typefd[et] && !btrue)
            a = ABMI;
        break;
```

```
    case OLE:
        a = ABLE;
        if(typefd[et] && !btrue)
            a = ABLS;
        break;
    case OGE:
        a = ABGE;
        if(typefd[et] && btrue)
            a = ABPL;
        break;
    case OGT:
        a = ABGT;
        if(typefd[et] && btrue)
            a = ABHI;
        break;
    case OLO:
        a = ABLO;
        break;
    case OLS:
        a = ABLS;
        break;
    case OHS:
        a = ABHS;
        break;
    case OHI:
        a = ABHI;
        break;

    case OCASE:
        nextpc();
        p->as = ACASE;
        p->scond = 0x9;
        naddr(f2, &p->from);
        a = ABHI;
        break;
    }
    f1 = Z;
    f2 = Z;
    break;
```

10.11 Function calls

⟨cgenrel() *switch node kind cases* 209⟩+≡ (179f) ◁204c 354c▷

```
  case OFUNC:
      ⟨cgenrel() when OFUNC, if complex indirect function call 212b⟩
      // else

      ⟨cgenrel() OFUNC case, if use REGARG, part one 280c⟩
      gargs(r, &nod, &nod1);
      ⟨cgenrel() OFUNC case, if l not addressable 212a⟩
      else
          gopcode(OFUNC, Z, Z, l);
      ⟨cgenrel() OFUNC case, if use REGARG, part two 280d⟩
      if(nn != Z) {
          // store result of function call (in REGRET) to dest nn
          regret(&nod, n);
          gopcode(OAS, &nod, Z, nn);
          regfree(&nod);
```

```
    }
    break;
```

⟨gopcode() *switch opcode cases* 210a⟩+≡ (184c) ◁208c

```
  case OFUNC:
      a = ABL;
      break;
```

10.11.1 gargs()

⟨*global* curarg 210b⟩≡ (460b)

```
  long curarg;
```

⟨*global* maxargsafe 210c⟩≡ (460b)

```
  long maxargsafe;
```

⟨codgen() *initialisation* 210d⟩≡ (156d) 210f▷

```
  curarg = 0;
  maxargsafe = 0;
```

⟨*global* cursafe 210e⟩≡ (460b)

```
  long cursafe;
```

⟨codgen() *initialisation* 210f⟩+≡ (156d) ◁210d

```
  cursafe = 0;
```

⟨*function* gargs 210g⟩≡ (462)

```
  /// cgenrel -> <>
  void
  gargs(Node *n, Node *tn1, Node *tn2)
  {
      Node fnxargs[20], *fnxp;
      long regs; // to save cursafe

      regs = cursafe; // save

      fnxp = fnxargs;
      // step1
      garg1(n, tn1, tn2, 0, &fnxp); /* compile fns to temps */

      curarg = 0;
      fnxp = fnxargs;
      // step2
      garg1(n, tn1, tn2, 1, &fnxp); /* compile normal args and temps */

      cursafe = regs; // restore
  }
```

⟨*function* garg1*(arm)* 210h⟩≡ (462)

```
  /// cgenrel -> gargs -> <>
  void
  garg1(Node *n, Node *tn1, Node *tn2, int f, Node **fnxp)
  {
      Node nod;

      if(n == Z)
          return;
      if(n->op == OLIST) {
          garg1(n->left, tn1, tn2, f, fnxp);
```

```
        garg1(n->right, tn1, tn2, f, fnxp);
        return;
    }
    // else, an argument

    if(f == 0) {
        ⟨garg() when f == 0, if n is complex part1 213a⟩
    }
    else

    ⟨garg1() if complex argument type 222f⟩
    else
    ⟨garg1() if use REGARG and curarg is zero and simple type 280e⟩
    else {
        regalloc(tn1, n, Z);
        ⟨garg() when f != 0, if n is complex part2 213b⟩
        else
            cgen(n, tn1);
        regaalloc(tn2, n);
        gopcode(OAS, tn1, Z, tn2);
        regfree(tn1);
    }
}
```

⟨*function* regaalloc*(arm)* 211a⟩≡ (462)

```
void
regaalloc(Node *n, Node *nn)
{
    curarg = align(curarg, nn->type, Aarg1);
    *n = *nn;
    n->op = OINDREG;
    n->reg = REGSP;
    n->xoffset = curarg + SZ_LONG;
    n->complex = 0;
    n->addable = 20;
    curarg = align(curarg, nn->type, Aarg2);
    maxargsafe = maxround(maxargsafe, cursafe+curarg);
}
```

10.11.2 Caller-save vs Callee-save

⟨*global* nodsafe 211b⟩≡ (460b)

```
Node* nodsafe;
```

⟨ginit() *special nodes initialisation* 211c⟩+≡ (35d) ◁195e 222h▷

```
nodsafe = new(ONAME, Z, Z);
nodsafe->sym = slookup(".safe");
nodsafe->type = types[TINT];
nodsafe->etype = types[TINT]->etype; // TINT
nodsafe->class = CAUTO;
complex(nodsafe);
```

⟨*function* regsalloc 211d⟩≡ (462)

```
/// garg1 -> <>
void
regsalloc(Node *n, Node *nn)
{
    cursafe = align(cursafe, nn->type, Aaut3);
    maxargsafe = maxround(maxargsafe, cursafe+curarg);
```

```
    *n = *nodsafe;
    n->xoffset = -(stkoff + cursafe);
    n->type = nn->type;
    n->etype = nn->type->etype;
    n->lineno = nn->lineno;
}
```

10.11.3 Indirect function calls

⟨cgenrel() *OFUNC case, if l not addressable* 212a⟩≡ (209)

```
if(l->addable < INDEXED) {
    reglcgen(&nod, l, Z);
    gopcode(OFUNC, Z, Z, &nod);
    regfree(&nod);
}
```

⟨cgenrel() *when OFUNC, if complex indirect function call* 212b⟩≡ (209)

```
if(l->complex >= FNX) {
    if(l->op != OIND)
        diag(n, "bad function call");

    regret(&nod, l->left);
    cgen(l->left, &nod);
    regsalloc(&nod1, l->left);
    gopcode(OAS, &nod, Z, &nod1);
    regfree(&nod);

    nod = *n;
    nod.left = &nod2;
    nod2 = *l;
    nod2.left = &nod1;
    nod2.complex = 1;
    cgen(&nod, nn);

    return;
}
```

10.11.4 Complex calls

⟨cgenrel() *if all complex fields more than FNX* 212c⟩≡ (179f)

```
if(n->complex >= FNX)
 if(l->complex >= FNX)
  if(r != Z && r->complex >= FNX)
    switch(o) {
    case OFUNC:
    case OCOMMA:
    case OANDAND:
    case OOROR:
    case OCOND:
    case ODOT:
        break;

    default:
        regret(&nod, r);
        cgen(r, &nod);

        regsalloc(&nod1, r);
        gopcode(OAS, &nod, Z, &nod1);
```

```
        regfree(&nod);
        nod = *n;
        nod.right = &nod1;
        cgen(&nod, nn);
        return;

    }
```

⟨garg() *when* f == 0, *if* n *is complex part1* 213a⟩≡ (210h)

```
  if(n->complex >= FNX) {
      regsalloc(*fnxp, n);
      nod = znode;
      nod.op = OAS;
      nod.left = *fnxp;
      nod.right = n;
      nod.type = n->type;
      cgen(&nod, Z);
      (*fnxp)++;
  }
```

⟨garg() *when* f != 0, *if* n *is complex part2* 213b⟩≡ (210h)

```
  if(n->complex >= FNX) {
      cgen(*fnxp, tn1);
      (*fnxp)++;
  }
```

10.12 Initializers

⟨*global* nodcast 213c⟩≡ (436d)

```
  Node* nodcast;
```

⟨*expression nodes* 213d⟩+≡ (16a) ◁93a 363a▷

```
  OASI, // appears during parsing
```

⟨doinit() *sanity check initlist after init1* 213e⟩≡ (93d)

```
  if(initlist != Z)
      diag(initlist, "more initializers than structure: %s",
          s->name);
```

⟨*function* init1 213f⟩≡ (450)

```
  /// yyparse -> xdlist -> doinit -> <>
  Node*
  init1(Sym *s, Type *t, long o, bool exflag)
  {
      Node *a, *l, *r, nod;
      Type *t1;
      long e, w, so, mw;

      a = peekinit();
      if(a == Z)
          return Z;

      ⟨init1() debug initialization 419b⟩

      if(exflag && a->op == OINIT)
          return doinit(s, t, o, nextinit());
```

```
// else
switch(t->etype) {
case TCHAR:
case TUCHAR:
case TINT:
case TUINT:
case TSHORT:
case TUSHORT:
case TLONG:
case TULONG:
case TVLONG:
case TUVLONG:
case TFLOAT:
case TDOUBLE:

case TIND:
single:
    if(a->op == OARRAY || a->op == OELEM)
        return Z;

    a = nextinit();
    if(a == Z)
        return Z;

    ⟨init1() sanity check t has no bitfields 354a⟩
    if(s->class == CAUTO) {
        l = new(ONAME, Z, Z);
        l->sym = s;
        l->type = t;
        l->etype = s->type ? s->type->etype : TVOID;
        l->xoffset = s->offset + o;
        l->class = s->class;

        l = new(OASI, l, a);
        return l;
    }

    complex(a);
    if(a->type == T)
        return Z;

    if(a->op == OCONST) {
        if(vconst(a) && t->etype == TIND && a->type && a->type->etype != TIND){
            diag(a, "initialize pointer to an integer: %s", s->name);
            return Z;
        }
        if(!sametype(a->type, t)) {
            /* hoop jumping to save malloc */
            if(nodcast == Z)
                nodcast = new(OCAST, Z, Z);
            nod = *nodcast;
            nod.left = a;
            nod.type = t;
            nod.lineno = a->lineno;
            complex(&nod);
            if(nod.type)
                *a = nod;
        }
        if(a->op != OCONST) {
            diag(a, "initializer is not a constant: %s",
```

```
                s->name);
            return Z;
        }
        if(vconst(a) == 0)
            return Z;
        goto gext;
    }
    if(t->etype == TIND) {
        while(a->op == OCAST) {
            warn(a, "CAST in initialization ignored");
            a = a->left;
        }
        if(!sametype(t, a->type)) {
            diag(a, "initialization of incompatible pointers: %s\n%T and %T",
                s->name, t, a->type);
        }
        if(a->op == OADDR)
            a = a->left;
        goto gext;
    }

    while(a->op == OCAST)
        a = a->left;
    if(a->op == OADDR) {
        warn(a, "initialize pointer to an integer: %s", s->name);
        a = a->left;
        goto gext;
    }
    diag(a, "initializer is not a constant: %s", s->name);
    return Z;

gext:
    gextern(s, a, o, t->width);

    return Z;

case TARRAY:
    w = t->link->width;
    if(a->op == OSTRING || a->op == OLSTRING)
    if(typei[t->link->etype]) {
        /*
         * get rid of null if sizes match exactly
         */
        a = nextinit();
        mw = t->width/w;
        so = a->type->width/a->type->link->width;
        if(mw && so > mw) {
            if(so != mw+1)
                diag(a, "string initialization larger than array");
            a->type->width -= a->type->link->width;
        }

        /*
         * arrange strings to be expanded
         * inside OINIT braces.
         */
        a = new(OUSED, a, Z);
        return doinit(s, t, o, a);
    }
```

```
    mw = -w;
    l = Z;
    for(e=0;;) {
        /*
         * peek ahead for element initializer
         */
        a = peekinit();
        if(a == Z)
            break;
        if(a->op == OELEM && t->link->etype != TSTRUCT)
            break;
        if(a->op == OARRAY) {
            if(e && exflag)
                break;
            a = nextinit();
            r = a->left;
            complex(r);
            if(r->op != OCONST) {
                diag(r, "initializer subscript must be constant");
                return Z;
            }
            e = r->vconst;
            if(t->width != 0)
                if(e < 0 || e*w >= t->width) {
                    diag(a, "initialization index out of range: %ld", e);
                    continue;
                }
        }

        so = e*w;
        if(so > mw)
            mw = so;
        if(t->width != 0)
            if(mw >= t->width)
                break;
        r = init1(s, t->link, o+so, true);
        l = newlist(l, r);
        e++;
    }
    if(t->width == 0)
        t->width = mw+w;
    return l;

case TUNION:
case TSTRUCT:
    /*
     * peek ahead to find type of rhs.
     * if its a structure, then treat
     * this element as a variable
     * rather than an aggregate.
     */
    if(isstruct(a, t))
        goto single;

    if(t->width <= 0) {
        diag(Z, "incomplete structure: %s", s->name);
        return Z;
    }
    l = Z;
```

```
    again:
        for(t1 = t->link; t1 != T; t1 = t1->down) {
            if(a->op == OARRAY && t1->etype != TARRAY)
                break;
            if(a->op == OELEM) {
                if(t1->sym != a->sym)
                    continue;
                nextinit();
            }
            r = init1(s, t1, o+t1->offset, true);
            l = newlist(l, r);
            a = peekinit();
            if(a == Z)
                break;
            if(a->op == OELEM)
                goto again;
        }
        if(a && a->op == OELEM)
            diag(a, "structure element not found %F", a);
        return l;

    default:
        diag(Z, "unknown type in initialization: %T to: %s", t, s->name);
        return Z;
    }
}
```

⟨*function* newlist 217a⟩≡ (450)

```
Node*
newlist(Node *l, Node *r)
{
    if(r == Z)
        return l;
    if(l == Z)
        return r;
    return new(OLIST, l, r);
}
```

⟨*function* isstruct 217b⟩≡ (450)

```
bool
isstruct(Node *a, Type *t)
{
    Node *n;

    switch(a->op) {
    case ODOTDOT:
        n = a->left;
        if(n && n->type && sametype(n->type, t))
            return true;
    case OSTRING: case OLSTRING:
    case OCONST:
    case OINIT:
    case OELEM:
        return false;
    }

    n = new(ODOTDOT, Z, Z);
    *n = *a;

    /*
```

```
     * ODOTDOT is a flag for tcom
     * a second tcom will not be performed
     */
    a->op = ODOTDOT;
    a->left = n;
    a->right = Z;

    if(tcom(n))
        return false;

    if(sametype(n->type, t))
        return true;
    return false;
}
```

⟨*function* peekinit 218a⟩≡ (450)

```
/*
 * get next major operator,
 * dont advance initlist.
 */
Node*
peekinit(void)
{
    Node *a;

    a = initlist;

loop:
    if(a == Z)
        return a;
    if(a->op == OLIST) {
        a = a->left;
        goto loop;
    }
    return a;
}
```

⟨*function* nextinit 218b⟩≡ (450)

```
/*
 * consume and return next element on
 * initlist. expand strings.
 */
Node*
nextinit(void)
{
    Node *a, *b, *n;

    a = initlist;
    n = Z;

    if(a == Z)
        return a;
    if(a->op == OLIST) {
        n = a->right;
        a = a->left;
    }
    if(a->op == OUSED) {
        a = a->left;
        b = new(OCONST, Z, Z);
        b->type = a->type->link;
```

```
        if(a->op == OSTRING) {
            b->vconst = convvtox(*a->cstring, TCHAR);
            a->cstring++;
        }
        if(a->op == OLSTRING) {
            b->vconst = convvtox(*a->rstring, TRUNE);
            a->rstring++;
        }
        a->type->width -= b->type->width;
        if(a->type->width <= 0)
            initlist = n;
        return b;
    }
    initlist = n;
    return a;
}
```

⟨*adlist rule, after doinit* 219a⟩≡ (111a)

```
$$ = contig($1->sym, $$, w);
```

⟨*function* contig 219b⟩≡ (450)

```
//@Scheck: used by cc.y
Node* contig(Sym *s, Node *n, long v)
{
    Node *p, *r, *q, *m;
    long w;
    Type *zt;

    ⟨contig() debug initialization 420⟩

    if(n == Z)
        goto no;
    w = s->type->width;

    /*
     * nightmare: an automatic array whose size
     * increases when it is initialized
     */
    if(v != w) {
        if(v != 0)
            diag(n, "automatic adjustable array: %s", s->name);
        v = s->offset;
        autoffset = align(autoffset, s->type, Aaut3);
        s->offset = -autoffset;
        stkoff = maxround(stkoff, autoffset);
        symadjust(s, n, v - s->offset);
    }
    if(w <= ewidth[TIND])
        goto no;
    if(n->op == OAS)
        diag(Z, "oops in contig");
/*ZZZ this appears incorrect
need to check if the list completely covers the data.
if not, bail
 */
    if(n->op == OLIST)
        goto no;
    if(n->op == OASI)
        if(n->left->type)
        if(n->left->type->width == w)
```

```
            goto no;
    while(w & (ewidth[TIND]-1))
        w++;
/*
 * insert the following code, where long becomes vlong if pointers are fat
 *
    *(long**)&X = (long*)((char*)X + sizeof(X));
    do {
        *(long**)&X -= 1;
        **(long**)&X = 0;
    } while(*(long**)&X);
 */

    for(q=n; q->op != ONAME; q=q->left)
        ;

    zt = ewidth[TIND] > ewidth[TLONG]? types[TVLONG]: types[TLONG];

    p = new(ONAME, Z, Z);
    *p = *q;
    p->type = typ(TIND, zt);
    p->xoffset = s->offset;

    r = new(ONAME, Z, Z);
    *r = *p;
    r = new(OPOSTDEC, r, Z);

    q = new(ONAME, Z, Z);
    *q = *p;
    q = new(OIND, q, Z);

    m = new(OCONST, Z, Z);
    m->vconst = 0;
    m->type = zt;

    q = new(OAS, q, m);

    r = new(OLIST, r, q);

    q = new(ONAME, Z, Z);
    *q = *p;
    r = new(ODWHILE, q, r);

    q = new(ONAME, Z, Z);
    *q = *p;
    q->type = q->type->link;
    q->xoffset += w;
    q = new(OADDR, q, 0);

    q = new(OASI, p, q);
    r = new(OLIST, q, r);

    n = new(OLIST, r, n);

no:
    return n;
}
```

⟨*function* symadjust 220⟩≡ (450)

```
void
```

```
symadjust(Sym *s, Node *n, long del)
{

    switch(n->op) {
    case ONAME:
        if(n->sym == s)
            n->xoffset -= del;
        return;

    case OCONST:
    case OSTRING: case OLSTRING:
    case OREGISTER: case OINDREG:
        return;

    default:
        if(n->left)
            symadjust(s, n->left, del);
        if(n->right)
            symadjust(s, n->right, del);
        return;

    }
}
```

10.13 Toplevel globals

⟨gclean() *locals* 221a⟩+≡ (34a) ◁167a

```
Sym *s;
```

⟨gclean() *generate all AGLOBL pseudo opcodes* 221b⟩≡ (34a)

```
for(i=0; i<NHASH; i++)
 for(s = hash[i]; s != S; s = s->link) {
    ⟨gclean() when generate all AGLOBL, filter and sanity check symbol 221c⟩
    // else
    gpseudo(AGLOBL, s, nodconst(s->type->width));
}
```

⟨gclean() *when generate all AGLOBL, filter and sanity check symbol* 221c⟩≡ (221b)

```
if(s->type == T)
    continue;
if(s->type->width == 0)
    continue;
if(s->class != CGLOBL && s->class != CSTATIC)
    continue;
if(s->type == types[TENUM])
    continue;
```

10.13.1 gextern()

⟨*function* gextern*(arm)* 221d⟩≡ (464b)

```
void
gextern(Sym *s, Node *a, long o, long w)
{
    ⟨gextern() if OCONST and a vlong type 223d⟩
    else {
        gpseudo(ADATA, s, a);
        p->from.offset += o;
```

```
        p->reg = w; // attributes
        if(p->to.type == D_OREG)
            p->to.type = D_CONST; // D_ADDR?
    }
}
```

10.14 Complex values

10.14.1 Complex types

⟨*global* `typecmplx` 222a⟩≡ (436d)

```
// set<type_kind>
char* typecmplx;
```

⟨`tinit()` *initialise 32 bits defaults type sets* 222b⟩+≡ (34b) ◁144e 280b▷

```
typecmplx = typesuv;
```

⟨*global* `typesuv` 222c⟩≡ (443b)

```
char typesuv[NTYPE];
```

⟨*global* `typesuvinit` 222d⟩≡ (443b)

```
int typesuvinit[] =
{
    TVLONG, TUVLONG, TSTRUCT, TUNION, -1,
};
```

10.14.2 Complex expression

⟨`cgenrel()` *if complex type node* 222e⟩≡ (179f)

```
if(typesuv[n->type->etype]) {
    sugen(n, nn, n->type->width);
}
```

10.14.3 Complex argument

⟨`garg1()` *if complex argument type* 222f⟩≡ (210h)

```
if(typesuv[n->type->etype]) {
    regaalloc(tn2, n);
    if(n->complex >= FNX) {
        sugen(*fnxp, tn2, n->type->width);
        (*fnxp)++;
    } else
        sugen(n, tn2, n->type->width);
}
```

10.14.4 Complex return value

⟨*global* `nodret` 222g⟩≡ (460b)

```
Node* nodret;
```

⟨`ginit()` *special nodes initialisation* 222h⟩+≡ (35d) ◁211c 383h▷

```
nodret = new(ONAME, Z, Z);
nodret->sym = slookup(".ret");
nodret->type = types[TIND];
nodret->etype = TIND;
nodret->class = CPARAM;
nodret = new(OIND, nodret, Z);
complex(nodret);
```

⟨codegen() *if complex return type* 223a⟩≡ (156d)

```
if(typecmplx[thisfntype->link->etype]) {
    n1 = nodret->left;
    if(n1->type == T || n1->type->link != thisfntype->link) {
        n1->type = typ(TIND, thisfntype->link);
        n1->etype = n1->type->etype; // TIND, dead instruction
        nodret = new(OIND, n1, Z); // useful? to force retyping?
        complex(nodret);
    }
}
```

⟨gen() *case ORETURN, if complex type* 223b⟩≡ (165g)

```
if(typecmplx[n->type->etype]) {
    nod = znode;
    nod.op = OAS;
    nod.left = nodret;
    nod.right = l;
    nod.type = n->type;
    nod.complex = l->complex;
    cgen(&nod, Z);
    noretval(1 | 2);
    gbranch(ORETURN);
    break;
}
```

⟨codegen() *if use REGARG, if complex return type* 223c⟩≡ (279e)

```
if(typecmplx[thisfntype->link->etype]) {
    nod1 = *nodret->left;
    nodreg(&nod, &nod1, REGARG);
    gmove(&nod, &nod1);
}
```

10.14.5 vlong constant

⟨gextern() *if OCONST and a vlong type* 223d⟩≡ (221d)

```
if(a->op == OCONST && typev[a->type->etype]) {
    if(align(0, types[TCHAR], Aarg1)) /* isbigendian */
        gpseudo(ADATA, s, nod32const(a->vconst>>32));
    else
        gpseudo(ADATA, s, nod32const(a->vconst));
    p->from.offset += o;
    p->reg = 4;
    if(align(0, types[TCHAR], Aarg1)) /* isbigendian */
        gpseudo(ADATA, s, nod32const(a->vconst));
    else
        gpseudo(ADATA, s, nod32const(a->vconst>>32));
    p->from.offset += o + 4;
    p->reg = 4;
}
```

⟨*function* nod32const*(arm)* 223e⟩≡ (462)

```
Node*
nod32const(vlong v)
{
    constnode.vconst = v & MASK(32);
    return &constnode;
}
```

10.15 Other topics

10.15.1 Sizes

⟨*global* ewidth 224a⟩≡ (462)

```
schar ewidth[NTYPE] =
{
    [TXXX] = -1,
    [TCHAR] = SZ_CHAR,

    [TUCHAR] = SZ_CHAR,
    [TSHORT] = SZ_SHORT,
    [TUSHORT] = SZ_SHORT,
    [TINT] = SZ_INT,
    [TUINT] = SZ_INT,
    [TLONG] = SZ_LONG,
    [TULONG] = SZ_LONG,
    [TVLONG] = SZ_VLONG,
    [TUVLONG] = SZ_VLONG,
    [TFLOAT] = SZ_FLOAT,
    [TDOUBLE] = SZ_DOUBLE,

    [TIND] = SZ_IND,
    [TFUNC] = 0,
    [TARRAY] = -1,
    [TVOID] = 0,
    [TSTRUCT] = -1,
    [TUNION] = -1,
    [TENUM] = SZ_INT,
};
```

⟨*constant* SZ_CHAR*(arm)* 224b⟩≡ (456)

```
#define SZ_CHAR  1
```

⟨*constant* SZ_SHORT*(arm)* 224c⟩≡ (456)

```
#define SZ_SHORT 2
```

⟨*constant* SZ_INT*(arm)* 224d⟩≡ (456)

```
#define SZ_INT  4
```

⟨*constant* SZ_LONG*(arm)* 224e⟩≡ (456)

```
#define SZ_LONG  4
```

⟨*constant* SZ_IND*(arm)* 224f⟩≡ (456)

```
#define SZ_IND  4
```

⟨*constant* SZ_FLOAT*(arm)* 224g⟩≡ (456)

```
#define SZ_FLOAT 4
```

⟨*constant* SZ_VLONG*(arm)* 224h⟩≡ (456)

```
#define SZ_VLONG 8
```

⟨*constant* SZ_DOUBLE*(arm)* 224i⟩≡ (456)

```
#define SZ_DOUBLE 8
```

10.15.2 First argument

⟨adecl() *if first parameter* 224j⟩≡ (180d)

```
if(autoffset == 0) {
    firstarg = s;
    firstargtype = t;
}
```

⟨*global* `firstarg` 225a⟩≡ (436d)

```
Sym* firstarg;
```

⟨*xdecl rule, initializations before processing a function* 225b⟩≡ (80c)

```
firstarg = S; // can mv after call to dodecl? No I think.
```

⟨*global* `firstargtype` 225c⟩≡ (436d)

```
Type* firstargtype;
```

10.15.3 Alignment

⟨*function* `align`*(arm)* 225d⟩≡ (464b)

```
/// ??? -> <>
long
align(long i, Type *t, int op)
{
    long o;
    Type *v;
    int w;

    o = i;
    w = 1;
    switch(op) {
    default:
        diag(Z, "unknown align opcode %d", op);
        break;

    case Asu2: /* padding at end of a struct */
        w = SZ_LONG;
        ⟨align() if packflg override w 399b⟩
        break;

    case Ael1: /* initial align of struct element */
        for(v=t; v->etype==TARRAY; v=v->link)
            ;
        w = ewidth[v->etype];
        if(w <= 0 || w >= SZ_LONG)
            w = SZ_LONG;
        ⟨align() if packflg override w 399b⟩
        break;

    case Ael2: /* width of a struct element */
        o += t->width;
        break;

    case Aarg0: /* initial passbyptr argument in arg list */
        if(typesuv[t->etype]) {
            o = align(o, types[TIND], Aarg1);
            o = align(o, types[TIND], Aarg2);
        }
        break;

    case Aarg1: /* initial align of parameter */
        w = ewidth[t->etype];
        if(w <= 0 || w >= SZ_LONG) {
            w = SZ_LONG;
            break;
        }
        w = 1;  /* little endian no adjustment */
```

```
        break;

    case Aarg2: /* width of a parameter */
        o += t->width;
        w = SZ_LONG;
        break;

    case Aaut3: /* total align of automatic */
        o = align(o, t, Ael2);
        o = align(o, t, Ael1);
        w = SZ_LONG; /* because of a pun in cc/dcl.c:contig() */
        break;
    }
    o = round(o, w);
    if(debug['A'])
        print("align %s %ld %T = %ld\n", bnames[op], i, t, o);
    return o;
}
```

⟨*function* maxround 226a⟩≡ (464b)

```
long
maxround(long max, long v)
{
    v = round(v, SZ_LONG);
    if(v > max)
        return v;
    return max;
}
```

⟨*enum* align 226b⟩≡ (429b)

```
enum align
{
    Axxx,

    Ael1,
    Ael2,
    Asu2,
    Aarg0,
    Aarg1,
    Aarg2,
    Aaut3,

    NALIGN,
};
```

⟨*global* bnames 226c⟩≡ (443b)

```
char* bnames[NALIGN];
```

⟨*global* bnamesinit 226d⟩≡ (443b)

```
Init bnamesinit[] =
{
    Axxx, 0, "Axxx",

    Ael1, 0, "el1",
    Ael2, 0, "el2",
    Asu2, 0, "su2",
    Aarg0, 0, "arg0",
    Aarg1, 0, "arg1",
    Aarg2, 0, "arg2",
    Aaut3, 0, "aut3",
    -1, 0, 0,
};
```

⟨Type *code generation fields* 227a⟩+≡ (19b) ◁21d 227c▷

```
long offset;
```

⟨*complex rule, when parse a structure definition, align the struct* 227b⟩≡ (102 101)

```
sualign($$);
```

⟨Type *code generation fields* 227c⟩+≡ (19b) ◁227a

```
schar shift;
char nbits;
```

⟨*function* sualign 227d⟩≡ (450)

```
//@Scheck: not dead, used by cc.y
void sualign(Type *t)
{
    Type *l;
    long o, w;

    o = 0;
    switch(t->etype) {

    case TSTRUCT:
        t->offset = 0;
        w = 0;
        for(l = t->link; l != T; l = l->down) {
            if(l->nbits) {
                if(l->shift <= 0) {
                    l->shift = -l->shift;
                    w = round(w, tfield->width);
                    o = w;
                    w += tfield->width;
                }
                l->offset = o;
            } else {
                if(l->width < 0 ||
                   l->width == 0 && l->down != T)
                    if(l->sym)
                        diag(Z, "incomplete structure element: %s",
                            l->sym->name);
                    else
                        diag(Z, "incomplete structure element");
                w = align(w, l, Ael1);
                l->offset = w;
                w = align(w, l, Ael2);
            }
        }
        w = align(w, t, Asu2);
        t->width = w;
        acidtype(t);
        pickletype(t);
        return;

    case TUNION:
        t->offset = 0;
        w = 0;
        for(l = t->link; l != T; l = l->down) {
            if(l->width <= 0)
                if(l->sym)
                    diag(Z, "incomplete union element: %s",
                        l->sym->name);
                else
```

```
                    diag(Z, "incomplete union element");
            l->offset = 0;
            l->shift = 0;
            o = align(align(0, l, Ael1), l, Ael2);
            if(o > w)
                w = o;
        }
        w = align(w, t, Asu2);
        t->width = w;
        acidtype(t);
        pickletype(t);
        return;

    default:
        diag(Z, "unknown type in sualign: %T", t);
        break;
    }
}
```

10.16 Advanced assembly generation

10.16.1 Switch

⟨gen() *locals* 228a⟩+≡ (158i) ◁165f

```
Case *cn;
```

⟨gen() *switch node kind cases* 228b⟩+≡ (158i) ◁179d 229c▷

```
case OSWITCH:
    l = n->left;
    complex(l);
    if(l->type == T)
        break;
    ⟨gen() when OSWITCH, typecheck condition 144c⟩

    gbranch(OGOTO);  /* entry */
    sp = p;

    cn = cases;
    cases = C;
    casf();

    sbc = breakpc;
    breakpc = pc;
    snbreak = nbreak;
    nbreak = 0;

    gbranch(OGOTO);
    spb = p;

    gen(n->right);  /* body */
    if(canreach){
        gbranch(OGOTO);
        patch(p, breakpc);

        nbreak++;
    }

    patch(sp, pc);
```

```
    regalloc(&nod, l, Z);
    /* always signed */
    if(typev[l->type->etype])
        nod.type = types[TVLONG];
    else
        nod.type = types[TLONG];
    cgen(l, &nod);
    doswit(&nod);
    regfree(&nod);
    patch(spb, pc);

    cases = cn;
    breakpc = sbc;
    canreach = nbreak!=0;
    if(canreach == false)
        warnreach = !suppress;
    nbreak = snbreak;
    break;
```

⟨*global* typev 229a⟩≡ (443b)

```
char typev[NTYPE];
```

⟨*global* typevinit 229b⟩≡ (443b)

```
int typevinit[] =
{
    TVLONG, TUVLONG, -1,
};
```

⟨gen() *switch node kind cases* 229c⟩+≡ (158i) ◁228b

```
case OCASE:
    canreach = true;
    l = n->left;
    if(cases == C)
        diag(n, "case/default outside a switch");
    if(l == Z) {
        casf();
        cases->val = 0;
        cases->def = 1;
        cases->label = pc;
        cases->isv = 0;
        goto rloop;
    }
    complex(l);
    if(l->type == T)
        goto rloop;
    if(l->op != OCONST || !typeswitch[l->type->etype]) {
        diag(n, "case expression must be integer constant");
        goto rloop;
    }
    casf();
    cases->val = l->vconst;
    cases->def = 0;
    cases->label = pc;
    cases->isv = typev[l->type->etype];
    goto rloop;
```

⟨*struct* Case 229d⟩≡ (456)

```
struct Case
{
    vlong val;
```

```
    long label;
    char def;
    char  isv;

    Case* link;
};
```

⟨*constant* C 230a⟩≡ (456)

```
#define C ((Case*)nil)
```

⟨*struct* C1 230b⟩≡ (456)

```
struct C1
{
    vlong val;
    long label;
};
```

⟨*global* cases 230c⟩≡ (460b) 230e ▷

```
Case* cases;
```

⟨ginit() *misc initialisations* 230d⟩+≡ (35d) ◁ 195h 353d ▷

```
cases = C;
```

⟨*global* cases 230e⟩+≡ (460b) ◁ 230c

```
Case* cases;
```

⟨*function* swcmp 230f⟩≡ (455a)

```
int
swcmp(const void *a1, const void *a2)
{
    C1 *p1, *p2;

    p1 = (C1*)a1;
    p2 = (C1*)a2;
    if(p1->val < p2->val)
        return -1;
    return p1->val > p2->val;
}
```

⟨*function* doswit 230g⟩≡ (455a)

```
void
doswit(Node *n)
{
    Case *c;
    C1 *q, *iq, *iqh, *iql;
    long def, nc, i, j, isv, nh;
    Prog *hsb;
    Node *vr[2];
    int dup;

    def = 0;
    nc = 0;
    isv = 0;
    for(c = cases; c->link != C; c = c->link) {
        if(c->def) {
            if(def)
                diag(n, "more than one default in switch");
            def = c->label;
            continue;
        }
```

```
        isv |= c->isv;
        nc++;
    }
    if(typev[n->type->etype])
        isv = 1;
    else if(isv){
        warn(n, "32-bit switch expression with 64-bit case constant");
        isv = 0;
    }

    iq = alloc(nc*sizeof(C1));
    q = iq;
    for(c = cases; c->link != C; c = c->link) {
        if(c->def)
            continue;
        if(c->isv && !isv)
            continue; /* can never match */
        q->label = c->label;
        if(isv)
            q->val = c->val;
        else
            q->val = (long)c->val; /* cast ensures correct value for 32-bit switch on 64-bit architecture */
        q++;
    }
    qsort(iq, nc, sizeof(C1), swcmp);

    if(debug['K'])
     for(i=0; i<nc; i++)
        print("case %2ld: = %.8llux\n", i, (vlong)iq[i].val);

   dup = 0;
    for(i=0; i<nc-1; i++)
        if(iq[i].val == iq[i+1].val) {
            diag(n, "duplicate cases in switch %lld", (vlong)iq[i].val);
            dup = 1;
        }
    if(dup)
        return;
    if(def == 0) {
        def = breakpc;
        nbreak++;
    }
    if(!isv || ewidth[TIND] > ewidth[TLONG] || n->op == OREGISTER) {
        swit1(iq, nc, def, n);
        return;
    }

    /*
     * 64-bit case on 32-bit machine:
     * switch on high-order words, and
     * in each of those, switch on low-order words
     */
    if(n->op != OREGPAIR)
        fatal(n, "internal: expected register pair");
    if(thechar == '8'){ /* TO DO: need an enquiry function */
        vr[0] = n->left; /* low */
        vr[1] = n->right; /* high */
    }else{
        vr[0] = n->right;
        vr[1] = n->left;
```

```
    }
    vr[0]->type = types[TLONG];
    vr[1]->type = types[TLONG];

    gbranch(OGOTO);
    hsb = p;

    iqh = alloc(nc*sizeof(C1));
    iql = alloc(nc*sizeof(C1));
    nh = 0;
    for(i=0; i<nc;){
        iqh[nh].val = iq[i].val >> 32;
        q = iql;
        /* iq is sorted, so equal top halves are adjacent */
        for(j = i; j < nc; j++){
            if((iq[j].val>>32) != iqh[nh].val)
                break;
            q->val = (long)iq[j].val;
            q->label = iq[j].label;
            q++;
        }
        qsort(iql,  q-iql, sizeof(C1), swcmp);
                //if(0){for(int k=0; k<(q-iql); k++)print("nh=%ld k=%d h=%#llux l=%#llux lab=%ld\n", nh, k, (vl
        iqh[nh].label = pc;
        nh++;
        swit1(iql, q-iql, def, vr[0]);
        i = j;
    }
    patch(hsb, pc);
        //if(0){for(int k=0; k<nh; k++)print("k*=%d h=%#llux lab=%ld\n", k, (vlong)iqh[k].val,  iqh[k].label);}
    swit1(iqh, nh, def, vr[1]);
}
```

⟨*function* casf 232a⟩≡ (455a)

```
void
casf(void)
{
    Case *c;

    c = alloc(sizeof(*c));
    c->link = cases;
    cases = c;
}
```

⟨*function* swit1*(arm)* 232b⟩≡ (464b)

```
void
swit1(C1 *q, int nc, long def, Node *n)
{
    Node tn;

    regalloc(&tn, &regnode, Z);
    swit2(q, nc, def, n, &tn);
    regfree(&tn);
}
```

⟨*function* swit2*(arm)* 232c⟩≡ (464b)

```
void
swit2(C1 *q, int nc, long def, Node *n, Node *tn)
{
    C1 *r;
```

```
    int i;
    long v;
    Prog *sp;

    if(nc >= 3) {
        i = (q+nc-1)->val - (q+0)->val;
        if(i > 0 && i < nc*2)
            goto direct;
    }
    if(nc < 5) {
        for(i=0; i<nc; i++) {
            if(debug['K'])
                print("case = %.8llux\n", q->val);
            gopcode(OEQ, nodconst(q->val), n, Z);
            patch(p, q->label);
            q++;
        }
        gbranch(OGOTO);
        patch(p, def);

        return;
    }

    i = nc / 2;
    r = q+i;
    if(debug['K'])
        print("case > %.8llux\n", r->val);
    gopcode(OGT, nodconst(r->val), n, Z);
    sp = p;
    gopcode(OEQ, nodconst(r->val), n, Z); /* just gen the B.EQ */
    patch(p, r->label);
    swit2(q, i, def, n, tn);

    if(debug['K'])
        print("case < %.8llux\n", r->val);
    patch(sp, pc);
    swit2(r+1, nc-i-1, def, n, tn);
    return;

direct:
    v = q->val;
    if(v != 0)
        gopcode(OSUB, nodconst(v), Z, n);
    gopcode(OCASE, nodconst((q+nc-1)->val - v), n, Z);
    patch(p, def);
    for(i=0; i<nc; i++) {
        if(debug['K'])
            print("case = %.8llux\n", q->val);
        while(q->val != v) {
            nextpc();
            p->as = ABCASE;
            patch(p, def);
            v++;
        }
        nextpc();
        p->as = ABCASE;
        patch(p, q->label);
        q++;
        v++;
    }
```

```
    gbranch(OGOTO);  /* so that regopt() won't be confused */
    patch(p, def);
}
```

Chapter 11

Object File Generation

11.1 Object format

11.2 Instruction output: outcode()

⟨*struct* Htab 235a⟩≡ (464b)

```
struct Htab {
    Sym *sym;
    short type;
};
```

⟨Sym *identifier value, code generation fields* 235b⟩+≡ (23b) ◁158g 333b▷

```
// index in h when the Sym is really a symbol, 0 when not a symbol
char symidx;
```

⟨*function* outcode*(arm)* 235c⟩≡ (464b)

```
/// main -> compile -> yyparse; gclean -> <>
void
outcode(void)
{
    struct Htab h[NSYM];
    Prog *p;
    Sym *s;
    int sf, st, t, sym;

    ⟨outcode() debug 417h⟩
    outhist(&outbuf);
    for(sym=0; sym<NSYM; sym++) {
        h[sym].sym = S;
        h[sym].type = 0;
    }
    sym = 1;
    for(p = firstp; p != P; p = p->link) {
    jackpot:
        sf = 0;
        s = p->from.sym;
        while(s != S) {
            sf = s->symidx;
            if(sf < 0 || sf >= NSYM)
                sf = 0;
            t = p->from.symkind;
            if(h[sf].type == t)
            if(h[sf].sym == s)
                break;
```

```
            s->symidx = sym;
            zname(&outbuf, s, t);
            h[sym].sym = s;
            h[sym].type = t;
            sf = sym;
            sym++;
            if(sym >= NSYM)
                sym = 1;
            break;
        }
        st = 0;
        s = p->to.sym;
        while(s != S) {
            st = s->symidx;
            if(st < 0 || st >= NSYM)
                st = 0;
            t = p->to.symkind;
            if(h[st].type == t)
            if(h[st].sym == s)
                break;
            s->symidx = sym;
            zname(&outbuf, s, t);
            h[sym].sym = s;
            h[sym].type = t;
            st = sym;
            sym++;
            if(sym >= NSYM)
                sym = 1;
            if(st == sf)
                goto jackpot;
            break;
        }
        zwrite(&outbuf, p, sf, st);
    }
    firstp = P;
    lastp = P;
}
```

⟨*function* `zwrite`*(arm)* 236a⟩≡ (464b)

```
void
zwrite(Biobuf *b, Prog *p, int sf, int st)
{
    char bf[100], *bp;

    bf[0] = p->as;
    bf[1] = p->scond;
    bf[2] = p->reg;
    bf[3] = p->lineno;
    bf[4] = p->lineno>>8;
    bf[5] = p->lineno>>16;
    bf[6] = p->lineno>>24;
    bp = zaddr(bf+7, &p->from, sf);
    bp = zaddr(bp, &p->to, st);
    Bwrite(b, bf, bp-bf);
}
```

11.3 Operand output: `zaddr()`

⟨*function* `zaddr`*(arm)* 236b⟩≡ (464b)

```
char*
zaddr(char *bp, Adr *a, int s)
{
    long l;
    Ieee e;

    bp[0] = a->type;
    bp[1] = a->reg;
    bp[2] = s;
    bp[3] = a->symkind;
    bp += 4;

    //if(a->type == D_CONST && a->symkind != N_NONE) {
    //    diag(Z, "missing D_CONST -> D_ADDR migration");
    //}
    //if(a->type == D_ADDR && a->symkind == N_NONE) {
    //    diag(Z, "wrong D_CONST -> D_ADDR migration");
    //}

    switch(a->type) {
    case D_NONE:
    case D_REG:
    case D_FREG:
    case D_PSR:
        break;

    case D_OREG:
    case D_CONST:
    case D_ADDR:
    case D_BRANCH:
    case D_SHIFT:
        l = a->offset;
        bp[0] = l;
        bp[1] = l>>8;
        bp[2] = l>>16;
        bp[3] = l>>24;
        bp += 4;
        break;

    case D_SCONST:
        memmove(bp, a->sval, NSNAME);
        bp += NSNAME;
        break;

    case D_FCONST:
        ieeedtod(&e, a->dval);
        l = e.l;
        bp[0] = l;
        bp[1] = l>>8;
        bp[2] = l>>16;
        bp[3] = l>>24;
        bp += 4;
        l = e.h;
        bp[0] = l;
        bp[1] = l>>8;
        bp[2] = l>>16;
        bp[3] = l>>24;
        bp += 4;
        break;
    default:
```

```
        diag(Z, "unknown type %d in zaddr", a->type);

    }
    return bp;
}
```

11.4 Symbol table, zname(), h and ANAME

⟨*function* zname*(arm)* 238a⟩≡ (464b)

```
void
zname(Biobuf *b, Sym *s, int t)
{
    char *n, bf[7];
    ulong sig;

    n = s->name;
    ⟨zname() if generate signature for symbol s 333c⟩
    else{
        bf[0] = ANAME;
        bf[1] = t; /* type */
        bf[2] = s->symidx; /* sym */
        Bwrite(b, bf, 3);
    }
    Bwrite(b, n, strlen(n)+1);
}
```

11.5 File and line information: outhist()

⟨*function* outhist*(arm)* 238b⟩≡ (464b)

```
void
outhist(Biobuf *b)
{
    Hist *h;
    char *p, *q, *op, c;
    Prog pg;
    int n;

    pg = zprog;
    pg.as = AHISTORY;
    c = pathchar();
    for(h = hist; h != H; h = h->link) {
        p = h->name;
        op = 0;
        /* on windows skip drive specifier in pathname */
        //if(systemtype(Windows) && p && p[1] == ':'){
        // p += 2;
        // c = *p;
        //}
        if(p && p[0] != c && h->offset == 0 && pathname){
            /* on windows skip drive specifier in pathname */
            //if(systemtype(Windows) && pathname[1] == ':') {
            // op = p;
            // p = pathname+2;
            // c = *p;
            //} else
            if(pathname[0] == c){
```

```
                op = p;
                p = pathname;
            }
        }
        while(p) {
            q = utfrune(p, c);
            if(q) {
                n = q-p;
                if(n == 0){
                    n = 1; /* leading "/" */
                    *p = '/'; /* don't emit "\" on windows */
                }
                q++;
            } else {
                n = strlen(p);
                q = 0;
            }
            if(n) {
                Bputc(b, ANAME);
                Bputc(b, N_FILE);
                Bputc(b, 1);
                Bputc(b, '<');
                Bwrite(b, p, n);
                Bputc(b, '\0');
            }
            p = q;
            if(p == 0 && op) {
                p = op;
                op = 0;
            }
        }
        pg.lineno = h->line;
        pg.to.type = zprog.to.type;
        pg.to.offset = h->offset;
        if(h->offset)
            pg.to.type = D_CONST;

        zwrite(b, &pg, 0, 0);
    }
}
```

Chapter 12

AST-level Optimizations

12.1 AST simplifications

12.1.1 General rewrites: ccom()

⟨*function* ccom 240a⟩≡ (454)

```
/// complex -> <>
/*
 * general rewrite
 * (IND(ADDR x)) ==> x
 * (ADDR(IND x)) ==> x
 * remove some zero operands
 * remove no op casts
 * evaluate constants // important one!
 */
void
ccom(Node *n)
{
    Node *l, *r;
    int t;

loop:
    if(n == Z)
        return;

    l = n->left;
    r = n->right;
    switch(n->op) {
    ⟨ccom() switch node kind cases 153c⟩
    default:
        // recurse
        if(l != Z)
            ccom(l);
        if(r != Z)
            ccom(r);
        // Fallthrough
    common:
        if(((l == Z)||(l->op == OCONST)) && ((r == Z)||(r->op == OCONST)))
            evconst(n);
    }
}
```

⟨ccom() *switch node kind cases* 240b⟩+≡ (240a) ◁153c 241a▷

```
case OCONST:
case ONAME:
```

```
    break;
```

⟨ccom() *switch node kind cases* 241a⟩+≡ (240a) ◁240b 241b▷

```
  case OREGISTER:
  case OINDREG:
      break;
```

No op casts

⟨ccom() *switch node kind cases* 241b⟩+≡ (240a) ◁241a 241c▷

```
  case OCAST:
      ccom(l);
      if(l->op == OCONST) {
          evconst(n);
          if(n->op == OCONST)
              break;
      }
      if(nocast(l->type, n->type) &&
         (!typefd[l->type->etype] ||
         typeu[l->type->etype] && typeu[n->type->etype]) ) {
          l->type = n->type;
          *n = *l;
      }
      break;
```

Pointers and addresses

⟨ccom() *switch node kind cases* 241c⟩+≡ (240a) ◁241b 241d▷

```
  case OADDR:
      ccom(l);
      l->etype = TVOID; // ????
      if(l->op == OIND) {
          l->left->type = n->type;
          *n = *l->left;
          break;
      }
      goto common;
```

⟨ccom() *switch node kind cases* 241d⟩+≡ (240a) ◁241c 241e▷

```
  case OIND:
      ccom(l);
      if(l->op == OADDR) {
          l->left->type = n->type;
          *n = *l->left;
          break;
      }
      goto common;
```

Zero operands

⟨ccom() *switch node kind cases* 241e⟩+≡ (240a) ◁241d 242a▷

```
  case OADD:
  case OOR:
  case OXOR:
      ccom(l);
      if(vconst(l) == 0) {
          *n = *r;
          goto loop;
```

```
    }
    ccom(r);
    if(vconst(r) == 0) {
        *n = *l;
        break;
    }
    goto commute;
```

⟨ccom() *switch node kind cases* 242a⟩+≡ (240a) ◁241e 242b▷

```
  commute:
    /* look for commutative constant */
    if(r->op == OCONST) {
        if(l->op == n->op) {
            if(l->left->op == OCONST) {
                n->right = l->right;
                l->right = r;
                goto loop;
            }
            if(l->right->op == OCONST) {
                n->right = l->left;
                l->left = r;
                goto loop;
            }
        }
    }
    if(l->op == OCONST) {
        if(r->op == n->op) {
            if(r->left->op == OCONST) {
                n->left = r->right;
                r->right = l;
                goto loop;
            }
            if(r->right->op == OCONST) {
                n->left = r->left;
                r->left = l;
                goto loop;
            }
        }
    }
    goto common;
```

⟨ccom() *switch node kind cases* 242b⟩+≡ (240a) ◁242a 243▷

```
  case OAND:
    ccom(l);
    ccom(r);
    if(vconst(l) == 0 && !side(r)) {
        *n = *l;
        break;
    }
    if(vconst(r) == 0 && !side(l)) {
        *n = *r;
        break;
    }
    // Fallthrough, goto commute
```

⟨*function* side 242c⟩≡ (443b)

```
  /// ccom -> <>
  bool
  side(Node *n)
  {
```

```
loop:
    if(n != Z)
    switch(n->op) {
    case OCAST:
    case ONOT:
    case OADDR:
    case OIND:
        n = n->left;
        goto loop;

    case OCOND:
        if(side(n->left))
            break;
        n = n->right;

    case OEQ:
    case ONE:
    case OLT:
    case OGE:
    case OGT:
    case OLE:
    case OADD:
    case OSUB:
    case OMUL:
    case OLMUL:
    case ODIV:
    case OLDIV:
    case OLSHR:
    case OASHL:
    case OASHR:
    case OAND:
    case OOR:
    case OXOR:
    case OMOD:
    case OLMOD:
    case OANDAND:
    case OOROR:
    case OCOMMA:
    case ODOT:
        if(side(n->left))
            break;
        n = n->right;
        goto loop;

    case OSIGN:
    case OSIZE:
    case OCONST:
    case OSTRING:
    case OLSTRING:
    case ONAME:
        return false;
    }
    // OFUNC/OAS/OASADD/...
    return true;
}
```

⟨ccom() *switch node kind cases* 243⟩+≡ (240a) ◁242b 244a▷

```
case OASHR:
case OASHL:
```

```
case OLSHR:
    ccom(l);
    if(vconst(l) == 0 && !side(r)) {
        *n = *l;
        break;
    }
    ccom(r);
    if(vconst(r) == 0) {
        *n = *l;
        break;
    }
    ⟨ccom() when OASHR/OASHL/OLSHR, check for stupid shift 153b⟩
    goto common;
```

⟨ccom() *switch node kind cases* 244a⟩+≡ (240a) ◁243 244b▷

```
case OMUL:
case OLMUL:
    ccom(l);
    t = vconst(l);
    if(t == 0 && !side(r)) {
        *n = *l;
        break;
    }
    if(t == 1) {
        *n = *r;
        goto loop;
    }
    ccom(r);
    t = vconst(r);
    if(t == 0 && !side(l)) {
        *n = *r;
        break;
    }
    if(t == 1) {
        *n = *l;
        break;
    }
    goto common;
```

⟨ccom() *switch node kind cases* 244b⟩+≡ (240a) ◁244a 245a▷

```
case ODIV:
case OLDIV:
    ccom(l);
    if(vconst(l) == 0 && !side(r)) {
        *n = *l;
        break;
    }
    ccom(r);
    t = vconst(r);
    if(t == 0) {
        diag(n, "divide check");
        *n = *r;
        break;
    }
    if(t == 1) {
        *n = *l;
        break;
    }
    goto common;
```

Sub to Add

⟨ccom() *switch node kind cases* 245a⟩+≡ (240a) ◁244b 245b▷

```
case OSUB:
    ccom(r);
    if(r->op == OCONST) {
        if(typefd[r->type->etype]) {
            n->op = OADD;
            r->fconst = -r->fconst;
            goto loop;
        } else {
            n->op = OADD;
            r->vconst = -r->vconst;
            goto loop;
        }
    }
    ccom(l);
    goto common;
```

Comparisons

⟨ccom() *switch node kind cases* 245b⟩+≡ (240a) ◁245a 250a▷

```
case OEQ:
case ONE:

case OLE:
case OGE:
case OLT:
case OGT:

case OLS:
case OHS:
case OLO:
case OHI:
    ccom(l);
    ccom(r);
    if(compar(n, 0) || compar(n, 1))
        break;
    relcon(l, r);
    relcon(r, l);
    goto common;
```

⟨*function* relcon 245c⟩≡ (443b)

```
/*
 * try to cast a constant down
 * rather than cast a variable up
 * example:
 * if(c == 'a')
 */
void
relcon(Node *l, Node *r)
{
    vlong v;

    if(l->op != OCONST)
        return;
    if(r->op != OCAST)
        return;
    if(!nilcast(r->left->type, r->type))
        return;
```

```
    switch(r->type->etype) {
    default:
        return;
    case TCHAR:
    case TUCHAR:
    case TSHORT:
    case TUSHORT:
        v = convvtox(l->vconst, r->type->etype);
        if(v != l->vconst)
            return;
        break;
    }
    l->type = r->left->type;
    *r = *r->left;
}
```

⟨*function* nilcast 246a⟩≡ (443b)

```
/*
 * a cast that has a noop semantic
 * (small to large, convert)
 */
bool
nilcast(Type *t1, Type *t2)
{
    int et1, et2;

    if(t1 == T)
        return false;
    if(t1->nbits)
        return false;
    if(t2 == T)
        return false;
    et1 = t1->etype;
    et2 = t2->etype;
    if(et1 == et2)
        return true;
    if(typefd[et1] && typefd[et2]) {
        if(ewidth[et1] < ewidth[et2])
            return true;
        return false;
    }
    if(typechlp[et1] && typechlp[et2]) {
        if(ewidth[et1] < ewidth[et2])
            return true;
        return false;
    }
    return false;
}
```

⟨*struct* Big 246b⟩≡ (454)

```
/* 128-bit numbers */
struct Big
{
    vlong a;
    uvlong b;
};
```

⟨*function* big 246c⟩≡ (454)

```
Big
big(vlong a, uvlong b)
```

```
{
    Big x;

    x.a = a;
    x.b = b;
    return x;
}
```

⟨*function* cmp 247a⟩≡ (454)

```
static int
cmp(Big x, Big y)
{
    if(x.a != y.a){
        if(x.a < y.a)
            return -1;
        return 1;
    }
    if(x.b != y.b){
        if(x.b < y.b)
            return -1;
        return 1;
    }
    return 0;
}
```

⟨*function* add 247b⟩≡ (454)

```
static Big
add(Big x, int y)
{
    uvlong ob;

    ob = x.b;
    x.b += y;
    if(y > 0 && x.b < ob)
        x.a++;
    if(y < 0 && x.b > ob)
        x.a--;
    return x;
}
```

⟨*after parsing nodes* 247c⟩+≡ (16a) ◁190f

```
OHI,
OHS,
OLO,
OLS,
```

⟨*global* cmps 247d⟩≡ (454)

```
/* OEQ, ONE, OLE, OLS, OLT, OLO, OGE, OHS, OGT, OHI */
static char *cmps[12] =
{
    "==", "!=", "<=", "<=", "<", "<", ">=", ">=", ">", ">",
};
```

⟨*function* compar 247e⟩≡ (454)

```
int
compar(Node *n, int reverse)
{
    Big lo, hi, x;
    int op;
    char xbuf[40], cmpbuf[50];
```

```
Node *l, *r;
Type *lt, *rt;

/*
 * The point of this function is to diagnose comparisons
 * that can never be true or that look misleading because
 * of the 'usual arithmetic conversions'.  As an example
 * of the latter, if x is a ulong, then if(x <= -1) really means
 * if(x <= 0xFFFFFFFF), while if(x <= -1LL) really means
 * what it says (but 8c compiles it wrong anyway).
 */

if(reverse){
    r = n->left;
    l = n->right;
    op = comrel[relindex(n->op)];
}else{
    l = n->left;
    r = n->right;
    op = n->op;
}

/*
 * Skip over left casts to find out the original expression range.
 */
while(l->op == OCAST)
    l = l->left;
if(l->op == OCONST)
    return 0;

lt = l->type;
if(l->op == ONAME && l->sym->type){
    lt = l->sym->type;
    if(lt->etype == TARRAY)
        lt = lt->link;
}
if(lt == T)
    return 0;
if(lt->etype == TXXX || lt->etype > TUVLONG)
    return 0;

/*
 * Skip over the right casts to find the on-screen value.
 */
if(r->op != OCONST)
    return 0;
while(r->oldop == OCAST && !r->xcast)
    r = r->left;
rt = r->type;
if(rt == T)
    return 0;

x.b = r->vconst;
x.a = 0;
if((rt->etype&1) && r->vconst < 0) /* signed negative */
    x.a = ~0ULL;

if((lt->etype&1)==0){
    /* unsigned */
    lo = big(0, 0);
```

```
        if(lt->width == 8)
            hi = big(0, ~0ULL);
        else
            hi = big(0, (1LL<<(l->type->width*8))-1);
    }else{
        lo = big(~0ULL, -(1LL<<(l->type->width*8-1)));
        hi = big(0, (1LL<<(l->type->width*8-1))-1);
    }

    switch(op){
    case OLT:
    case OLO:
    case OGE:
    case OHS:
        if(cmp(x, lo) <= 0)
            goto useless;
        if(cmp(x, add(hi, 1)) >= 0)
            goto useless;
        break;
    case OLE:
    case OLS:
    case OGT:
    case OHI:
        if(cmp(x, add(lo, -1)) <= 0)
            goto useless;
        if(cmp(x, hi) >= 0)
            goto useless;
        break;
    case OEQ:
    case ONE:
        /*
         * Don't warn about comparisons if the expression
         * is as wide as the value: the compiler-supplied casts
         * will make both outcomes possible.
         */
        if(lt->width >= rt->width && debug['w'] < 2)
            return 0;
        if(cmp(x, lo) < 0 || cmp(x, hi) > 0)
            goto useless;
        break;
    }
    return 0;

useless:
    if((x.a==0 && x.b<=9) || (x.a==~0LL && x.b >= -9ULL))
        snprint(xbuf, sizeof xbuf, "%lld", x.b);
    else if(x.a == 0)
        snprint(xbuf, sizeof xbuf, "%#llux", x.b);
    else
        snprint(xbuf, sizeof xbuf, "%#llx", x.b);
    if(reverse)
        snprint(cmpbuf, sizeof cmpbuf, "%s %s %T",
            xbuf, cmps[relindex(n->op)], lt);
    else
        snprint(cmpbuf, sizeof cmpbuf, "%T %s %s",
            lt, cmps[relindex(n->op)], xbuf);
    if(debug['y'])
        prtree(n, "strange");
    warn(n, "useless or misleading comparison: %s", cmpbuf);
    return 0;
```

```
}
```

Boolean expressions

⟨ccom() *switch node kind cases* 250a⟩+≡ (240a) ◁245b 250b▷

```
case OANDAND:
    ccom(l);
    if(vconst(l) == 0) {
        *n = *l;
        break;
    }
    ccom(r);
    goto common;
```

⟨ccom() *switch node kind cases* 250b⟩+≡ (240a) ◁250a 250c▷

```
case OOROR:
    ccom(l);
    if(l->op == OCONST && l->vconst != 0) {
        *n = *l;
        n->vconst = 1;
        break;
    }
    ccom(r);
    goto common;
```

⟨ccom() *switch node kind cases* 250c⟩+≡ (240a) ◁250b

```
case OCOND:
    ccom(l);
    ccom(r);
    if(l->op == OCONST)
        if(vconst(l) == 0)
            *n = *r->right;
        else
            *n = *r->left;
    break;
```

12.1.2 Constant evaluation: `evconst()`

⟨*function* evconst 250d⟩≡ (452b)

```
/// gen | ... -> complex -> ccom -> <>
void
evconst(Node *n)
{
    Node *l, *r;
    int et;
    bool isf;
    vlong v = 0;
    double d = 0;

    if(n == Z || n->type == T)
        return;

    et = n->type->etype;
    isf = typefd[et];

    l = n->left;
    r = n->right;
```

```
    switch(n->op) {
    ⟨evconst() switch node kind cases 251b⟩
    default:
        return;
    }
    // reach this point if the node is indeed a candidate to "constification"
    if(isf)
        n->fconst = d;
    else
        n->vconst = convvtox(v, n->type->etype);
    n->oldop = n->op;
    n->op = OCONST;
}
```

⟨Node *origin tracking fields* 251a⟩+≡ (16b) ◁91f

```
// enum<node_kind>
char oldop;
```

⟨evconst() *switch node kind cases* 251b⟩≡ (250d) 251c▷

```
case OCONST:
    break;
```

⟨evconst() *switch node kind cases* 251c⟩+≡ (250d) ◁251b 251d▷

```
case OCAST:
    if(et == TVOID)
        return;
    et = l->type->etype;
    if(isf) {
        if(typefd[et])
            d = l->fconst;
        else
            d = l->vconst;
    } else {
        if(typefd[et])
            v = l->fconst;
        else
            v = convvtox(l->vconst, n->type->etype);
    }
    break;
```

⟨evconst() *switch node kind cases* 251d⟩+≡ (250d) ◁251c 252a▷

```
case OADD:
    if(isf)
        d = l->fconst + r->fconst;
    else
        v = l->vconst + r->vconst;
    break;

case OSUB:
    if(isf)
        d = l->fconst - r->fconst;
    else
        v = l->vconst - r->vconst;
    break;

case OMUL:
    if(isf)
        d = l->fconst * r->fconst;
    else
        v = l->vconst * r->vconst;
```

```
        break;

    case ODIV:
        if(vconst(r) == 0) {
            warn(n, "divide by zero");
            return;
        }
        if(isf)
            d = l->fconst / r->fconst;
        else
            v = l->vconst / r->vconst;
        break;

    case OMOD:
        if(vconst(r) == 0) {
            warn(n, "modulo by zero");
            return;
        }
        v = l->vconst % r->vconst;
        break;
```

⟨evconst() *switch node kind cases* 252a⟩+≡ (250d) ◁251d 252b▷

```
    case OLMUL:
        v = (uvlong)l->vconst * (uvlong)r->vconst;
        break;

    case OLDIV:
        if(vconst(r) == 0) {
            warn(n, "divide by zero");
            return;
        }
        v = (uvlong)l->vconst / (uvlong)r->vconst;
        break;

    case OLMOD:
        if(vconst(r) == 0) {
            warn(n, "modulo by zero");
            return;
        }
        v = (uvlong)l->vconst % (uvlong)r->vconst;
        break;
```

⟨evconst() *switch node kind cases* 252b⟩+≡ (250d) ◁252a 252c▷

```
    case ONEG:
        if(isf)
            d = -l->fconst;
        else
            v = -l->vconst;
        break;

    case OCOM:
        v = ~l->vconst;
        break;
```

⟨evconst() *switch node kind cases* 252c⟩+≡ (250d) ◁252b 253a▷

```
    case OAND:
        v = l->vconst & r->vconst;
        break;

    case OOR:
```

```
        v = l->vconst | r->vconst;
        break;

    case OXOR:
        v = l->vconst ^ r->vconst;
        break;
```

⟨evconst() *switch node kind cases* 253a⟩+≡ (250d) ◁252c 253b▷

```
    case OLSHR:
        v = (uvlong)l->vconst >> r->vconst;
        break;

    case OASHR:
        v = l->vconst >> r->vconst;
        break;

    case OASHL:
        v = l->vconst << r->vconst;
        break;
```

⟨evconst() *switch node kind cases* 253b⟩+≡ (250d) ◁253a 253c▷

```
    case OLT:
        if(typefd[l->type->etype])
            v = l->fconst < r->fconst;
        else
            v = l->vconst < r->vconst;
        break;

    case OGT:
        if(typefd[l->type->etype])
            v = l->fconst > r->fconst;
        else
            v = l->vconst > r->vconst;
        break;

    case OLE:
        if(typefd[l->type->etype])
            v = l->fconst <= r->fconst;
        else
            v = l->vconst <= r->vconst;
        break;

    case OGE:
        if(typefd[l->type->etype])
            v = l->fconst >= r->fconst;
        else
            v = l->vconst >= r->vconst;
        break;
```

⟨evconst() *switch node kind cases* 253c⟩+≡ (250d) ◁253b 254a▷

```
    case OLO:
        v = (uvlong)l->vconst < (uvlong)r->vconst;
        break;

    case OHI:
        v = (uvlong)l->vconst > (uvlong)r->vconst;
        break;

    case OLS:
        v = (uvlong)l->vconst <= (uvlong)r->vconst;
```

```
        break;

    case OHS:
        v = (uvlong)l->vconst >= (uvlong)r->vconst;
        break;
```

⟨evconst() *switch node kind cases* 254a⟩+≡ (250d) ◁253c 254b▷

```
    case OEQ:
        if(typefd[l->type->etype])
            v = l->fconst == r->fconst;
        else
            v = l->vconst == r->vconst;
        break;

    case ONE:
        if(typefd[l->type->etype])
            v = l->fconst != r->fconst;
        else
            v = l->vconst != r->vconst;
        break;
```

⟨evconst() *switch node kind cases* 254b⟩+≡ (250d) ◁254a

```
    case OANDAND:
        if(typefd[l->type->etype])
            v = l->fconst && r->fconst;
        else
            v = l->vconst && r->vconst;
        break;

    case OOROR:
        if(typefd[l->type->etype])
            v = l->fconst || r->fconst;
        else
            v = l->vconst || r->vconst;
        break;
    case ONOT:
        if(typefd[l->type->etype])
            v = !l->fconst;
        else
            v = !l->vconst;
        break;
```

12.2 Comma hoisting: comma()

⟨*struct* Com 254c⟩≡ (454)

```
    struct Com
    {
        int n;
        Node *t[500];
    };
```

⟨*function* comma 254d⟩≡ (454)

```
    /// complex -> <>
    static void
    comma(Node *n)
    {
        Com com;
        Node *nn;
```

```
    com.n = 0;
    nn = commas(&com, n);

    if(com.n > 0){

if(debug['y'])print("n=%d\n", com.n);
if(debug['y']) prtree(nn, "res");

        if(nn != n)
            *n = *nn;
        while(com.n > 0){

if(debug['y']) prtree(com.t[com.n-1], "tree");

            nn = new1(OXXX, Z, Z);
            *nn = *n;
            n->op = OCOMMA;
            n->type = nn->type;
            n->left = com.t[--com.n];
            n->right = nn;
            n->lineno = n->left->lineno;
        }
if(debug['y']) prtree(n, "final");

    }else if(n != nn)
        fatal(n, "odd tree");
}
```

⟨*function* commas 255⟩≡ (454)

```
static Node*
commas(Com *com, Node *n)
{
    Node *t;

    if(n == Z)
        return n;
    switch(n->op){
    case OREGISTER:
    case OINDREG:
    case OCONST:
    case ONAME:
    case OSTRING:
        /* leaf */
        return n;

    case OCOMMA:
        t = commas(com, n->left);
        if(com->n >= nelem(com->t))
            fatal(n, "comma list overflow");
        com->t[com->n++] = t;
        return commas(com, n->right);

    case OFUNC:
        n->left = commas(com, n->left);
        n->right = comargs(com, n->right);
        return n;

    case OCOND:
        n->left = commas(com, n->left);
```

```
        comma(n->right->left);
        comma(n->right->right);
        return n;

    case OANDAND:
    case OOROR:
        n->left = commas(com, n->left);
        comma(n->right);
        return n;

    case ORETURN:
        comma(n->left);
        return n;
    }
    n->left = commas(com, n->left);
    if(n->right != Z)
        n->right = commas(com, n->right);
    return n;
}
```

⟨*function* comargs 256a⟩≡ (454)

```
/*
 * hoist comma operators out of expressions
 * (a,b) OP c => (a, b OP c)
 * OP(a,b) => (a, OP b)
 * a OP (b,c) => (b, a OP c)
 */

static Node*
comargs(Com *com, Node *n)
{
    if(n != Z && n->op == OLIST){
        n->left = comargs(com, n->left);
        n->right = comargs(com, n->right);
    }
    return commas(com, n);
}
```

12.3 Bitshifting opportunities

⟨*function* vlog 256b⟩≡ (443b)

```
/// arith -> <>
int
vlog(Node *n)
{
    if(n->op != OCONST)
        goto bad;
    if(typefd[n->type->etype])
        goto bad;

    return log2(n->vconst);

bad:
    return -1;
}
```

⟨arith() *when substract two pointers, convert DIV to SHR if can* 256c⟩≡ (130d)

```
w = vlog(n1);
```

```
if(w >= 0) {
    n->op = OASHR;
    n1->vconst = w;
}
```

⟨xcom() *switch op cases to set addable* 257a⟩+≡ (168c) ◁170d 257b▷

```
case OMUL:
case OLMUL:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OASHL;
        r->vconst = t;
        r->type = types[TINT];
    }
    t = vlog(l);
    if(t >= 0) {
        n->op = OASHL;
        n->left = r;
        n->right = l;
        r = l;
        l = n->left;
        r->vconst = t;
        r->type = types[TINT];
    }
    break;
```

⟨xcom() *switch op cases to set addable* 257b⟩+≡ (168c) ◁257a 257c▷

```
case OASLMUL:
case OASMUL:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OASASHL;
        r->vconst = t;
        r->type = types[TINT];
    }
    break;
```

⟨xcom() *switch op cases to set addable* 257c⟩+≡ (168c) ◁257b 257d▷

```
case OLDIV:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OLSHR;
        r->vconst = t;
        r->type = types[TINT];
    }
    break;
```

⟨xcom() *switch op cases to set addable* 257d⟩+≡ (168c) ◁257c 258b▷

```
case OASLDIV:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OASLSHR;
```

```
        r->vconst = t;
        r->type = types[TINT];
    }
    break;
```

⟨cgenrel() *when ODIV/OMOD, convert to SHR if possible* 258a⟩≡ (184a)

```
 if(nn != Z)
  if((t = vlog(r)) >= 0) {
    /* signed div/mod by constant power of 2 */
    cgen(l, nn);
    gopcode(OGE, nodconst(0), nn, Z);
    p1 = p;
    if(o == ODIV) {
        gopcode(OADD, nodconst((1<<t)-1), Z, nn);
        patch(p1, pc);
        gopcode(OASHR, nodconst(t), Z, nn);
    } else {
        gopcode(OSUB, nn, nodconst(0), nn);
        gopcode(OAND, nodconst((1<<t)-1), Z, nn);
        gopcode(OSUB, nn, nodconst(0), nn);
        gbranch(OGOTO);
        patch(p1, pc);
        p1 = p;
        gopcode(OAND, nodconst((1<<t)-1), Z, nn);
        patch(p1, pc);
    }
    break;
 }
```

12.4 And opportunities

⟨xcom() *switch op cases to set addable* 258b⟩+≡ (168c) ◁257d 258c▷

```
 case OLMOD:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OAND;
        r->vconst--;
    }
    break;
```

⟨xcom() *switch op cases to set addable* 258c⟩+≡ (168c) ◁258b

```
 case OASLMOD:
    xcom(l);
    xcom(r);
    t = vlog(r);
    if(t >= 0) {
        n->op = OASAND;
        r->vconst--;
    }
    break;
```

12.5 Immediate operands

12.5.1 Substraction

⟨cgenrel() *when OSUB, if l is OCONST* 258d⟩≡ (183c)

```
if(nn != Z)
 if(l->op == OCONST)
  if(!typefd[n->type->etype]) {
    cgen(r, nn);
    gopcode(o, Z, l, nn);
    break;
}
```

12.5.2 Addition, or, etc.

⟨cgenrel() *when OADD/OAND/etc, if r is OCONST* 259a⟩≡ (183c)

```
/*
 * immediate operands
 */
if(nn != Z)
 if(r->op == OCONST)
  if(!typefd[n->type->etype]) {
    cgen(l, nn);
    if((r->vconst == 0) && (o != OAND))
        break;
    if(nn != Z)
        gopcode(o, r, Z, nn);
    break;
}
```

⟨xcom() *switch node opkind cases to set complexity* 259b⟩+≡ (170e) ◁171a

```
case OADD:
case OAND:
case OOR:
case OXOR:
case OEQ:
case ONE:
    /*
     * immediate operators, make const on right
     */
    if(l->op == OCONST) {
        n->left = r;
        n->right = l;
    }
    break;
```

12.5.3 Multiplication

⟨cgenrel() *when OMUL/OLMUL, mulcon opportunity* 259c⟩≡ (183c)

```
if(o == OMUL || o == OLMUL) {
    if(mulcon(n, nn))
        break;
}
```

⟨*function* mulcon*(arm)* 259d⟩≡ (464b)

```
/// cgen -> cgenrel -> <>
bool
mulcon(Node *n, Node *nn)
{
    Node *l, *r, nod1, nod2;
    Multab *m;
    long v, vs;
    int o;
```

```
    char code[sizeof(m->code)+2], *p;

    if(typefd[n->type->etype])
        return false;

    l = n->left;
    r = n->right;
    if(l->op == OCONST) {
        l = r;
        r = n->left;
    }
    if(r->op != OCONST)
        return false;


    v = convvtox(r->vconst, n->type->etype);
    if(v != r->vconst) {
        if(debug['M'])
            print("%L multiply conv: %lld\n", n->lineno, r->vconst);
        return false;
    }
    m = mulcon0(v);
    if(!m) {
        if(debug['M'])
            print("%L multiply table: %lld\n", n->lineno, r->vconst);
        return false;
    }
    if(debug['M'] && debug['v'])
        print("%L multiply: %ld\n", n->lineno, v);

    memmove(code, m->code, sizeof(m->code));
    code[sizeof(m->code)] = 0;

    p = code;
    if(p[1] == 'i')
        p += 2;
    regalloc(&nod1, n, nn);
    cgen(l, &nod1);
    vs = v;
    regalloc(&nod2, n, Z);

loop:
    switch(*p) {
    case 0:
        regfree(&nod2);
        if(vs < 0) {
            gopcode(OAS, &nod1, Z, &nod1);
            gopcode(OSUB, &nod1, nodconst(0), nn);
        } else
            gopcode(OAS, &nod1, Z, nn);
        regfree(&nod1);
        return true;
    case '+':
        o = OADD;
        goto addsub;
    case '-':
        o = OSUB;
    addsub: /* number is r,n,l */
        v = p[1] - '0';
        r = &nod1;
```

```
        if(v&4)
            r = &nod2;
        n = &nod1;
        if(v&2)
            n = &nod2;
        l = &nod1;
        if(v&1)
            l = &nod2;
        gopcode(o, l, n, r);
        break;
    default: /* op is shiftcount, number is r,l */
        v = p[1] - '0';
        r = &nod1;
        if(v&2)
            r = &nod2;
        l = &nod1;
        if(v&1)
            l = &nod2;
        v = *p - 'a';
        if(v < 0 || v >= 32) {
            diag(n, "mulcon unknown op: %c%c", p[0], p[1]);
            break;
        }
        gopcode(OASHL, nodconst(v), l, r);
        break;
    }
    p += 2;
    goto loop;
}
```

⟨*struct* Multab*(arm)* 261a⟩≡ (456)

```
struct Multab
{
    long val;
    char code[20];
};
```

⟨*global* multab*(arm)* 261b⟩≡ (460b)

```
Multab multab[20];
```

⟨*global* maxmulops*(arm)* 261c⟩≡ (465)

```
static  int maxmulops = 3; /* max # of ops to replace mul with */
```

⟨*global* multabp*(arm)* 261d⟩≡ (465)

```
static int multabp;
```

⟨*global* mulval*(arm)* 261e⟩≡ (465)

```
static long mulval;
```

⟨*global* mulcp*(arm)* 261f⟩≡ (465)

```
static char* mulcp;
```

⟨*global* valmax*(arm)* 261g⟩≡ (465)

```
static long valmax;
```

⟨*global* shmax*(arm)* 261h⟩≡ (465)

```
static int shmax;
```

⟨*enum* _anon_ (5c/mul.c)*(arm)* 262a⟩≡ (465)

```
enum
{
    SR1 = 1<<0,  /* r1 has been shifted */
    SR0 = 1<<1,  /* r0 has been shifted */
    UR1 = 1<<2,  /* r1 has not been used */
    UR0 = 1<<3,  /* r0 has not been used */
};
```

⟨*struct* Hintab*(arm)* 262b⟩≡ (456)

```
struct Hintab
{
    ushort val;
    char hint[10];
};
```

⟨*function* mulcon0*(arm)* 262c⟩≡ (465)

```
Multab*
mulcon0(long v)
{
    int a1, a2, g;
    Multab *m, *m1;
    char hint[10];

    if(v < 0)
        v = -v;

    /*
     * look in cache
     */
    m = multab;
    for(g=0; g<nelem(multab); g++) {
        if(m->val == v) {
            if(m->code[0] == 0)
                return 0;
            return m;
        }
        m++;
    }

    /*
     * select a spot in cache to overwrite
     */
    multabp++;
    if(multabp < 0 || multabp >= nelem(multab))
        multabp = 0;
    m = multab+multabp;
    m->val = v;
    mulval = v;

    /*
     * look in execption hint table
     */
    a1 = 0;
    a2 = hintabsize;
    for(;;) {
        if(a1 >= a2)
            goto no;
        g = (a2 + a1)/2;
        if(v < hintab[g].val) {
```

```
            a2 = g;
            continue;
        }
        if(v > hintab[g].val) {
            a1 = g+1;
            continue;
        }
        break;
    }

    if(docode(hintab[g].hint, m->code, 1, 0))
        return m;
    print("multiply table failure %ld\n", v);
    m->code[0] = 0;
    return 0;

no:
    /*
     * try to search
     */
    hint[0] = 0;
    for(g=1; g<=maxmulops; g++) {
        if(g >= maxmulops && v >= 65535)
            break;
        mulcp = hint+g;
        *mulcp = 0;
        if(gen1(g)) {
            if(docode(hint, m->code, 1, 0))
                return m;
            print("multiply table failure %ld\n", v);
            break;
        }
    }

    /*
     * try a recur followed by a shift
     */
    g = 0;
    while(!(v & 1)) {
        g++;
        v >>= 1;
    }
    if(g) {
        m1 = mulcon0(v);
        if(m1) {
            strcpy(m->code, m1->code);
            sprint(strchr(m->code, 0), "%c0", g+'a');
            return m;
        }
    }
    m->code[0] = 0;
    return 0;
}
```

⟨*function* docode*(arm)* 263⟩≡ (465)

```
static int
docode(char *hp, char *cp, int r0, int r1)
{
    int c, i;
```

```
c = *hp++;
*cp = c;
cp += 2;
switch(c) {
default:
    c -= 'a';
    if(c < 1 || c >= 30)
        break;
    for(i=0; i<4; i++) {
        switch(i) {
        case 0:
            if(docode(hp, cp, r0<<c, r1))
                goto out;
            break;
        case 1:
            if(docode(hp, cp, r1<<c, r1))
                goto out;
            break;
        case 2:
            if(docode(hp, cp, r0, r0<<c))
                goto out;
            break;
        case 3:
            if(docode(hp, cp, r0, r1<<c))
                goto out;
            break;
        }
    }
    break;

case '+':
    for(i=0; i<8; i++) {
        cp[-1] = i+'0';
        switch(i) {
        case 1:
            if(docode(hp, cp, r0+r1, r1))
                goto out;
            break;
        case 5:
            if(docode(hp, cp, r0, r0+r1))
                goto out;
            break;
        }
    }
    break;

case '-':
    for(i=0; i<8; i++) {
        cp[-1] = i+'0';
        switch(i) {
        case 1:
            if(docode(hp, cp, r0-r1, r1))
                goto out;
            break;
        case 2:
            if(docode(hp, cp, r1-r0, r1))
                goto out;
            break;
        case 5:
            if(docode(hp, cp, r0, r0-r1))
```

```
                    goto out;
                break;
            case 6:
                if(docode(hp, cp, r0, r1-r0))
                    goto out;
                break;
            }
        }
        break;

    case 0:
        if(r0 == mulval)
            return 1;
    }
    return 0;

out:
    cp[-1] = i+'0';
    return 1;
}
```

⟨*function* gen1*(arm)* 265a⟩≡ (465)

```
static int
gen1(int len)
{
    int i;

    for(shmax=1; shmax<30; shmax++) {
        valmax = 1<<shmax;
        if(valmax >= mulval)
            break;
    }
    if(mulval == 1)
        return 1;

    len--;
    for(i=1; i<=shmax; i++)
        if(gen2(len, 1<<i)) {
            *--mulcp = 'a'+i;
            return 1;
        }
    return 0;
}
```

⟨*function* gen2*(arm)* 265b⟩≡ (465)

```
static int
gen2(int len, long r1)
{
    int i;

    if(len <= 0) {
        if(r1 == mulval)
            return 1;
        return 0;
    }

    len--;
    if(len == 0)
        goto calcr0;
```

```
    if(gen3(len, r1, r1+1, UR1)) {
        i = '+';
        goto out;
    }
    if(gen3(len, r1-1, r1, UR0)) {
        i = '-';
        goto out;
    }
    if(gen3(len, 1, r1+1, UR1)) {
        i = '+';
        goto out;
    }
    if(gen3(len, 1, r1-1, UR1)) {
        i = '-';
        goto out;
    }

    return 0;

calcr0:
    if(mulval == r1+1) {
        i = '+';
        goto out;
    }
    if(mulval == r1-1) {
        i = '-';
        goto out;
    }
    return 0;

out:
    *--mulcp = i;
    return 1;
}
```

⟨*function* gen3*(arm)* 266⟩≡ (465)

```
static int
gen3(int len, long r0, long r1, int flag)
{
    int i, f1, f2;
    long x;

    if(r0 <= 0 ||
       r0 >= r1 ||
       r1 > valmax)
        return 0;

    len--;
    if(len == 0)
        goto calcr0;

    if(!(flag & UR1)) {
        f1 = UR1|SR1;
        for(i=1; i<=shmax; i++) {
            x = r0<<i;
            if(x > valmax)
                break;
            if(gen3(len, r0, x, f1)) {
                i += 'a';
                goto out;
```

```
        }
    }
}

if(!(flag & UR0)) {
    f1 = UR1|SR1;
    for(i=1; i<=shmax; i++) {
        x = r1<<i;
        if(x > valmax)
            break;
        if(gen3(len, r1, x, f1)) {
            i += 'a';
            goto out;
        }
    }
}

if(!(flag & SR1)) {
    f1 = UR1|SR1|(flag&UR0);
    for(i=1; i<=shmax; i++) {
        x = r1<<i;
        if(x > valmax)
            break;
        if(gen3(len, r0, x, f1)) {
            i += 'a';
            goto out;
        }
    }
}

if(!(flag & SR0)) {
    f1 = UR0|SR0|(flag&(SR1|UR1));

    f2 = UR1|SR1;
    if(flag & UR1)
        f2 |= UR0;
    if(flag & SR1)
        f2 |= SR0;

    for(i=1; i<=shmax; i++) {
        x = r0<<i;
        if(x > valmax)
            break;
        if(x > r1) {
            if(gen3(len, r1, x, f2)) {
                i += 'a';
                goto out;
            }
        } else
            if(gen3(len, x, r1, f1)) {
                i += 'a';
                goto out;
            }
    }
}

x = r1+r0;
if(gen3(len, r0, x, UR1)) {
    i = '+';
    goto out;
```

```
    }

    if(gen3(len, r1, x, UR1)) {
        i = '+';
        goto out;
    }

    x = r1-r0;
    if(gen3(len, x, r1, UR0)) {
        i = '-';
        goto out;
    }

    if(x > r0) {
        if(gen3(len, r0, x, UR1)) {
            i = '-';
            goto out;
        }
    } else
        if(gen3(len, x, r0, UR0)) {
            i = '-';
            goto out;
        }

    return 0;

calcr0:
    f1 = flag & (UR0|UR1);
    if(f1 == UR1) {
        for(i=1; i<=shmax; i++) {
            x = r1<<i;
            if(x >= mulval) {
                if(x == mulval) {
                    i += 'a';
                    goto out;
                }
                break;
            }
        }
    }

    if(mulval == r1+r0) {
        i = '+';
        goto out;
    }
    if(mulval == r1-r0) {
        i = '-';
        goto out;
    }

    return 0;

out:
    *--mulcp = i;
    return 1;
}
```

⟨*global* hintab*(arm)* 268⟩≡ (465)

```
/*
 * hint table has numbers that
```

```
 * the search algorithm fails on.
 * <1000:
 * all numbers
 * <5000:
 *  ÷ by 5
 * <10000:
 *  ÷ by 50
 * <65536:
 *  ÷ by 250
 */
Hintab hintab[] =
{
    683, "b++d+e+",
    687, "b+e++e-",
    691, "b++d+e+",
    731, "b++d+e+",
    811, "b++d+i+",
    821, "b++e+e+",
    843, "b+d++e+",
    851, "b+f-+e-",
    853, "b++e+e+",
    877, "c++++g-",
    933, "b+c++g-",
    981, "c-+e-d+",
    1375, "b+c+b+h-",
    1675, "d+b++h+",
    2425, "c++f-e+",
    2675, "c+d++f-",
    2750, "b+d-b+h-",
    2775, "c-+g-e-",
    3125, "b++e+g+",
    3275, "b+c+g+e+",
    3350, "c++++i+",
    3475, "c-+e-f-",
    3525, "c-+d+g-",
    3625, "c-+e-j+",
    3675, "b+d+d+e+",
    3725, "b+d-+h+",
    3925, "b+d+f-d-",
    4275, "b+g++e+",
    4325, "b+h-+d+",
    4425, "b+b+g-j-",
    4525, "b+d-d+f+",
    4675, "c++d-g+",
    4775, "b+d+b+g-",
    4825, "c+c-+i-",
    4850, "c++++i-",
    4925, "b++e-g-",
    4975, "c+f++e-",
    5500, "b+g-c+d+",
    6700, "d+b++i+",
    9700, "d++++j-",
    11000, "b+f-c-h-",
    11750, "b+d+g+j-",
    12500, "b+c+e-k+",
    13250, "b+d+e-f+",
    13750, "b+h-c-d+",
    14250, "b+g-c+e-",
    14500, "c+f+j-d-",
    14750, "d-g--f+",
```

```
16750, "b+e-d-n+",
17750, "c+h-b+e+",
18250, "d+b+h-d+",
18750, "b+g-++f+",
19250, "b+e+b+h+",
19750, "b++h--f-",
20250, "b+e-l-c+",
20750, "c++bi+e-",
21250, "b+i+l+c+",
22000, "b+e+d-g-",
22250, "b+d-h+k-",
22750, "b+d-e-g+",
23250, "b+c+h+e-",
23500, "b+g-c-g-",
23750, "b+g-b+h-",
24250, "c++g+m-",
24750, "b+e+e+j-",
25000, "b++dh+g+",
25250, "b+e+d-g-",
25750, "b+e+b+j+",
26250, "b+h+c+e+",
26500, "b+h+c+g+",
26750, "b+d+e+g-",
27250, "b+e+e+f+",
27500, "c-i-c-d+",
27750, "b+bd++j+",
28250, "d-d-++i-",
28500, "c+c-h-e-",
29000, "b+g-d-f+",
29500, "c+h+++e-",
29750, "b+g+f-c+",
30250, "b+f-g-c+",
33500, "c-f-d-n+",
33750, "b+d-b+j-",
34250, "c+e+++i+",
35250, "e+b+d+k+",
35500, "c+e+d-g-",
35750, "c+i-++e+",
36250, "b+bh-d+e+",
36500, "c+c-h-e-",
36750, "d+e--i+",
37250, "b+g+g+b+",
37500, "b+h-b+f+",
37750, "c+be++j-",
38500, "b+e+b+i+",
38750, "d+i-b+d+",
39250, "b+g-l-+d+",
39500, "b+g-c+g-",
39750, "b+bh-c+f-",
40250, "b+bf+d+g-",
40500, "b+g-c+g+",
40750, "c+b+i-e+",
41250, "d++bf+h+",
41500, "b+j+c+d-",
41750, "c+f+b+h-",
42500, "c+h++g+",
42750, "b+g+d-f-",
43250, "b+l-e+d-",
43750, "c+bd+h+f-",
44000, "b+f+g-d-",
```

```
    44250, "b+d-g--f+",
    44500, "c+e+c+h+",
    44750, "b+e+d-h-",
    45250, "b++g+j-g+",
    45500, "c+d+e-g+",
    45750, "b+d-h-e-",
    46250, "c+bd++j+",
    46500, "b+d-c-j-",
    46750, "e-e-b+g-",
    47000, "b+c+d-j-",
    47250, "b+e+e-g-",
    47500, "b+g-c-h-",
    47750, "b+f-c+h-",
    48250, "d--h+n-",
    48500, "b+c-g+m-",
    48750, "b+e+e-g+",
    49500, "c-f+e+j-",
    49750, "c+c+g++f-",
    50000, "b+e+e+k+",
    50250, "b++i++g+",
    50500, "c+g+f-i+",
    50750, "b+e+d+k-",
    51500, "b+i+c-f+",
    51750, "b+bd+g-e-",
    52250, "b+d+g-j+",
    52500, "c+c+f+g+",
    52750, "b+c+e+i+",
    53000, "b+i+c+g+",
    53500, "c+g+g-n+",
    53750, "b+j+d-c+",
    54250, "b+d-g-j-",
    54500, "c-f+e+f+",
    54750, "b+f-+c+g+",
    55000, "b+g-d-g-",
    55250, "b+e+e+g+",
    55500, "b+cd++j+",
    55750, "b+bh-d-f-",
    56250, "c+d-b+j-",
    56500, "c+d+c+i+",
    56750, "b+e+d++h-",
    57000, "b+d+g-f+",
    57250, "b+f-m+d-",
    57750, "b+i+c+e-",
    58000, "b+e+d+h+",
    58250, "c+b+g+g+",
    58750, "d-e-j--e+",
    59000, "d-i-+e+",
    59250, "e--h-m+",
    59500, "c+c-h+f-",
    59750, "b+bh-e+i-",
    60250, "b+bh-e-e-",
    60500, "c+c-g-g-",
    60750, "b+e-l-e-",
    61250, "b+g-g-c+",
    61750, "b+g-c+g+",
    62250, "f--+c-i-",
    62750, "e+f--+g+",
    64750, "b+f+d+p-",
};
```

⟨*global hintabsize (5c/mul.c)(arm)* 272a⟩≡ (465)
```
int hintabsize = nelem(hintab);
```

⟨*global* hintabsize*(arm)* 272b⟩≡ (460b)
```
int hintabsize;
```

12.6 Associative-commutative arithmetic optimisations

12.6.1 Term

⟨*constant* NTERM 272c⟩≡ (429b)
```
#define NTERM  10
```

⟨*struct* Term 272d⟩≡ (429b)
```
struct Term
{
    vlong mult;
    Node *node;
};
```

⟨*global* term 272e⟩≡ (436d)
```
Term term[NTERM];
```

⟨*global* nterm 272f⟩≡ (436d)
```
int nterm;
```

12.6.2 acom()

⟨*function* acom 272g⟩≡ (452b)
```
void
acom(Node *n)
{
    Type *t;
    Node *l, *r;
    int i;

    switch(n->op)
    {

    case ONAME:
    case OCONST:
    case OSTRING:
    case OINDREG:
    case OREGISTER:
        return;

    case ONEG:
        l = n->left;
        if(addo(n) && addo(l))
            break;
        acom(l);
        return;

    case OADD:
    case OSUB:
    case OMUL:
        l = n->left;
```

```
        r = n->right;
        if(addo(n)) {
            if(addo(r))
                break;
            if(addo(l))
                break;
        }
        acom(l);
        acom(r);
        return;

    default:
        l = n->left;
        r = n->right;
        if(l != Z)
            acom(l);
        if(r != Z)
            acom(r);
        return;
    }

    /* bust terms out */
    t = n->type;
    term[0].mult = 0;
    term[0].node = Z;
    nterm = 1;
    acom1(1, n);

    if(debug['m'])
     for(i=0; i<nterm; i++) {
        print("%d %3lld ", i, term[i].mult);
        prtree1(term[i].node, 1, 0);
    }

    if(nterm < NTERM)
        acom2(n, t);
    n->type = t;
}
```

⟨*function* addo 273⟩≡ (452b)

```
int
addo(Node *n)
{

    if(n != Z)
    if(!typefd[n->type->etype])
    if(!typev[n->type->etype] || ewidth[TVLONG] == ewidth[TIND])
    switch(n->op) {

    case OCAST:
        if(nilcast(n->left->type, n->type))
            return 1;
        break;

    case ONEG:
    case OADD:
    case OSUB:
        return 1;

    case OMUL:
```

```
        if(n->left->op == OCONST)
            return 1;
        if(n->right->op == OCONST)
            return 1;
    }
    return 0;
}
```

⟨*function* acom1 274⟩≡ (452b)

```
void
acom1(vlong v, Node *n)
{
    Node *l, *r;

    if(v == 0 || nterm >= NTERM)
        return;
    if(!addo(n)) {
        if(n->op == OCONST)
        if(!typefd[n->type->etype]) {
            term[0].mult += v*n->vconst;
            return;
        }
        term[nterm].mult = v;
        term[nterm].node = n;
        nterm++;
        return;
    }
    switch(n->op) {

    case OCAST:
        acom1(v, n->left);
        break;

    case ONEG:
        acom1(-v, n->left);
        break;

    case OADD:
        acom1(v, n->left);
        acom1(v, n->right);
        break;

    case OSUB:
        acom1(v, n->left);
        acom1(-v, n->right);
        break;

    case OMUL:
        l = n->left;
        r = n->right;
        if(l->op == OCONST)
        if(!typefd[n->type->etype]) {
            acom1(v*l->vconst, r);
            break;
        }
        if(r->op == OCONST)
        if(!typefd[n->type->etype]) {
            acom1(v*r->vconst, l);
            break;
        }
```

```
        break;

    default:
        diag(n, "not addo");
    }
}
```

⟨*function* acast 275a⟩≡ (452b)

```
static Node*
acast(Type *t, Node *n)
{
    if(n->type->etype != t->etype || n->op == OBIT) {
        n = new1(OCAST, n, Z);
        if(nocast(n->left->type, t))
            *n = *n->left;
        n->type = t;
    }
    return n;
}
```

⟨*function* acomcmp1 275b⟩≡ (452b)

```
int
acomcmp1(const void *a1, const void *a2)
{
    vlong c1, c2;
    Term *t1, *t2;

    t1 = (Term*)a1;
    t2 = (Term*)a2;
    c1 = t1->mult;
    if(c1 < 0)
        c1 = -c1;
    c2 = t2->mult;
    if(c2 < 0)
        c2 = -c2;
    if(c1 > c2)
        return 1;
    if(c1 < c2)
        return -1;
    c1 = 1;
    if(t1->mult < 0)
        c1 = 0;
    c2 = 1;
    if(t2->mult < 0)
        c2 = 0;
    if(c2 -= c1)
        return c2;
    if(t2 > t1)
        return 1;
    return -1;
}
```

⟨*function* acomcmp2 275c⟩≡ (452b)

```
int
acomcmp2(const void *a1, const void *a2)
{
    vlong c1, c2;
    Term *t1, *t2;

    t1 = (Term*)a1;
```

```
    t2 = (Term*)a2;
    c1 = t1->mult;
    c2 = t2->mult;
    if(c1 > c2)
        return 1;
    if(c1 < c2)
        return -1;
    if(t2 > t1)
        return 1;
    return -1;
}
```

⟨*function* acom2 276⟩≡ (452b)

```
void
acom2(Node *n, Type *t)
{
    Node *l, *r;
    Term trm[NTERM];
    int et, nt, i, j;
    vlong c1, c2;

    /*
     * copy into automatic
     */
    c2 = 0;
    nt = nterm;
    for(i=0; i<nt; i++)
        trm[i] = term[i];
    /*
     * recur on subtrees
     */
    j = 0;
    for(i=1; i<nt; i++) {
        c1 = trm[i].mult;
        if(c1 == 0)
            continue;
        l = trm[i].node;
        if(l != Z) {
            j = 1;
            acom(l);
        }
    }
    c1 = trm[0].mult;
    if(j == 0) {
        n->oldop = n->op;
        n->op = OCONST;
        n->vconst = c1;
        return;
    }
    et = t->etype;

    /*
     * prepare constant term,
     * combine it with an addressing term
     */
    if(c1 != 0) {
        l = new1(OCONST, Z, Z);
        l->type = t;
        l->vconst = c1;
        trm[0].mult = 1;
```

```
    for(i=1; i<nt; i++) {
        if(trm[i].mult != 1)
            continue;
        r = trm[i].node;
        if(r->op != OADDR)
            continue;
        r->type = t;
        l = new1(OADD, r, l);
        l->type = t;
        trm[i].mult = 0;
        break;
    }
    trm[0].node = l;
}
/*
 * look for factorable terms
 * c1*i + c1*c2*j -> c1*(i + c2*j)
 */
qsort(trm+1, nt-1, sizeof(trm[0]), acomcmp1);
for(i=nt-1; i>=0; i--) {
    c1 = trm[i].mult;
    if(c1 < 0)
        c1 = -c1;
    if(c1 <= 1)
        continue;
    for(j=i+1; j<nt; j++) {
        c2 = trm[j].mult;
        if(c2 < 0)
            c2 = -c2;
        if(c2 <= 1)
            continue;
        if(c2 % c1)
            continue;
        r = trm[j].node;
        if(r->type->etype != et)
            r = acast(t, r);
        c2 = trm[j].mult/trm[i].mult;
        if(c2 != 1 && c2 != -1) {
            r = new1(OMUL, r, new(OCONST, Z, Z));
            r->type = t;
            r->right->type = t;
            r->right->vconst = c2;
        }
        l = trm[i].node;
        if(l->type->etype != et)
            l = acast(t, l);
        r = new1(OADD, l, r);
        r->type = t;
        if(c2 == -1)
            r->op = OSUB;
        trm[i].node = r;
        trm[j].mult = 0;
    }
}
if(debug['m']) {
    print("\n");
    for(i=0; i<nt; i++) {
        print("%d %3lld ", i, trm[i].mult);
        prtree1(trm[i].node, 1, 0);
    }
```

```
    }

    /*
     * put it all back together
     */
    qsort(trm+1, nt-1, sizeof(trm[0]), acomcmp2);
    l = Z;
    for(i=nt-1; i>=0; i--) {
        c1 = trm[i].mult;
        if(c1 == 0)
            continue;
        r = trm[i].node;
        if(r->type->etype != et || r->op == OBIT)
            r = acast(t, r);
        if(c1 != 1 && c1 != -1) {
            r = new1(OMUL, r, new(OCONST, Z, Z));
            r->type = t;
            r->right->type = t;
            if(c1 < 0) {
                r->right->vconst = -c1;
                c1 = -1;
            } else {
                r->right->vconst = c1;
                c1 = 1;
            }
        }
        if(l == Z) {
            l = r;
            c2 = c1;
            continue;
        }
        if(c1 < 0)
            if(c2 < 0)
                l = new1(OADD, l, r);
            else
                l = new1(OSUB, l, r);
        else
            if(c2 < 0) {
                l = new1(OSUB, r, l);
                c2 = 1;
            } else
                l = new1(OADD, l, r);
        l->type = t;
    }
    if(c2 < 0) {
        r = new1(OCONST, 0, 0);
        r->vconst = 0;
        r->type = t;
        l = new1(OSUB, r, l);
        l->type = t;
    }
    *n = *l;
}
```

Chapter 13

Assembly-level Optimisations

13.1 Nop detection

⟨gmove() *if samaddr return* 279a⟩≡ (171d)
```
  if(a == AMOVW || a == AMOVF || a == AMOVD)
   if(samaddr(f, t))
      return;
```

⟨*function* samaddr 279b⟩≡ (462)
```
  bool
  samaddr(Node *f, Node *t)
  {

      if(f->op != t->op)
          return false;
      switch(f->op) {
      case OREGISTER:
          return (f->reg == t->reg);
      }
      return false;
  }
```

13.2 ARM special instructions

⟨gopcode() *when OSUB, ARSB opportunity* 279c⟩≡ (185c)
```
  if(f2 && f2->op == OCONST) {
      Node *t = f1;
      f1 = f2;
      f2 = t;
      a = ARSB;
  }
```

13.3 Register passing argument: REGARG

⟨codgen() *locals* 279d⟩+≡ (156d) ◁156e
```
  Node nod, nod1;
```

⟨codgen() *if use REGARG* 279e⟩≡ (156d)
```
  /*
   * isolate first argument
   */
```

```
if(REGARG >= 0) {
    ⟨codegen() if use REGARG, if complex return type 223c⟩
    else
    if(firstarg && typeword[firstargtype->etype]) {
        nod1 = znode;
        nod1.op = ONAME;
        nod1.sym = firstarg;
        nod1.type = firstargtype;
        nod1.etype = firstargtype->etype;
        nod1.class = CPARAM;
        nod1.xoffset = align(0, firstargtype, Aarg1);
        xcom(&nod1);
        nodreg(&nod, &nod1, REGARG);
        gmove(&nod, &nod1);
    }
}
```

⟨*global* typeword 280a⟩≡ (436d)

```
// set<type_kind>
char* typeword;
```

⟨tinit() *initialise 32 bits defaults type sets* 280b⟩+≡ (34b) ◁222b

```
/* 32-bit defaults */
typeword = typechlp;
```

⟨cgenrel() *OFUNC case, if use REGARG, part one* 280c⟩≡ (209)

```
if(REGARG >= 0)
    o = reg[REGARG];
```

⟨cgenrel() *OFUNC case, if use REGARG, part two* 280d⟩≡ (209)

```
if(REGARG >= 0)
    if(o != reg[REGARG])
        reg[REGARG]--;
```

⟨garg1() *if use REGARG and curarg is zero and simple type* 280e⟩≡ (210h)

```
if(REGARG >= 0 && curarg == 0 && typechlp[n->type->etype]) {
    regaalloc1(tn1, n);
    if(n->complex >= FNX) {
        cgen(*fnxp, tn1);
        (*fnxp)++;
    } else
        cgen(n, tn1);
}
```

⟨*function* regaalloc1*(arm)* 280f⟩≡ (462)

```
void
regaalloc1(Node *n, Node *nn)
{
    nodreg(n, nn, REGARG);
    reg[REGARG]++;
    curarg = align(curarg, nn->type, Aarg1);
    curarg = align(curarg, nn->type, Aarg2);
    maxargsafe = maxround(maxargsafe, cursafe+curarg);
}
```

13.4 Register allocation optimisations

13.4.1 regopt()

⟨codgen() *register optimisation* 281a⟩≡ (156d)
```
if(!debug['N'] || debug['R'] || debug['P'])
    regopt(sp);
```

⟨*function* regopt*(arm)* 281b⟩≡ (466)
```
void
regopt(Prog *p)
{
    ⟨regopt() locals 282a⟩

    ⟨regopt() initialisation 282b⟩

    ⟨regopt() pass 1 283⟩
    ⟨regopt() pass 2 285a⟩
    ⟨regopt() pass 2.5 285b⟩
    ⟨regopt() pass 3 286a⟩
    ⟨regopt() pass 4 286b⟩
    ⟨regopt() pass 5 287⟩
    ⟨regopt() pass 6 288⟩
    ⟨regopt() pass 7 289a⟩
    ⟨regopt() pass 8 289b⟩
    ⟨regopt() pass 9 290a⟩
}
```

⟨*global* externs 281c⟩≡ (460b)
```
Bits externs;
```

⟨*global* params 281d⟩≡ (460b)
```
Bits params;
```

⟨*global* consts 281e⟩≡ (460b)
```
Bits consts;
```

⟨*global* addrs 281f⟩≡ (460b)
```
Bits addrs;
```

⟨*global* regbits 281g⟩≡ (460b)
```
long regbits;
```

⟨*global* exregbits 281h⟩≡ (460b)
```
long exregbits;
```

⟨*global* change 281i⟩≡ (460b)
```
int change;
```

⟨*global* firstr 281j⟩≡ (460b)
```
Reg* firstr;
```

⟨*global* lastr 281k⟩≡ (460b)
```
Reg* lastr;
```

⟨*global* zreg 281l⟩≡ (460b)
```
Reg zreg;
```

⟨*global* freer 281m⟩≡ (460b)
```
Reg* freer;
```

⟨regopt() *locals* 282a⟩≡ (281b)

```
Reg *r, *r1, *r2;
Prog *p1;
int i, z;
long initpc, val, npc;
ulong vreg;
Bits bit;
struct
{
    long m;
    long c;
    Reg* p;
} log5[6], *lp;
```

⟨regopt() *initialisation* 282b⟩≡ (281b)

```
firstr = R;
lastr = R;
nvar = 0;
regbits = 0;
for(z=0; z<BITS; z++) {
    externs.b[z] = 0;
    params.b[z] = 0;
    consts.b[z] = 0;
    addrs.b[z] = 0;
}
```

13.4.2 noretval()

⟨codgen() *before RET* 282c⟩≡ (156d)

```
noretval(1 | 2);
```

⟨gen() *case ORETURN with no argument, before RET* 282d⟩≡ (165g)

```
noretval(1 | 2);
```

⟨gen() *case ORETURN with argument, before RET* 282e⟩≡ (165g)

```
if(typefd[n->type->etype])
    noretval(1);
else
    noretval(2);
```

⟨*function* noretval*(arm)* 282f⟩≡ (464a)

```
void
noretval(int n)
{

    if(n & 1) {
        gins(ANOP, Z, Z);
        p->to.type = D_REG;
        p->to.reg = REGRET;
    }
    if(n & 2) {
        gins(ANOP, Z, Z);
        p->to.type = D_FREG;
        p->to.reg = FREGRET;
    }
}
```

13.4.3 Pass 1

⟨regopt() *pass 1* 283⟩≡ (281b)

```
/*
 * pass 1
 * build aux data structure
 * allocate pcs
 * find use and set of variables
 */
val = 5L * 5L * 5L * 5L * 5L;
lp = log5;
for(i=0; i<5; i++) {
    lp->m = val;
    lp->c = 0;
    lp->p = R;
    val /= 5L;
    lp++;
}
val = 0;
for(; p != P; p = p->link) {
    switch(p->as) {
    case ADATA:
    case AGLOBL:
    case ANAME:
    case ASIGNAME:
        continue;
    }
    r = rega();
    if(firstr == R) {
        firstr = r;
        lastr = r;
    } else {
        lastr->link = r;
        r->p1 = lastr;
        lastr->s1 = r;
        lastr = r;
    }
    r->prog = p;
    r->pc = val;
    val++;

    lp = log5;
    for(i=0; i<5; i++) {
        lp->c--;
        if(lp->c <= 0) {
            lp->c = lp->m;
            if(lp->p != R)
                lp->p->log5 = r;
            lp->p = r;
            (lp+1)->c = 0;
            break;
        }
        lp++;
    }

    r1 = r->p1;
    if(r1 != R)
    switch(r1->prog->as) {
    case ARET:
    case AB:
```

```
    case ARFE:
        r->p1 = R;
        r1->s1 = R;
    }

    /*
     * left side always read
     */
    bit = mkvar(&p->from, p->as==AMOVW);
    for(z=0; z<BITS; z++)
        r->use1.b[z] |= bit.b[z];

    /*
     * right side depends on opcode
     */
    bit = mkvar(&p->to, 0);
    if(bany(&bit))
    switch(p->as) {
    default:
        diag(Z, "reg: unknown asop: %A", p->as);
        break;

    /*
     * right side write
     */
    case ANOP:
    case AMOVB:
    case AMOVBU:
    case AMOVH:
    case AMOVHU:
    case AMOVW:
    case AMOVF:
    case AMOVD:
        for(z=0; z<BITS; z++)
            r->set.b[z] |= bit.b[z];
        break;

    /*
     * funny
     */
    case ABL:
        for(z=0; z<BITS; z++)
            addrs.b[z] |= bit.b[z];
        break;
    }

    if(p->as == AMOVM) {
        if(p->from.type == D_CONST)
            z = p->from.offset;
        else
            z = p->to.offset;
        for(i=0; z; i++) {
            if(z&1)
                regbits |= RtoB(i);
            z >>= 1;
        }
    }
}
if(firstr == R)
    return;
```

```
initpc = pc - val;
npc = val;
```

13.4.4 Pass 2

⟨regopt() *pass 2* 285a⟩≡ (281b)

```
/*
 * pass 2
 * turn branch references to pointers
 * build back pointers
 */
for(r = firstr; r != R; r = r->link) {
    p = r->prog;
    if(p->to.type == D_BRANCH) {
        val = p->to.offset - initpc;
        r1 = firstr;
        while(r1 != R) {
            r2 = r1->log5;
            if(r2 != R && val >= r2->pc) {
                r1 = r2;
                continue;
            }
            if(r1->pc == val)
                break;
            r1 = r1->link;
        }
        if(r1 == R) {
            nearln = p->lineno;
            diag(Z, "ref not found\n%P", p);
            continue;
        }
        if(r1 == r) {
            nearln = p->lineno;
            diag(Z, "ref to self\n%P", p);
            continue;
        }
        r->s2 = r1;
        r->p2link = r1->p2;
        r1->p2 = r;
    }
}
if(debug['R']) {
    p = firstr->prog;
    print("\n%L %D\n", p->lineno, &p->from);
}
```

⟨regopt() *pass 2.5* 285b⟩≡ (281b)

```
/*
 * pass 2.5
 * find looping structure
 */
for(r = firstr; r != R; r = r->link)
    r->active = 0;
change = 0;
loopit(firstr, npc);
```

13.4.5 Pass 3

⟨regopt() *pass 3* 286a⟩≡ (281b)

```
    /*
     * pass 3
     * iterate propagating usage
     *  back until flow graph is complete
     */
loop1:
    change = 0;
    for(r = firstr; r != R; r = r->link)
        r->active = 0;
    for(r = firstr; r != R; r = r->link)
        if(r->prog->as == ARET)
            prop(r, zbits, zbits);
loop11:
    /* pick up unreachable code */
    i = 0;
    for(r = firstr; r != R; r = r1) {
        r1 = r->link;
        if(r1 && r1->active && !r->active) {
            prop(r, zbits, zbits);
            i = 1;
        }
    }
    if(i)
        goto loop11;
    if(change)
        goto loop1;
```

13.4.6 Pass 4

⟨regopt() *pass 4* 286b⟩≡ (281b)

```
    /*
     * pass 4
     * iterate propagating register/variable synchrony
     *  forward until graph is complete
     */
loop2:
    change = 0;
    for(r = firstr; r != R; r = r->link)
        r->active = 0;
    synch(firstr, zbits);
    if(change)
        goto loop2;

    addsplits();

    if(debug['R'] && debug['v']) {
        print("\nprop structure:\n");
        for(r = firstr; r != R; r = r->link) {
            print("%ld:%P", r->loop, r->prog);
            for(z=0; z<BITS; z++)
                bit.b[z] = r->set.b[z] |
                    r->refahead.b[z] | r->calahead.b[z] |
                    r->refbehind.b[z] | r->calbehind.b[z] |
                    r->use1.b[z] | r->use2.b[z];
            if(bany(&bit)) {
                print("\t");
```

```
                if(bany(&r->use1))
                    print(" u1=%B", r->use1);
                if(bany(&r->use2))
                    print(" u2=%B", r->use2);
                if(bany(&r->set))
                    print(" st=%B", r->set);
                if(bany(&r->refahead))
                    print(" ra=%B", r->refahead);
                if(bany(&r->calahead))
                    print(" ca=%B", r->calahead);
                if(bany(&r->refbehind))
                    print(" rb=%B", r->refbehind);
                if(bany(&r->calbehind))
                    print(" cb=%B", r->calbehind);
            }
            print("\n");
        }
    }
```

13.4.7 pass 5

⟨regopt() *pass 5* 287⟩≡ (281b)

```
    /*
     * pass 5
     * isolate regions
     * calculate costs (paint1)
     */
    r = firstr;
    if(r) {
        for(z=0; z<BITS; z++)
            bit.b[z] = (r->refahead.b[z] | r->calahead.b[z]) &
              ~(externs.b[z] | params.b[z] | addrs.b[z] | consts.b[z]);
        if(bany(&bit)) {
            nearln = r->prog->lineno;
            warn(Z, "used and not set: %B", bit);
            if(debug['R'] && !debug['w'])
                print("used and not set: %B\n", bit);
        }
    }

    for(r = firstr; r != R; r = r->link)
        r->act = zbits;
    rgp = region;
    nregion = 0;
    for(r = firstr; r != R; r = r->link) {
        for(z=0; z<BITS; z++)
            bit.b[z] = r->set.b[z] &
              ~(r->refahead.b[z] | r->calahead.b[z] | addrs.b[z]);
        if(bany(&bit)) {
            nearln = r->prog->lineno;
            warn(Z, "set and not used: %B", bit);
            if(debug['R'])
                print("set and not used: %B\n", bit);
            excise(r);
        }
        for(z=0; z<BITS; z++)
            bit.b[z] = LOAD(r) & ~(r->act.b[z] | addrs.b[z]);
        while(bany(&bit)) {
            i = bnum(bit);
```

```
            rgp->enter = r;
            rgp->varno = i;
            change = 0;
            if(debug['R'] && debug['v'])
                print("\n");
            paint1(r, i);
            bit.b[i/32] &= ~(1L<<(i%32));
            if(change <= 0) {
                if(debug['R'])
                    print("%L $%d: %B\n",
                        r->prog->lineno, change, blsh(i));
                continue;
            }
            rgp->cost = change;
            nregion++;
            if(nregion >= NRGN) {
                warn(Z, "too many regions");
                goto brk;
            }
            rgp++;
        }
    }
brk:
    qsort(region, nregion, sizeof(region[0]), rcmp);
```

13.4.8 pass 6

⟨regopt() *pass 6* 288⟩≡ (281b)

```
/*
 * pass 6
 * determine used registers (paint2)
 * replace code (paint3)
 */
rgp = region;
for(i=0; i<nregion; i++) {
    bit = blsh(rgp->varno);
    vreg = paint2(rgp->enter, rgp->varno);
    vreg = allreg(vreg, rgp);
    if(debug['R']) {
        if(rgp->regno >= NREG)
            print("%L $%d F%d: %B\n",
                rgp->enter->prog->lineno,
                rgp->cost,
                rgp->regno-NREG,
                bit);
        else
            print("%L $%d R%d: %B\n",
                rgp->enter->prog->lineno,
                rgp->cost,
                rgp->regno,
                bit);
    }
    if(rgp->regno != 0)
        paint3(rgp->enter, rgp->varno, vreg, rgp->regno);
    rgp++;
}
```

13.4.9 pass 7, peep hole optimizations

⟨regopt() *pass 7* 289a⟩≡ (281b)

```
/*
 * pass 7
 * peep-hole on basic block
 */
if(!debug['R'] || debug['P'])
    peep();
```

13.4.10 pass 8

⟨regopt() *pass 8* 289b⟩≡ (281b)

```
/*
 * pass 8
 * recalculate pc
 */
val = initpc;
for(r = firstr; r != R; r = r1) {
    r->pc = val;
    p = r->prog;
    p1 = P;
    r1 = r->link;
    if(r1 != R)
        p1 = r1->prog;
    for(; p != p1; p = p->link) {
        switch(p->as) {
        default:
            val++;
            break;

        case ANOP:
        case ADATA:
        case AGLOBL:
        case ANAME:
        case ASIGNAME:
            break;
        }
    }
}
pc = val;

/*
 * fix up branches
 */
if(debug['R'])
    if(bany(&addrs))
        print("addrs: %B\n", addrs);

r1 = 0; /* set */
for(r = firstr; r != R; r = r->link) {
    p = r->prog;
    if(p->to.type == D_BRANCH)
        p->to.offset = r->s2->pc;
    r1 = r;
}
```

13.4.11 pass 9

⟨regopt() *pass 9* 290a⟩≡ (281b)

```
/*
 * last pass
 * eliminate nops
 * free aux structures
 */
for(p = firstr->prog; p != P; p = p->link){
    while(p->link && p->link->as == ANOP)
        p->link = p->link->link;
}
if(r1 != R) {
    r1->link = freer;
    freer = firstr;
}
```

13.4.12 Bits

⟨*constant* BITS 290b⟩≡ (429b)

```
#define BITS 5
```

⟨*struct* Bits 290c⟩≡ (429b)

```
struct Bits
{
    ulong b[BITS];
};
```

⟨*global* zbits 290d⟩≡ (436d)

```
Bits zbits;
```

⟨*function* bor 290e⟩≡ (446a)

```
Bits
bor(Bits a, Bits b)
{
    Bits c;
    int i;

    for(i=0; i<BITS; i++)
        c.b[i] = a.b[i] | b.b[i];
    return c;
}
```

⟨*function* bany 290f⟩≡ (446a)

```
/*
Bits
bnot(Bits a)
{
    Bits c;
    int i;

    for(i=0; i<BITS; i++)
        c.b[i] = ~a.b[i];
    return c;
}
*/

int
bany(Bits *a)
```

```
{
    int i;

    for(i=0; i<BITS; i++)
        if(a->b[i])
            return 1;
    return 0;
}
```

⟨*function* beq 291a⟩≡ (446a)

```
int
beq(Bits a, Bits b)
{
    int i;

    for(i=0; i<BITS; i++)
        if(a.b[i] != b.b[i])
            return 0;
    return 1;
}
```

⟨*function* bnum 291b⟩≡ (446a)

```
int
bnum(Bits a)
{
    int i;
    long b;

    for(i=0; i<BITS; i++)
        if(b = a.b[i])
            return 32*i + bitno(b);
    diag(Z, "bad in bnum");
    return 0;
}
```

⟨*function* blsh 291c⟩≡ (446a)

```
Bits
blsh(uint n)
{
    Bits c;

    c = zbits;
    c.b[n/32] = 1L << (n%32);
    return c;
}
```

⟨*function* bset 291d⟩≡ (446a)

```
//@Scheck: also defined as a macro in gc.h, TODO fix that!
int bset(Bits a, uint n)
{
    if(a.b[n/32] & (1L << (n%32)))
        return 1;
    return 0;
}
```

⟨*function* bset*(arm)* 291e⟩≡ (456)

```
#define bset(a,n) ((a).b[(n)/32]&(1L<<(n)%32))
```

13.4.13 Var

⟨*constant* NVAR 292a⟩≡ (429b)
```
#define NVAR (BITS*sizeof(ulong)*8)
```

⟨*global* nvar 292b⟩≡ (460b)
```
int nvar;
```

⟨*global* var 292c⟩≡ (460b)
```
Var var[NVAR];
```

⟨*struct* Var 292d⟩≡ (456)
```
struct Var
{
    Sym* sym;
    //enum<sym_kind>
    char symkind;
    //enum<type_kind>
    char etype;
    long offset;
};
```

13.4.14 Reg

⟨*struct* Reg 292e⟩≡ (456)
```
struct Reg
{
    long pc;
    long rpo;  /* reverse post ordering */

    Bits set;
    Bits use1;
    Bits use2;

    Bits refbehind;
    Bits refahead;
    Bits calbehind;
    Bits calahead;
    Bits regdiff;
    Bits act;

    long regu;
    long loop;  /* could be shorter */


    Reg* log5;
    long active;

    Reg* p1;
    Reg* p2;
    Reg* p2link;
    Reg* s1;
    Reg* s2;
    Reg* link;

    Prog* prog;
};
```

⟨*constant* R 292f⟩≡ (456)
```
#define R ((Reg*)nil)
```

13.4.15 XXX

⟨*function* rega 293a⟩≡ (466)

```
Reg*
rega(void)
{
    Reg *r;

    r = freer;
    if(r == R) {
        r = alloc(sizeof(*r));
    } else
        freer = r->link;

    *r = zreg;
    return r;
}
```

⟨*function* rcmp 293b⟩≡ (466)

```
int
rcmp(const void *a1, const void *a2)
{
    Rgn *p1, *p2;
    int c1, c2;

    p1 = (Rgn*)a1;
    p2 = (Rgn*)a2;
    c1 = p2->cost;
    c2 = p1->cost;
    if(c1 -= c2)
        return c1;
    return p2->varno - p1->varno;
}
```

⟨*function* LOAD*(arm)* 293c⟩≡ (456)

```
#define LOAD(r)  (~r->refbehind.b[z] & r->refahead.b[z])
```

⟨*function* STORE*(arm)* 293d⟩≡ (456)

```
#define STORE(r) (~r->calbehind.b[z] & r->calahead.b[z])
```

⟨*constant* NRGN*(arm)* 293e⟩≡ (456)

```
#define NRGN 1000  /* was 600; raised for paranoia.c */
```

⟨*global* region 293f⟩≡ (460b)

```
Rgn region[NRGN];
```

⟨*global* rgp 293g⟩≡ (460b)

```
Rgn* rgp;
```

⟨*struct* Rgn 293h⟩≡ (456)

```
struct Rgn
{
    Reg* enter;
    short cost;
    short varno;
    short regno;
};
```

⟨*global* nregion 293i⟩≡ (460b)

```
int nregion;
```

⟨*function* addsplits*(arm)* 294a⟩≡ (466)

```
void
addsplits(void)
{
    Reg *r, *r1;
    int z, i;
    Bits bit;

    for(r = firstr; r != R; r = r->link) {
        if(r->loop > 1)
            continue;
        if(r->prog->as == ABL)
            continue;
        for(r1 = r->p2; r1 != R; r1 = r1->p2link) {
            if(r1->loop <= 1)
                continue;
            for(z=0; z<BITS; z++)
                bit.b[z] = r1->calbehind.b[z] &
                    (r->refahead.b[z] | r->use1.b[z] | r->use2.b[z]) &
                    ~(r->calahead.b[z] & addrs.b[z]);
            while(bany(&bit)) {
                i = bnum(bit);
                bit.b[i/32] &= ~(1L << (i%32));
            }
        }
    }
}
```

⟨*function* addmove*(arm)* 294b⟩≡ (466)

```
/*
 * add mov b,rn
 * just after r
 */
void
addmove(Reg *r, int bn, int rn, int f)
{
    Prog *p, *p1;
    Adr *a;
    Var *v;

    p1 = alloc(sizeof(Prog));
    *p1 = zprog;
    p = r->prog;

    p1->link = p->link;
    p->link = p1;
    p1->lineno = p->lineno;

    v = var + bn;

    a = &p1->to;
    a->sym = v->sym;
    a->symkind = v->symkind;
    a->offset = v->offset;
    a->etype = v->etype;
    a->type = D_OREG;
    if(v->etype == TARRAY || a->sym == S)
        a->type = D_CONST;

    p1->as = AMOVW;
```

```
    if(v->etype == TCHAR || v->etype == TUCHAR)
        p1->as = AMOVB;
    if(v->etype == TSHORT || v->etype == TUSHORT)
        p1->as = AMOVH;
    if(v->etype == TFLOAT)
        p1->as = AMOVF;
    if(v->etype == TDOUBLE)
        p1->as = AMOVD;

    p1->from.type = D_REG;
    p1->from.reg = rn;
    if(rn >= NREG) {
        p1->from.type = D_FREG;
        p1->from.reg = rn-NREG;
    }
    if(!f) {
        p1->from = *a;
        *a = zprog.from;
        a->type = D_REG;
        a->reg = rn;
        if(rn >= NREG) {
            a->type = D_FREG;
            a->reg = rn-NREG;
        }
        if(v->etype == TUCHAR)
            p1->as = AMOVBU;
        if(v->etype == TUSHORT)
            p1->as = AMOVHU;
    }
    if(debug['R'])
        print("%P\t.a%P\n", p, p1);
}
```

⟨*function* mkvar *(arm)* 295⟩≡ (466)

```
Bits
mkvar(Adr *a, int docon)
{
    Var *v;
    int i, n, z;
    long o;
    Bits bit;
    Sym *s;
    //enum<operand_kind>
    int t;
    //enum<type_kind>
    int et;

    t = a->type;
    if(t == D_REG && a->reg != R_NONE)
        regbits |= RtoB(a->reg);
    if(t == D_FREG && a->reg != R_NONE)
        regbits |= FtoB(a->reg);
    s = a->sym;
    o = a->offset;
    et = a->etype;
    if(s == S) {
        if(t != D_CONST || !docon || a->reg != R_NONE)
            goto none;
        et = TLONG;
    }
```

```
    if(t == D_CONST) {
        if(s == S && sval(o))
            goto none;
    }

    n = a->symkind;
    v = var;
    for(i=0; i<nvar; i++) {
        if(s == v->sym)
        if(n == v->symkind)
        if(o == v->offset)
            goto out;
        v++;
    }
    if(s)
        if(s->name[0] == '.')
            goto none;
    if(nvar >= NVAR) {
        if(debug['w'] > 1 && s)
            warn(Z, "variable not optimized: %s", s->name);
        goto none;
    }
    i = nvar;
    nvar++;
    v = &var[i];
    v->sym = s;
    v->offset = o;
    v->etype = et;
    v->symkind = n;
    if(debug['R'])
        print("bit=%2d et=%2d %D\n", i, et, a);
out:
    bit = blsh(i);
    if(n == N_EXTERN || n == N_INTERN)
        for(z=0; z<BITS; z++)
            externs.b[z] |= bit.b[z];
    if(n == N_PARAM)
        for(z=0; z<BITS; z++)
            params.b[z] |= bit.b[z];
    if(v->etype != et || !typechlpfd[et]) /* funny punning */
        for(z=0; z<BITS; z++)
            addrs.b[z] |= bit.b[z];
    if(t == D_CONST) {
        if(s == S) {
            for(z=0; z<BITS; z++)
                consts.b[z] |= bit.b[z];
            return bit;
        }
        if(et != TARRAY)
            for(z=0; z<BITS; z++)
                addrs.b[z] |= bit.b[z];
        for(z=0; z<BITS; z++)
            params.b[z] |= bit.b[z];
        return bit;
    }
    if(t == D_OREG)
        return bit;

none:
    return zbits;
```

```
}
```

⟨*global* typechlpfd 297a⟩≡ (443b)

```
char typechlpfd[NTYPE];
```

⟨*global* typechlpfdinit 297b⟩≡ (443b)

```
int typechlpfdinit[] =
{
    TCHAR, TUCHAR, TSHORT, TUSHORT, TINT, TUINT, TLONG, TULONG, TFLOAT, TDOUBLE, TIND, -1,
};
```

⟨*function* sval*(arm)* 297c⟩≡ (462)

```
int
sval(long v)
{
    int i;

    for(i=0; i<16; i++) {
        if((v & ~0xff) == 0)
            return 1;
        if((~v & ~0xff) == 0)
            return 1;
        v = (v<<2) | ((ulong)v>>30);
    }
    return 0;
}
```

⟨*function* prop*(arm)* 297d⟩≡ (466)

```
void
prop(Reg *r, Bits ref, Bits cal)
{
    Reg *r1, *r2;
    int z;

    for(r1 = r; r1 != R; r1 = r1->p1) {
        for(z=0; z<BITS; z++) {
            ref.b[z] |= r1->refahead.b[z];
            if(ref.b[z] != r1->refahead.b[z]) {
                r1->refahead.b[z] = ref.b[z];
                change++;
            }
            cal.b[z] |= r1->calahead.b[z];
            if(cal.b[z] != r1->calahead.b[z]) {
                r1->calahead.b[z] = cal.b[z];
                change++;
            }
        }
        switch(r1->prog->as) {
        case ABL:
            for(z=0; z<BITS; z++) {
                cal.b[z] |= ref.b[z] | externs.b[z];
                ref.b[z] = 0;
            }
            break;

        case ATEXT:
            for(z=0; z<BITS; z++) {
                cal.b[z] = 0;
                ref.b[z] = 0;
            }
```

```
            break;

        case ARET:
            for(z=0; z<BITS; z++) {
                cal.b[z] = externs.b[z];
                ref.b[z] = 0;
            }
        }
        for(z=0; z<BITS; z++) {
            ref.b[z] = (ref.b[z] & ~r1->set.b[z]) |
                r1->use1.b[z] | r1->use2.b[z];
            cal.b[z] &= ~(r1->set.b[z] | r1->use1.b[z] | r1->use2.b[z]);
            r1->refbehind.b[z] = ref.b[z];
            r1->calbehind.b[z] = cal.b[z];
        }
        if(r1->active)
            break;
        r1->active = 1;
    }
    for(; r != r1; r = r->p1)
        for(r2 = r->p2; r2 != R; r2 = r2->p2link)
            prop(r2, r->refbehind, r->calbehind);
}
```

⟨*function* rpolca 298a⟩≡ (466)

```
long
rpolca(long *idom, long rpo1, long rpo2)
{
    long t;

    if(rpo1 == -1)
        return rpo2;
    while(rpo1 != rpo2){
        if(rpo1 > rpo2){
            t = rpo2;
            rpo2 = rpo1;
            rpo1 = t;
        }
        while(rpo1 < rpo2){
            t = idom[rpo2];
            if(t >= rpo2)
                fatal(Z, "bad idom");
            rpo2 = t;
        }
    }
    return rpo1;
}
```

⟨*function* doms 298b⟩≡ (466)

```
int
doms(long *idom, long r, long s)
{
    while(s > r)
        s = idom[s];
    return s == r;
}
```

⟨*function* loophead 298c⟩≡ (466)

```
int
loophead(long *idom, Reg *r)
```

```
{
    long src;

    src = r->rpo;
    if(r->p1 != R && doms(idom, src, r->p1->rpo))
        return 1;
    for(r = r->p2; r != R; r = r->p2link)
        if(doms(idom, src, r->rpo))
            return 1;
    return 0;
}
```

⟨*constant* LOOP*(arm)* 299a⟩≡ (456)

```
#define LOOP 3
```

⟨*function* loopmark 299b⟩≡ (466)

```
void
loopmark(Reg **rpo2r, long head, Reg *r)
{
    if(r->rpo < head || r->active == head)
        return;
    r->active = head;
    r->loop += LOOP;
    if(r->p1 != R)
        loopmark(rpo2r, head, r->p1);
    for(r = r->p2; r != R; r = r->p2link)
        loopmark(rpo2r, head, r);
}
```

⟨*global* idom 299c⟩≡ (460b)

```
long* idom;
```

⟨*global* rpo2r 299d⟩≡ (460b)

```
Reg** rpo2r;
```

⟨*global* maxnr 299e⟩≡ (460b)

```
long maxnr;
```

⟨*function* loopit 299f⟩≡ (466)

```
void
loopit(Reg *r, long nr)
{
    Reg *r1;
    long i, d, me;

    if(nr > maxnr) {
        rpo2r = alloc(nr * sizeof(Reg*));
        idom = alloc(nr * sizeof(long));
        maxnr = nr;
    }

    d = postorder(r, rpo2r, 0);
    if(d > nr)
        fatal(Z, "too many reg nodes");
    nr = d;
    for(i = 0; i < nr / 2; i++){
        r1 = rpo2r[i];
        rpo2r[i] = rpo2r[nr - 1 - i];
        rpo2r[nr - 1 - i] = r1;
    }
```

```
    for(i = 0; i < nr; i++)
        rpo2r[i]->rpo = i;

    idom[0] = 0;
    for(i = 0; i < nr; i++){
        r1 = rpo2r[i];
        me = r1->rpo;
        d = -1;
        if(r1->p1 != R && r1->p1->rpo < me)
            d = r1->p1->rpo;
        for(r1 = r1->p2; r1 != nil; r1 = r1->p2link)
            if(r1->rpo < me)
                d = rpolca(idom, d, r1->rpo);
        idom[i] = d;
    }

    for(i = 0; i < nr; i++){
        r1 = rpo2r[i];
        r1->loop++;
        if(r1->p2 != R && loophead(idom, r1))
            loopmark(rpo2r, i, r1);
    }
}
```

⟨*function* synch 300a⟩≡ (466)

```
void
synch(Reg *r, Bits dif)
{
    Reg *r1;
    int z;

    for(r1 = r; r1 != R; r1 = r1->s1) {
        for(z=0; z<BITS; z++) {
            dif.b[z] = (dif.b[z] &
                ~(~r1->refbehind.b[z] & r1->refahead.b[z])) |
                    r1->set.b[z] | r1->regdiff.b[z];
            if(dif.b[z] != r1->regdiff.b[z]) {
                r1->regdiff.b[z] = dif.b[z];
                change++;
            }
        }
        if(r1->active)
            break;
        r1->active = 1;
        for(z=0; z<BITS; z++)
            dif.b[z] &= ~(~r1->calbehind.b[z] & r1->calahead.b[z]);
        if(r1->s2 != R)
            synch(r1->s2, dif);
    }
}
```

⟨*function* allreg*(arm)* 300b⟩≡ (466)

```
ulong
allreg(ulong b, Rgn *r)
{
    Var *v;
    int i;

    v = var + r->varno;
    r->regno = 0;
```

```
    switch(v->etype) {

    case TCHAR:
    case TUCHAR:
    case TSHORT:
    case TUSHORT:
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:
    case TIND:
    case TARRAY:
        i = BtoR(~b);
        if(i && r->cost >= 0) {
            r->regno = i;
            return RtoB(i);
        }
        break;

    case TVLONG:
    case TDOUBLE:
    case TFLOAT:
        i = BtoF(~b);
        if(i && r->cost >= 0) {
            r->regno = i+NREG;
            return FtoB(i);
        }
        break;

    default:
        diag(Z, "unknown etype %d/%d", bitno(b), v->etype);
        break;

    }
    return 0;
}
```

⟨*constant* CLOAD*(arm)* 301a⟩≡ (456)

```
#define CLOAD 4
```

⟨*constant* CREF*(arm)* 301b⟩≡ (456)

```
#define CREF 5
```

⟨*function* paint1*(arm)* 301c⟩≡ (466)

```
void
paint1(Reg *r, int bn)
{
    Reg *r1;
    Prog *p;
    int z;
    ulong bb;

    z = bn/32;
    bb = 1L<<(bn%32);
    if(r->act.b[z] & bb)
        return;
    for(;;) {
        if(!(r->refbehind.b[z] & bb))
            break;
        r1 = r->p1;
```

```
        if(r1 == R)
            break;
        if(!(r1->refahead.b[z] & bb))
            break;
        if(r1->act.b[z] & bb)
            break;
        r = r1;
    }

    if(LOAD(r) & ~(r->set.b[z] & ~(r->use1.b[z]|r->use2.b[z])) & bb) {
        change -= CLOAD * r->loop;
        if(debug['R'] && debug['v'])
            print("%ld%P\tld %B $%d\n", r->loop,
                r->prog, blsh(bn), change);
    }
    for(;;) {
        r->act.b[z] |= bb;
        p = r->prog;

        if(r->use1.b[z] & bb) {
            change += CREF * r->loop;
            if(debug['R'] && debug['v'])
                print("%ld%P\tu1 %B $%d\n", r->loop,
                    p, blsh(bn), change);
        }

        if((r->use2.b[z]|r->set.b[z]) & bb) {
            change += CREF * r->loop;
            if(debug['R'] && debug['v'])
                print("%ld%P\tu2 %B $%d\n", r->loop,
                    p, blsh(bn), change);
        }

        if(STORE(r) & r->regdiff.b[z] & bb) {
            change -= CLOAD * r->loop;
            if(debug['R'] && debug['v'])
                print("%ld%P\tst %B $%d\n", r->loop,
                    p, blsh(bn), change);
        }

        if(r->refbehind.b[z] & bb)
            for(r1 = r->p2; r1 != R; r1 = r1->p2link)
                if(r1->refahead.b[z] & bb)
                    paint1(r1, bn);

        if(!(r->refahead.b[z] & bb))
            break;
        r1 = r->s2;
        if(r1 != R)
            if(r1->refbehind.b[z] & bb)
                paint1(r1, bn);
        r = r->s1;
        if(r == R)
            break;
        if(r->act.b[z] & bb)
            break;
        if(!(r->refbehind.b[z] & bb))
            break;
    }
}
```

⟨*function* paint2*(arm)* 303a⟩≡ (466)

```
ulong
paint2(Reg *r, int bn)
{
    Reg *r1;
    int z;
    ulong bb, vreg;

    z = bn/32;
    bb = 1L << (bn%32);
    vreg = regbits;
    if(!(r->act.b[z] & bb))
        return vreg;
    for(;;) {
        if(!(r->refbehind.b[z] & bb))
            break;
        r1 = r->p1;
        if(r1 == R)
            break;
        if(!(r1->refahead.b[z] & bb))
            break;
        if(!(r1->act.b[z] & bb))
            break;
        r = r1;
    }
    for(;;) {
        r->act.b[z] &= ~bb;

        vreg |= r->regu;

        if(r->refbehind.b[z] & bb)
            for(r1 = r->p2; r1 != R; r1 = r1->p2link)
                if(r1->refahead.b[z] & bb)
                    vreg |= paint2(r1, bn);

        if(!(r->refahead.b[z] & bb))
            break;
        r1 = r->s2;
        if(r1 != R)
            if(r1->refbehind.b[z] & bb)
                vreg |= paint2(r1, bn);
        r = r->s1;
        if(r == R)
            break;
        if(!(r->act.b[z] & bb))
            break;
        if(!(r->refbehind.b[z] & bb))
            break;
    }
    return vreg;
}
```

⟨*function* paint3 303b⟩≡ (466)

```
void
paint3(Reg *r, int bn, long rb, int rn)
{
    Reg *r1;
    Prog *p;
    int z;
    ulong bb;
```

```
z = bn/32;
bb = 1L << (bn%32);
if(r->act.b[z] & bb)
    return;
for(;;) {
    if(!(r->refbehind.b[z] & bb))
        break;
    r1 = r->p1;
    if(r1 == R)
        break;
    if(!(r1->refahead.b[z] & bb))
        break;
    if(r1->act.b[z] & bb)
        break;
    r = r1;
}

if(LOAD(r) & ~(r->set.b[z] & ~(r->use1.b[z]|r->use2.b[z])) & bb)
    addmove(r, bn, rn, 0);
for(;;) {
    r->act.b[z] |= bb;
    p = r->prog;

    if(r->use1.b[z] & bb) {
        if(debug['R'])
            print("%P", p);
        addreg(&p->from, rn);
        if(debug['R'])
            print("\t.c%P\n", p);
    }
    if((r->use2.b[z]|r->set.b[z]) & bb) {
        if(debug['R'])
            print("%P", p);
        addreg(&p->to, rn);
        if(debug['R'])
            print("\t.c%P\n", p);
    }

    if(STORE(r) & r->regdiff.b[z] & bb)
        addmove(r, bn, rn, 1);
    r->regu |= rb;

    if(r->refbehind.b[z] & bb)
        for(r1 = r->p2; r1 != R; r1 = r1->p2link)
            if(r1->refahead.b[z] & bb)
                paint3(r1, bn, rb, rn);

    if(!(r->refahead.b[z] & bb))
        break;
    r1 = r->s2;
    if(r1 != R)
        if(r1->refbehind.b[z] & bb)
            paint3(r1, bn, rb, rn);
    r = r->s1;
    if(r == R)
        break;
    if(r->act.b[z] & bb)
        break;
    if(!(r->refbehind.b[z] & bb))
```

```
            break;
        }
    }
```

⟨*function* addreg*(arm)* 305a⟩≡ (466)

```
void
addreg(Adr *a, int rn)
{

    a->sym = 0;
    a->symkind = N_NONE;
    a->type = D_REG;
    a->reg = rn;
    if(rn >= NREG) {
        a->type = D_FREG;
        a->reg = rn-NREG;
    }
}
```

⟨*function* RtoB*(arm)* 305b⟩≡ (466)

```
/*
 * bit reg
 * 0 R0
 * 1 R1
 * ... ...
 * 10 R10
 */
long
RtoB(int r)
{

    if(r >= REGMIN && r <= REGMAX)
        return 1L << r;
    return 0;
}
```

⟨*function* BtoR*(arm)* 305c⟩≡ (466)

```
int
BtoR(long b)
{
    b &= 0x01fcL; // excluded R9 and R10 for m and g
    if(b == 0)
        return 0;
    return bitno(b);
}
```

⟨*function* FtoB*(arm)* 305d⟩≡ (466)

```
/*
 * bit reg
 * 18 F2
 * 19 F3
 * ... ...
 * 23 F7
 */
long
FtoB(int f)
{

    if(f < 2 || f > NFREG-1)
        return 0;
    return 1L << (f + 16);
}
```

⟨*function* BtoF*(arm)* 306a⟩≡ (466)

```
int
BtoF(long b)
{

    b &= 0xfc0000L;
    if(b == 0)
        return 0;
    return bitno(b) - 16;
}
```

13.5 Peephole optimizer: peep()

⟨*function* peep*(arm)* 306b⟩≡ (463)

```
/// codgen -> regopt -> <>
void
peep(void)
{
    Reg *r, *r1, *r2;
    Prog *p, *p1;
    int t;
/*
 * complete R structure
 */
    t = 0;
    for(r=firstr; r!=R; r=r1) {
        r1 = r->link;
        if(r1 == R)
            break;
        p = r->prog->link;
        while(p != r1->prog)
        switch(p->as) {
        default:
            r2 = rega();
            r->link = r2;
            r2->link = r1;

            r2->prog = p;
            r2->p1 = r;
            r->s1 = r2;
            r2->s1 = r1;
            r1->p1 = r2;

            r = r2;
            t++;

        case ADATA:
        case AGLOBL:
        case ANAME:
        case ASIGNAME:
            p = p->link;
        }
    }

loop1:
    t = 0;
    for(r=firstr; r!=R; r=r->link) {
        p = r->prog;
```

```
    if(p->as == ASLL || p->as == ASRL || p->as == ASRA) {
        /*
         * elide shift into D_SHIFT operand of subsequent instruction
         */
        if(shiftprop(r)) {
            excise(r);
            t++;
        }
    }
    if(p->as == AMOVW || p->as == AMOVF || p->as == AMOVD)
    if(regtyp(&p->to)) {
        if(p->from.type == D_CONST)
            constprop(&p->from, &p->to, r->s1);
        else if(regtyp(&p->from))
        if(p->from.type == p->to.type) {
            if(copyprop(r)) {
                excise(r);
                t++;
            } else
            if(subprop(r) && copyprop(r)) {
                excise(r);
                t++;
            }
        }
    }
}
if(t)
    goto loop1;
/*
 * look for MOVB x,R; MOVB R,R
 */
for(r=firstr; r!=R; r=r->link) {
    p = r->prog;
    switch(p->as) {
    default:
        continue;
    case AEOR:
        /*
         * EOR -1,x,y => MVN x,y
         */
        if(p->from.type == D_CONST && p->from.offset == -1) {
            p->as = AMVN;
            p->from.type = D_REG;
            if(p->reg != R_NONE)
                p->from.reg = p->reg;
            else
                p->from.reg = p->to.reg;
            p->reg = R_NONE;
        }
        continue;
    case AMOVH:
    case AMOVHU:
    case AMOVB:
    case AMOVBU:
        if(p->to.type != D_REG)
            continue;
        break;
    }
    r1 = r->link;
    if(r1 == R)
```

```
        continue;
    p1 = r1->prog;
    if(p1->as != p->as)
        continue;
    if(p1->from.type != D_REG || p1->from.reg != p->to.reg)
        continue;
    if(p1->to.type != D_REG || p1->to.reg != p->to.reg)
        continue;
    excise(r1);
}

for(r=firstr; r!=R; r=r->link) {
    p = r->prog;
    switch(p->as) {
    case AMOVW:
    case AMOVB:
    case AMOVBU:
        if(p->from.type == D_OREG && p->from.offset == 0)
            xtramodes(r, &p->from);
        else if(p->to.type == D_OREG && p->to.offset == 0)
            xtramodes(r, &p->to);
        else
            continue;
        break;
    case ACMP:
        /*
         * elide CMP $0,x if calculation of x can set condition codes
         */
        if(p->from.type != D_CONST || p->from.offset != 0)
            continue;
        r2 = r->s1;
        if(r2 == R)
            continue;
        t = r2->prog->as;
        switch(t) {
        default:
            continue;
        case ABEQ:
        case ABNE:
        case ABMI:
        case ABPL:
            break;
        case ABGE:
            t = ABPL;
            break;
        case ABLT:
            t = ABMI;
            break;
        case ABHI:
            t = ABNE;
            break;
        case ABLS:
            t = ABEQ;
            break;
        }
        r1 = r;
        do
            r1 = uniqp(r1);
        while (r1 != R && r1->prog->as == ANOP);
        if(r1 == R)
```

```
                continue;
            p1 = r1->prog;
            if(p1->to.type != D_REG)
                continue;
            if(p1->to.reg != p->reg)
            if(!(p1->as == AMOVW && p1->from.type == D_REG && p1->from.reg == p->reg))
                continue;
            switch(p1->as) {
            default:
                continue;
            case AMOVW:
                if(p1->from.type != D_REG)
                    continue;
            case AAND:
            case AEOR:
            case AORR:
            case ABIC:
            case AMVN:
            case ASUB:
            case ARSB:
            case AADD:
            case AADC:
            case ASBC:
            case ARSC:
                break;
            }
            p1->scond |= C_SBIT;
            r2->prog->as = t;
            excise(r);
            continue;
        }
    }

    predicate();
}
```

⟨*function* excise*(arm)* 309a⟩≡ (463)

```
void
excise(Reg *r)
{
    Prog *p;

    p = r->prog;
    p->as = ANOP;
    p->scond = zprog.scond;
    p->from = zprog.from;
    p->to = zprog.to;
    p->reg = zprog.reg; /**/
}
```

⟨*function* uniqp 309b⟩≡ (463)

```
Reg*
uniqp(Reg *r)
{
    Reg *r1;

    r1 = r->p1;
    if(r1 == R) {
        r1 = r->p2;
        if(r1 == R || r1->p2link != R)
```

```
            return R;
    } else
        if(r->p2 != R)
            return R;
    return r1;
}
```

⟨*function* uniqs 310a⟩≡ (463)

```
Reg*
uniqs(Reg *r)
{
    Reg *r1;

    r1 = r->s1;
    if(r1 == R) {
        r1 = r->s2;
        if(r1 == R)
            return R;
    } else
        if(r->s2 != R)
            return R;
    return r1;
}
```

⟨*function* regtyp*(arm)* 310b⟩≡ (463)

```
int
regtyp(Adr *a)
{

    if(a->type == D_REG)
        return 1;
    if(a->type == D_FREG)
        return 1;
    return 0;
}
```

⟨*function* subprop*(arm)* 310c⟩≡ (463)

```
/*
 * the idea is to substitute
 * one register for another
 * from one MOV to another
 * MOV a, R0
 * ADD b, R0 / no use of R1
 * MOV R0, R1
 * would be converted to
 * MOV a, R1
 * ADD b, R1
 * MOV R1, R0
 * hopefully, then the former or latter MOV
 * will be eliminated by copy propagation.
 */
int
subprop(Reg *r0)
{
    Prog *p;
    Adr *v1, *v2;
    Reg *r;
    int t;

    p = r0->prog;
```

```
v1 = &p->from;
if(!regtyp(v1))
    return 0;
v2 = &p->to;
if(!regtyp(v2))
    return 0;
for(r=uniqp(r0); r!=R; r=uniqp(r)) {
    if(uniqs(r) == R)
        break;
    p = r->prog;
    switch(p->as) {
    case ABL:
        return 0;

    case ACMP:
    case ACMN:
    case AADD:
    case ASUB:
    case ARSB:
    case ASLL:
    case ASRL:
    case ASRA:
    case AORR:
    case AAND:
    case AEOR:
    case AMUL:
    case ADIV:
    case ADIVU:

    case ACMPF:
    case ACMPD:
    case AADDD:
    case AADDF:
    case ASUBD:
    case ASUBF:
    case AMULD:
    case AMULF:
    case ADIVD:
    case ADIVF:
        if(p->to.type == v1->type)
        if(p->to.reg == v1->reg) {
            if(p->reg == R_NONE)
                p->reg = p->to.reg;
            goto gotit;
        }
        break;

    case AMOVF:
    case AMOVD:
    case AMOVW:
        if(p->to.type == v1->type)
        if(p->to.reg == v1->reg)
            goto gotit;
        break;

    case AMOVM:
        t = 1<<v2->reg;
        if((p->from.type == D_CONST && (p->from.offset&t)) ||
           (p->to.type == D_CONST && (p->to.offset&t)))
            return 0;
```

```
            break;
        }
        if(copyau(&p->from, v2) ||
           copyau1(p, v2) ||
           copyau(&p->to, v2))
            break;
        if(copysub(&p->from, v1, v2, 0) ||
           copysub1(p, v1, v2, 0) ||
           copysub(&p->to, v1, v2, 0))
            break;
    }
    return 0;

gotit:
    copysub(&p->to, v1, v2, 1);
    if(debug['P']) {
        print("gotit: %D->%D\n%P", v1, v2, r->prog);
        if(p->from.type == v2->type)
            print(" excise");
        print("\n");
    }
    for(r=uniqs(r); r!=r0; r=uniqs(r)) {
        p = r->prog;
        copysub(&p->from, v1, v2, 1);
        copysub1(p, v1, v2, 1);
        copysub(&p->to, v1, v2, 1);
        if(debug['P'])
            print("%P\n", r->prog);
    }
    t = v1->reg;
    v1->reg = v2->reg;
    v2->reg = t;
    if(debug['P'])
        print("%P last\n", r->prog);
    return 1;
}
```

⟨*function* copyprop 312⟩≡ (463)

```
/*
 * The idea is to remove redundant copies.
 * v1->v2 F=0
 * (use v2 s/v2/v1/)*
 * set v1 F=1
 * use v2 return fail
 * -----------------
 * v1->v2 F=0
 * (use v2 s/v2/v1/)*
 * set v1 F=1
 * set v2 return success
 */
int
copyprop(Reg *r0)
{
    Prog *p;
    Adr *v1, *v2;
    Reg *r;

    p = r0->prog;
    v1 = &p->from;
    v2 = &p->to;
```

```
    if(copyas(v1, v2))
        return 1;
    for(r=firstr; r!=R; r=r->link)
        r->active = 0;
    return copy1(v1, v2, r0->s1, 0);
}
```

⟨*function* copy1 313⟩≡ (463)

```
int
copy1(Adr *v1, Adr *v2, Reg *r, int f)
{
    int t;
    Prog *p;

    if(r->active) {
        if(debug['P'])
            print("act set; return 1\n");
        return 1;
    }
    r->active = 1;
    if(debug['P'])
        print("copy %D->%D f=%d\n", v1, v2, f);
    for(; r != R; r = r->s1) {
        p = r->prog;
        if(debug['P'])
            print("%P", p);
        if(!f && uniqp(r) == R) {
            f = 1;
            if(debug['P'])
                print("; merge; f=%d", f);
        }
        t = copyu(p, v2, A);
        switch(t) {
        case 2: /* rar, cant split */
            if(debug['P'])
                print("; %Drar; return 0\n", v2);
            return 0;

        case 3: /* set */
            if(debug['P'])
                print("; %Dset; return 1\n", v2);
            return 1;

        case 1: /* used, substitute */
        case 4: /* use and set */
            if(f) {
                if(!debug['P'])
                    return 0;
                if(t == 4)
                    print("; %Dused+set and f=%d; return 0\n", v2, f);
                else
                    print("; %Dused and f=%d; return 0\n", v2, f);
                return 0;
            }
            if(copyu(p, v2, v1)) {
                if(debug['P'])
                    print("; sub fail; return 0\n");
                return 0;
            }
            if(debug['P'])
```

```
                print("; sub%D/%D", v2, v1);
            if(t == 4) {
                if(debug['P'])
                    print("; %Dused+set; return 1\n", v2);
                return 1;
            }
            break;
        }
        if(!f) {
            t = copyu(p, v1, A);
            if(!f && (t == 2 || t == 3 || t == 4)) {
                f = 1;
                if(debug['P'])
                    print("; %Dset and !f; f=%d", v1, f);
            }
        }
        if(debug['P'])
            print("\n");
        if(r->s2)
            if(!copy1(v1, v2, r->s2, f))
                return 0;
    }
    return 1;
}
```

⟨*function* constprop*(arm)* 314⟩≡ (463)

```
/*
 * The idea is to remove redundant constants.
 * $c1->v1
 * ($c1->v2 s/$c1/v1)*
 * set v1  return
 * The v1->v2 should be eliminated by copy propagation.
 */
void
constprop(Adr *c1, Adr *v1, Reg *r)
{
    Prog *p;

    if(debug['C'])
        print("constprop %D->%D\n", c1, v1);
    for(; r != R; r = r->s1) {
        p = r->prog;
        if(debug['C'])
            print("%P", p);
        if(uniqp(r) == R) {
            if(debug['C'])
                print("; merge; return\n");
            return;
        }
        if(p->as == AMOVW && copyas(&p->from, c1)) {
                if(debug['C'])
                    print("; sub%D/%D", &p->from, v1);
                p->from = *v1;
        } else if(copyu(p, v1, A) > 1) {
            if(debug['C'])
                print("; %Dset; return\n", v1);
            return;
        }
        if(debug['C'])
            print("\n");
```

```
        if(r->s2)
            constprop(c1, v1, r->s2);
    }
}
```

⟨*function* FAIL*(arm)* 315a⟩≡ (463)

```
/*
 * ASLL x,y,w
 * .. (not use w, not set x y w)
 * AXXX w,a,b (a != w)
 * .. (not use w)
 * (set w)
 * ----------- changed to
 * ..
 * AXXX (x<<y),a,b
 * ..
 */
#define FAIL(msg) { if(debug['H']) print("\t%s; FAILURE\n", msg); return 0; }
```

⟨*function* shiftprop*(arm)* 315b⟩≡ (463)

```
int
shiftprop(Reg *r)
{
    Reg *r1;
    Prog *p, *p1, *p2;
    int n, o;
    Adr a;

    p = r->prog;
    if(p->to.type != D_REG)
        FAIL("BOTCH: result not reg");
    n = p->to.reg;
    a = zprog.from;
    if(p->reg != R_NONE && p->reg != p->to.reg) {
        a.type = D_REG;
        a.reg = p->reg;
    }
    if(debug['H'])
        print("shiftprop\n%P", p);
    r1 = r;
    for(;;) {
        /* find first use of shift result; abort if shift operands or result are changed */
        r1 = uniqs(r1);
        if(r1 == R)
            FAIL("branch");
        if(uniqp(r1) == R)
            FAIL("merge");
        p1 = r1->prog;
        if(debug['H'])
            print("\n%P", p1);
        switch(copyu(p1, &p->to, A)) {
        case 0: /* not used or set */
            if((p->from.type == D_REG && copyu(p1, &p->from, A) > 1) ||
               (a.type == D_REG && copyu(p1, &a, A) > 1))
                FAIL("args modified");
            continue;
        case 3: /* set, not used */
            FAIL("BOTCH: noref");
        }
        break;
```

```
}
/* check whether substitution can be done */
switch(p1->as) {
default:
    FAIL("non-dpi");
case AAND:
case AEOR:
case AADD:
case AADC:
case AORR:
case ASUB:
case ARSB:
case ASBC:
case ARSC:
    if(p1->reg == n || (p1->reg == R_NONE && p1->to.type == D_REG && p1->to.reg == n)) {
        if(p1->from.type != D_REG)
            FAIL("can't swap");
        p1->reg = p1->from.reg;
        p1->from.reg = n;
        switch(p1->as) {
        case ASUB:
            p1->as = ARSB;
            break;
        case ARSB:
            p1->as = ASUB;
            break;
        case ASBC:
            p1->as = ARSC;
            break;
        case ARSC:
            p1->as = ASBC;
            break;
        }
        if(debug['H'])
            print("\t=>%P", p1);
    }
case ABIC:
case ACMP:
case ACMN:
    if(p1->reg == n)
        FAIL("can't swap");
    if(p1->reg == R_NONE && p1->to.reg == n)
        FAIL("shift result used twice");
case AMVN:
    if(p1->from.type == D_SHIFT)
        FAIL("shift result used in shift");
    if(p1->from.type != D_REG || p1->from.reg != n)
        FAIL("BOTCH: where is it used?");
    break;
}
/* check whether shift result is used subsequently */
p2 = p1;
if(p1->to.reg != n)
for (;;) {
    r1 = uniqs(r1);
    if(r1 == R)
        FAIL("inconclusive");
    p1 = r1->prog;
    if(debug['H'])
        print("\n%P", p1);
```

```
        switch(copyu(p1, &p->to, A)) {
        case 0: /* not used or set */
            continue;
        case 3: /* set, not used */
            break;
        default:/* used */
            FAIL("reused");
        }
        break;
    }
    /* make the substitution */
    p2->from.type = D_SHIFT;
    p2->from.reg = R_NONE;
    o = p->reg;
    if(o == R_NONE)
        o = p->to.reg;
    switch(p->from.type){
    case D_CONST:
        o |= (p->from.offset&0x1f)<<7;
        break;
    case D_REG:
        o |= (1<<4) | (p->from.reg<<8);
        break;
    }
    switch(p->as){
    case ASLL:
        o |= 0<<5;
        break;
    case ASRL:
        o |= 1<<5;
        break;
    case ASRA:
        o |= 2<<5;
        break;
    }
    p2->from.offset = o;
    if(debug['H'])
        print("\t=>%P\tSUCCEED\n", p2);
    return 1;
}
```

⟨*function* findpre*(arm)* 317⟩≡ (463)

```
Reg*
findpre(Reg *r, Adr *v)
{
    Reg *r1;

    for(r1=uniqp(r); r1!=R; r=r1,r1=uniqp(r)) {
        if(uniqs(r1) != r)
            return R;
        switch(copyu(r1->prog, v, A)) {
        case 1: /* used */
        case 2: /* read-alter-rewrite */
            return R;
        case 3: /* set */
        case 4: /* set and used */
            return r1;
        }
    }
    return R;
```

```
}
```

⟨*function* findinc*(arm)* 318a⟩≡ (463)

```
Reg*
findinc(Reg *r, Reg *r2, Adr *v)
{
    Reg *r1;
    Prog *p;


    for(r1=uniqs(r); r1!=R && r1!=r2; r=r1,r1=uniqs(r)) {
        if(uniqp(r1) != r)
            return R;
        switch(copyu(r1->prog, v, A)) {
        case 0: /* not touched */
            continue;
        case 4: /* set and used */
            p = r1->prog;
            if(p->as == AADD)
            if(p->from.type == D_CONST)
            if(p->from.offset > -4096 && p->from.offset < 4096)
                return r1;
        default:
            return R;
        }
    }
    return R;
}
```

⟨*function* nochange*(arm)* 318b⟩≡ (463)

```
int
nochange(Reg *r, Reg *r2, Prog *p)
{
    Adr a[3];
    int i, n;

    if(r == r2)
        return 1;
    n = 0;
    if(p->reg != R_NONE && p->reg != p->to.reg) {
        a[n].type = D_REG;
        a[n++].reg = p->reg;
    }
    switch(p->from.type) {
    case D_SHIFT:
        a[n].type = D_REG;
        a[n++].reg = p->from.offset&0xf;
    case D_REG:
        a[n].type = D_REG;
        a[n++].reg = p->from.reg;
    }
    if(n == 0)
        return 1;
    for(; r!=R && r!=r2; r=uniqs(r)) {
        p = r->prog;
        for(i=0; i<n; i++)
            if(copyu(p, &a[i], A) > 1)
                return 0;
    }
    return 1;
}
```

⟨*function* findu1*(arm)* 319a⟩≡ (463)

```
int
findu1(Reg *r, Adr *v)
{
    for(; r != R; r = r->s1) {
        if(r->active)
            return 0;
        r->active = 1;
        switch(copyu(r->prog, v, A)) {
        case 1: /* used */
        case 2: /* read-alter-rewrite */
        case 4: /* set and used */
            return 1;
        case 3: /* set */
            return 0;
        }
        if(r->s2)
            if (findu1(r->s2, v))
                return 1;
    }
    return 0;
}
```

⟨*function* finduse*(arm)* 319b⟩≡ (463)

```
int
finduse(Reg *r, Adr *v)
{
    Reg *r1;

    for(r1=firstr; r1!=R; r1=r1->link)
        r1->active = 0;
    return findu1(r, v);
}
```

⟨*function* xtramodes*(arm)* 319c⟩≡ (463)

```
int
xtramodes(Reg *r, Adr *a)
{
    Reg *r1, *r2, *r3;
    Prog *p, *p1;
    Adr v;

    p = r->prog;
    if(debug['h'] && p->as == AMOVB && p->from.type == D_OREG) /* byte load */
        return 0;
    v = *a;
    v.type = D_REG;
    r1 = findpre(r, &v);
    if(r1 != R) {
        p1 = r1->prog;
        if(p1->to.type == D_REG && p1->to.reg == v.reg)
        switch(p1->as) {
        case AADD:
            if(p1->from.type == D_REG ||
               (p1->from.type == D_SHIFT && (p1->from.offset&(1<<4)) == 0 &&
                (p->as != AMOVB || (a == &p->from && (p1->from.offset&~0xf) == 0))) ||
               (p1->from.type == D_CONST &&
                p1->from.offset > -4096 && p1->from.offset < 4096))
            if(nochange(uniqs(r1), r, p1)) {
                if(a != &p->from || v.reg != p->to.reg)
```

```
                if (finduse(r->s1, &v)) {
                    if(p1->reg == R_NONE || p1->reg == v.reg)
                        /* pre-indexing */
                        p->scond |= C_WBIT;
                    else return 0;
                }
                switch (p1->from.type) {
                case D_REG:
                    /* register offset */
                    a->type = D_SHIFT;
                    a->offset = p1->from.reg;
                    break;
                case D_SHIFT:
                    /* scaled register offset */
                    a->type = D_SHIFT;
                case D_CONST:
                    /* immediate offset */
                    a->offset = p1->from.offset;
                    break;
                }
                if(p1->reg != R_NONE)
                    a->reg = p1->reg;
                excise(r1);
                return 1;
            }
            break;
        case AMOVW:
            if(p1->from.type == D_REG)
            if((r2 = findinc(r1, r, &p1->from)) != R) {
            for(r3=uniqs(r2); r3->prog->as==ANOP; r3=uniqs(r3))
                ;
            if(r3 == r) {
                /* post-indexing */
                p1 = r2->prog;
                a->reg = p1->to.reg;
                a->offset = p1->from.offset;
                p->scond |= C_PBIT;
                if(!finduse(r, &r1->prog->to))
                    excise(r1);
                excise(r2);
                return 1;
            }
            }
            break;
        }
    }
    if(a != &p->from || a->reg != p->to.reg)
    if((r1 = findinc(r, R, &v)) != R) {
        /* post-indexing */
        p1 = r1->prog;
        a->offset = p1->from.offset;
        p->scond |= C_PBIT;
        excise(r1);
        return 1;
    }
    return 0;
}
```

⟨*function* copyu*(arm)* 320⟩≡ (463)

```
/*
```

```
 * return
 * 1 if v only used (and substitute),
 * 2 if read-alter-rewrite
 * 3 if set
 * 4 if set and used
 * 0 otherwise (not touched)
 */
int
copyu(Prog *p, Adr *v, Adr *s)
{

    switch(p->as) {

    default:
        if(debug['P'])
            print(" (???)");
        return 2;

    case AMOVM:
        if(v->type != D_REG)
            return 0;
        if(p->from.type == D_CONST) { /* read reglist, read/rar */
            if(s != A) {
                if(p->from.offset&(1<<v->reg))
                    return 1;
                if(copysub(&p->to, v, s, 1))
                    return 1;
                return 0;
            }
            if(copyau(&p->to, v)) {
                if(p->scond&C_WBIT)
                    return 2;
                return 1;
            }
            if(p->from.offset&(1<<v->reg))
                return 1;
        } else {   /* read/rar, write reglist */
            if(s != A) {
                if(p->to.offset&(1<<v->reg))
                    return 1;
                if(copysub(&p->from, v, s, 1))
                    return 1;
                return 0;
            }
            if(copyau(&p->from, v)) {
                if(p->scond&C_WBIT)
                    return 2;
                if(p->to.offset&(1<<v->reg))
                    return 4;
                return 1;
            }
            if(p->to.offset&(1<<v->reg))
                return 3;
        }
        return 0;

    case ANOP: /* read, write */
    case AMOVW:
    case AMOVF:
    case AMOVD:
```

```
case AMOVH:
case AMOVHU:
case AMOVB:
case AMOVBU:
case AMOVDW:
case AMOVWD:
case AMOVFD:
case AMOVDF:
    if(p->scond&(C_WBIT|C_PBIT))
    if(v->type == D_REG) {
        if(p->from.type == D_OREG || p->from.type == D_SHIFT) {
            if(p->from.reg == v->reg)
                return 2;
        } else {
        if(p->to.reg == v->reg)
            return 2;
        }
    }
    if(s != A) {
        if(copysub(&p->from, v, s, 1))
            return 1;
        if(!copyas(&p->to, v))
            if(copysub(&p->to, v, s, 1))
                return 1;
        return 0;
    }
    if(copyas(&p->to, v)) {
        if(copyau(&p->from, v))
            return 4;
        return 3;
    }
    if(copyau(&p->from, v))
        return 1;
    if(copyau(&p->to, v))
        return 1;
    return 0;


case AADD: /* read, read, write */
case ASUB:
case ARSB:
case ASLL:
case ASRL:
case ASRA:
case AORR:
case AAND:
case AEOR:
case AMUL:
case ADIV:
case ADIVU:
case AADDF:
case AADDD:
case ASUBF:
case ASUBD:
case AMULF:
case AMULD:
case ADIVF:
case ADIVD:

case ACMPF:
```

```
case ACMPD:
case ACMP:
case ACMN:
case ACASE:
    if(s != A) {
        if(copysub(&p->from, v, s, 1))
            return 1;
        if(copysub1(p, v, s, 1))
            return 1;
        if(!copyas(&p->to, v))
            if(copysub(&p->to, v, s, 1))
                return 1;
        return 0;
    }
    if(copyas(&p->to, v)) {
        if(p->reg == R_NONE)
            p->reg = p->to.reg;
        if(copyau(&p->from, v))
            return 4;
        if(copyau1(p, v))
            return 4;
        return 3;
    }
    if(copyau(&p->from, v))
        return 1;
    if(copyau1(p, v))
        return 1;
    if(copyau(&p->to, v))
        return 1;
    return 0;

case ABEQ: /* read, read */
case ABNE:
case ABHS:
case ABLO:
case ABMI:
case ABPL:
case ABVS:
case ABVC:
case ABHI:
case ABLS:
case ABGE:
case ABLT:
case ABGT:
case ABLE:
    if(s != A) {
        if(copysub(&p->from, v, s, 1))
            return 1;
        return copysub1(p, v, s, 1);
    }
    if(copyau(&p->from, v))
        return 1;
    if(copyau1(p, v))
        return 1;
    return 0;

case AB: /* funny */
    if(s != A) {
        if(copysub(&p->to, v, s, 1))
            return 1;
```

```
            return 0;
        }
        if(copyau(&p->to, v))
            return 1;
        return 0;

    case ARET: /* funny */
        if(v->type == D_REG)
        if(v->reg == REGRET)
            return 2;
        if(v->type == D_FREG)
        if(v->reg == FREGRET)
            return 2;

    case ABL: /* funny */
        if(v->type == D_REG) {
            if(v->reg <= REGEXT && v->reg > exregoffset)
                return 2;
            if(v->reg == (uchar)REGARG)
                return 2;
        }
        if(v->type == D_FREG)
            if(v->reg <= FREGEXT && v->reg > exfregoffset)
                return 2;

        if(s != A) {
            if(copysub(&p->to, v, s, 1))
                return 1;
            return 0;
        }
        if(copyau(&p->to, v))
            return 4;
        return 3;

    case ATEXT: /* funny */
        if(v->type == D_REG)
            if(v->reg == (uchar)REGARG)
                return 3;
        return 0;
    }
}
```

⟨*function* a2type*(arm)* 324⟩≡ (463)

```
int
a2type(Prog *p)
{

    switch(p->as) {

    case ACMP:
    case ACMN:

    case AADD:
    case ASUB:
    case ARSB:
    case ASLL:
    case ASRL:
    case ASRA:
    case AORR:
    case AAND:
```

```
    case AEOR:
    case AMUL:
    case ADIV:
    case ADIVU:
        return D_REG;

    case ACMPF:
    case ACMPD:

    case AADDF:
    case AADDD:
    case ASUBF:
    case ASUBD:
    case AMULF:
    case AMULD:
    case ADIVF:
    case ADIVD:
        return D_FREG;
    }
    return D_NONE;
}
```

⟨*function* copyas*(arm)* 325a⟩≡ (463)

```
/*
 * direct reference,
 * could be set/use depending on
 * semantics
 */
int
copyas(Adr *a, Adr *v)
{

    if(regtyp(v)) {
        if(a->type == v->type)
        if(a->reg == v->reg)
            return 1;
    } else if(v->type == D_CONST) {  /* for constprop */
        if(a->type == v->type)
        if(a->symkind == v->symkind)
        if(a->sym == v->sym)
        if(a->reg == v->reg)
        if(a->offset == v->offset)
            return 1;
    }
    return 0;
}
```

⟨*function* copyau*(arm)* 325b⟩≡ (463)

```
/*
 * either direct or indirect
 */
int
copyau(Adr *a, Adr *v)
{

    if(copyas(a, v))
        return 1;
    if(v->type == D_REG) {
        if(a->type == D_OREG) {
            if(v->reg == a->reg)
```

```
                return 1;
        } else if(a->type == D_SHIFT) {
            if((a->offset&0xf) == v->reg)
                return 1;
            if((a->offset&(1<<4)) &&
                (a->offset>>8) == v->reg)
                return 1;
        }
    }
    return 0;
}
```

⟨*function* copyau1*(arm)* 326a⟩≡ (463)

```
int
copyau1(Prog *p, Adr *v)
{

    if(regtyp(v)) {
        if(a2type(p) == v->type)
        if(p->reg == v->reg) {
            if(a2type(p) != v->type)
                print("botch a2type %P\n", p);
            return 1;
        }
    }
    return 0;
}
```

⟨*function* copysub*(arm)* 326b⟩≡ (463)

```
/*
 * substitute s for v in a
 * return failure to substitute
 */
int
copysub(Adr *a, Adr *v, Adr *s, int f)
{

    if(f)
    if(copyau(a, v)) {
        if(a->type == D_SHIFT) {
            if((a->offset&0xf) == v->reg)
                a->offset = (a->offset&~0xf)|s->reg;
            if((a->offset&(1<<4)) &&
                          (a->offset>>8) == v->reg)
                a->offset = (a->offset&~(0xf<<8))|(s->reg<<8);
        } else
            a->reg = s->reg;
    }
    return 0;
}
```

⟨*function* copysub1*(arm)* 326c⟩≡ (463)

```
int
copysub1(Prog *p1, Adr *v, Adr *s, int f)
{

    if(f)
    if(copyau1(p1, v))
        p1->reg = s->reg;
    return 0;
}
```

⟨*global* predinfo*(arm)* 327a⟩≡ (463)

```
struct {
    int opcode;
    int notopcode;
    int scond;
    int notscond;
} predinfo[]  = {
    { ABEQ, ABNE, 0x0, 0x1, },
    { ABNE, ABEQ, 0x1, 0x0, },
    { ABHS, ABLO, 0x2, 0x3, },
    { ABLO, ABHS, 0x3, 0x2, },
    { ABMI, ABPL, 0x4, 0x5, },
    { ABPL, ABMI, 0x5, 0x4, },
    { ABVS, ABVC, 0x6, 0x7, },
    { ABVC, ABVS, 0x7, 0x6, },
    { ABHI, ABLS, 0x8, 0x9, },
    { ABLS, ABHI, 0x9, 0x8, },
    { ABGE, ABLT, 0xA, 0xB, },
    { ABLT, ABGE, 0xB, 0xA, },
    { ABGT, ABLE, 0xC, 0xD, },
    { ABLE, ABGT, 0xD, 0xC, },
};
```

⟨*enum* _anon_*(arm)* 327b⟩≡ (463)

```
enum {
    Join,
    Split,
    End,
    Branch,
    Setcond,
    Toolong
};
```

⟨*enum* _anon_ (5c/peep.c)*(arm)* 327c⟩≡ (463)

```
enum {
    Falsecond,
    Truecond,
    Delbranch,
    Keepbranch
};
```

⟨*function* isbranch*(arm)* 327d⟩≡ (463)

```
int
isbranch(Prog *p)
{
    return (ABEQ <= p->as) && (p->as <= ABLE);
}
```

⟨*function* predicable*(arm)* 327e⟩≡ (463)

```
int
predicable(Prog *p)
{
    if (isbranch(p)
        || p->as == ANOP
        || p->as == AXXX
        || p->as == ADATA
        || p->as == AGLOBL
        || p->as == AGOK
        || p->as == AHISTORY
        || p->as == ANAME
```

```
        || p->as == ASIGNAME
        || p->as == ATEXT
        || p->as == AWORD
//        || p->as == ADYNT
//        || p->as == AINIT
        || p->as == ABCASE
        || p->as == ACASE)
        return 0;
    return 1;
}
```

⟨*function* modifiescpsr*(arm)* 328a⟩≡ (463)

```
/*
 * Depends on an analysis of the encodings performed by 5l.
 * These seem to be all of the opcodes that lead to the "S" bit
 * being set in the instruction encodings.
 *
 * C_SBIT may also have been set explicitly in p->scond.
 */
int
modifiescpsr(Prog *p)
{
    return (p->scond&C_SBIT)
        || p->as == ATST
        || p->as == ATEQ
        || p->as == ACMN
        || p->as == ACMP
        || p->as == AMULU
        || p->as == ADIVU
        || p->as == AMUL
        || p->as == ADIV
        || p->as == AMOD
        || p->as == AMODU
        || p->as == ABL;
}
```

⟨*struct* Joininfo 328b⟩≡ (463)

```
typedef struct Joininfo {
    Reg *start;
    Reg *last;
    Reg *end;
    int len;
} Joininfo;
```

⟨*function* joinsplit*(arm)* 328c⟩≡ (463)

```
/*
 * Find the maximal chain of instructions starting with r which could
 * be executed conditionally
 */
int
joinsplit(Reg *r, Joininfo *j)
{
    j->start = r;
    j->last = r;
    j->len = 0;
    do {
        if (r->p2 && (r->p1 || r->p2->p2link)) {
            j->end = r;
            return Join;
        }
```

```
        if (r->s1 && r->s2) {
            j->end = r;
            return Split;
        }
        j->last = r;
        if (r->prog->as != ANOP)
            j->len++;
        if (!r->s1 && !r->s2) {
            j->end = r->link;
            return End;
        }
        if (r->s2) {
            j->end = r->s2;
            return Branch;
        }
        if (modifiescpsr(r->prog)) {
            j->end = r->s1;
            return Setcond;
        }
        r = r->s1;
    } while (j->len < 4);
    j->end = r;
    return Toolong;
}
```

⟨*function* successor*(arm)* 329a⟩≡ (463)

```
Reg *
successor(Reg *r)
{
    if (r->s1)
        return r->s1;
    else
        return r->s2;
}
```

⟨*function* applypred*(arm)* 329b⟩≡ (463)

```
void
applypred(Reg *rstart, Joininfo *j, int cond, int branch)
{
    int pred;
    Reg *r;

    if(j->len == 0)
        return;
    if (cond == Truecond)
        pred = predinfo[rstart->prog->as - ABEQ].scond;
    else
        pred = predinfo[rstart->prog->as - ABEQ].notscond;

    for (r = j->start; ; r = successor(r)) {
        if (r->prog->as == AB) {
            if (r != j->last || branch == Delbranch)
                excise(r);
            else {
              if (cond == Truecond)
                r->prog->as = predinfo[rstart->prog->as - ABEQ].opcode;
              else
                r->prog->as = predinfo[rstart->prog->as - ABEQ].notopcode;
            }
        }
```

```
        else if (predicable(r->prog))
            r->prog->scond = (r->prog->scond&~C_SCOND)|pred;
        if (r->s1 != r->link) {
            r->s1 = r->link;
            r->link->p1 = r;
        }
        if (r == j->last)
            break;
    }
}
```

⟨*function* predicate*(arm)* 330a⟩≡ (463)

```
void
predicate(void)
{
    Reg *r;
    int t1, t2;
    Joininfo j1, j2;

    for(r=firstr; r!=R; r=r->link) {
        if (isbranch(r->prog)) {
            t1 = joinsplit(r->s1, &j1);
            t2 = joinsplit(r->s2, &j2);
            if(j1.last->link != j2.start)
                continue;
            if(j1.end == j2.end)
            if((t1 == Branch && (t2 == Join || t2 == Setcond)) ||
               (t2 == Join && (t1 == Join || t1 == Setcond))) {
                applypred(r, &j1, Falsecond, Delbranch);
                applypred(r, &j2, Truecond, Delbranch);
                excise(r);
                continue;
            }
            if(t1 == End || t1 == Branch) {
                applypred(r, &j1, Falsecond, Keepbranch);
                excise(r);
                continue;
            }
        }
    }
}
```

13.6 Dominators

⟨*function* postorder 330b⟩≡ (466)

```
/*
 * find looping structure
 *
 * 1) find reverse postordering
 * 2) find approximate dominators,
 * the actual dominators if the flow graph is reducible
 * otherwise, dominators plus some other non-dominators.
 * See Matthew S. Hecht and Jeffrey D. Ullman,
 * "Analysis of a Simple Algorithm for Global Data Flow Problems",
 * Conf.  Record of ACM Symp. on Principles of Prog. Langs, Boston, Massachusetts,
 * Oct. 1-3, 1973, pp.  207-217.
 * 3) find all nodes with a predecessor dominated by the current node.
 * such a node is a loop head.
```

```
 * recursively, all preds with a greater rpo number are in the loop
 */
long
postorder(Reg *r, Reg **rpo2r, long n)
{
    Reg *r1;

    r->rpo = 1;
    r1 = r->s1;
    if(r1 && !r1->rpo)
        n = postorder(r1, rpo2r, n);
    r1 = r->s2;
    if(r1 && !r1->rpo)
        n = postorder(r1, rpo2r, n);
    rpo2r[n] = r;
    n++;
    return n;
}
```

Chapter 14

Linking Support

14.1 #pragma lib and automagic linking

⟨macprag() *other locals* 332a⟩≡ (74a)

```
int c0, c;
char *hp;
Hist *h;
```

⟨macprag() *if pragma xxx* 332b⟩≡ (74a) 336c ▷

```
if(s && strcmp(s->name, "lib") == 0) {
    c0 = getnsc();
    if(c0 != '"') {
        c = c0;
        if(c0 != '<')
            goto bad;
        c0 = '>';
    }
    for(hp = symb;;) {
        c = getc();
        if(c == c0)
            break;
        if(c == '\n')
            goto bad;
        *hp++ = c;
    }
    *hp = '\0';
    c = getcom();
    if(c != '\n')
        goto bad;

    /*
     * put pragma-line in as a funny history
     */
    c = strlen(symb) + 1;
    while(c & 3)
        c++;

    while(nhunk < c)
        gethunk();
    hp = hunk;
    memcpy(hunk, symb, c);
    nhunk -= c;
    hunk += c;

    h = alloc(sizeof(Hist));
```

```
    h->name = hp;
    h->line = lineno;
    h->offset = -1; // pragma

    // add_list(h, hist)
    h->link = H;
    if(ehist == H) {
        hist = h;
        ehist = h;
        return;
    }
    ehist->link = h;
    ehist = h;

    return;

bad:
    unget(c);
    yyerror("syntax in #pragma lib");
    macend();

}
```

14.2 Safe linking with type signatures: `5c -T`

⟨*enum* `signature` 333a⟩≡ (429b)

```
enum signature {
    SIGNONE = 0,
    SIGDONE = 1,
    SIGINTERN = 2,

    // ???
    SIGNINTERN = 1729*325*1729,
};
```

⟨`Sym` *identifier value, code generation fields* 333b⟩+≡ (23b) ◁235b

```
// enum<signature>
char sig;
```

⟨`zname()` *if generate signature for symbol s* 333c⟩≡ (238a)

```
if(debug['T'] &&
   t == N_EXTERN &&
   s->sig != SIGDONE &&
   s->type != types[TENUM] &&
   s != symrathole){

    sig = sign(s);
    bf[0] = ASIGNAME;
    bf[1] = sig;
    bf[2] = sig>>8;
    bf[3] = sig>>16;
    bf[4] = sig>>24;
    bf[5] = t;
    bf[6] = s->symidx;
    Bwrite(b, bf, 7);
    s->sig = SIGDONE;
}
```

14.2.1 sign()

⟨*function* sign 334a⟩≡ (450)

```
/// main -> compile -> gclean -> outcode -> zname -> <>
ulong
sign(Sym *s)
{
    ulong v;
    Type *t;

    if(s->sig == SIGINTERN)
        return SIGNINTERN;
    if((t = s->type) == T)
        return 0;
    v = signature(t);
    if(v == 0)
        v = SIGNINTERN;
    return v;
}
```

⟨*function* signature 334b⟩≡ (450)

```
/// sign -> <>
ulong
signature(Type *t)
{
    ulong s;
    Typetab tt;

    tt.n = 0;
    tt.a = nil;
    s = signat(t, &tt);
    free(tt.a);
    return s;
}
```

⟨*function* signat 334c⟩≡ (450)

```
/// sign -> signature -> <>
static ulong
signat(Type *t, Typetab *tt)
{
    int i;
    Type *t1;
    long s;

    s = 0;
    for(; t; t=t->link) {
        s = s*thash1 + thash[t->etype];
        if(t->garb&GINCOMPLETE)
            return s;
        switch(t->etype) {
        case TARRAY:
            s = s*thash2 + 0; /* was t->width */
            break;
        case TFUNC:
            for(t1=t->down; t1; t1=t1->down)
                s = s*thash3 + signat(t1, tt);
            break;
        case TSTRUCT:
        case TUNION:
            if((i = sigind(t, tt)) >= 0){
```

```
                s = s*thash2 + i;
                return s;
            }
            for(t1=t->link; t1; t1=t1->down)
                s = s*thash3 + signat(t1, tt);
            return s;
        case TIND:
            break;
        default:
            return s;
        }
    }
    return s;
}
```

⟨*global* thash1 335a⟩≡ (443b)

```
ulong thash1 = 0x2edab8c9;
```

⟨*global* thash2 335b⟩≡ (443b)

```
ulong thash2 = 0x1dc74fb8;
```

⟨*global* thash3 335c⟩≡ (443b)

```
ulong thash3 = 0x1f241331;
```

⟨*global* thash 335d⟩≡ (443b)

```
ulong thash[NALLTYPES];
```

⟨*global* thashinit 335e⟩≡ (443b)

```
Init thashinit[] =
{
    TXXX,  0x17527bbd, 0,

    TCHAR,  0x5cedd32b, 0,
    TUCHAR,  0x552c4454, 0,
    TSHORT,  0x63040b4b, 0,
    TUSHORT, 0x32a45878, 0,
    TINT,  0x4151d5bd, 0,
    TUINT,  0x5ae707d6, 0,
    TLONG,  0x5ef20f47, 0,
    TULONG,  0x36d8eb8f, 0,
    TVLONG,  0x6e5e9590, 0,
    TUVLONG, 0x75910105, 0,
    TFLOAT,  0x25fd7af1, 0,
    TDOUBLE, 0x7c40a1b2, 0,

    TIND,  0x1b832357, 0,
    TFUNC,  0x6babc9cb, 0,
    TARRAY,  0x7c50986d, 0,
    TVOID,  0x44112eff, 0,
    TSTRUCT, 0x7c2da3bf, 0,
    TUNION,  0x3eb25e98, 0,
    TENUM,  0x44b54f61, 0,

    TOLD,  0x22b15988, 0,
    TDOT,  0x0204f6b3, 0,

    -1,  0,  0,
};
```

⟨*struct* Typetab 336a⟩≡ (450)

```
struct Typetab {
    int n;
    Type **a;
};
```

⟨*function* sigind 336b⟩≡ (450)

```
/// signat -> <>
static int
sigind(Type *t, Typetab *tt)
{
    int n;
    Type **a, **na, **p, **e;

    n = tt->n;
    a = tt->a;
    e = a+n;
    /* linear search seems ok */
    for(p = a ; p < e; p++)
        if(sametype(*p, t))
            return p-a;
    if((n&15) == 0){
        na = malloc((n+16)*sizeof(Type*));
        memmove(na, a, n*sizeof(Type*));
        free(a);
        a = tt->a = na;
    }
    a[tt->n++] = t;
    return -1;
}
```

14.2.2 #pragma incomplete

⟨macprag() *if pragma xxx* 336c⟩+≡ (74a) ◁332b 348d▷

```
if(s && strcmp(s->name, "incomplete") == 0) {
    pragincomplete();
    return;
}
```

⟨*function* pragincomplete 336d⟩≡ (453)

```
void
pragincomplete(void)
{
    Sym *s;
    Type *t;
    bool istag;
    int  w, et;

    istag = false;
    s = getsym();
    if(s == nil)
        goto out;
    et = 0;
    w = s->lexical;
    if(w == LSTRUCT)
        et = TSTRUCT;
    else if(w == LUNION)
        et = TUNION;
    if(et != 0){
```

```
        s = getsym();
        if(s == nil){
            yyerror("missing struct/union tag in pragma incomplete");
            goto out;
        }
        if(s->lexical != LNAME && s->lexical != LTYPE){
            yyerror("invalid struct/union tag: %s", s->name);
            goto out;
        }
        dotag(s, et, 0);
        istag = true;
    }else if(strcmp(s->name, "_off_") == 0){
        debug['T'] = false;
        goto out;
    }else if(strcmp(s->name, "_on_") == 0){
        debug['T'] = true;
        goto out;
    }

    t = s->type;
    if(istag)
        t = s->suetag;
    if(t == T)
        yyerror("unknown type %s in pragma incomplete", s->name);
    else if(!typesu[t->etype])
        yyerror("not struct/union type in pragma incomplete: %s", s->name);
    else
        t->garb |= GINCOMPLETE;
out:
    while(getnsc() != '\n')
        ;
    if(debug['f'])
        print("%s incomplete\n", s->name);
}
```

⟨Qualifier *other cases* 337⟩≡ (20d)

```
GINCOMPLETE = 1<<2,
```

Chapter 15

Debugging Support

15.1 General debugging metadata

15.2 Acid debugger metadata: `5c -a`

⟨`compile()` *if writing acid to standard output* 338a⟩≡ (32e)

```
/*
 * if we're writing acid to standard output, don't keep scratching
 * outbuf.
 */
if((debug['a'] || debug['Z']) && !debug['n']) {
    if (first) {
        outfile = nil;
        Binit(&outbuf, dup(1, -1), OWRITE);
        dup(2, 1);
    }
}
```

⟨`compile()` *adjust p for outfile if acid option* 338b⟩≡ (33e)

```
if(debug['a'] && debug['n'])
    strcat(p, ".acid");
```

15.2.1 Entities

⟨`dodecl()` *case ONAME, debugging support* 338c⟩≡ (79f)

```
acidvar(s);
```

⟨`doenum()` *debugging support* 338d⟩≡ (105d)

```
acidvar(s);
```

⟨*function* `acidvar` 338e⟩≡ (448)

```
/// doenum? | dodecl? -> <>
void
acidvar(Sym *s)
{
    int n;
    Io *i;
    Type *t;
    Sym *s1, *s2;

    ⟨acidvar() return if no -a or -s 339a⟩
    ⟨acidvar() if -aa 339b⟩
```

```
    t = s->type;
    while(t && t->etype == TIND)
        t = t->link;
    if(t == T)
        return;
    if(t->etype == TENUM) {
        Bprint(&outbuf, "%s = ", amap(s->name));
        if(!typefd[t->etype])
            Bprint(&outbuf, "%lld;\n", s->vconst);
        else
            Bprint(&outbuf, "%f\n;", s->fconst);
        return;
    }
    if(!typesu[t->etype])
        return;
    s1 = acidsue(t->link);
    if(s1 == S)
        return;
    switch(s->class) {
    case CAUTO:
    case CPARAM:
        s2 = acidfun(thisfntype);
        if(s2)
            Bprint(&outbuf, "complex %s %s:%s;\n",
                amap(s1->name), amap(s2->name), amap(s->name));
        break;

    case CEXTERN:
    case CGLOBL:
    case CSTATIC: case CLOCAL:
        Bprint(&outbuf, "complex %s %s;\n",
            amap(s1->name), amap(s->name));
        break;
    }
}
```

⟨acidvar() *return if no -a or -s* 339a⟩≡ (338e)

```
if(!debug['a'] || debug['s'])
    return;
```

⟨acidvar() *if -aa* 339b⟩≡ (338e)

```
if(debug['a'] > 1) {
    n = 0;
    for(i=iostack; i; i=i->link)
        n++;
    if(n > 1)
        return;
}
```

⟨*function* acidsue 339c⟩≡ (448)

```
/// acidvar -> <>
Sym*
acidsue(Type *t)
{
    int h;
    Sym *s;

    if(t != T)
    for(h=0; h<nelem(hash); h++)
        for(s = hash[h]; s != S; s = s->link)
```

```
            if(s->suetag && s->suetag->link == t)
                return s;
    return 0;
}
```

⟨*function* acidfun 340a⟩≡ (448)

```
/// acidvar -> <>
Sym*
acidfun(Type *t)
{
    int h;
    Sym *s;

    for(h=0; h<nelem(hash); h++)
        for(s = hash[h]; s != S; s = s->link)
            if(s->type == t)
                return s;
    return 0;
}
```

⟨*global* kwd 340b⟩≡ (448)

```
static char *kwd[] =
{
    "$adt", "$aggr", "$append", "$builtin", "$complex", "$defn",
    "$delete", "$do", "$else", "$eval", "$head", "$if",
    "$local", "$loop", "$return", "$tail", "$then",
    "$union", "$whatis", "$while",
};
```

⟨*function* amap 340c⟩≡ (448)

```
char*
amap(char *s)
{
    int i, bot, top, new;

    bot = 0;
    top = bot + nelem(kwd) - 1;
    while(bot <= top){
        new = bot + (top - bot)/2;
        i = strcmp(kwd[new]+1, s);
        if(i == 0)
            return kwd[new];

        if(i < 0)
            bot = new + 1;
        else
            top = new - 1;
    }
    return s;
}
```

15.2.2 Structure/union definitions

⟨*function* acidtype 340d⟩≡ (448)

```
void
acidtype(Type *t)
{
    Sym *s;
    Type *l;
```

```
    Io *i;
    int n;
    char *an;

    ⟨acidtype() return if no -a 341a⟩
    ⟨acidtype() if -aa 341b⟩

    s = acidsue(t->link);
    if(s == S)
        return;
    switch(t->etype) {
    case TUNION:
    case TSTRUCT:
        if(debug['s'])
            goto asmstr;
        an = amap(s->name);
        Bprint(&outbuf, "sizeof%s = %ld;\n", an, t->width);
        Bprint(&outbuf, "aggr %s\n{\n", an);
        for(l = t->link; l != T; l = l->down)
            acidmember(l, 0, 1);
        Bprint(&outbuf, "};\n\n");

        Bprint(&outbuf, "defn\n%s(addr) {\n\tcomplex %s addr;\n", an, an);
        for(l = t->link; l != T; l = l->down)
            acidmember(l, 0, 0);
        Bprint(&outbuf, "};\n\n");
        break;

    asmstr:
        if(s == S)
            break;
        for(l = t->link; l != T; l = l->down)
            if(l->sym != S)
                Bprint(&outbuf, "#define\t%s.%s\t%ld\n",
                    s->name,
                    l->sym->name,
                    l->offset);
        break;

    default:
        Bprint(&outbuf, "T%d\n", t->etype);
        return;

    }
}
```

⟨acidtype() *return if no -a* 341a⟩≡ (340d)

```
if(!debug['a'])
    return;
```

⟨acidtype() *if -aa* 341b⟩≡ (340d)

```
if(debug['a'] > 1) {
    n = 0;
    for(i=iostack; i; i=i->link)
        n++;
    if(n > 1)
        return;
}
```

⟨*global* acidchar 341c⟩≡ (448)

```
char acidchar[NTYPE];
```

⟨*global* acidcinit 342a⟩≡ (448)

```
Init acidcinit[] =
{
    TCHAR,  'C', 0,
    TUCHAR,  'b', 0,
    TSHORT,  'd', 0,
    TUSHORT, 'u', 0,
    TLONG,  'D', 0,
    TULONG,  'U', 0,
    TVLONG,  'V', 0,
    TUVLONG, 'W', 0,
    TFLOAT,  'f', 0,
    TDOUBLE, 'F', 0,
    TARRAY,  'a', 0,
    TIND,  'X', 0,
    -1,  0, 0,
};
```

⟨*function* acidinit 342b⟩≡ (448)

```
static void
acidinit(void)
{
    Init *p;

    for(p=acidcinit; p->code >= 0; p++)
        acidchar[p->code] = p->value;

    acidchar[TINT] = acidchar[TLONG];
    acidchar[TUINT] = acidchar[TULONG];
    if(types[TINT]->width != types[TLONG]->width) {
        acidchar[TINT] = acidchar[TSHORT];
        acidchar[TUINT] = acidchar[TUSHORT];
        if(types[TINT]->width != types[TSHORT]->width)
            warn(Z, "acidmember int not long or short");
    }
    if(types[TIND]->width == types[TUVLONG]->width)
        acidchar[TIND] = 'Y';

}
```

⟨*function* acidmember 342c⟩≡ (448)

```
void
acidmember(Type *t, long off, int flag)
{
    Sym *s, *s1;
    Type *l;
    static bool acidcharinit = false;

    if(acidcharinit == false) {
        acidinit();
        acidcharinit = true;
    }
    s = t->sym;
    switch(t->etype) {
    default:
        Bprint(&outbuf, " T%d\n", t->etype);
        break;

    case TIND:
        if(s == S)
```

```
        break;
    if(flag) {
        for(l=t; l->etype==TIND; l=l->link)
            ;
        if(typesu[l->etype]) {
            s1 = acidsue(l->link);
            if(s1 != S) {
                Bprint(&outbuf, " 'A' %s %ld %s;\n",
                    amap(s1->name),
                    t->offset+off, amap(s->name));
                break;
            }
        }
    } else {
        Bprint(&outbuf,
            "\tprint(\"\t%s\t\", addr.%s\\X, \"\\n\");\n",
            amap(s->name), amap(s->name));
        break;
    }

case TINT:
case TUINT:
case TCHAR:
case TUCHAR:
case TSHORT:
case TUSHORT:
case TLONG:
case TULONG:
case TVLONG:
case TUVLONG:
case TFLOAT:
case TDOUBLE:
case TARRAY:
    if(s == S)
        break;
    if(flag) {
        Bprint(&outbuf, " '%c' %ld %s;\n",
        acidchar[t->etype], t->offset+off, amap(s->name));
    } else {
        Bprint(&outbuf, "\tprint(\"\t%s\t\", addr.%s, \"\\n\");\n",
            amap(s->name), amap(s->name));
    }
    break;

case TSTRUCT:
case TUNION:
    s1 = acidsue(t->link);
    if(s1 == S)
        break;
    if(flag) {
        if(s == S) {
            Bprint(&outbuf, " {\n");
            for(l = t->link; l != T; l = l->down)
                acidmember(l, t->offset+off, flag);
            Bprint(&outbuf, " };\n");
        } else {
            Bprint(&outbuf, " %s %ld %s;\n",
                amap(s1->name),
                t->offset+off, amap(s->name));
        }
```

```
        } else {
            if(s != S) {
                Bprint(&outbuf, "\tprint(\"%s %s {\\n\");\n",
                    amap(s1->name), amap(s->name));
                Bprint(&outbuf, "\t%s(addr.%s);\n",
                    amap(s1->name), amap(s->name));
                Bprint(&outbuf, "\tprint(\"}\\n\");\n");
            } else {
                Bprint(&outbuf, "\tprint(\"%s {\\n\");\n",
                    amap(s1->name));
                Bprint(&outbuf, "\t\t%s(addr+%ld);\n",
                    amap(s1->name), t->offset+off);
                Bprint(&outbuf, "\tprint(\"}\\n\");\n");
            }
        }
        break;
    }
}
```

15.3 Pickle: 5c -Z

⟨compile() *adjust p for outfile if pickle option* 344a⟩≡ (33e)

```
else if(debug['Z'] && debug['n'])
    strcat(p, "_pickle.c");
```

⟨*global kwd (cc/pickle.c)* 344b⟩≡ (449a)

```
static char *kwd[] =
{
    "$adt", "$aggr", "$append", "$complex", "$defn",
    "$delete", "$do", "$else", "$eval", "$head", "$if",
    "$local", "$loop", "$return", "$tail", "$then",
    "$union", "$whatis", "$while",
};
```

⟨*global* picklestr 344c⟩≡ (449a)

```
static char picklestr[] = "\tpickle(s, un, ";
```

⟨*function* pmap 344d⟩≡ (449a)

```
static char*
pmap(char *s)
{
    int i, bot, top, new;

    bot = 0;
    top = bot + nelem(kwd) - 1;
    while(bot <= top){
        new = bot + (top - bot)/2;
        i = strcmp(kwd[new]+1, s);
        if(i == 0)
            return kwd[new];

        if(i < 0)
            bot = new + 1;
        else
            top = new - 1;
    }
    return s;
}
```

⟨*function* picklesue 345a⟩≡ (449a)

```
Sym*
picklesue(Type *t)
{
    int h;
    Sym *s;

    if(t != T)
    for(h=0; h<nelem(hash); h++)
        for(s = hash[h]; s != S; s = s->link)
            if(s->suetag && s->suetag->link == t)
                return s;
    return 0;
}
```

⟨*global* picklechar 345b⟩≡ (449a)

```
char picklechar[NTYPE];
```

⟨*global* picklecinit 345c⟩≡ (449a)

```
Init picklecinit[] =
{
    TCHAR,  'C', 0,
    TUCHAR,  'b', 0,
    TSHORT,  'd', 0,
    TUSHORT,  'u', 0,
    TLONG,  'D', 0,
    TULONG,  'U', 0,
    TVLONG,  'V', 0,
    TUVLONG, 'W', 0,
    TFLOAT,  'f', 0,
    TDOUBLE,  'F', 0,
    TARRAY,  'a', 0,
    TIND,  'X', 0,
    -1,  0, 0,
};
```

⟨*function* pickleinit 345d⟩≡ (449a)

```
static void
pickleinit(void)
{
    Init *p;

    for(p=picklecinit; p->code >= 0; p++)
        picklechar[p->code] = p->value;

    picklechar[TINT] = picklechar[TLONG];
    picklechar[TUINT] = picklechar[TULONG];
    if(types[TINT]->width != types[TLONG]->width) {
        picklechar[TINT] = picklechar[TSHORT];
        picklechar[TUINT] = picklechar[TUSHORT];
        if(types[TINT]->width != types[TSHORT]->width)
            warn(Z, "picklemember int not long or short");
    }

}
```

⟨*function* picklemember 345e⟩≡ (449a)

```
void
picklemember(Type *t, long off)
{
```

```
Sym *s, *s1;
static int picklecharinit = 0;

if(picklecharinit == 0) {
    pickleinit();
    picklecharinit = 1;
}
s = t->sym;
switch(t->etype) {
default:
    Bprint(&outbuf, " T%d\n", t->etype);
    break;

case TIND:
    if(s == S)
        Bprint(&outbuf,
            "%s\"p\", (char*)addr+%ld+_i*%ld);\n",
            picklestr, t->offset+off, t->width);
    else
        Bprint(&outbuf,
            "%s\"p\", &addr->%s);\n",
            picklestr, pmap(s->name));
    break;

case TINT:
case TUINT:
case TCHAR:
case TUCHAR:
case TSHORT:
case TUSHORT:
case TLONG:
case TULONG:
case TVLONG:
case TUVLONG:
case TFLOAT:
case TDOUBLE:
    if(s == S)
        Bprint(&outbuf, "%s\"%c\", (char*)addr+%ld+_i*%ld);\n",
            picklestr, picklechar[t->etype], t->offset+off, t->width);
    else
        Bprint(&outbuf, "%s\"%c\", &addr->%s);\n",
            picklestr, picklechar[t->etype], pmap(s->name));
    break;
case TARRAY:
    Bprint(&outbuf, "\tfor(_i = 0; _i < %ld; _i++) {\n\t",
        t->width/t->link->width);
    picklemember(t->link, t->offset+off);
    Bprint(&outbuf, "\t}\n\t_i = 0;\n\tUSED(_i);\n");
    break;

case TSTRUCT:
case TUNION:
    s1 = picklesue(t->link);
    if(s1 == S)
        break;
    if(s == S) {
        Bprint(&outbuf, "\tpickle_%s(s, un, (%s*)((char*)addr+%ld+_i*%ld));\n",
            pmap(s1->name), pmap(s1->name), t->offset+off, t->width);
    } else {
        Bprint(&outbuf, "\tpickle_%s(s, un, &addr->%s);\n",
```

```
                pmap(s1->name), pmap(s->name));
        }
        break;
    }
}
```

⟨*function* pickletype 347⟩≡ (449a)

```
void
pickletype(Type *t)
{
    Sym *s;
    Type *l;
    Io *i;
    int n;
    char *an;

    if(!debug['P'])
        return;
    if(debug['P'] > 1) {
        n = 0;
        for(i=iostack; i; i=i->link)
            n++;
        if(n > 1)
            return;
    }
    s = picklesue(t->link);
    if(s == S)
        return;
    switch(t->etype) {
    default:
        Bprint(&outbuf, "T%d\n", t->etype);
        return;

    case TUNION:
    case TSTRUCT:
        if(debug['s'])
            goto asmstr;
        an = pmap(s->name);

        Bprint(&outbuf, "void\npickle_%s(void *s, int un, %s *addr)\n{\n\tint _i = 0;\n\n\tUSED(_i);\n", an, an
        for(l = t->link; l != T; l = l->down)
            picklemember(l, 0);
        Bprint(&outbuf, "}\n\n");
        break;
    asmstr:
        if(s == S)
            break;
        for(l = t->link; l != T; l = l->down)
            if(l->sym != S)
                Bprint(&outbuf, "#define\t%s.%s\t%ld\n",
                    s->name,
                    l->sym->name,
                    l->offset);
        break;
    }
}
```

Chapter 16

Profiling Support

⟨*global* `profileflg` 348a⟩≡ (436d)
```
bool profileflg;
```

⟨`main()` *remaining initialisation* 348b⟩+≡ (30d) ◁31i 353e▷
```
profileflg = true; /* #pragma can turn it off */
```

⟨`gpseudo()` *if TEXT, set possible TEXT attributes* 348c⟩≡ (157d)
```
if(a == ATEXT)
    p->reg = (profileflg ? 0 : NOPROF);
```

16.1 Text attributes

16.2 #pragma profile

⟨`macprag()` *if pragma xxx* 348d⟩+≡ (74a) ◁336c 379c▷
```
if(s && strcmp(s->name, "profile") == 0) {
    pragprofile();
    return;
}
```

⟨*function* `pragprofile` 348e⟩≡ (453)
```
void
pragprofile(void)
{
    Sym *s;

    profileflg = false;
    s = getsym();
    if(s) {
        profileflg = atoi(s->name+1);
        if(strcmp(s->name, "on") == 0 ||
           strcmp(s->name, "yes") == 0)
            profileflg = true;
    }
    while(getnsc() != '\n')
        ;
    ⟨pragprofile() debug if -f 349⟩
}
```

⟨pragprofile() *debug if* -f 349⟩≡ (348e)

```
if(debug['f'])
    if(profileflg)
        print("%4ld: profileflg %d\n", lineno, profileflg);
    else
        print("%4ld: profileflg off\n", lineno);
```

Chapter 17

Advanced Topics TODO

17.1 Advanced C features

17.1.1 Function and function pointer automatic conversions

⟨tcomo() *when OFUNC case, add OIND if pointer function call* 350a⟩≡ (138b)

```
if(l->type->etype == TIND && l->type->link->etype == TFUNC) {
    l = new1(OIND, l, Z);
    l->type = l->left->type->link;
    n->left = l;
}
```

⟨paramconv() *switch etype cases* 350b⟩≡ (146e) 350c ▷

```
case TFUNC:
    t = typ(TIND, t);
    t->width = types[TIND]->width;
    break;
```

17.1.2 Array and pointer automatic conversions

⟨paramconv() *switch etype cases* 350c⟩+≡ (146e) ◁ 350b

```
case TARRAY:
    t = typ(TIND, t->link);
    t->width = types[TIND]->width;
    break;
```

17.1.3 Local static

⟨Storage_class *cases* 350d⟩+≡ (20e) ◁ 108j 365a ▷

```
CLOCAL, // local static
```

⟨adecl() *adjust storage to CLOCAL if static local variable* 350e⟩≡ (111b)

```
if(c == CSTATIC)
    c = CLOCAL;
```

⟨dodecl() *case ONAME, if local static variable* 350f⟩≡ (79f)

```
if(s->class == CLOCAL)
    s = mkstatic(s);
```

⟨*name rule, if local static variable* 350g⟩≡ (88b)

```
if($1->class == CLOCAL)
    $1 = mkstatic($1);
```

⟨*function* mkstatic 351a⟩≡ (450)

```
Sym*
mkstatic(Sym *s)
{
    Sym *s1;

    ⟨mkstatic() sanity check s 351b⟩
    snprint(symb, NSYMB, "%s$%d", s->name, s->block);
    s1 = lookup();

    if(s1->class != CSTATIC) {
        s1->type = s->type;
        s1->offset = s->offset;
        s1->block = s->block;
        s1->class = CSTATIC;
    }
    return s1;
}
```

⟨mkstatic() *sanity check s* 351b⟩≡ (351a)

```
if(s->class != CLOCAL) // should never happen, use errorexit()?
    return s;
```

17.1.4 Local volatile

⟨revertdcl() *other locals* 351c⟩≡ (26d)

```
Node *n1;
```

⟨reverdcl() *if volatile symbol* 351d⟩≡ (27c)

```
if(s->type && (s->type->garb & GVOLATILE)) {
    // add USED(&local_volatile);
    n1 = new(ONAME, Z, Z);

    n1->sym = s;
    n1->type = s->type;
    n1->etype = (s->type != T) ? s->type->etype : TVOID;
    n1->xoffset = s->offset;
    n1->class = s->class;

    n1 = new(OADDR, n1, Z);
    n1 = new(OUSED, n1, Z);

    // add_list(n1, n)
    if(n == Z)
        n = n1;
    else
        n = new(OLIST, n1, n);
}
```

⟨*xdecl rule, adjust block body with possible hidden generated nodes* 351e⟩≡ (80c)

```
if(n)
    $6 = new(OLIST, n, $6);
```

⟨*ulstmt rule, adjust block body with possible hidden generated nodes* 351f⟩≡ (81f)

```
if(n)
    $$ = new(OLIST, n, $2);
```

⟨*ulstmt rule, adjust forexpr with possible hidden generated nodes* 352a⟩≡ (82e)

```
if(n){
    if($4)
        $4 = new(OLIST, n, $4);
    else
        $4 = n;
}
```

⟨`usedset()` *switch node op cases* 352b⟩≡ (148i)

```
case OADDR: /* volatile */
    gins(ANOP, n, Z);
    break;
```

17.1.5 Bitfields

⟨*global* `firstbit` 352c⟩≡ (436d)

```
bool firstbit;
```

⟨*global* `lastbit` 352d⟩≡ (436d)

```
long lastbit;
```

⟨`dodecl()` *case ONAME, other initializations* 352e⟩≡ (79f)

```
firstbit = false;
```

⟨*sbody rule, initializations before parsing the fields* 352f⟩≡ (103a)

```
lastbit = 0;
firstbit = true;
```

⟨*edecor rule, set fields after parsed a field declarator* 352g⟩≡ (103d)

```
lastbit = 0;
firstbit = true;
```

⟨*global* `lastfield` 352h⟩≡ (436d)

```
// option<long> None = -1 | 0
long lastfield;
```

⟨`dodecl()` *other initializations* 352i⟩≡ (78d)

```
lastfield = 0;
```

⟨*declaration nodes* 352j⟩+≡ (16a) ◁108d

```
OBIT,
```

⟨*edecor rule* 352k⟩+≡ (103c) ◁103d

```
|   tag ':' lexpr   { $$ = new(OBIT, $1, $3); }
|   ':' lexpr       { $$ = new(OBIT, Z, $2); }
```

⟨`dodecl()` *switch node kind cases* 352l⟩+≡ (78d) ◁107b

```
case OBIT:
    n1 = n->right;
    complex(n1);

    lastfield = -1; // None
    if(n1->op == OCONST)
        lastfield = n1->vconst;
    if(lastfield < 0) {
        diag(n, "field width must be non-negative constant");
        lastfield = 1;
    }

    if(lastfield == 0) {
```

```
        lastbit = 0;
        firstbit = true;
        if(n->left != Z) {
            diag(n, "zero width named field");
            lastfield = 1;
        }
    }
    if(!typei[t->etype]) {
        diag(n, "field type must be int-like");
        t = types[TINT];
        lastfield = 1;
    }
    if(lastfield > tfield->width*8) {
        diag(n, "field width larger than field unit");
        lastfield = 1;
    }

    lastbit += lastfield;
    if(lastbit > tfield->width*8) {
        lastbit = lastfield;
        firstbit = true;
    }
    n = n->left;
    goto loop;
```

⟨edecl() *if lastfield* 353a⟩≡ (104a)

```
if(lastfield) {
    t->shift = lastbit - lastfield;
    t->nbits = lastfield;
    if(firstbit)
        t->shift = -t->shift;

    if(typeu[t->etype])
        t->etype = tufield->etype;
    else
        t->etype = tfield->etype;
}
```

⟨*global* tfield 353b⟩≡ (436d)

```
Type* tfield;
```

⟨*global* tufield 353c⟩≡ (436d)

```
Type* tufield;
```

⟨ginit() *misc initialisations* 353d⟩+≡ (35d) ◁230d 365g▷

```
tfield = types[TLONG];
```

⟨main() *remaining initialisation* 353e⟩+≡ (30d) ◁348b

```
tufield = simplet((1L<<tfield->etype) | BUNSIGNED);
```

⟨tcomo() *when OADDR, check if bitfield* 353f⟩≡ (136b)

```
if(l->type->nbits) {
    diag(n, "address of a bit field");
    goto bad;
}
```

⟨tcomo() *when OSIZE, check if sizeof bitfield* 353g⟩≡ (143b)

```
if(l->op == OBIT) {
    diag(n, "sizeof bitfield");
    goto bad;
}
```

⟨init1() *sanity check t has no bitfields* 354a⟩≡ (213f)

```
if(t->nbits)
    diag(Z, "cannot initialize bitfields");
```

⟨makedot() *if bitfields* 354b⟩≡ (194b)

```
if(t->nbits) {
    n1 = new(OXXX, Z, Z);
    *n1 = *n;
    n->op = OBIT;
    n->left = n1;
    n->right = Z;
    n->type = t;
    n->addable = n1->left->addable;
    n = n1;
}
```

⟨cgenrel() *switch node kind cases* 354c⟩+≡ (179f) ◁209 354g▷

```
case OBIT:
    ⟨cgenrel() nullwarn check if nn is null 152c⟩
    bitload(n, &nod, Z, Z, nn);
    gopcode(OAS, &nod, Z, nn);
    regfree(&nod);
    break;
```

⟨cgenrel() *when OAS, if bitfield assignment* 354d⟩≡ (188e)

```
if(l->op == OBIT)
    goto bitas;
```

⟨cgenrel() *when OASMUL/OASDIV/etc, if bitfield assignment* 354e⟩≡ (193a)

```
if(l->op == OBIT)
    goto asbitop;
```

⟨cgenrel() *when OASADD/OASSUB/etc, if bitfield assignment* 354f⟩≡ (193a)

```
if(l->op == OBIT)
    goto asbitop;
```

⟨cgenrel() *switch node kind cases* 354g⟩+≡ (179f) ◁354c 355b▷

```
asbitop:
    regalloc(&nod4, n, nn);
    if(l->complex >= r->complex) {
        bitload(l, &nod, &nod1, &nod2, &nod4);
        regalloc(&nod3, r, Z);
        cgen(r, &nod3);
    } else {
        regalloc(&nod3, r, Z);
        cgen(r, &nod3);
        bitload(l, &nod, &nod1, &nod2, &nod4);
    }
    gmove(&nod, &nod4);
    gopcode(o, &nod3, Z, &nod4);
    regfree(&nod3);
    gmove(&nod4, &nod);
    regfree(&nod4);
    bitstore(l, &nod, &nod1, &nod2, nn);
    break;
```

⟨cgenrel() *when OPOSTINC/OPOSTDEC, if bitfield* 354h⟩≡ (203a)

```
if(l->op == OBIT)
    goto bitinc;
```

⟨cgenrel() *when OPREINC/OPREDEC, if bitfield* 355a⟩≡ (203b)

```
if(l->op == OBIT)
    goto bitinc;
```

⟨cgenrel() *switch node kind cases* 355b⟩+≡ (179f) ◁354g 355c▷

```
bitinc:
    if(nn != Z && (o == OPOSTINC || o == OPOSTDEC)) {
        bitload(l, &nod, &nod1, &nod2, Z);
        gopcode(OAS, &nod, Z, nn);
        gopcode(OADD, nodconst(v), Z, &nod);
        bitstore(l, &nod, &nod1, &nod2, Z);
        break;
    }
    bitload(l, &nod, &nod1, &nod2, nn);
    gopcode(OADD, nodconst(v), Z, &nod);
    bitstore(l, &nod, &nod1, &nod2, nn);
    break;
```

⟨cgenrel() *switch node kind cases* 355c⟩+≡ (179f) ◁355b

```
bitas:
    n = l->left;
    regalloc(&nod, r, nn);
    if(l->complex >= r->complex) {
        reglcgen(&nod1, n, Z);
        cgen(r, &nod);
    } else {
        cgen(r, &nod);
        reglcgen(&nod1, n, Z);
    }
    regalloc(&nod2, n, Z);
    gopcode(OAS, &nod1, Z, &nod2);
    bitstore(l, &nod, &nod1, &nod2, nn);
    break;
```

⟨*function* bitload*(arm)* 355d⟩≡ (464b)

```
void
bitload(Node *b, Node *n1, Node *n2, Node *n3, Node *nn)
{
    int sh;
    long v;
    Node *l;

    /*
     * n1 gets adjusted/masked value
     * n2 gets address of cell
     * n3 gets contents of cell
     */
    l = b->left;
    if(n2 != Z) {
        regalloc(n1, l, nn);
        reglcgen(n2, l, Z);
        regalloc(n3, l, Z);
        gopcode(OAS, n2, Z, n3);
        gopcode(OAS, n3, Z, n1);
    } else {
        regalloc(n1, l, nn);
        cgen(l, n1);
    }
    if(b->type->shift == 0 && typeu[b->type->etype]) {
        v = ~0 + (1L << b->type->nbits);
```

```
        gopcode(OAND, nodconst(v), Z, n1);
    } else {
        sh = 32 - b->type->shift - b->type->nbits;
        if(sh > 0)
            gopcode(OASHL, nodconst(sh), Z, n1);
        sh += b->type->shift;
        if(sh > 0)
            if(typeu[b->type->etype])
                gopcode(OLSHR, nodconst(sh), Z, n1);
            else
                gopcode(OASHR, nodconst(sh), Z, n1);
    }
}
```

⟨*function* bitstore*(arm)* 356a⟩≡ (464b)

```
void
bitstore(Node *b, Node *n1, Node *n2, Node *n3, Node *nn)
{
    long v;
    Node nod, *l;
    int sh;

    /*
     * n1 has adjusted/masked value
     * n2 has address of cell
     * n3 has contents of cell
     */
    l = b->left;
    regalloc(&nod, l, Z);
    v = ~0 + (1L << b->type->nbits);
    gopcode(OAND, nodconst(v), Z, n1);
    gopcode(OAS, n1, Z, &nod);
    if(nn != Z)
        gopcode(OAS, n1, Z, nn);
    sh = b->type->shift;
    if(sh > 0)
        gopcode(OASHL, nodconst(sh), Z, &nod);
    v <<= sh;
    gopcode(OAND, nodconst(~v), Z, n3);
    gopcode(OOR, n3, Z, &nod);
    gopcode(OAS, &nod, Z, n2);

    regfree(&nod);
    regfree(n1);
    regfree(n2);
    regfree(n3);
}
```

17.1.6 Old style prototypes

⟨*parameter declarator rules* 356b⟩+≡ (77b) ◁107e

```
/*
 * parameter declarator
 */
pdecl:
  /* empty */
|   pdecl ctlist pdlist ';'

pdlist:
```

```
    xdecor              { dodecl(pdecl, lastclass, lasttype, $1); }
|   pdlist ',' pdlist
```

⟨*constant* OLDPROTO 357a⟩≡ (450)

```
#define OLDPROTO 2
```

⟨argmark() *if old proto style* 357b⟩≡ (109a)

```
if(pass != 0 && anyproto(n->right) == OLDPROTO) {
    t = typ(TFUNC, n->left->sym->type->link);
    t->down = typ(TOLD, T);
    t->down->down = ofnproto(n->right);
    tmerge(t, n->left->sym);
    n->left->sym->type = t;
}
```

⟨walkparam() *when OPROTO case, if no name found and old style decl* 357c⟩≡ (110b)

```
dodecl(NODECL, CPARAM, n->type, n->left);
pdecl(CPARAM, lastdcltype, S);
```

⟨*function* ofnproto 357d⟩≡ (450)

```
Type*
ofnproto(Node *n)
{
    Type *tl, *tr, *t;

    if(n == Z)
        return T;
    switch(n->op) {
    case OLIST:
        tl = ofnproto(n->left);
        tr = ofnproto(n->right);
        if(tl == T)
            return tr;
        tl->down = tr;
        return tl;

    case ONAME:
        t = copytyp(n->sym->type);
        t->down = T;
        return t;
    }
    return T;
}
```

⟨*constant* ANSIPROTO 357e⟩≡ (450)

```
#define ANSIPROTO 1
```

⟨fnproto() *old prototype style handling* 357f⟩≡ (107c)

```
int r;

r = anyproto(n->right);
if(r == 0 || (r & OLDPROTO)) {
    if(r & ANSIPROTO)
        diag(n, "mixed ansi/old function declaration: %F", n->left);
    return T;
}
```

⟨*function* anyproto 358a⟩≡ (450)

```
bool
anyproto(Node *n)
{
    int r;

    r = false;

loop:
    if(n == Z)
        return r;
    switch(n->op) {
    case OLIST:
        r |= anyproto(n->left);
        n = n->right;
        goto loop;

    case ODOTDOT:
    case OPROTO:
        return r | ANSIPROTO;
    }
    return r | OLDPROTO;
}
```

⟨walkparam() *switch op cases* 358b⟩+≡ (109b) ◁110a

```
case ONAME:
    s = n->sym;
    if(pass == 0) {
        push1(s);
        s->offset = -1;
        break;
    }
    if(s->offset != -1) {
        if(autoffset == 0) {
            firstarg = s;
            firstargtype = s->type;
        }
        autoffset = align(autoffset, s->type, Aarg1);
        s->offset = autoffset;
        autoffset = align(autoffset, s->type, Aarg2);
    } else
        dodecl(pdecl, CXXX, types[TINT], n);
    break;
```

⟨Type_kind_bis *misc cases* 358c⟩≡ (93f)

```
TOLD,
```

⟨rsametype() *continue if old style type* 358d⟩≡ (116b)

```
if(t1->etype == TOLD) {
    t1 = t1->down;
    continue;
}
if(t2->etype == TOLD) {
    t2 = t2->down;
    continue;
}
```

⟨tmerge() *switch etype t1 cases* 358e⟩≡ (117d)

```
case TFUNC:
    ta = t1->down;
```

```
    tb = t2->down;
    if(ta == T) {
        t1->down = tb;
        break;
    }
    if(tb == T)
        break;
    while(ta != T && tb != T) {
        if(ta == tb)
            break;
        /* ignore old-style flag */
        if(ta->etype == TOLD) {
            ta = ta->down;
            continue;
        }
        if(tb->etype == TOLD) {
            tb = tb->down;
            continue;
        }
        /* checking terminated by ... */
        if(ta->etype == TDOT && tb->etype == TDOT) {
            ta = T;
            tb = T;
            break;
        }
        if(!sametype(ta, tb))
            break;
        ta = ta->down;
        tb = tb->down;
    }
    if(ta != tb)
        diag(Z, "function inconsistently declared: %s", s->name);

    /* take new-style over old-style */
    ta = t1->down;
    tb = t2->down;
    if(ta != T && ta->etype == TOLD)
        if(tb != T && tb->etype != TOLD)
            t1->down = tb;
    break;
```

⟨`tcomo()` *when OFUNC case, warn function args not checked if old proto* 359a⟩≡ (138b)

```
  if(!debug['B'])
      if(l->type->down == T || l->type->down->etype == TOLD) {
          nerrors--;
          diag(n, "function args not checked: %F", l);
      }
```

17.1.7 `_Noreturn()`

17.2 C extensions

17.2.1 Typing extensions: `typeext()`

⟨`tcomo()` *when OEQ/ONE, typing extensions* 359b⟩≡ (131a)

```
  typeext(l->type, r);
  typeext(r->type, l);
```

⟨tcomo() *when OAS, typing extension* 360a⟩≡ (134b 133a)
```
typeext(l->type, r);
```

⟨tcomo() *when OASADD, typing extension* 360b⟩≡ (134c)
```
typeext(l->type, r);
```

⟨tcomo() *when ORETURN, typing extensions* 360c⟩≡ (144a)
```
typeext(n->type, l);
```

⟨tcoma() *typing extension on argument* 360d⟩≡ (138d)
```
typeext(t, n);
```

⟨*function* typeext 360e⟩≡ (443b)
```
void
typeext(Type *st, Node *l)
{
    Type *lt = l->type;
    ⟨typeext() other locals 361b⟩

    ⟨typeext() sanity check lt 360f⟩

    ⟨typeext() nil pointer 360g⟩
    ⟨typeext() float handling 382a⟩
    ⟨typeext() unnamed substructure handling 361c⟩
}
```

⟨typeext() *sanity check lt* 360f⟩≡ (360e)
```
if(lt == T)
    return;
```

⟨typeext() *nil pointer* 360g⟩≡ (360e)
```
if(st->etype == TIND && vconst(l) == 0) {
    l->type = st;
    l->vconst = 0;
    return;
}
```

17.2.2 Unnamed structure elements

⟨*zedlist rule* 360h⟩≡ (103b)
```
zedlist:                    /* extension */
 /* empty */
    {
        lastfield = 0;
        edecl(CXXX, lasttype, S);
    }
|   edlist
```

⟨edecl() *if unnamed structure element* 360i⟩≡ (104a)
```
if(s == S) {
    if(!typesu[t->etype])
        diag(Z, "unnamed structure element must be struct/union");
    if(c != CXXX)
        diag(Z, "unnamed structure element cannot have class");
}
```

⟨dotsearch() *if unnamed substructures* 361a⟩≡ (137d)

```
/*
 * look it up in unnamed substructures
 */
for(t1 = t; t1 != T; t1 = t1->down)
    if(t1->sym == S && typesu[t1->etype]){
        // recurse!
        rt = dotsearch(s, t1->link, n, off);
        if(rt != T) {
            if(xt != T)
                goto ambig;
            xt = rt;
            *off += t1->offset;
        }
    }
```

⟨typeext() *other locals* 361b⟩≡ (360e)

```
Node *n1, *n2;
long o;
```

⟨typeext() *unnamed substructure handling* 361c⟩≡ (360e)

```
/*
 * extension of C
 * if assign of struct containing unnamed sub-struct
 * to type of sub-struct, insert the DOT.
 * if assign of *struct containing unnamed substruct
 * to type of *sub-struct, insert the add-offset
 */
if(typesu[st->etype] && typesu[lt->etype]) {
    o = dotoffset(st, lt, l);
    if(o >= 0) {
        n1 = new1(OXXX, Z, Z);
        *n1 = *l;
        l->op = ODOT;
        l->left = n1;
        l->right = Z;
        makedot(l, st, o);
    }
    return;
}
// else

if(st->etype == TIND && typesu[st->link->etype])
if(lt->etype == TIND && typesu[lt->link->etype]) {
    o = dotoffset(st->link, lt->link, l);
    if(o >= 0) {
        l->type = st;
        if(o == 0)
            return;
        // else
        n1 = new1(OXXX, Z, Z);
        *n1 = *l;
        n2 = new1(OCONST, Z, Z);
        n2->vconst = o;
        n2->type = st;
        l->op = OADD;
        l->left = n1;
        l->right = n2;
    }
    return;
}
```

⟨*function* dotoffset 362⟩≡ (443b)

```
/// typeext -> <>
long
dotoffset(Type *st, Type *lt, Node *n)
{
    Type *t;
    Sym *g;
    long o, o1;

    o = -1;
    /*
     * first try matching at the top level
     * for matching tag names
     */
    g = st->tag;
    if(g != S)
        for(t=lt->link; t!=T; t=t->down)
            if(t->sym == S)
                if(g == t->tag) {
                    if(o >= 0)
                        goto ambig;
                    o = t->offset;
                }
    if(o >= 0)
        return o;

    /*
     * second try matching at the top level
     * for similar types
     */
    for(t=lt->link; t!=T; t=t->down)
        if(t->sym == S)
            if(sametype(st, t)) {
                if(o >= 0)
                    goto ambig;
                o = t->offset;
            }
    if(o >= 0)
        return o;

    /*
     * last try matching sub-levels
     */
    for(t=lt->link; t!=T; t=t->down)
        if(t->sym == S)
        if(typesu[t->etype]) {
            // recurse
            o1 = dotoffset(st, t, n);
            if(o1 >= 0) {
                if(o >= 0)
                    goto ambig;
                o = o1 + t->offset;
            }
        }
    return o;

ambig:
    diag(n, "ambiguous unnamed structure element");
    return o;
}
```

17.2.3 Struct constructors

⟨*expression nodes* 363a⟩+≡ (16a) ◁213d 374a▷

```
OSTRUCT,
OUNION,
```

⟨*xuexpr rule* 363b⟩+≡ (84d) ◁91e

```
|   '(' tlist abdecor ')' '{' ilist '}' /* extension */
    {
        $$ = new(OSTRUCT, $6, Z);
        dodecl(NODECL, CXXX, $2, $3);
        $$->type = lastdcltype;
    }
```

⟨`tcomo()` *switch node kind cases* 363c⟩+≡ (121c) ◁183a 374f▷

```
case OSTRUCT:
    if(tcomx(n))
        goto bad;
    break;
```

⟨*function* `tcomx` 363d⟩≡ (454)

```
bool
tcomx(Node *n)
{
    Type *t;
    Node *l, *r, **ar, **al;
    bool e;

    e = false;
    if(n->type->etype != TSTRUCT) {
        diag(n, "constructor must be a structure");
        return true;
    }
    l = invert(n->left);
    n->left = l;
    al = &n->left;
    for(t = n->type->link; t != T; t = t->down) {
        if(l == Z) {
            diag(n, "constructor list too short");
            return true;
        }
        if(l->op == OLIST) {
            r = l->left;
            ar = &l->left;
            al = &l->right;
            l = l->right;
        } else {
            r = l;
            ar = al;
            l = Z;
        }
        if(tcom(r))
            e++;
        typeext(t, r);
        if(tcompat(n, t, r->type, tasign))
            e++;
        constas(n, t, r->type);
        if(!e && !sametype(t, r->type)) {
            r = new1(OCAST, r, Z);
            r->type = t;
```

```
            *ar = r;
        }
    }
    if(l != Z) {
        diag(n, "constructor list too long");
        return true;
    }
    return e;
}
```

⟨sugen() *switch op cases* 364⟩≡ (196)

```
case OSTRUCT:
    /*
     * rewrite so lhs has no fn call
     */
    if(nn != Z && nn->complex >= FNX) {
        nod1 = *n;
        nod1.type = typ(TIND, n->type);
        regret(&nod2, &nod1);
        lcgen(nn, &nod2);
        regsalloc(&nod0, &nod1);
        gopcode(OAS, &nod2, Z, &nod0);
        regfree(&nod2);

        nod1 = *n;
        nod1.op = OIND;
        nod1.left = &nod0;
        nod1.right = Z;
        nod1.complex = 1;

        sugen(n, &nod1, w);
        return;
    }

    r = n->left;
    for(t = n->type->link; t != T; t = t->down) {
        l = r;
        if(r->op == OLIST) {
            l = r->left;
            r = r->right;
        }
        if(nn == Z) {
            cgen(l, nn);
            continue;
        }
        /*
         * hand craft *(&nn + o) = l
         */
        nod0 = znode;
        nod0.op = OAS;
        nod0.type = t;
        nod0.left = &nod1;
        nod0.right = l;

        nod1 = znode;
        nod1.op = OIND;
        nod1.type = t;
        nod1.left = &nod2;

        nod2 = znode;
```

```
        nod2.op = OADD;
        nod2.type = typ(TIND, t);
        nod2.left = &nod3;
        nod2.right = &nod4;

        nod3 = znode;
        nod3.op = OADDR;
        nod3.type = nod2.type;
        nod3.left = nn;

        nod4 = znode;
        nod4.op = OCONST;
        nod4.type = nod2.type;
        nod4.vconst = t->offset;

        ccom(&nod0);
        acom(&nod0);
        xcom(&nod0);
        nod0.addable = 0;

        cgen(&nod0, Z);
    }
    break;
```

17.2.4 Per-processor storage: `extern register`

⟨`Storage_class` *cases* 365a⟩+≡ (20e) ◁350d 367c▷

```
CEXREG, // extern register, kenccext (used in kernel for mips)
```

⟨`ginit()` *reg and resvreg initialisation* 365b⟩+≡ (35d) ◁166g

```
/* keep two external registers */
reg[REGEXT] = 1; // R10
reg[REGEXT-1] = 1; // R9
```

⟨`simplec()` *cases* 365c⟩+≡ (99a) ◁108i 367d▷

```
case BEXTERN|BREGISTER:
    return CEXREG;
```

⟨`xdecl()` *switch class cases* 365d⟩+≡ (79g) ◁114b 367e▷

```
case CEXREG:
    o = exreg(t);
    if(o == 0)
        class = CEXTERN;
    if(s->class == CGLOBL)
        class = CGLOBL;
    break;
```

⟨*global* `exregoffset` 365e⟩≡ (460b)

```
long exregoffset;
```

⟨*global* `exfregoffset` 365f⟩≡ (460b)

```
long exfregoffset;
```

⟨`ginit()` *misc initialisations* 365g⟩+≡ (35d) ◁353d 406a▷

```
exregoffset = REGEXT;
exfregoffset = FREGEXT;
```

⟨*function* `exreg`*(arm)* 366a⟩≡ (462)

```
long
exreg(Type *t)
{
    long o;

    if(typechlp[t->etype]) {
        if(exregoffset <= REGEXT-2)
            return 0;

        o = exregoffset;
        if(reg[o] && !resvreg[o])
            return 0;
        resvreg[o] = reg[o] = 1;
        exregoffset--;
        return o;
    }
    if(typefd[t->etype]) {
        if(exfregoffset <= NFREG-1)
            return 0;

        o = exfregoffset + NREG;
        if(reg[o] && !resvreg[o])
            return 0;
        resvreg[o] = reg[o] = 1;
        exfregoffset--;
        return o;
    }
    return 0;
}
```

⟨`tcomo()` *in ONAME case, if extern register* 366b⟩≡ (123f)

```
if(n->class == CEXREG) {
    n->op = OREGISTER;
    ⟨tcomo() in ONAME case, if extern register, if x86 366c⟩
    n->reg = n->sym->offset;
    n->xoffset = 0;
    break;
}
```

⟨`tcomo()` *in ONAME case, if extern register, if x86* 366c⟩≡ (366b)

⟨`tcomo()` *when OADDR, check if register* 366d⟩≡ (136b)

```
if(l->op == OREGISTER) {
    diag(n, "address of a register");
    goto bad;
}
```

17.2.5 Type reflection: `typestr()`

⟨`itab` *entries, kencc extensions* 366e⟩+≡ (42a) ◁148e 374c▷

```
"typestr", LTYPESTR, 0,
```

⟨*token declarations* 366f⟩+≡ (76a) ◁148d 374b▷

```
%token  LTYPESTR
```

⟨*cname rule* 366g⟩+≡ (95b) ◁108h

```
|   LTYPESTR  { $$ = BTYPESTR; }
```

⟨Bxxx *cases* 367a⟩≡ (94a)

```
BTYPESTR = 1L<<TTYPESTR,
```

⟨Type_kind_bis *storage cases* 367b⟩+≡ (93f) ◁98b

```
TTYPESTR,
```

⟨Storage_class *cases* 367c⟩+≡ (20e) ◁365a

```
CTYPESTR,
```

⟨simplec() *cases* 367d⟩+≡ (99a) ◁365c

```
case BTYPESTR:
    return CTYPESTR;
```

⟨xdecl() *switch class cases* 367e⟩+≡ (79g) ◁365d

```
case CTYPESTR:
    if(!typesuv[t->etype]) {
        diag(Z, "typestr must be struct/union: %s", s->name);
        break;
    }
    dclfunct(t, s);
    break;
```

⟨*function* dclfunct 367f⟩≡ (452a)

```
/// dodecl(xdecl, ...) ->  <>
void
dclfunct(Type *t, Sym *s)
{
    Funct *f;
    Node *n;
    Type *f1, *f2, *f3, *f4;
    int o, i, c;
    char str[100];

    if(t->funct)
        return;

    // recognize generated tag of dorm _%d_
    if(t->tag == S)
        goto bad;
    for(i=0; c = t->tag->name[i]; i++) {
        if(c == '_') {
            if(i == 0 || t->tag->name[i+1] == 0)
                continue;
            break;
        }
        if(c < '0' || c > '9')
            break;
    }
    if(c == 0)
        goto bad;

    f = alloc(sizeof(*f));
    for(o=0; o<sizeof(f->sym); o++) // bugfix?nelem(f->sym) in latest plan9
        f->sym[o] = S;

    t->funct = f;

    f1 = typ(TFUNC, t);
    f1->down = copytyp(t);
    f1->down->down = t;
```

```
f2 = typ(TFUNC, types[TINT]);
f2->down = copytyp(t);
f2->down->down = t;

f3 = typ(TFUNC, t);
f3->down = typ(TIND, t);
f3->down->down = t;

f4 = typ(TFUNC, t);
f4->down = t;

for(i=0;; i++) {
    o = ftabinit[i].op;
    if(o == OXXX)
        break;
    sprint(str, "%s_%s_", t->tag->name, ftabinit[i].name);
    n = new(ONAME, Z, Z);
    n->sym = slookup(str);
    f->sym[o] = n->sym;

    switch(ftabinit[i].typ) {

    case 1: // T f(T,T) +
        dodecl(xdecl, CEXTERN, f1, n);
        break;

    case 2: // int f(T,T) ==
        dodecl(xdecl, CEXTERN, f2, n);
        break;

    case 3: // void f(T*,T) +=
        dodecl(xdecl, CEXTERN, f3, n);
        break;

    case 4: // T f(T) ~
        dodecl(xdecl, CEXTERN, f4, n);
        break;

    default:
        diag(Z, "dclfunct op missing %d\n", ftabinit[i].typ);
        break;
    }
}
for(i=0;; i++) {
    o = gtabinit[i].etype;
    if(o == TXXX)
        break;

    /*
     * OCAST types T1 _T2_T1_(T2)
     */
    sprint(str, "_%s%s_", gtabinit[i].name, t->tag->name);
    n = new(ONAME, Z, Z);
    n->sym = slookup(str);
    f->castto[o] = n->sym;

    f1 = typ(TFUNC, t);
    f1->down = types[o];
    dodecl(xdecl, CEXTERN, f1, n);
```

```
        sprint(str, "%s_%s_", t->tag->name, gtabinit[i].name);
        n = new(ONAME, Z, Z);
        n->sym = slookup(str);
        f->castfr[o] = n->sym;

        f1 = typ(TFUNC, types[o]);
        f1->down = t;
        dodecl(xdecl, CEXTERN, f1, n);
    }
    return;
bad:
    diag(Z, "dclfunct bad %T %s\n", t, s->name);
}
```

⟨*struct* Ftab 369a⟩≡ (452a)

```
struct Ftab
{
    char op;
    char* name;
    char typ;
};
```

⟨*struct* Gtab 369b⟩≡ (452a)

```
struct Gtab
{
    char etype;
    char* name;
};
```

⟨*global* gtabinit 369c⟩≡ (452a)

```
Gtab gtabinit[NTYPE] =
{
    TCHAR,  "c",
    TUCHAR,  "uc",
    TSHORT,  "h",
    TUSHORT, "uh",
    TINT,  "i",
    TUINT,  "ui",
    TLONG,  "l",
    TULONG,  "ul",
    TVLONG,  "v",
    TUVLONG, "uv",
    TFLOAT,  "f",
    TDOUBLE, "d",

    TXXX
};
```

⟨*global* ftabinit 369d⟩≡ (452a)

```
Ftab ftabinit[OEND] =
{
    OADD,  "add",  1,
    OAND,  "and",  1,
    OASHL,  "ashl",  1,
    OASHR,  "ashr",  1,
    ODIV,  "div",  1,
    OLDIV,  "ldiv",  1,
    OLMOD,  "lmod",  1,
    OLMUL,  "lmul",  1,
```

```
    OLSHR,  "lshr",  1,
    OMOD,  "mod",  1,
    OMUL,  "mul",  1,
    OOR,  "or",  1,
    OSUB,  "sub",  1,
    OXOR,  "xor",  1,

    OEQ,  "eq",  2,
    OGE,  "ge",  2,
    OGT,  "gt",  2,
    OHI,  "hi",  2,
    OHS,  "hs",  2,
    OLE,  "le",  2,
    OLO,  "lo",  2,
    OLS,  "ls",  2,
    OLT,  "lt",  2,
    ONE,  "ne",  2,

    OASADD,  "asadd", 3,
    OASAND,  "asand", 3,
    OASASHL, "asashl", 3,
    OASASHR, "asashr", 3,
    OASDIV,  "asdiv", 3,
    OASLDIV, "asldiv", 3,
    OASLMOD, "aslmod", 3,
    OASLMUL, "aslmul", 3,
    OASLSHR, "aslshr", 3,
    OASMOD,  "asmod", 3,
    OASMUL,  "asmul", 3,
    OASOR,  "asor",  3,
    OASSUB,  "assub", 3,
    OASXOR,  "asxor", 3,

    OPOS,  "pos",  4,
    ONEG,  "neg",  4,
    OCOM,  "com",  4,
    ONOT,  "not",  4,

// OPOSTDEC,
// OPOSTINC,
// OPREDEC,
// OPREINC,

    OXXX,
};
```

⟨tcomo() *break if isfunct* 370a⟩≡ (143a 141a 136a 135 134c 133a 132 131 127c 126 125 124)

```
if(isfunct(n))
    break;
```

⟨Type *other fields* 370b⟩+≡ (19b) ◁19d

```
Funct* funct;
```

⟨*struct* Funct 370c⟩≡ (429b)

```
struct Funct
{
    Sym* sym[OEND];
    Sym* castto[NTYPE];
    Sym* castfr[NTYPE];
};
```

⟨*function* isfunct 371⟩≡ (452a)

```
bool
isfunct(Node *n)
{
    Type *t, *t1;
    Funct *f;
    Node *l;
    Sym *s;
    int o;

    o = n->op;
    if(n->left == Z)
        goto no;
    t = n->left->type;
    if(t == T)
        goto no;
    f = t->funct;

    switch(o) {
    case OAS: // put cast on rhs
    case OASI:
    case OASADD:
    case OASSUB:
    case OASMUL:
    case OASDIV:
    case OASMOD:
    case OASASHR:
    case OASLMUL:
    case OASLDIV:
    case OASLMOD:
    case OASLSHR:
    case OASASHL:
    case OASAND:
    case OASOR:
    case OASXOR:
        if(n->right == Z)
            goto no;
        t1 = n->right->type;
        if(t1 == T)
            goto no;
        if(t1->funct == f)
            break;

        l = new(OXXX, Z, Z);
        *l = *n->right;

        n->right->left = l;
        n->right->right = Z;
        n->right->type = t;
        n->right->op = OCAST;

        if(!isfunct(n->right))
            prtree(n, "isfunc !");
        break;

    case OCAST: // t f(T) or T f(t)
        t1 = n->type;
        if(t1 == T)
            goto no;
        if(f != nil) {
```

```
        s = f->castfr[t1->etype];
        if(s == S)
            goto no;
        n->right = n->left;
        goto build;
    }
    f = t1->funct;
    if(f != nil) {
        s = f->castto[t->etype];
        if(s == S)
            goto no;
        n->right = n->left;
        goto build;
    }
    goto no;
}

if(f == nil)
    goto no;
s = f->sym[o];
if(s == S)
    goto no;



/*
 * the answer is yes,
 * now we rewrite the node
 * and give diagnostics
 */
switch(o) {
case OADD: // T f(T, T)
case OAND:
case OASHL:
case OASHR:
case ODIV:
case OLDIV:
case OLMOD:
case OLMUL:
case OLSHR:
case OMOD:
case OMUL:
case OOR:
case OSUB:
case OXOR:

case OEQ: // int f(T, T)
case OGE:
case OGT:
case OHI:
case OHS:
case OLE:
case OLO:
case OLS:
case OLT:
case ONE:
    if(n->right == Z)
        goto bad;
    t1 = n->right->type;
    if(t1 == T)
```

```
            goto bad;
        if(t1->funct != f)
            goto bad;
        n->right = new(OLIST, n->left, n->right);
        break;

    case OAS: // structure copies done by the compiler
    case OASI:
        goto no;

    case OASADD: // T f(T*, T)
    case OASAND:
    case OASASHL:
    case OASASHR:
    case OASDIV:
    case OASLDIV:
    case OASLMOD:
    case OASLMUL:
    case OASLSHR:
    case OASMOD:
    case OASMUL:
    case OASOR:
    case OASSUB:
    case OASXOR:
        if(n->right == Z)
            goto bad;
        t1 = n->right->type;
        if(t1 == T)
            goto bad;
        if(t1->funct != f)
            goto bad;
        n->right = new(OLIST, new(OADDR, n->left, Z), n->right);
        break;

    case OPOS: // T f(T)
    case ONEG:
    case ONOT:
    case OCOM:
        n->right = n->left;
        break;

    default:
        diag(n, "isfunct op missing %O\n", o);
        goto bad;


    }

build:
    l = new(ONAME, Z, Z);
    l->sym = s;
    l->type = s->type;
    l->etype = s->type->etype;
    l->xoffset = s->offset;
    l->class = s->class;
    tcomo(l, 0);

    n->op = OFUNC;
    n->left = l;
    n->type = l->type->link;
```

```
    if(tcompat(n, T, l->type, tfunct))
        goto bad;
    if(tcoma(n->left, n->right, l->type->down, true))
        goto bad;
    return true;

no:
    return false;

bad:
    diag(n, "cant rewrite typestr for op %O\n", o);
    prtree(n, "isfunct");
    n->type = T;
    return true;
}
```

17.2.6 Signature reflection: `signof()`

⟨*expression nodes* 374a⟩+≡ (16a) ◁363a

```
OSIGN,
```

⟨*token declarations* 374b⟩+≡ (76a) ◁366f

```
%token  LSIGNOF
```

⟨`itab` *entries, kencc extensions* 374c⟩+≡ (42a) ◁366e

```
"signof", LSIGNOF, 0,
```

⟨*uexpr rule* 374d⟩+≡ (84d) ◁92e

```
|   LSIGNOF uexpr { $$ = new(OSIGN, $2, Z); }
```

⟨*pexpr rule* 374e⟩+≡ (84d) ◁92d

```
|   LSIGNOF '(' tlist abdecor ')'
    {
        $$ = new(OSIGN, Z, Z);
        dodecl(NODECL, CXXX, $3, $4);
        $$->type = lastdcltype;
    }
```

⟨`tcomo()` *switch node kind cases* 374f⟩+≡ (121c) ◁363c

```
case OSIGN: /* extension signof(type) returns a hash */
    if(l != Z) {
        if(l->op != OSTRING && l->op != OLSTRING)
            if(tcomo(l, 0))
                goto bad;
        ⟨tcomo() when OSIGN, sanity check no bitfield 374g⟩
        n->type = l->type;
    }
    ⟨tcomo() when OSIGN, typecheck and sanity check 375a⟩
    n->op = OCONST;
    n->left = Z;
    n->right = Z;
    n->vconst = convvtox(signature(n->type), TULONG);
    n->type = types[TULONG];
    break;
```

⟨`tcomo()` *when OSIGN, sanity check no bitfield* 374g⟩≡ (374f)

```
if(l->op == OBIT) {
    diag(n, "signof bitfield");
    goto bad;
}
```

⟨tcomo() *when OSIGN, typecheck and sanity check* 375a⟩≡ (374f)

```
if(n->type == T)
    goto bad;
if(n->type->width < 0) {
    diag(n, "signof undefined type");
    goto bad;
}
```

17.2.7 Format's arguments checking

⟨*enum* fxxx 375b⟩≡ (453)

```
enum fxxx
{
    Fnone = 0,

    Fl,
    Fvl,
    Fignor,
    Fstar,
    Fadj,

    Fverb = 10,
};
```

⟨*struct* Tprot 375c⟩≡ (453)

```
struct Tprot
{
    Type* type;
    Bits flag;
    Tprot* link;
};
```

⟨*struct* Tname 375d⟩≡ (453)

```
struct Tname
{
    char* name;
    int  param;
    Tname* link;
};
```

⟨*global* indchar 375e⟩≡ (453)

```
static Type* indchar;
```

⟨*global* flagbits 375f⟩≡ (453)

```
static uchar flagbits[512];
```

⟨*global* fmtbuf 375g⟩≡ (453)

```
static char fmtbuf[100];
```

⟨*global* lastadj 375h⟩≡ (453)

```
static int lastadj;
```

⟨*global* lastverb 375i⟩≡ (453)

```
static int lastverb;
```

⟨*global* nstar 375j⟩≡ (453)

```
static int nstar;
```

⟨*global* tprot 375k⟩≡ (453)

```
static Tprot* tprot;
```

⟨*global* tname 376a⟩≡ (453)

```
static Tname* tname;
```

⟨*function* argflag 376b⟩≡ (453)

```
void
argflag(int c, int v)
{

    switch(v) {
    case Fignor:
    case Fstar:
    case Fl:
    case Fvl:
        flagbits[c] = v;
        break;
    case Fverb:
        flagbits[c] = lastverb;
/*print("flag-v %c %d\n", c, lastadj);*/
        lastverb++;
        break;
    case Fadj:
        flagbits[c] = lastadj;
/*print("flag-l %c %d\n", c, lastadj);*/
        lastadj++;
        break;
    }
}
```

⟨*function* getflag 376c⟩≡ (453)

```
Bits
getflag(char *s)
{
    Bits flag;
    int f;
    char *fmt;
    Rune c;

    fmt = fmtbuf;
    flag = zbits;
    nstar = 0;
    for(;;) {
        s += chartorune(&c, s);
        fmt += runetochar(fmt, &c);
        if(c == 0 || c >= nelem(flagbits))
            break;
        f = flagbits[c];
        switch(f) {
        case Fnone:
            argflag(c, Fverb);
            f = flagbits[c];
            break;
        case Fstar:
            nstar++;
        case Fignor:
            continue;
        case Fl:
            if(bset(flag, Fl))
                flag = bor(flag, blsh(Fvl));
        }
        flag = bor(flag, blsh(f));
```

```
        if(f >= Fverb)
            break;
    }
    *fmt = 0;
    return flag;
}
```

⟨*function* arginit 377a⟩≡ (453)

```
void
arginit(void)
{
    int i;

/* debug['F'] = 1;*/
/* debug['w'] = 1;*/

    lastadj = Fadj;
    lastverb = Fverb;
    indchar = typ(TIND, types[TCHAR]);

    memset(flagbits, Fnone, sizeof(flagbits));

    for(i='0'; i<='9'; i++)
        argflag(i, Fignor);
    argflag('.', Fignor);
    argflag('#', Fignor);
    argflag('u', Fignor);
    argflag('h', Fignor);
    argflag('+', Fignor);
    argflag('-', Fignor);

    argflag('*', Fstar);
    argflag('l', Fl);

    argflag('o', Fverb);
    flagbits['x'] = flagbits['o'];
    flagbits['X'] = flagbits['o'];
}
```

⟨*function* nextarg 377b⟩≡ (453)

```
Node*
nextarg(Node *n, Node **a)
{
    if(n == Z) {
        *a = Z;
        return Z;
    }
    if(n->op == OLIST) {
        *a = n->left;
        return n->right;
    }
    *a = n;
    return Z;
}
```

⟨*function* checkargs 377c⟩≡ (453)

```
void
checkargs(Node *nn, char *s, int pos)
{
    Node *a, *n;
```

```
    Bits flag;
    Tprot *l;

    if(!debug['F'])
        return;
    n = nn;
    for(;;) {
        s = strchr(s, '%');
        if(s == 0) {
            nextarg(n, &a);
            if(a != Z)
                warn(nn, "more arguments than format %T",
                    a->type);
            return;
        }
        s++;
        flag = getflag(s);
        while(nstar > 0) {
            n = nextarg(n, &a);
            pos++;
            nstar--;
            if(a == Z) {
                warn(nn, "more format than arguments %s",
                    fmtbuf);
                return;
            }
            if(a->type == T)
                continue;
            if(!sametype(types[TINT], a->type) &&
               !sametype(types[TUINT], a->type))
                warn(nn, "format mismatch '*' in %s %T, arg %d",
                    fmtbuf, a->type, pos);
        }
        for(l=tprot; l; l=l->link)
            if(sametype(types[TVOID], l->type)) {
                if(beq(flag, l->flag)) {
                    s++;
                    goto loop;
                }
            }

        n = nextarg(n, &a);
        pos++;
        if(a == Z) {
            warn(nn, "more format than arguments %s",
                fmtbuf);
            return;
        }
        if(a->type == 0)
            continue;
        for(l=tprot; l; l=l->link)
            if(sametype(a->type, l->type)) {
/*print("checking %T/%ulx %T/%ulx\n", a->type, flag.b[0], l->type, l->flag.b[0]);*/
                if(beq(flag, l->flag))
                    goto loop;
            }
        warn(nn, "format mismatch %s %T, arg %d", fmtbuf, a->type, pos);
    loop:;
    }
}
```

⟨tcomo() *when OFUNC case, format check* 379a⟩≡ (138b)

```
dpcheck(n);
```

⟨*function* dpcheck 379b⟩≡ (453)

```
void
dpcheck(Node *n)
{
    char *s;
    Node *a, *b;
    Tname *l;
    int i;

    if(n == Z)
        return;
    b = n->left;
    if(b == Z || b->op != ONAME)
        return;
    s = b->sym->name;
    for(l=tname; l; l=l->link)
        if(strcmp(s, l->name) == 0)
            break;
    if(l == 0)
        return;

    i = l->param;
    b = n->right;
    while(i > 0) {
        b = nextarg(b, &a);
        i--;
    }
    if(a == Z) {
        warn(n, "cant find format arg");
        return;
    }
    if(!sametype(indchar, a->type)) {
        warn(n, "format arg type %T", a->type);
        return;
    }
    if(a->op != OADDR || a->left->op != ONAME || a->left->sym != symstring) {
/*  warn(n, "format arg not constant string");*/
        return;
    }
    s = a->left->cstring;
    checkargs(b, s, l->param);
}
```

#pragma varargck xxx

⟨macprag() *if pragma xxx* 379c⟩+≡ (74a) ◁348d 398a▷

```
if(s && strcmp(s->name, "varargck") == 0) {
    pragvararg();
    return;
}
```

⟨*function* pragvararg 379d⟩≡ (453)

```
void
pragvararg(void)
{
    Sym *s;
```

```
    int n, c;
    char *t;
    Rune r;
    Type *ty;

    if(!debug['F'])
        goto out;

    s = getsym();
    if(s && strcmp(s->name, "argpos") == 0)
        goto ckpos;
    if(s && strcmp(s->name, "type") == 0)
        goto cktype;
    if(s && strcmp(s->name, "flag") == 0)
        goto ckflag;
    yyerror("syntax in #pragma varargck");
    goto out;

ckpos:
/*#pragma varargck argpos warn 2*/
    s = getsym();
    if(s == S)
        goto bad;
    n = getnsn();
    if(n < 0)
        goto bad;
    newname(s->name, n);
    goto out;

ckflag:
/*#pragma varargck flag 'c'*/
    c = getnsc();
    if(c != '\'')
        goto bad;
    c = getr();
    if(c == '\\')
        c = getr();
    else if(c == '\'')
        goto bad;
    if(c == '\n')
        goto bad;
    if(getc() != '\'')
        goto bad;
    argflag(c, Fignor);
    goto out;

cktype:
/*#pragma varargck type O int*/
    c = getnsc();
    if(c != '"')
        goto bad;
    t = fmtbuf;
    for(;;) {
        r = getr();
        if(r == ' ' || r == '\n')
            goto bad;
        if(r == '"')
            break;
        t += runetochar(t, &r);
    }
```

```
    *t = 0;
    t = strdup(fmtbuf);
    s = getsym();
    if(s == S)
        goto bad;
    ty = s->type;
    while((c = getnsc()) == '*')
        ty = typ(TIND, ty);
    unget(c);
    newprot(s, ty, t);
    goto out;

bad:
    yyerror("syntax in #pragma varargck");

out:
    while(getnsc() != '\n')
        ;
}
```

⟨*function* newname 381a⟩≡ (453)

```
void
newname(char *s, int p)
{
    Tname *l;

    for(l=tname; l; l=l->link)
        if(strcmp(l->name, s) == 0) {
            if(l->param != p)
                yyerror("vargck %s already defined\n", s);
            return;
        }
    l = alloc(sizeof(*l));
    l->name = s;
    l->param = p;
    l->link = tname;
    tname = l;
}
```

⟨*function* newprot 381b⟩≡ (453)

```
void
newprot(Sym *m, Type *t, char *s)
{
    Bits flag;
    Tprot *l;

    if(t == T) {
        warn(Z, "%s: newprot: type not defined", m->name);
        return;
    }
    flag = getflag(s);
    for(l=tprot; l; l=l->link)
        if(beq(flag, l->flag) && sametype(t, l->type))
            return;
    l = alloc(sizeof(*l));
    l->type = t;
    l->flag = flag;
    l->link = tprot;
    tprot = l;
}
```

17.3 Floats

⟨typeext() *float handling* 382a⟩≡ (360e)
```
typeext1(st, l);
```

⟨tcomo() *when OASMUL/OASDIV, typing extensions* 382b⟩≡ (135c)
```
typeext1(l->type, r);
```

⟨tcomo() *when OLT/OGE/OGT/OLE, typing extensions* 382c⟩≡ (131d)
```
typeext1(l->type, r);
typeext1(r->type, l);
```

⟨*function* typeext1 382d⟩≡ (443b)
```
void
typeext1(Type *st, Node *l)
{
    if(st->etype == TFLOAT && allfloat(l, false))
        allfloat(l, true);
}
```

⟨*function* allfloat 382e⟩≡ (443b)
```
/*
 * look into tree for floating point constant expressions
 */
bool
allfloat(Node *n, bool flag)
{

    if(n != Z) {
        if(n->type->etype != TDOUBLE)
            return true;
        switch(n->op) {
        case OCONST:
            if(flag)
                n->type = types[TFLOAT];
            return true;
        case OADD: /* no need to get more exotic than this */
        case OSUB:
        case OMUL:
        case ODIV:
            if(!allfloat(n->right, flag))
                break;
        case OCAST:
            if(!allfloat(n->left, flag))
                break;
            if(flag)
                n->type = types[TFLOAT];
            return true;
        }
    }
    return false;
}
```

⟨gopcode() *when OADD, adjust a if float type* 382f⟩≡ (185b)
```
if(et == TFLOAT)
    a = AADDF;
else
if(et == TDOUBLE || et == TVLONG)
    a = AADDD;
```

⟨gopcode() *when OSUB, adjust a if float type* 383a⟩≡ (185c)

```
if(et == TFLOAT)
    a = ASUBF;
else
if(et == TDOUBLE || et == TVLONG)
    a = ASUBD;
```

⟨gopcode() *when OMUL, adjust a if float type* 383b⟩≡ (186b)

```
if(et == TFLOAT)
    a = AMULF;
else
if(et == TDOUBLE || et == TVLONG)
    a = AMULD;
```

⟨gopcode() *when ODIV, adjust a if float type* 383c⟩≡ (186c)

```
if(et == TFLOAT)
    a = ADIVF;
else
if(et == TDOUBLE || et == TVLONG)
    a = ADIVD;
```

⟨naddr() *if float type* 383d⟩≡ (176c)

```
if(typefd[n->type->etype]) {
    a->type = D_FCONST;
    a->dval = n->fconst;
}
```

⟨naddr() *if float register, adjust a* 383e⟩≡ (177a)

```
if(a->reg >= NREG) {
    a->type = D_FREG;
    a->reg -= NREG;
}
```

⟨regret() *if float expression, adjust return register* 383f⟩≡ (166c)

```
if(typefd[nn->type->etype])
    r = FREGRET+NREG;
```

⟨*global* fconstnode 383g⟩≡ (460b)

```
Node fconstnode;
```

⟨ginit() *special nodes initialisation* 383h⟩+≡ (35d) ◁222h

```
fconstnode.op = OCONST;
fconstnode.class = CXXX;
fconstnode.complex = 0;
fconstnode.addable = 20;
fconstnode.type = types[TDOUBLE];
```

⟨*function* nodfconst 383i⟩≡ (462)

```
Node*
nodfconst(double d)
{
    fconstnode.fconst = d;
    return &fconstnode;
}
```

⟨*function* ieeedtod 384a⟩≡ (455a)

```
void
ieeedtod(Ieee *ieee, double native)
{
    double fr, ho, f;
    int exp;

    if(native < 0) {
        ieeedtod(ieee, -native);
        ieee->h |= 0x80000000L;
        return;
    }
    if(native == 0) {
        ieee->l = 0;
        ieee->h = 0;
        return;
    }
    fr = frexp(native, &exp);
    f = 2097152L;  /* shouldnt use fp constants here */
    fr = modf(fr*f, &ho);
    ieee->h = ho;
    ieee->h &= 0xfffffL;
    ieee->h |= (exp+1022L) << 20;
    f = 65536L;
    fr = modf(fr*f, &ho);
    ieee->l = ho;
    ieee->l <<= 16;
    ieee->l |= (long)(fr*f);
}
```

⟨gclean() *sanity check reg* 384b⟩+≡ (34a) ◁167b

```
for(i=NREG; i<NREG+NFREG; i++)
    if(reg[i] && !resvreg[i])
        diag(Z, "freg %d left allocated", i-NREG);
```

⟨*function* raddr*(arm)* 384c⟩≡ (462)

```
/// gmove -> <>
void
raddr(Node *n, Prog *p)
{
    Adr a;

    naddr(n, &a);
    if(a.type != D_REG && a.type != D_FREG) {
        if(n)
            diag(n, "bad in raddr: %O", n->op);
        else
            diag(n, "bad in raddr: <null>");
        p->reg = R_NONE;
    } else
        p->reg = a.reg;
}
```

⟨gmove() *switch from type cases* 384d⟩+≡ (171d) ◁175c

```
case TDOUBLE:
case TVLONG:
case TFLOAT:
    switch(tt) {
    case TDOUBLE:
    case TVLONG:
```

```
        a = AMOVD;
        if(ft == TFLOAT)
            a = AMOVFD;
        break;
    case TFLOAT:
        a = AMOVDF;
        if(ft == TFLOAT)
            a = AMOVF;
        break;
    case TINT:
    case TUINT:
    case TLONG:
    case TULONG:
    case TIND:
        a = AMOVDW;
        if(ft == TFLOAT)
            a = AMOVFW;
        break;
    case TSHORT:
    case TUSHORT:
    case TCHAR:
    case TUCHAR:
        a = AMOVDW;
        if(ft == TFLOAT)
            a = AMOVFW;
        break;
    }
    break;
```

⟨gmove() *switch from type cases, TUINT/TULONG case, if float target* 385⟩≡ (173f)

```
 if(tt == TFLOAT || tt == TDOUBLE) {
    // ugly and probably longer than necessary,
    // but vfp has a single instruction for this,
    // so hopefully it won't last long.
    //
    // tmp = f
    // tmp1 = tmp & 0x80000000
    // tmp ^= tmp1
    // t = float(int32(tmp))
    // if(tmp1)
    //  t += 2147483648.
    //
    regalloc(&nod, f, Z);
    regalloc(&nod1, f, Z);
    gins(AMOVW, f, &nod);
    gins(AMOVW, &nod, &nod1);
    gins(AAND, nodconst(0x80000000), &nod1);
    gins(AEOR, &nod1, &nod);
    if(tt == TFLOAT)
        gins(AMOVWF, &nod, t);
    else
        gins(AMOVWD, &nod, t);
    gins(ACMP, nodconst(0), Z);
    raddr(&nod1, p);
    gins(ABEQ, Z, Z);
    regfree(&nod);
    regfree(&nod1);
    p1 = p;
    regalloc(&nod, t, Z);
    if(tt == TFLOAT) {
```

```
        gins(AMOVF, nodfconst(2147483648.), &nod);
        gins(AADDF, &nod, t);
    } else {
        gins(AMOVD, nodfconst(2147483648.), &nod);
        gins(AADDD, &nod, t);
    }
    regfree(&nod);
    patch(p1, pc);
    return;
}
```

⟨gmove() *switch from type cases, TINT/TLONG case, if float target* 386a⟩≡ (173f)

```
case TDOUBLE:
    gins(AMOVWD, f, t);
    return;
case TFLOAT:
    gins(AMOVWF, f, t);
    return;
```

⟨gmove() *switch from type cases, TSHORT case, if float target* 386b⟩≡ (174c)

```
case TDOUBLE:
    regalloc(&nod, f, Z);
    gins(AMOVH, f, &nod);
    gins(AMOVWD, &nod, t);
    regfree(&nod);
    return;
case TFLOAT:
    regalloc(&nod, f, Z);
    gins(AMOVH, f, &nod);
    gins(AMOVWF, &nod, t);
    regfree(&nod);
    return;
```

⟨gmove() *switch from type cases, TUSHORT case, if float target* 386c⟩≡ (175a)

```
case TDOUBLE:
    regalloc(&nod, f, Z);
    gins(AMOVHU, f, &nod);
    gins(AMOVWD, &nod, t);
    regfree(&nod);
    return;
case TFLOAT:
    regalloc(&nod, f, Z);
    gins(AMOVHU, f, &nod);
    gins(AMOVWF, &nod, t);
    regfree(&nod);
    return;
```

⟨gmove() *switch from type cases, TCHAR case, if float target* 386d⟩≡ (175b)

```
case TDOUBLE:
    regalloc(&nod, f, Z);
    gins(AMOVB, f, &nod);
    gins(AMOVWD, &nod, t);
    regfree(&nod);
    return;
case TFLOAT:
    regalloc(&nod, f, Z);
    gins(AMOVB, f, &nod);
    gins(AMOVWF, &nod, t);
    regfree(&nod);
    return;
```

⟨gmove() *switch from type cases, TUCHAR case, if float target* 387a⟩≡ (175c)

```
case TDOUBLE:
    regalloc(&nod, f, Z);
    gins(AMOVBU, f, &nod);
    gins(AMOVWD, &nod, t);
    regfree(&nod);
    return;
case TFLOAT:
    regalloc(&nod, f, Z);
    gins(AMOVBU, f, &nod);
    gins(AMOVWF, &nod, t);
    regfree(&nod);
    return;
```

⟨regalloc() *switch tn type, float or vlong case* 387b⟩≡ (178b)

```
case TFLOAT:
case TDOUBLE:

case TVLONG:
    if(o != Z && o->op == OREGISTER) {
        i = o->reg;
        if(i >= NREG && i < NREG+NFREG)
            goto out;
    }
    j = 0*2 + NREG;
    for(i=NREG; i<NREG+NFREG; i++) {
        if(j >= NREG+NFREG)
            j = NREG;
        if(reg[j] == 0) {
            i = j;
            goto out;
        }
        j++;
    }
    diag(tn, "out of float registers");
    goto err;
```

17.4 64 bits operations

17.4.1 com64init()

⟨ginit() *com64init initialisation* 387c⟩≡ (35d)

```
com64init();
```

⟨*function* com64init 387d⟩≡ (449b)

```
void
com64init(void)
{
    Init *p;

    nodaddv = fvn("_addv", TVLONG);
    nodsubv = fvn("_subv", TVLONG);
    nodmulv = fvn("_mulv", TVLONG);
    noddivv = fvn("_divv", TVLONG);
    noddivvu = fvn("_divvu", TVLONG);
    nodmodv = fvn("_modv", TVLONG);
    nodmodvu = fvn("_modvu", TVLONG);
    nodlshv = fvn("_lshv", TVLONG);
```

```
    nodrshav = fvn("_rshav", TVLONG);
    nodrshlv = fvn("_rshlv", TVLONG);
    nodandv = fvn("_andv", TVLONG);
    nodorv = fvn("_orv", TVLONG);
    nodxorv = fvn("_xorv", TVLONG);
    nodnegv = fvn("_negv", TVLONG);
    nodcomv = fvn("_comv", TVLONG);

    nodtestv = fvn("_testv", TLONG);
    nodeqv = fvn("_eqv", TLONG);
    nodnev = fvn("_nev", TLONG);
    nodlev = fvn("_lev", TLONG);
    nodltv = fvn("_ltv", TLONG);
    nodgev = fvn("_gev", TLONG);
    nodgtv = fvn("_gtv", TLONG);
    nodhiv = fvn("_hiv", TLONG);
    nodhsv = fvn("_hsv", TLONG);
    nodlov = fvn("_lov", TLONG);
    nodlsv = fvn("_lsv", TLONG);

    nodf2v = fvn("_f2v", TVLONG);
    nodd2v = fvn("_d2v", TVLONG);
    nodp2v = fvn("_p2v", TVLONG);
    nodsi2v = fvn("_si2v", TVLONG);
    nodui2v = fvn("_ui2v", TVLONG);
    nodsl2v = fvn("_sl2v", TVLONG);
    nodul2v = fvn("_ul2v", TVLONG);
    nodsh2v = fvn("_sh2v", TVLONG);
    noduh2v = fvn("_uh2v", TVLONG);
    nodsc2v = fvn("_sc2v", TVLONG);
    noduc2v = fvn("_uc2v", TVLONG);

    nodv2f = fvn("_v2f", TFLOAT);
    nodv2d = fvn("_v2d", TDOUBLE);
    nodv2sl = fvn("_v2sl", TLONG);
    nodv2ul = fvn("_v2ul", TULONG);
    nodv2si = fvn("_v2si", TINT);
    nodv2ui = fvn("_v2ui", TUINT);
    nodv2sh = fvn("_v2sh", TSHORT);
    nodv2uh = fvn("_v2ul", TUSHORT);
    nodv2sc = fvn("_v2sc", TCHAR);
    nodv2uc = fvn("_v2uc", TUCHAR);

    nodvpp = fvn("_vpp", TVLONG);
    nodppv = fvn("_ppv", TVLONG);
    nodvmm = fvn("_vmm", TVLONG);
    nodmmv = fvn("_mmv", TVLONG);

    nodvasop = fvn("_vasop", TVLONG);

    for(p = initetconv; p->code >= 0; p++)
        etconv[p->code] = p->value;
}
```

⟨*global* etconv 388a⟩≡ (449b)

```
char etconv[NTYPE]; /* for _vasop */
```

⟨*global* initetconv 388b⟩≡ (449b)

```
Init initetconv[] =
{
```

```
    TCHAR,  1, 0,
    TUCHAR,  2, 0,
    TSHORT,  3, 0,
    TUSHORT, 4, 0,
    TLONG,  5, 0,
    TULONG,  6, 0,
    TVLONG,  7, 0,
    TUVLONG, 8, 0,
    TINT,  9, 0,
    TUINT,  10, 0,
    -1,  0, 0,
};
```

⟨*global* fntypes 389a⟩≡ (436d)

```
Type* fntypes[NTYPE];
```

⟨*function* fvn 389b⟩≡ (449b)

```
Node*
fvn(char *name, int type)
{
    Node *n;

    n = new(ONAME, Z, Z);
    n->sym = slookup(name);
    n->sym->sig = SIGINTERN;
    if(fntypes[type] == nil)
        fntypes[type] = typ(TFUNC, types[type]);
    n->type = fntypes[type];
    n->etype = type; // type of return?
    n->class = CGLOBL;
    n->addable = 10;
    n->complex = 0;
    return n;
}
```

17.4.2 Rewriting: com64()

⟨xcom() *if 64 bits operation, transform and return* 389c⟩≡ (170e)

```
if(com64(n))
    return;
```

⟨*function* com64 389d⟩≡ (449b)

```
bool
com64(Node *n)
{
    Node *l, *r;
    Node *a, *t;
    bool lv, rv;

    if(n->type == nil)
        return false;

    l = n->left;
    r = n->right;

    lv = false;
    if(l && l->type && typev[l->type->etype])
        lv = true;
    rv = false;
```

```
if(r && r->type && typev[r->type->etype])
    rv = true;

if(lv) {
    switch(n->op) {
    case OEQ:
        a = nodeqv;
        goto setbool;
    case ONE:
        a = nodnev;
        goto setbool;
    case OLE:
        a = nodlev;
        goto setbool;
    case OLT:
        a = nodltv;
        goto setbool;
    case OGE:
        a = nodgev;
        goto setbool;
    case OGT:
        a = nodgtv;
        goto setbool;
    case OHI:
        a = nodhiv;
        goto setbool;
    case OHS:
        a = nodhsv;
        goto setbool;
    case OLO:
        a = nodlov;
        goto setbool;
    case OLS:
        a = nodlsv;
        goto setbool;

    case OANDAND:
    case OOROR:
        if(machcap(n))
            return true;

        if(rv) {
            r = new(OFUNC, nodtestv, r);
            n->right = r;
            r->complex = FNX;
            r->op = OFUNC;
            r->type = types[TLONG];
        }

    case OCOND:
    case ONOT:
        if(machcap(n))
            return true;

        l = new(OFUNC, nodtestv, l);
        n->left = l;
        l->complex = FNX;
        l->op = OFUNC;
        l->type = types[TLONG];
        n->complex = FNX;
```

```
        return 1;
    }
}

if(rv) {
    if(machcap(n))
        return true;
    switch(n->op) {
    case OANDAND:
    case OOROR:
        r = new(OFUNC, nodtestv, r);
        n->right = r;
        r->complex = FNX;
        r->op = OFUNC;
        r->type = types[TLONG];
        return 1;
    case OCOND:
        return 1;
    }
}

if(typev[n->type->etype]) {
    if(machcap(n))
        return true;
    switch(n->op) {
    default:
        diag(n, "unknown vlong %O", n->op);
    case OFUNC:
        n->complex = FNX;
    case ORETURN:
    case OAS:
    case OIND:
    case OLIST:
    case OCOMMA:
        return 1;
    case OADD:
        a = nodaddv;
        goto setbop;
    case OSUB:
        a = nodsubv;
        goto setbop;
    case OMUL:
    case OLMUL:
        a = nodmulv;
        goto setbop;
    case ODIV:
        a = noddivv;
        goto setbop;
    case OLDIV:
        a = noddivvu;
        goto setbop;
    case OMOD:
        a = nodmodv;
        goto setbop;
    case OLMOD:
        a = nodmodvu;
        goto setbop;
    case OASHL:
        a = nodlshv;
        goto setbop;
```

```
case OASHR:
    a = nodrshav;
    goto setbop;
case OLSHR:
    a = nodrshlv;
    goto setbop;
case OAND:
    a = nodandv;
    goto setbop;
case OOR:
    a = nodorv;
    goto setbop;
case OXOR:
    a = nodxorv;
    goto setbop;
case OPOSTINC:
    a = nodvpp;
    goto setvinc;
case OPOSTDEC:
    a = nodvmm;
    goto setvinc;
case OPREINC:
    a = nodppv;
    goto setvinc;
case OPREDEC:
    a = nodmmv;
    goto setvinc;
case ONEG:
    a = nodnegv;
    goto setfnx;
case OCOM:
    a = nodcomv;
    goto setfnx;
case OCAST:
    switch(l->type->etype) {
    case TCHAR:
        a = nodsc2v;
        goto setfnxl;
    case TUCHAR:
        a = noduc2v;
        goto setfnxl;
    case TSHORT:
        a = nodsh2v;
        goto setfnxl;
    case TUSHORT:
        a = noduh2v;
        goto setfnxl;
    case TINT:
        a = nodsi2v;
        goto setfnx;
    case TUINT:
        a = nodui2v;
        goto setfnx;
    case TLONG:
        a = nodsl2v;
        goto setfnx;
    case TULONG:
        a = nodul2v;
        goto setfnx;
    case TFLOAT:
```

```
            a = nodf2v;
            goto setfnx;
        case TDOUBLE:
            a = nodd2v;
            goto setfnx;
        case TIND:
            a = nodp2v;
            goto setfnx;
        }
        diag(n, "unknown %T->vlong cast", l->type);
        return true;
    case OASADD:
        a = nodaddv;
        goto setasop;
    case OASSUB:
        a = nodsubv;
        goto setasop;
    case OASMUL:
    case OASLMUL:
        a = nodmulv;
        goto setasop;
    case OASDIV:
        a = noddivv;
        goto setasop;
    case OASLDIV:
        a = noddivvu;
        goto setasop;
    case OASMOD:
        a = nodmodv;
        goto setasop;
    case OASLMOD:
        a = nodmodvu;
        goto setasop;
    case OASASHL:
        a = nodlshv;
        goto setasop;
    case OASASHR:
        a = nodrshav;
        goto setasop;
    case OASLSHR:
        a = nodrshlv;
        goto setasop;
    case OASAND:
        a = nodandv;
        goto setasop;
    case OASOR:
        a = nodorv;
        goto setasop;
    case OASXOR:
        a = nodxorv;
        goto setasop;
    }
}

if(typefd[n->type->etype] && l && l->op == OFUNC) {
    switch(n->op) {
    case OASADD:
    case OASSUB:
    case OASMUL:
    case OASLMUL:
```

```
    case OASDIV:
    case OASLDIV:
    case OASMOD:
    case OASLMOD:
    case OASASHL:
    case OASASHR:
    case OASLSHR:
    case OASAND:
    case OASOR:
    case OASXOR:
        if(l->right && typev[l->right->etype]) {
            diag(n, "sorry float <asop> vlong not implemented\n");
        }
    }
}

if(n->op == OCAST) {
    if(l->type && typev[l->type->etype]) {
        if(machcap(n))
            return true;
        switch(n->type->etype) {
        case TDOUBLE:
            a = nodv2d;
            goto setfnx;
        case TFLOAT:
            a = nodv2f;
            goto setfnx;
        case TLONG:
            a = nodv2sl;
            goto setfnx;
        case TULONG:
            a = nodv2ul;
            goto setfnx;
        case TINT:
            a = nodv2si;
            goto setfnx;
        case TUINT:
            a = nodv2ui;
            goto setfnx;
        case TSHORT:
            a = nodv2sh;
            goto setfnx;
        case TUSHORT:
            a = nodv2uh;
            goto setfnx;
        case TCHAR:
            a = nodv2sc;
            goto setfnx;
        case TUCHAR:
            a = nodv2uc;
            goto setfnx;
        case TIND: // small pun here
            a = nodv2ul;
            goto setfnx;
        }
        diag(n, "unknown vlong->%T cast", n->type);
        return true;
    }
}
```

```
    return false;

setbop:
    n->left = a;
    n->right = new(OLIST, l, r);
    n->complex = FNX;
    n->op = OFUNC;
    return true;

setfnxl:
    l = new(OCAST, l, 0);
    l->type = types[TLONG];
    l->complex = l->left->complex;

setfnx:
    n->left = a;
    n->right = l;
    n->complex = FNX;
    n->op = OFUNC;
    return true;

setvinc:
    n->left = a;
    l = new(OADDR, l, Z);
    l->type = typ(TIND, l->left->type);
    l->complex = l->left->complex;
    n->right = new(OLIST, l, r);
    n->complex = FNX;
    n->op = OFUNC;
    return true;

setbool:
    if(machcap(n))
        return true;
    n->left = a;
    n->right = new(OLIST, l, r);
    n->complex = FNX;
    n->op = OFUNC;
    n->type = types[TLONG];
    return true;

setasop:
    if(l->op == OFUNC) {
        l = l->right;
        goto setasop;
    }

    t = new(OCONST, 0, 0);
    t->vconst = etconv[l->type->etype];
    t->type = types[TLONG];
    t->addable = 20;
    r = new(OLIST, t, r);

    t = new(OADDR, a, 0);
    t->type = typ(TIND, a->type);
    r = new(OLIST, t, r);

    t = new(OADDR, l, 0);
    t->type = typ(TIND, l->type);
    t->complex = l->complex;
```

```
    r = new(OLIST, t, r);

    n->left = nodvasop;
    n->right = r;
    n->complex = FNX;
    n->op = OFUNC;

    return true;
}
```

17.4.3 bool64()

⟨bcomplex() *possibly convert node to funcall if unsupported 64 bit op* 396a⟩≡ (160a)

```
bool64(n);
```

⟨*function* bool64 396b⟩≡ (449b)

```
void
bool64(Node *n)
{
    Node *n1;

    if(machcap(Z))
        return;

    if(typev[n->type->etype]) {
        n1 = new(OXXX, 0, 0);
        *n1 = *n;

        n->right = n1;
        n->left = nodtestv;
        n->complex = FNX;
        n->addable = 0;
        n->op = OFUNC;
        n->type = types[TLONG];
    }
}
```

17.5 Endianness

17.6 Using an external preprocessor: /bin/cpp

⟨*constant* CPP 396c⟩≡ (446b)

```
#define CPP "/bin/cpp"
```

⟨compile() *locals* 396d⟩+≡ (32e) ◁33f

```
char **av, opt[256];
int i, c, fd[2];
```

⟨compile() *if use ANSI preprocessor* 396e⟩≡ (32e)

```
/* Use an ANSI preprocessor */
if(debug['p']) {
    if(myaccess(infile) < 0) {
        diag(Z, "%s does not exist", infile);
        errorexit();
    }
    if(pipe(fd) < 0) {
```

```
        diag(Z, "pipe failed");
        errorexit();
    }
    switch(fork()) {
    case -1:
        diag(Z, "fork failed");
        errorexit();

    case 0:
        close(fd[0]);
        dup(fd[1], 1);
        close(fd[1]);
        av = alloc((3 + ndef + ninclude + 2) * sizeof *av);
        av[0] = CPP;
        i = 1;
        if(debug['.'])
            av[i++] = strdup("-.");
        /* 1999 ANSI C requires recognising // comments */
        av[i++] = strdup("-+");
        for(c = 0; c < ndef; c++) {
            sprint(opt, "-D%s", defs[c]);
            av[i++] = strdup(opt);
        }
        for(c = 0; c < ninclude; c++) {
            sprint(opt, "-I%s", include[c]);
            av[i++] = strdup(opt);
        }
        if(strcmp(infile, "stdin") != 0)
            av[i++] = infile;
        av[i] = 0;
        if(debug['p'] > 1) {
            for(c = 0; c < i; c++)
                fprint(2, "%s ", av[c]);
            fprint(2, "\n");
        }

        exec(av[0], av);
        fprint(2, "can't exec C preprocessor %s: %r\n", CPP);
        errorexit();

    default:
        close(fd[1]);
        newfile(infile, fd[0]);
        break;
    }
}
```

17.7 Processing multiple files

⟨main() *locals* 397a⟩+≡ (30d) ◁31j

```
int nproc, nout, status, i;
```

⟨main() *multiple files handling* 397b⟩≡ (30d)

```
if(argc > 1) {
    nproc = 1;
    /*
     * if we're writing acid to standard output, don't compile
     * concurrently, to avoid interleaving output.
```

```
     */
    if(((!debug['a'] && !debug['Z']) || debug['n']) &&
        (p = getenv("NPROC")) != nil)
        nproc = atol(p); /* */
    c = 0;
    nout = 0;
    for(;;) {
        while(nout < nproc && argc > 0) {
            i = fork();
            if(i < 0) {
                i = mywait(&status);
                if(i < 0) {
                    print("cannot create a process\n");
                    errorexit();
                }
                if(status)
                    c++;
                nout--;
                continue;
            }
            if(i == 0) {
                fprint(2, "%s:\n", *argv);
                if (compile(*argv, defs, ndef))
                    errorexit();
                exits(0);
            }
            nout++;
            argc--;
            argv++;
        }
        i = mywait(&status);
        if(i < 0) {
            if(c)
                errorexit();
            exits(0);
        }
        if(status)
            c++;
        nout--;
    }
}
```

17.8 Other optimizations

17.8.1 Packing: #pragma pack

⟨macprag() *if pragma xxx* 398a⟩+≡ (74a) ◁379c 436b▷

```
if(s && strcmp(s->name, "pack") == 0) {
    pragpack();
    return;
}
```

⟨*global* packflg 398b⟩≡ (436d)

```
int packflg;
```

⟨*function* pragpack 398c⟩≡ (453)

```
void
pragpack(void)
```

```
{
    Sym *s;

    packflg = 0;
    s = getsym();
    if(s) {
        packflg = atoi(s->name+1);
        if(strcmp(s->name, "on") == 0 ||
           strcmp(s->name, "yes") == 0)
            packflg = 1;
    }
    while(getnsc() != '\n')
        ;
    ⟨pragpack() debug if -f 399a⟩
}
```

⟨pragpack() *debug if* -f 399a⟩≡ (398c)

```
if(debug['f'])
    if(packflg)
        print("%4ld: pack %d\n", lineno, packflg);
    else
        print("%4ld: pack off\n", lineno);
```

⟨align() *if packflg override w* 399b⟩≡ (225)

```
if(packflg)
    w = packflg;
```

17.8.2 Minimizing #line history

⟨linehist() *possibly overwrite last line directive* 399c⟩≡ (60g)

```
/*
 * overwrite the last #line directive if
 * no alloc has happened since the last one
 */
if(!newflag && ehist != H && offset != 0 && ehist->offset != 0)
    if(f && ehist->name && strcmp(f, ehist->name) == 0) {
        ehist->line = lineno;
        ehist->offset = offset;
        return;
    }
newflag = false;
```

⟨*global* newflag 399d⟩≡ (436d)

```
bool newflag;
```

⟨new() *set lineno* 399e⟩+≡ (17d) ◁17e

```
newflag = true;
```

Chapter 18

Conclusion

Appendix A

Debugging

⟨*global* debug 401a⟩≡ (436d)
```
char debug[256];
```

A.1 Dumpers

A.1.1 cc/ Fmt data structures

⟨cinit() *fmtinstall* 401b⟩≡ (35a) 423b ▷
```
fmtinstall('O', Oconv);
fmtinstall('T', Tconv);
fmtinstall('F', FNconv);
fmtinstall('Q', Qconv);
fmtinstall('|', VBconv);
```

⟨*function* Oconv 401c⟩≡ (446b)
```
// enum<Node_kind> -> unit
int Oconv(Fmt *fp)
{
    int a;

    a = va_arg(fp->args, int);
    if(a < OXXX || a > OEND)
        return fmtprint(fp, "***badO %d***", a);

    return fmtstrcpy(fp, onames[a]);
}
```

⟨*global* onames 401d⟩≡ (443b)
```
char* onames[OEND+1];
```

⟨*global* onamesinit 401e⟩≡ (443b)
```
Init onamesinit[] =
{
    OXXX,  0, "OXXX",
    OADD,  0, "ADD",
    OADDR,  0, "ADDR",
    OAND,  0, "AND",
    OANDAND, 0, "ANDAND",
    OARRAY,  0, "ARRAY",
    OAS,  0, "AS",
    OASI,  0, "ASI",
    OASADD,  0, "ASADD",
    OASAND,  0, "ASAND",
```

```
OASASHL, 0, "ASASHL",
OASASHR, 0, "ASASHR",
OASDIV,  0, "ASDIV",
OASHL,  0, "ASHL",
OASHR,  0, "ASHR",
OASLDIV, 0, "ASLDIV",
OASLMOD, 0, "ASLMOD",
OASLMUL, 0, "ASLMUL",
OASLSHR, 0, "ASLSHR",
OASMOD,  0, "ASMOD",
OASMUL,  0, "ASMUL",
OASOR,  0, "ASOR",
OASSUB,  0, "ASSUB",
OASXOR,  0, "ASXOR",
OBIT,  0, "BIT",
OBREAK,  0, "BREAK",
OCASE,  0, "CASE",
OCAST,  0, "CAST",
OCOMMA,  0, "COMMA",
OCOND,  0, "COND",
OCONST,  0, "CONST",
OCONTINUE, 0, "CONTINUE",
ODIV,  0, "DIV",
ODOT,  0, "DOT",
ODOTDOT, 0, "DOTDOT",
ODWHILE, 0, "DWHILE",
OEQ,  0, "EQ",
OFOR,  0, "FOR",
OFUNC,  0, "FUNC",
OGE,  0, "GE",
OGOTO,  0, "GOTO",
OGT,  0, "GT",
OHI,  0, "HI",
OHS,  0, "HS",
OIF,  0, "IF",
OIND,  0, "IND",
OINDREG, 0, "INDREG",
OINIT,  0, "INIT",
OLABEL,  0, "LABEL",
OLDIV,  0, "LDIV",
OLE,  0, "LE",
OLIST,  0, "LIST",
OLMOD,  0, "LMOD",
OLMUL,  0, "LMUL",
OLO,  0, "LO",
OLS,  0, "LS",
OLSHR,  0, "LSHR",
OLT,  0, "LT",
OMOD,  0, "MOD",
OMUL,  0, "MUL",
ONAME,  0, "NAME",
ONE,  0, "NE",
ONOT,  0, "NOT",
OOR,  0, "OR",
OOROR,  0, "OROR",
OPOSTDEC, 0, "POSTDEC",
OPOSTINC, 0, "POSTINC",
OPREDEC, 0, "PREDEC",
OPREINC, 0, "PREINC",
OPROTO,  0, "PROTO",
```

```
    OREGISTER, 0, "REGISTER",
    ORETURN, 0, "RETURN",
    OSET,  0, "SET",
    OSIGN,  0, "SIGN",
    OSIZE,  0, "SIZE",
    OSTRING, 0, "STRING",
    OLSTRING, 0, "LSTRING",
    OSTRUCT, 0, "STRUCT",
    OSUB,  0, "SUB",
    OSWITCH, 0, "SWITCH",
    OUNION,  0, "UNION",
    OUSED,  0, "USED",
    OWHILE,  0, "WHILE",
    OXOR,  0, "XOR",
    OPOS,  0, "POS",
    ONEG,  0, "NEG",
    OCOM,  0, "COM",
    OELEM,  0, "ELEM",
    OINDEX,  0, "INDEX",
    OREGPAIR, 0, "REGPAIR",
    OEXREG,  0, "EXREG",
    OEND,  0, "END",
    -1,  0, 0,
};
```

⟨*function* Tconv 403⟩≡ (446b)

```
// option<Type> -> string
int
Tconv(Fmt *fp)
{
    char str[STRINGSZ+20], s[STRINGSZ+20];
    Type *t, *t1;
    int et;
    long n;

    str[0] = '\0';
    for(t = va_arg(fp->args, Type*); t != T; t = t->link) {
        et = t->etype;
        if(str[0])
            strcat(str, " ");
        if(t->garb&~GINCOMPLETE) {
            sprint(s, "%s ", gnames[t->garb&~GINCOMPLETE]);
            if(strlen(str) + strlen(s) < STRINGSZ)
                strcat(str, s);
        }
        sprint(s, "%s", tnames[et]);
        if(strlen(str) + strlen(s) < STRINGSZ)
            strcat(str, s);
        if(et == TFUNC && (t1 = t->down)) {
            sprint(s, "(%T", t1);
            if(strlen(str) + strlen(s) < STRINGSZ)
                strcat(str, s);
            while(t1 = t1->down) {
                sprint(s, ", %T", t1);
                if(strlen(str) + strlen(s) < STRINGSZ)
                    strcat(str, s);
            }
            if(strlen(str) + strlen(s) < STRINGSZ)
                strcat(str, ")");
        }
```

```
        if(et == TARRAY) {
            n = t->width;
            if(t->link && t->link->width)
                n /= t->link->width;
            sprint(s, "[%ld]", n);
            if(strlen(str) + strlen(s) < STRINGSZ)
                strcat(str, s);
        }
        if(t->nbits) {
            sprint(s, " %d:%d", t->shift, t->nbits);
            if(strlen(str) + strlen(s) < STRINGSZ)
                strcat(str, s);
        }
        if(typesu[et]) {
            if(t->tag) {
                strcat(str, " ");
                if(strlen(str) + strlen(t->tag->name) < STRINGSZ)
                    strcat(str, t->tag->name);
            } else
                strcat(str, " {}");
            break;
        }
    }
    return fmtstrcpy(fp, str);
}
```

⟨*global* tnames 404a⟩≡ (443b)

```
char* tnames[NALLTYPES];
```

⟨*global* tnamesinit 404b⟩≡ (443b)

```
Init tnamesinit[] =
{
    TXXX,  0, "TXXX",

    TCHAR,  0, "CHAR",
    TUCHAR,  0, "UCHAR",
    TSHORT,  0, "SHORT",
    TUSHORT, 0, "USHORT",
    TINT,  0, "INT",
    TUINT,  0, "UINT",
    TLONG,  0, "LONG",
    TULONG,  0, "ULONG",
    TVLONG,  0, "VLONG",
    TUVLONG, 0, "UVLONG",
    TFLOAT,  0, "FLOAT",
    TDOUBLE, 0, "DOUBLE",
    TIND,  0, "IND",
    TFUNC,  0, "FUNC",
    TARRAY,  0, "ARRAY",
    TVOID,  0, "VOID",
    TSTRUCT, 0, "STRUCT",
    TUNION,  0, "UNION",
    TENUM,  0, "ENUM",
    TOLD,  0, "OLD",
    TDOT,  0, "DOT",

    -1,  0, 0,
};
```

⟨*function* FNconv 405a⟩≡ (446b)

```
// option<Node identifier cases> -> string
int
FNconv(Fmt *fp)
{
    char *str;
    Node *n;

    n = va_arg(fp->args, Node*);
    str = "<indirect>";
    if(n != Z && (n->op == ONAME || n->op == ODOT || n->op == OELEM))
        str = n->sym->name;
    return fmtstrcpy(fp, str);
}
```

⟨*function* Qconv 405b⟩≡ (446b)

```
// ??
int
Qconv(Fmt *fp)
{
    char str[STRINGSZ+20], *s;
    long b;
    int i;

    str[0] = 0;
    for(b = va_arg(fp->args, long); b;) {
        i = bitno(b);
        if(str[0])
            strcat(str, " ");
        s = qnames[i];
        if(strlen(str) + strlen(s) >= STRINGSZ)
            break;
        strcat(str, s);
        b &= ~(1L << i);
    }
    return fmtstrcpy(fp, str);
}
```

⟨*function* VBconv 405c⟩≡ (446b)

```
// ??
int
VBconv(Fmt *fp)
{
    char str[STRINGSZ];
    int i, n, t, pc;

    n = va_arg(fp->args, int);
    pc = 0; /* BUG: was printcol */
    i = 0;
    while(pc < n) {
        t = (pc+4) & ~3;
        if(t <= n) {
            str[i++] = '\t';
            pc = t;
            continue;
        }
        str[i++] = ' ';
        pc++;
    }
    str[i] = '\0';
```

```
    return fmtstrcpy(fp, str);
}
```

A.1.2 5c/ Fmt data structures

⟨ginit() *misc initialisations* 406a⟩+≡ (35d) ◁365g

```
listinit();
```

⟨*function* listinit*(arm)* 406b⟩≡ (461)

```
void
listinit(void)
{

    fmtinstall('A', Aconv);
    fmtinstall('P', Pconv);
    fmtinstall('S', Sconv);
    fmtinstall('N', Nconv);
    fmtinstall('B', Bconv);
    fmtinstall('D', Dconv);
    fmtinstall('R', Rconv);
}
```

⟨*function* Bconv 406c⟩≡ (461)

```
// Bits -> string
int
Bconv(Fmt *fp)
{
    char str[STRINGSZ], ss[STRINGSZ], *s;
    Bits bits;
    int i;

    str[0] = 0;
    bits = va_arg(fp->args, Bits);
    while(bany(&bits)) {
        i = bnum(bits);
        if(str[0])
            strcat(str, " ");
        if(var[i].sym == S) {
            snprint(ss, sizeof(ss), "$%ld", var[i].offset);
            s = ss;
        } else
            s = var[i].sym->name;
        if(strlen(str) + strlen(s) + 1 >= STRINGSZ)
            break;
        strcat(str, s);
        bits.b[i/32] &= ~(1L << (i%32));
    }
    return fmtstrcpy(fp, str);
}
```

⟨*function* Pconv*(arm)* 406d⟩≡ (461)

```
// Prog -> string
int
Pconv(Fmt *fp)
{
    char str[STRINGSZ], sc[20];
    Prog *p;
    int a, s;
```

```
    p = va_arg(fp->args, Prog*);
    a = p->as;
    s = p->scond;
    strcpy(sc, extra[s & C_SCOND]);
    if(s & C_SBIT)
        strcat(sc, ".S");
    if(s & C_PBIT)
        strcat(sc, ".P");
    if(s & C_WBIT)
        strcat(sc, ".W");
    if(s & C_UBIT)  /* ambiguous with FBIT */
        strcat(sc, ".U");
    if(a == AMOVM) {
        if(p->from.type == D_CONST)
            snprint(str, sizeof(str), " %A%s %R,%D", a, sc, &p->from, &p->to);
        else
        if(p->to.type == D_CONST)
            snprint(str, sizeof(str), " %A%s %D,%R", a, sc, &p->from, &p->to);
        else
            snprint(str, sizeof(str), " %A%s %D,%D", a, sc, &p->from, &p->to);
    } else
    if(a == ADATA)
        snprint(str, sizeof(str), " %A %D/%d,%D", a, &p->from, p->reg, &p->to);
    else
    if(p->as == ATEXT)
        snprint(str, sizeof(str), " %A %D,%d,%D", a, &p->from, p->reg, &p->to);
    else
    if(p->reg == R_NONE)
        snprint(str, sizeof(str), " %A%s %D,%D", a, sc, &p->from, &p->to);
    else
    if(p->from.type != D_FREG)
        snprint(str, sizeof(str), " %A%s %D,R%d,%D", a, sc, &p->from, p->reg, &p->to);
    else
        snprint(str, sizeof(str), " %A%s %D,F%d,%D", a, sc, &p->from, p->reg, &p->to);
    return fmtstrcpy(fp, str);
}
```

⟨*global* extra*(arm)* 407a⟩≡ (461)

```
char *extra [] = {
    ".EQ", ".NE", ".CS", ".CC",
    ".MI", ".PL", ".VS", ".VC",
    ".HI", ".LS", ".GE", ".LT",
    ".GT", ".LE", "", ".NV",
};
```

⟨*function* Aconv 407b⟩≡ (461)

```
// enum<opcode_kind> -> string
int
Aconv(Fmt *fp)
{
    char *s;
    int a;

    a = va_arg(fp->args, int);
    s = "???";
    if(a >= AXXX && a < ALAST)
        s = anames[a];
    return fmtstrcpy(fp, s);
}
```

⟨*function* Dconv*(arm)* 408⟩≡ (461)

```
int
Dconv(Fmt *fp)
{
    char str[STRINGSZ];
    Adr *a;
    char *op;
    int v;

    a = va_arg(fp->args, Adr*);
    switch(a->type) {

    default:
        snprint(str, sizeof(str), "GOK-type(%d)", a->type);
        break;

    case D_NONE:
        str[0] = 0;
        if(a->symkind != N_NONE || a->reg != R_NONE || a->sym != S)
            snprint(str, sizeof(str), "%N(R%d)(NONE)", a, a->reg);
        break;

    case D_CONST:
        if(a->reg != R_NONE)
            snprint(str, sizeof(str), "$%N(R%d)", a, a->reg);
        else
            snprint(str, sizeof(str), "$%N", a);
        break;

    case D_SHIFT:
        v = a->offset;
        op = "<<>>->@>" + (((v>>5) & 3) << 1);
        if(v & (1<<4))
            snprint(str, sizeof(str), "R%d%c%cR%d", v&15, op[0], op[1], (v>>8)&15);
        else
            snprint(str, sizeof(str), "R%d%c%c%d", v&15, op[0], op[1], (v>>7)&31);
        if(a->reg != R_NONE)
            sprint(str+strlen(str), "(R%d)", a->reg);
        break;

    case D_OREG:
        if(a->reg != R_NONE)
            snprint(str, sizeof(str), "%N(R%d)", a, a->reg);
        else
            snprint(str, sizeof(str), "%N", a);
        break;

    case D_REG:
        snprint(str, sizeof(str), "R%d", a->reg);
        if(a->symkind != N_NONE || a->sym != S)
            snprint(str, sizeof(str), "%N(R%d)(REG)", a, a->reg);
        break;

    case D_FREG:
        snprint(str, sizeof(str), "F%d", a->reg);
        if(a->symkind != N_NONE || a->sym != S)
            snprint(str, sizeof(str), "%N(R%d)(REG)", a, a->reg);
        break;

    case D_PSR:
```

```
        snprint(str, sizeof(str), "PSR");
        if(a->symkind != N_NONE || a->sym != S)
            snprint(str, sizeof(str), "%N(PSR)(REG)", a);
        break;

    case D_BRANCH:
        snprint(str, sizeof(str), "%ld(PC)", a->offset-pc);
        break;

    case D_FCONST:
        snprint(str, sizeof(str), "$%.17e", a->dval);
        break;

    case D_SCONST:
        snprint(str, sizeof(str), "$\"%S\"", a->sval);
        break;
    }
    return fmtstrcpy(fp, str);
}
```

⟨*function* Rconv *(arm)* 409a⟩≡ (461)

```
// enum<operand_kind(register-only??)> -> string
int
Rconv(Fmt *fp)
{
    char str[STRINGSZ], *p, *e;
    Adr *a;
    int i, v;

    a = va_arg(fp->args, Adr*);
    snprint(str, sizeof(str), "GOK-reglist");
    switch(a->type) {
    case D_CONST:
        if(a->reg != R_NONE)
            break;
        if(a->sym != S)
            break;
        v = a->offset;
        p = str;
        e = str+sizeof(str);
        for(i=0; i<NREG; i++) {
            if(v & (1<<i)) {
                if(p == str)
                    p = seprint(p, e, "[R%d", i);
                else
                    p = seprint(p, e, ",R%d", i);
            }
        }
        seprint(p, e, "]");
    }
    return fmtstrcpy(fp, str);
}
```

⟨*function* Sconv *(arm)* 409b⟩≡ (461)

```
int
Sconv(Fmt *fp)
{
    int i, c;
    char str[STRINGSZ], *p, *a;
```

```
    a = va_arg(fp->args, char*);
    p = str;
    for(i=0; i<NSNAME; i++) {
        c = a[i] & 0xff;
        if(c >= 'a' && c <= 'z' ||
           c >= 'A' && c <= 'Z' ||
           c >= '0' && c <= '9' ||
           c == ' ' || c == '%') {
            *p++ = c;
            continue;
        }
        *p++ = '\\';
        switch(c) {
        case 0:
            *p++ = 'z';
            continue;
        case '\\':
        case '"':
            *p++ = c;
            continue;
        case '\n':
            *p++ = 'n';
            continue;
        case '\t':
            *p++ = 't';
            continue;
        case '\r':
            *p++ = 'r';
            continue;
        case '\f':
            *p++ = 'f';
            continue;
        }
        *p++ = (c>>6) + '0';
        *p++ = ((c>>3) & 7) + '0';
        *p++ = (c & 7) + '0';
    }
    *p = 0;
    return fmtstrcpy(fp, str);
}
```

⟨*function* Nconv *(arm)* 410⟩≡ (461)

```
int
Nconv(Fmt *fp)
{
    char str[STRINGSZ];
    Adr *a;
    Sym *s;

    a = va_arg(fp->args, Adr*);
    s = a->sym;
    if(s == S) {
        snprint(str, sizeof(str), "%ld", a->offset);
        goto out;
    }
    switch(a->symkind) {
    case N_NONE:
        snprint(str, sizeof(str), "%ld", a->offset);
        break;
```

```
    case N_EXTERN:
        snprint(str, sizeof(str), "%s+%ld(SB)", s->name, a->offset);
        break;

    case N_INTERN:
        snprint(str, sizeof(str), "%s<>+%ld(SB)", s->name, a->offset);
        break;

    case N_LOCAL:
        snprint(str, sizeof(str), "%s-%ld(SP)", s->name, -a->offset);
        break;

    case N_PARAM:
        snprint(str, sizeof(str), "%s+%ld(FP)", s->name, a->offset);
        break;

    default:
        snprint(str, sizeof(str), "GOK-name(%d)", a->symkind);
        break;

    }
out:
    return fmtstrcpy(fp, str);
}
```

A.1.3 xxxnames[]

⟨*global* cnames 411a⟩≡ (443b)

```
char* cnames[NCTYPES];
```

⟨*global* cnamesinit 411b⟩≡ (443b)

```
Init cnamesinit[] =
{
    CXXX     ,  0, "CXXX",
    CAUTO    ,  0, "AUTO",
    CEXTERN  ,  0, "EXTERN",
    CGLOBL   ,  0, "GLOBL",
    CSTATIC  ,  0, "STATIC",
    CLOCAL   ,  0, "LOCAL",
    CTYPEDEF ,  0, "TYPEDEF",
    CTYPESTR ,  0, "TYPESTR",
    CPARAM   ,  0, "PARAM",
    CEXREG   ,  0, "EXREG",
    -1,   0, 0,
};
```

⟨*global* qnames 411c⟩≡ (443b)

```
char* qnames[NALLTYPES];
```

⟨*global* qnamesinit 411d⟩≡ (443b)

```
Init qnamesinit[] =
{
    TXXX      ,  0, "TXXX",

    TCHAR     ,  0, "CHAR",
    TUCHAR    ,  0, "UCHAR",
    TSHORT    ,  0, "SHORT",
    TUSHORT   ,  0, "USHORT",
    TINT      ,  0, "INT",
```

```
    TUINT     ,  0, "UINT",
    TLONG     ,  0, "LONG",
    TULONG    ,  0, "ULONG",
    TVLONG    ,  0, "VLONG",
    TUVLONG   ,  0, "UVLONG",
    TFLOAT    ,  0, "FLOAT",
    TDOUBLE   ,  0, "DOUBLE",
    TIND      ,  0, "IND",
    TFUNC     ,  0, "FUNC",
    TARRAY    ,  0, "ARRAY",
    TVOID     ,  0, "VOID",
    TSTRUCT   ,  0, "STRUCT",
    TUNION    ,  0, "UNION",
    TENUM     ,  0, "ENUM",

    TAUTO     ,  0, "AUTO",
    TEXTERN   ,  0, "EXTERN",
    TSTATIC   ,  0, "STATIC",
    TTYPEDEF  ,  0, "TYPEDEF",
    TTYPESTR  ,  0, "TYPESTR",
    TREGISTER ,  0, "REGISTER",
    TCONSTNT  ,  0, "CONSTNT",
    TVOLATILE ,  0, "VOLATILE",
    TUNSIGNED ,  0, "UNSIGNED",
    TSIGNED   ,  0, "SIGNED",
    TDOT      ,  0, "DOT",
    TOLD      ,  0, "OLD",

    -1,  0, 0,
};
```

⟨*global* gnames 412a⟩≡ (443b)

```
char* gnames[NGTYPES];
```

⟨*global* gnamesinit 412b⟩≡ (443b)

```
Init gnamesinit[] =
{
    GXXX,  0, "GXXX",
    GCONSTNT,  0, "CONST",
    GVOLATILE,  0, "VOLATILE",
    GVOLATILE|GCONSTNT, 0, "CONST-VOLATILE",
    -1,   0, 0,
};
```

⟨tinit() *initialise xxxnames debugging arrays* 412c⟩≡ (34b)

```
for(p=bnamesinit; p->code >= 0; p++) {
    urk("bnames", nelem(bnames), p->code);
    bnames[p->code] = p->s;
}
for(p=tnamesinit; p->code >= 0; p++) {
    urk("tnames", nelem(tnames), p->code);
    tnames[p->code] = p->s;
}
for(p=gnamesinit; p->code >= 0; p++) {
    urk("gnames", nelem(gnames), p->code);
    gnames[p->code] = p->s;
}
for(p=qnamesinit; p->code >= 0; p++) {
    urk("qnames", nelem(qnames), p->code);
    qnames[p->code] = p->s;
```

```
}
for(p=cnamesinit; p->code >= 0; p++) {
    urk("cnames", nelem(cnames), p->code);
    cnames[p->code] = p->s;
}
for(p=onamesinit; p->code >= 0; p++) {
    urk("onames", nelem(onames), p->code);
    onames[p->code] = p->s;
}
```

A.1.4 AST dumper: prtree() and 5c -x

⟨*xdecl rule, debug function body* 413a⟩≡ (80c)

```
if(debug['x']) {
    prtree($2, "func");
    prtree($6, "body");
}
```

⟨*function* prtree 413b⟩≡ (443b)

```
void
prtree(Node *n, char *s)
{

    print(" == %s ==\n", s);
    prtree1(n, 0, false);
    print("\n");
}
```

⟨*function* prtree1 413c⟩≡ (443b)

```
void
prtree1(Node *n, int d, bool f)
{
    //bitset<Left|Right> to decide whether to recurse left and right child
    int i;

    if(f)
      for(i=0; i<d; i++)
        print("   ");
    if(n == Z) {
        print("Z\n");
        return;
    }

    if(n->op == OLIST) {
        prtree1(n->left, d, false);
        prtree1(n->right, d, true);
        return;
    }
    d++;
    print("%O", n->op);
    i = 3; // 1 | 2

    switch(n->op) {
    case OCONST:
        if(typefd[n->type->etype])
            print(" \"%.8e\"", n->fconst);
        else
            print(" \"%lld\"", n->vconst);
        i = 0;
```

```
        break;

    case OSTRING:
        print(" \"%s\"", n->cstring);
        i = 0;
        break;
    case OLSTRING:
        if(sizeof(TRune) == sizeof(Rune))
            print(" \"%S\"", (Rune*)n->rstring);
        else
            print(" \"...\"");
        i = 0;
        break;

    case ONAME:
        print(" \"%F\"", n);
        print(" %ld", n->xoffset);
        i = 0;
        break;
    case ODOT:
    case OELEM:
        print(" \"%F\"", n);
        break;

    case OREGISTER:
        if(n->xoffset)
            print(" %ld+R%d", n->xoffset, n->reg);
        else
            print(" R%d", n->reg);
        i = 0;
        break;
    case OINDREG:
        print(" %ld(R%d)", n->xoffset, n->reg);
        i = 0;
        break;

    }

    if(n->type != T)
        print(" %T", n->type);

    if(n->addable != 0)
        print(" <%d>", n->addable);
    if(n->complex != 0)
        print(" (%d)", n->complex);

    print(" %L\n", n->lineno);

    if(i & 2)
        prtree1(n->left, d, true);
    if(i & 1)
        prtree1(n->right, d, true);
}
```

A.2 5c -v, verbose mode

A.3 Preprocessing debugging

A.3.1 Macro debugging: 5c -m

⟨dodefine() *debug* 415a⟩≡ (63c)
```
if(debug['m'])
    print("#define (-D) %s %s\n", s->name, s->macro+1);
```

⟨macdef() *debug* 415b⟩≡ (64b)
```
if(debug['m'])
    print("#define %s %s\n", s->name, s->macro+1);
```

⟨macexpand() *debug macro expansion* 415c⟩≡ (68)
```
if(debug['m'])
    print("#expand %s %s\n", s->name, ob);
```

A.3.2 File inclusion debugging: 5c -e

⟨newfile() *debug* 415d⟩≡ (37e)
```
if(debug['e'])
    print("%L: %s\n", lineno, s);
```

A.3.3 Line information debugging: 5c -f:

⟨linehist() *debug* 415e⟩≡ (60g)
```
if(debug['f'])
    if(f) {
        if(offset)
            print("%4ld: %s (#line %d)\n", lineno, f, offset);
        else
            print("%4ld: %s\n", lineno, f);
    } else
        print("%4ld: <pop>\n", lineno);
```

A.4 Parsing debugging

A.4.1 Printing names: 5c -L

⟨yylex() *in talph case, debug symbol* 415f⟩≡ (43f)
```
if(debug['L'])
    print("%L: %s\n", lineno, symb);
```

A.4.2 Printing declarations: 5c -d

⟨dodecl() *debug declaration* 415g⟩≡ (79f)
```
if(debug['d'])
    dbgdecl(s);
```

⟨dcllabel() *debug declaration* 415h⟩≡ (83h)
```
if(debug['d'])
    dbgdecl(s);
```

⟨doinit() *debug declaration* 416a⟩≡ (114d)
```
if(debug['d'])
    dbgdecl(s);
```

⟨doenum() *debug declaration* 416b⟩≡ (105d)
```
if(debug['d'])
    dbgdecl(s);
```

⟨*function* dbgdecl 416c⟩≡ (450)
```
void
dbgdecl(Sym *s)
{
    print("decl \"%s\": C=%s [B=%d:O=%ld] T=%T\n",
        s->name, cnames[s->class], s->block, s->offset, s->type);
}
```

⟨revertdcl() *debug revert DSUE* 416d⟩≡ (27a)
```
if(debug['d'])
    print("revert2 \"%s\"\n", s->name);
```

⟨revertdcl() *debug revert DLABEL* 416e⟩≡ (27b)
```
if(debug['d'])
    print("revert3 \"%s\"\n", s->name);
```

⟨revertdcl() *debug revert DAUTO* 416f⟩≡ (27c)
```
if(debug['d'])
    print("revert1 \"%s\"\n", s->name);
```

A.5 Typing debugging

A.5.1 Printing expression type trees: 5c -t

⟨complex() *debug tree pre complex* 416g⟩≡ (168b)
```
if(debug['t'])
    if(n->op != OCONST)
        prtree(n, "pre complex");
```

⟨complex() *debug tree after tcom and comma* 416h⟩≡ (168b)
```
if(debug['t'])
    if(n->op != OCONST)
        prtree(n, "t complex");
```

⟨complex() *debug tree after ccom* 416i⟩≡ (168b)
```
if(debug['t'])
    if(n->op != OCONST)
        prtree(n, "c complex");
```

⟨complex() *debug tree after acom* 416j⟩≡ (168b)
```
if(debug['t'])
    if(n->op != OCONST)
        prtree(n, "a complex");
```

⟨complex() *debug tree after xcom* 416k⟩≡ (168b)
```
if(debug['t'])
    if(n->op != OCONST)
        prtree(n, "x complex");
```

A.6 Code generation debugging

A.6.1 Printing processed opcodes: `5c -G`

⟨gen() *debug opcode* 417a⟩≡ (158i)

```
if(debug['G'] && (o != OLIST))
    print("%L %O\n", nearln, o);
```

A.6.2 Printing code generation information: `5c -g`

⟨cgenrel() *debug* 417b⟩≡ (179f)

```
if(debug['g']) {
    prtree(nn, "cgen lhs");
    prtree(n, "cgen");
}
```

⟨boolgen() *debug* 417c⟩≡ (205d)

```
if(debug['g']) {
    prtree(nn, "boolgen lhs");
    prtree(n, "boolgen");
}
```

⟨gins() *debug* 417d⟩≡ (176a)

```
if(debug['g'])
    print("%P\n", p);
```

⟨gopcode() *debug* 417e⟩≡ (184c)

```
if(debug['g'])
    print("%P\n", p);
```

⟨lcgen() *debug* 417f⟩≡ (189b)

```
if(debug['g']) {
    prtree(nn, "lcgen lhs");
    prtree(n, "lcgen");
}
```

⟨sugen() *debug* 417g⟩≡ (196)

```
if(debug['g']) {
    prtree(nn, "sugen lhs");
    prtree(n, "sugen");
}
```

A.6.3 Printing generated assembly: `5c -S`

⟨outcode() *debug* 417h⟩≡ (235c)

```
if(debug['S']) {
    for(p = firstp; p != P; p = p->link)
        if(p->as != ADATA && p->as != AGLOBL)
            pc--;
    for(p = firstp; p != P; p = p->link) {
        print("%P\n", p);
        if(p->as != ADATA && p->as != AGLOBL)
            pc++;
    }
}
```

⟨*global* anames *(arm)* 418⟩≡ (455c)

```
char* anames[] =
{
    "XXX",
    "NOP",
    "AND",
    "ORR",
    "EOR",
    "BIC",
    "ADD",
    "SUB",
    "RSB",
    "ADC",
    "SBC",
    "RSC",
    "MVN",
    "MUL",
    "DIV",
    "MOD",
    "MULA",
    "MULL",
    "MULAL",
    "MULLU",
    "MULALU",
    "MULU",
    "DIVU",
    "MODU",
    "SLL",
    "SRL",
    "SRA",
    "MOVW",
    "MOVB",
    "MOVBU",
    "MOVH",
    "MOVHU",
    "MOVM",
    "SWPW",
    "SWPBU",
    "CMP",
    "TST",
    "TEQ",
    "CMN",
    "B",
    "BEQ",
    "BNE",
    "BHS",
    "BLO",
    "BMI",
    "BPL",
    "BVS",
    "BVC",
    "BHI",
    "BLS",
    "BGE",
    "BLT",
    "BGT",
    "BLE",
    "BL",
    "RET",
    "SWI",
```

```
    "RFE",
    "MOVWD",
    "MOVWF",
    "MOVDW",
    "MOVFW",
    "MOVFD",
    "MOVDF",
    "MOVF",
    "MOVD",
    "CMPF",
    "CMPD",
    "ADDF",
    "ADDD",
    "SUBF",
    "SUBD",
    "MULF",
    "MULD",
    "DIVF",
    "DIVD",
    "SQRTF",
    "SQRTD",
    "TEXT",
    "GLOBL",
    "DATA",
    "WORD",
    "END",
    "NAME",
    "HISTORY",
    "CASE",
    "BCASE",
    "SIGNAME",
    "GOK",
    "DYNT",
    "INIT",
    "LAST",
};
```

A.7 Optimization debugging

A.7.1 Printing arithmetic trees: `5c -m`

A.7.2 Printing registerization: `5c -r`

A.8 Printing initializations: `5c -i`

⟨`doinit()` *debug initialization* 419a⟩≡ (114d)

```
if(debug['i']) {
    print("t = %T; o = %ld; n = %s\n", t, o, s->name);
    prtree(a, "doinit value");
}
```

⟨`init1()` *debug initialization* 419b⟩≡ (213f)

```
if(debug['i']) {
    print("t = %T; o = %ld; n = %s\n", t, o, s->name);
    prtree(a, "init1 value");
}
```

⟨contig() *debug initialization* 420⟩≡ (219b)

```
if(debug['i']) {
    print("contig v = %ld; s = %s\n", v, s->name);
    prtree(n, "doinit value");
}
```

Appendix B

Error Management

⟨*global* `nerrors` 421a⟩≡ (436d)

```
int nerrors;
```

B.0.1 errorexit()

⟨*function* `errorexit` 421b⟩≡ (439a)

```
/// main | compile | newio | newfile | yylex | ... -> <>
void
errorexit(void)
{
    if(outfile)
        remove(outfile);
    exits("error");
}
```

B.0.2 yyerror()

⟨*function* `yyerror` 421c⟩≡ (439a)

```
void
yyerror(char *fmt, ...)
{
    char buf[STRINGSZ];
    va_list arg;

    ⟨yyerror() when called from yyparse 421d⟩

    va_start(arg, fmt);
    vseprint(buf, buf+sizeof(buf), fmt, arg);
    va_end(arg);

    Bprint(&diagbuf, "%L %s\n", lineno, buf);
    nerrors++;

    if(nerrors > 10) {
        Bprint(&diagbuf, "too many errors\n");
        errorexit();
    }
}
```

⟨`yyerror()` *when called from yyparse* 421d⟩≡ (421c)

```
/*
 * hack to intercept message from yaccpar
 */
```

```
if(strcmp(fmt, "syntax error") == 0) {
    yyerror("syntax error, last name: %s", symb);
    return;
}
```

B.0.3 diag()

⟨*function* diag 422a⟩≡ (443b)

```
void
diag(Node *n, char *fmt, ...)
{
    char buf[STRINGSZ];
    va_list arg;

    va_start(arg, fmt);
    vseprint(buf, buf+sizeof(buf), fmt, arg);
    va_end(arg);

    Bprint(&diagbuf, "%L %s\n", (n==Z)? nearln: n->lineno, buf);

    ⟨fatal() or diag() abort if -X 422c⟩
    ⟨fatal() or diag() print diagnostic if -v 423a⟩
    nerrors++;
    if(nerrors > 10) {
        Bprint(&diagbuf, "too many errors\n");
        errorexit();
    }
}
```

B.0.4 fatal()

⟨*function* fatal 422b⟩≡ (443b)

```
/// code generation stuff -> <>
void
fatal(Node *n, char *fmt, ...)
{
    char buf[STRINGSZ];
    va_list arg;

    va_start(arg, fmt);
    vseprint(buf, buf+sizeof(buf), fmt, arg);
    va_end(arg);
    Bprint(&diagbuf, "%L %s\n", (n==Z)? nearln: n->lineno, buf);

    ⟨fatal() or diag() abort if -X 422c⟩
    ⟨fatal() or diag() print diagnostic if -v 423a⟩
    nerrors++;
    errorexit();
}
```

⟨fatal() or diag() *abort if -X* 422c⟩≡ (422)

```
if(debug['X']){
    Bflush(&diagbuf);
    abort();
}
```

⟨fatal() or diag() *print diagnostic if -v* 423a⟩≡ (422)

```
if(n != Z)
  if(debug['v'])
    prtree(n, "diagnostic");
```

B.0.5 Error location

⟨cinit() *fmtinstall* 423b⟩+≡ (35a) ◁401b

```
fmtinstall('L', Lconv);
```

⟨*constant* HISTSZ 423c⟩≡ (429b)

```
#define HISTSZ  20
```

⟨*struct* Atab 423d⟩≡ (446b)

```
struct Atab
    {
        Hist* incl; /* start of this include file */
        long idel; /* delta line number to apply to include */
        Hist* line; /* start of this #line directive */
        long ldel; /* delta line number to apply to #line */
};
```

⟨*function* Lconv 423e⟩≡ (446b)

```
// int -> string?
int
Lconv(Fmt *fp)
{
    char str[STRINGSZ], s[STRINGSZ];
    Hist *h;
    long l, d;
    int i, n;
    struct Atab  a[HISTSZ];

    l = va_arg(fp->args, long);
    n = 0;
    for(h = hist; h != H; h = h->link) {
        if(l < h->line)
            break;
        if(h->name) {
            if(h->offset != 0) {  /* #line directive, not #pragma */
                if(n > 0 && n < HISTSZ && h->offset >= 0) {
                    a[n-1].line = h;
                    a[n-1].ldel = h->line - h->offset + 1;
                }
            } else {
                if(n < HISTSZ) { /* beginning of file */
                    a[n].incl = h;
                    a[n].idel = h->line;
                    a[n].line = 0;
                }
                n++;
            }
            continue;
        }
        n--;
        if(n > 0 && n < HISTSZ) {
            d = h->line - a[n].incl->line;
            a[n-1].ldel += d;
            a[n-1].idel += d;
```

```
        }
    }
    if(n > HISTSZ)
        n = HISTSZ;
    str[0] = 0;
    for(i=n-1; i>=0; i--) {
        if(i != n-1) {
            if(fp->flags & ~(FmtWidth|FmtPrec)) /* BUG ROB - was f3 */
                break;
            strcat(str, " ");
        }
        if(a[i].line)
            snprint(s, STRINGSZ, "%s:%ld[%s:%ld]",
                a[i].line->name, l-a[i].ldel+1,
                a[i].incl->name, l-a[i].idel+1);
        else
            snprint(s, STRINGSZ, "%s:%ld",
                a[i].incl->name, l-a[i].idel+1);
        if(strlen(s)+strlen(str) >= STRINGSZ-10)
            break;
        strcat(str, s);
        l = a[i].incl->line - 1; /* now print out start of this file */
    }
    if(n == 0)
        strcat(str, "<eof>");

    return fmtstrcpy(fp, str);
}
```

Appendix C

Libc

C.1 Memory management

⟨*constant* NHUNK 425a⟩≡ (429b)
```
#define NHUNK  50000L
```

⟨*global* hunk 425b⟩≡ (436d)
```
char* hunk;
```

⟨*global* nhunk 425c⟩≡ (436d)
```
long nhunk;
```

⟨*global* thunk 425d⟩≡ (436d)
```
long thunk;
```

⟨cinit() *memory globals initialization* 425e⟩≡ (35a)
```
nhunk = 0;
```

⟨*function* gethunk 425f⟩≡ (439a)
```
void
gethunk(void)
{
    char *h;
    long nh;

    nh = NHUNK;
    if(thunk >= 10L*NHUNK)
        nh = 10L*NHUNK;

    h = (char*)sbrk(nh);
    if(h == (char*)-1) {
        yyerror("out of memory");
        errorexit();
    }
    hunk = h;
    nhunk = nh;
    thunk += nh;
}
```

⟨*constant* MAXALIGN 425g⟩≡ (429b)
```
#define MAXALIGN 7
```

⟨*function* alloc 426a⟩≡ (439a)

```
/*
 * real allocs
 */
void*
alloc(long n)
{
    void *p;

    while((uintptr)hunk & MAXALIGN) {
        hunk++;
        nhunk--;
    }
    while(nhunk < n)
        gethunk();
    p = hunk;
    nhunk -= n;
    hunk += n;
    return p;
}
```

⟨*function* allocn 426b⟩≡ (439a)

```
void*
allocn(void *p, long on, long n)
{
    void *q;

    q = (uchar*)p + on;
    if(q != hunk || nhunk < n) {
        while(nhunk < on+n)
            gethunk();
        memmove(hunk, p, on);
        p = hunk;
        hunk += on;
        nhunk -= on;
    }
    hunk += n;
    nhunk -= n;
    return p;
}
```

⟨*function* malloc 426c⟩≡ (438b)

```
/*
 * fake mallocs
 */
void*
malloc(ulong n)
{
    return alloc(n);
}
```

⟨*function* free 426d⟩≡ (438b)

```
void
free(void*)
{
}
```

⟨*function* setmalloctag 426e⟩≡ (438b)

```
//@Scheck: looks dead, but because we redefine malloc/free we must also redefine that
void setmalloctag(void*, ulong)
{
}
```

C.2 String management

C.3 Bit operations

⟨*function* topbit 427a⟩≡ (443b)

```
int
topbit(ulong v)
{
    int i;

    for(i = -1; v; i++)
        v >>= 1;
    return i;
}
```

⟨*function* bitno 427b⟩≡ (443b)

```
int
bitno(long b)
{
    int i;

    for(i=0; i<32; i++)
        if(b & (1L<<i))
            return i;
    diag(Z, "bad in bitno");
    return 0;
}
```

⟨*function* SIGN 427c⟩≡ (429b)

```
#define SIGN(n)  (1ULL<<(n-1))
```

⟨*function* MASK 427d⟩≡ (429b)

```
#define MASK(n)  (SIGN(n)|(SIGN(n)-1))
```

C.4 Mathematic functions

⟨*function* round 427e⟩≡ (450)

```
long
round(long v, int w)
{
    int r;

    if(w <= 0 || w > 8) {
        diag(Z, "rounding by %d", w);
        w = 1;
    }
    r = v%w;
    if(r)
        v += w-r;
    return v;
}
```

⟨*function* log2 427f⟩≡ (443b)

```
/*
 * return log(n) if n is a power of 2 constant
 */
int
```

```
log2(uvlong v)
{
    int s, i;
    uvlong m;

    s = 0;
    m = MASK(8*sizeof(uvlong));
    for(i=32; i; i>>=1) {
        m >>= i;
        if(!(v & m)) {
            v >>= i;
            s += i;
        }
    }
    if(v == 1)
        return s;
    return -1;
}
```

C.5 Key/value data structure

⟨*struct* Init 428a⟩≡ (429b)

```
struct Init   /* general purpose initialization */
{
    int  code;
    ulong value;
    char* s;
};
```

C.6 Portability

⟨*enum* os 428b⟩≡ (429b)

```
enum os    /* also in ../{8a,5a,0a}.h */
{
    Plan9 = 1<<0,
    Unix = 1<<1,
    //Windows = 1<<2,
};
```

Appendix D

Extra Code

D.1 cc/

D.1.1 cc/cc.h

⟨*constant* YYMAXDEPTH 429a⟩≡ (429b)

```
#define YYMAXDEPTH 1500
```

⟨cc/cc.h 429b⟩≡

```
#include <u.h>
#include <libc.h>
#include <bio.h>
#include <ctype.h>

#pragma lib "../cc/cc.a$O" //$

typedef struct Node Node;
typedef struct Type Type;
typedef struct Sym  Sym;
typedef struct Decl Decl;
typedef struct Funct Funct;
typedef struct Io  Io;
typedef struct Hist Hist;
typedef struct Term Term;
typedef struct Init Init;
typedef struct Bits Bits;

typedef Rune TRune; /* target system type */

⟨constant NHUNK 425a⟩
⟨constant BUFSIZ 37f⟩
⟨constant NSYMB 22h⟩
⟨constant NHASH 22b⟩
⟨constant STRINGSZ 60h⟩
⟨constant HISTSZ 423c⟩
⟨constant YYMAXDEPTH 429a⟩
⟨constant NTERM 272c⟩
⟨constant MAXALIGN 425g⟩

⟨function SIGN 427c⟩
⟨function MASK 427d⟩

⟨constant BITS 290b⟩
⟨constant NVAR 292a⟩
⟨struct Bits 290c⟩
```

⟨*struct* Node 16b⟩
⟨*constant* Z 17a⟩

⟨*struct* Sym 21e⟩
⟨*constant* S 22d⟩

⟨*enum* signature 333a⟩

⟨*struct* Decl 24h⟩
⟨*constant* D 25c⟩

⟨*struct* Type 19b⟩
⟨*constant* T 19g⟩
⟨*constant* NODECL 91g⟩

⟨*struct* Init 428a⟩

⟨*struct* Fi 38a⟩
```
extern struct Fi fi;
```

⟨*struct* Io 36b⟩
⟨*constant* I 36e⟩

⟨*struct* Hist 59f⟩
⟨*constant* H 60a⟩
```
extern Hist* hist;
```

⟨*struct* Term 272d⟩

⟨*enum* os 428b⟩
⟨*enum* node_kind 16a⟩
⟨*enum* type_kind 18d⟩
⟨*enum* type_kind_bis 93f⟩
⟨*enum* align 226b⟩
⟨*enum* dxxx 24g⟩
⟨*enum* storage_class 20e⟩
⟨*enum* qualifier 20d⟩
⟨*enum* bxxx 94a⟩

⟨*struct* Funct 370c⟩

⟨*struct* En 104c⟩
```
extern struct En en;

extern int autobn;
extern long autoffset;
extern int blockno;
extern Decl* dclstack;
extern char debug[256];
extern Hist* ehist;
extern bool firstbit;
extern Sym* firstarg;
extern Type* firstargtype;
extern Decl* firstdcl;
extern Sym* hash[NHASH];
extern char* hunk;
extern char** include;
extern Io* iofree;
```

```
extern Io* ionext;
extern Io* iostack;
extern long lastbit;
extern char lastclass;
extern Type* lastdcltype;
extern long lastfield;
extern Type* lasttype;
extern long lineno;
extern long nearln;
extern int maxinclude;
extern int nerrors;
extern bool newflag;
extern long nhunk;
extern int ninclude;
extern Node* nodproto;
extern Node* nodcast;
extern Biobuf outbuf;
extern Biobuf diagbuf;
extern char* outfile;
extern char* pathname;
extern int peekc;
extern long stkoff;
extern Type* strf;
extern Type* strl;
extern char symb[NSYMB];
extern Sym* symstring;
extern int taggen;
extern Type* tfield;
extern Type* tufield;
extern int thechar;
extern char* thestring;
extern Type* thisfntype;
extern long thunk;
extern Type* types[NTYPE];

extern Type* fntypes[NTYPE];
extern Node* initlist;
extern Term term[NTERM];
extern int nterm;
extern int packflg;
extern int fproundflg;
extern bool profileflg;
extern int ncontin;
extern int canreach;
extern int warnreach;
extern Bits zbits;

extern char *onames[], *tnames[], *gnames[];
extern char *cnames[], *qnames[], *bnames[];

extern char comrel[], invrel[], logrel[];
extern long ncast[], tadd[], tand[];
extern long targ[], tasadd[], tasign[], tcast[];
extern long tdot[], tfunct[], tindir[], tmul[];
extern long tnot[], trel[], tsub[];

extern char typeaf[];
extern char typefd[];
extern char typei[];
extern char typesu[];
```

```
extern char typesuv[];
extern char typeu[];
extern char typev[];
extern char typeilp[];
extern char typechl[];

// not used on ARM
extern char typechlv[];
extern char typeil[];

extern char typechlp[];
extern char typechlpfd[];

extern char* typeswitch;
extern char* typeword;
extern char* typecmplx;

extern ulong thash1;
extern ulong thash2;
extern ulong thash3;
extern ulong thash[];

/*
 * compat.c/unix.c/windows.c
 */
int mywait(int*);
int mycreat(char*, int);
int myaccess(char*);
int pathchar(void);
bool systemtype(int);

// utils.c
void gethunk(void);
void* allocn(void*, long, long);
void* alloc(long);
void errorexit(void);
void yyerror(char*, ...);

/*
 * parser
 */
//@Scheck: def in y.tab.c from cc.y
int yyparse(void);

/*
 * lex.c
 */
long yylex(void);

Sym* lookup(void);
Sym* slookup(char*);

int filbuf(void);
int getc(void);
int getnsc(void);
void newfile(char*, int);
void newio(void);
void pushio(void);
void unget(int);
```

```
// used by dpchk.c
long getr(void);

//!!!! (hmmm in lex.c, as well as cinit(), compile())
void main(int, char*[]);


/*
 * mac.c
 */
void dodefine(char*);
void domacro(void);
void linehist(char*, int);
void macexpand(Sym*, char*);

// for dpchk.c
Sym* getsym(void);
long getnsn(void);

/*
 * dcl.c
 */
//@Scheck: useful, used by cc.y
Type* tcopy(Type*);
//@Scheck: useful, used by cc.y
Node* doinit(Sym*, Type*, long, Node*);
//@Scheck: useful, used by cc.y
void adecl(int, Type*, Sym*);
void argmark(Node*, int);
Node* dcllabel(Sym*, int);
Node* dodecl(void(*)(int, Type*, Sym*), int, Type*, Node*);
//@Scheck: useful, used by cc.y
Sym* mkstatic(Sym*);
void doenum(Sym*, Node*);
void snap(Type*);
Type* dotag(Sym*, int, int);
void edecl(int, Type*, Sym*);
void markdcl(void);
//@Scheck: useful, used by cc.y
void pdecl(int, Type*, Sym*);
Node* revertdcl(void);
long round(long, int);
int sametype(Type*, Type*);
ulong sign(Sym*);
ulong signature(Type*);
void sualign(Type*);
void xdecl(int, Type*, Sym*);
Node* contig(Sym*, Node*, long);

/*
 * com.c
 */
void ccom(Node*);
void complex(Node*);
int tcom(Node*);
int tcoma(Node*, Node*, Type*, int);
int tcomo(Node*, int);
void constas(Node*, Type*, Type*);
Node* uncomma(Node*);
```

```
/*
 * con.c
 */
void acom(Node*);
void evconst(Node*);

/*
 * funct.c
 */
int isfunct(Node*);
void dclfunct(Type*, Sym*);

/*
 * sub.c
 */
void arith(Node*, int);
int deadheads(Node*);
Type* dotsearch(Sym*, Type*, Node*, long*);
Node* invert(Node*);
int bitno(long);
void makedot(Node*, Type*, long);
int mixedasop(Type*, Type*);
Node* new(int, Node*, Node*);
Node* new1(int, Node*, Node*);
int nilcast(Type*, Type*);
int nocast(Type*, Type*);
void prtree(Node*, char*);
void prtree1(Node*, int, int);
void relcon(Node*, Node*);
int relindex(int);
//@Scheck: useful, used by cc.y
int simpleg(long);
Type* garbt(Type*, long);
int simplec(long);
Type* simplet(long);
int stcompat(Node*, Type*, Type*, long[]);
int tcompat(Node*, Type*, Type*, long[]);
void tinit(void);
Type* typ(int, Type*);
Type* copytyp(Type*);
void typeext(Type*, Node*);
void typeext1(Type*, Node*);
int side(Node*);
int vconst(Node*);
int log2(uvlong);
int vlog(Node*);
int topbit(ulong);
void simplifyshift(Node*);
long typebitor(long, long);
void diag(Node*, char*, ...);
void warn(Node*, char*, ...);
void fatal(Node*, char*, ...);

/*
 * acid.c
 */
void acidtype(Type*);
void acidvar(Sym*);

/*
```

```
 * pickle.c
 */
void pickletype(Type*);

/*
 * bits.c
 */
Bits bor(Bits, Bits);
int bany(Bits*);
int bnum(Bits);
Bits blsh(uint);
int beq(Bits, Bits);
int bset(Bits, uint);
//Bits band(Bits, Bits);
//Bits bnot(Bits);

/*
 * dpchk.c
 */
void dpcheck(Node*);
void arginit(void);

void pragvararg(void);
void pragpack(void);
void pragfpround(void);
void pragprofile(void);
void pragincomplete(void);

/*
 * calls to machine depend part
 */
//todo: could define an interface instantiated by each xc
void codgen(Node*, Node*);
void gclean(void);
void gextern(Sym*, Node*, long, long);
void ginit(void);
long outstring(char*, long);
long outlstring(TRune*, long);
void xcom(Node*);
long exreg(Type*);
long align(long, Type*, int);
long maxround(long, long);

extern schar ewidth[];

/*
 * com64
 */
int com64(Node*);
void com64init(void);
void bool64(Node*);
vlong convvtox(vlong, int);
//double convvtof(vlong);
//vlong  convftov(double);
//double convftox(double, int);

/*
 * machcap
 */
int machcap(Node*);
```

```
#pragma varargck argpos warn 2
#pragma varargck argpos diag 2
#pragma varargck argpos yyerror 1

#pragma varargck type "F" Node*
#pragma varargck type "L" long
#pragma varargck type "Q" long
#pragma varargck type "O" int
#pragma varargck type "T" Type*
#pragma varargck type "|" int
```

D.1.2 cc/globals.c

⟨*global* fproundflg 436a⟩≡ (436d)

```
int fproundflg;
```

⟨macprag() *if pragma xxx* 436b⟩+≡ (74a) ◁398a

```
if(s && strcmp(s->name, "fpround") == 0) {
    pragfpround();
    return;
}
```

⟨*function* pragfpround 436c⟩≡ (453)

```
void
pragfpround(void)
{
    Sym *s;

    fproundflg = 0;
    s = getsym();
    if(s) {
        fproundflg = atoi(s->name+1);
        if(strcmp(s->name, "on") == 0 ||
           strcmp(s->name, "yes") == 0)
            fproundflg = 1;
    }
    while(getnsc() != '\n')
        ;
    if(debug['f'])
        if(fproundflg)
            print("%4ld: fproundflg %d\n", lineno, fproundflg);
        else
            print("%4ld: fproundflg off\n", lineno);
}
```

⟨cc/globals.c 436d⟩≡

```
#include "cc.h"

⟨global fi 38b⟩
⟨global hist 60c⟩
⟨global en 104d⟩

⟨global autobn 25h⟩
⟨global autoffset 158c⟩
⟨global blockno 25g⟩
⟨global dclstack 25a⟩
⟨global debug 401a⟩
⟨global ehist 60d⟩
```

⟨*global* `firstbit` 352c⟩
⟨*global* `firstarg` 225a⟩
⟨*global* `firstargtype` 225c⟩
⟨*global* `firstdcl` 80a⟩
⟨*global* `hash` 22a⟩
⟨*global* `hunk` 425b⟩
⟨*global* `include` 59a⟩
⟨*global* `iofree` 37a⟩
⟨*global* `ionext` 36f⟩
⟨*global* `iostack` 36c⟩
⟨*global* `lastbit` 352d⟩
⟨*global* `lastclass` 77e⟩
⟨*global* `lastdcl` 78e⟩
⟨*global* `lastfield` 352h⟩
⟨*global* `lasttype` 77f⟩
⟨*global* `lineno` 17f⟩
⟨*global* `nearln` 18a⟩
⟨*global* `maxinclude` 59c⟩
⟨*global* `nerrors` 421a⟩
⟨*global* `newflag` 399d⟩
⟨*global* `nhunk` 425c⟩
⟨*global* `ninclude` 59b⟩
⟨*global* `nodproto` 140e⟩
⟨*global* `nodcast` 213c⟩
⟨*global* `outbuf` 32b⟩
⟨*global* `diagbuf` 32c⟩
⟨*global* `outfile` 30c⟩
⟨*global* `pathname` 35b⟩
⟨*global* `peekc` 40c⟩
⟨*global* `stkoff` 158b⟩
⟨*global* `strf` 102f⟩
⟨*global* `strl` 102g⟩
⟨*global* `symb` 22g⟩
⟨*global* `symstring` 181g⟩
⟨*global* `taggen` 102c⟩
⟨*global* `tfield` 353b⟩
⟨*global* `tufield` 353c⟩
⟨*global* `thechar` 30a⟩
⟨*global* `thestring` 30b⟩
⟨*global* `thisfn` 80b⟩
⟨*global* `thunk` 425d⟩
⟨*global* `types` 20a⟩
⟨*global* `fntypes` 389a⟩
⟨*global* `initlist` 93c⟩
⟨*global* `term` 272e⟩
⟨*global* `nterm` 272f⟩
⟨*global* `packflg` 398b⟩
⟨*global* `fproundflg` 436a⟩
⟨*global* `profileflg` 348a⟩
⟨*global* `ncontin` 165c⟩
⟨*global* `canreach` 149d⟩
⟨*global* `warnreach` 149e⟩
⟨*global* `zbits` 290d⟩

⟨*global* `typeswitch` 144d⟩
⟨*global* `typeword` 280a⟩
⟨*global* `typecmplx` 222a⟩

D.1.3 cc/omachcap.c

⟨cc/omachcap.c 438a⟩≡

```
#include "cc.h"

/* default, like old cc */
//int
//machcap(Node*)
//{
// return 0;
//}
```

D.1.4 cc/compat.c

⟨cc/compat.c 438b⟩≡

```
#include "cc.h"

//#include "compat"
//TODO copy paste with aa/, maybe could factorize with another lib?
//

int
myaccess(char *f)
{
 return access(f, AEXIST);
}


int
mycreat(char *n, int p)
{

 return create(n, 1, p);
}

int
mywait(int *s)
{
 int p;
 Waitmsg *w;

 if((w = wait()) == nil)
  return -1;
 else{
  p = w->pid;
  *s = 0;
  if(w->msg[0])
   *s = 1;
  free(w);
  return p;
 }
}

bool
systemtype(int sys)
{
 return sys & Plan9;
}
```

```
int
pathchar(void)
{
 return '/';
}
```

⟨*function* malloc 426c⟩

⟨*function* free 426d⟩

⟨*function* setmalloctag 426e⟩

D.1.5 cc/utils.c

⟨cc/utils.c 439a⟩≡

```
#include "cc.h"
```

⟨*function* errorexit 421b⟩

⟨*function* gethunk 425f⟩

⟨*function* alloc 426a⟩

⟨*function* allocn 426b⟩

⟨*function* yyerror 421c⟩

D.1.6 cc/mac.c

⟨cc/mac.c 439b⟩≡

```
#include "cc.h"

//old: #include "macbody"
//TODO copy paste with aa/, maybe could factorize with another lib?

void macdef(void);
void macinc(void);
void macprag(void);
void maclin(void);
void macif(int);
void macend(void);
void macund(void);

long getnsn(void);
Sym* getsym(void);
```

⟨*constant* VARMAC 64c⟩

⟨*function* getnsn 75⟩

⟨*function* getsym 57b⟩

⟨*function* getsymdots 65b⟩

⟨*function* getcom 62c⟩

⟨*function* dodefine 63c⟩

⟨*global* mactab 56b⟩

⟨*function* domacro 56c⟩

⟨*function* macund 72a⟩

⟨*constant* NARG 64a⟩
⟨*function* macdef 64b⟩

⟨*function* macexpand 68e⟩

⟨*function* macinc 61a⟩

⟨*function* maclin 74b⟩

⟨*function* macif 72b⟩

⟨*function* macprag 74a⟩

⟨*function* macend 58c⟩

⟨*function* linehist 60g⟩

D.1.7 cc/sub.c

⟨*function* simplifyshift 440⟩≡ (443b)

```
/*
 * try to rewrite shift & mask
 */
void
simplifyshift(Node *n)
{
    ulong c3;
    int o, s1, s2, c1, c2;

    if(!typechlp[n->type->etype])
        return;
    switch(n->op) {
    default:
        return;
    case OASHL:
        s1 = 0;
        break;
    case OLSHR:
        s1 = 1;
        break;
    case OASHR:
        s1 = 2;
        break;
    }
    if(n->right->op != OCONST)
        return;
    if(n->left->op != OAND)
        return;
    if(n->left->right->op != OCONST)
        return;
    switch(n->left->left->op) {
```

```
    default:
        return;
    case OASHL:
        s2 = 0;
        break;
    case OLSHR:
        s2 = 1;
        break;
    case OASHR:
        s2 = 2;
        break;
    }
    if(n->left->left->right->op != OCONST)
        return;

    c1 = n->right->vconst;
    c2 = n->left->left->right->vconst;
    c3 = n->left->right->vconst;

/*
    if(debug['h'])
        print("%.3o %ld %ld %d #%.lux\n",
            (s1<<3)|s2, c1, c2, topbit(c3), c3);
*/

    o = n->op;
    switch((s1<<3)|s2) {
    case 000: /* (((e <<u c2) & c3) <<u c1) */
        c3 >>= c2;
        c1 += c2;
        if(c1 >= 32)
            break;
        goto rewrite1;

    case 002: /* (((e >>s c2) & c3) <<u c1) */
        if(topbit(c3) >= (32-c2))
            break;
    case 001: /* (((e >>u c2) & c3) <<u c1) */
        if(c1 > c2) {
            c3 <<= c2;
            c1 -= c2;
            o = OASHL;
            goto rewrite1;
        }
        c3 <<= c1;
        if(c1 == c2)
            goto rewrite0;
        c1 = c2-c1;
        o = OLSHR;
        goto rewrite2;

    case 022: /* (((e >>s c2) & c3) >>s c1) */
        if(c2 <= 0)
            break;
    case 012: /* (((e >>s c2) & c3) >>u c1) */
        if(topbit(c3) >= (32-c2))
            break;
        goto s11;
    case 021: /* (((e >>u c2) & c3) >>s c1) */
        if(topbit(c3) >= 31 && c2 <= 0)
```

```
            break;
        goto s11;
    case 011: /* (((e >>u c2) & c3) >>u c1) */
    s11:
        c3 <<= c2;
        c1 += c2;
        if(c1 >= 32)
            break;
        o = OLSHR;
        goto rewrite1;

    case 020: /* (((e <<u c2) & c3) >>s c1) */
        if(topbit(c3) >= 31)
            break;
    case 010: /* (((e <<u c2) & c3) >>u c1) */
        c3 >>= c1;
        if(c1 == c2)
            goto rewrite0;
        if(c1 > c2) {
            c1 -= c2;
            goto rewrite2;
        }
        c1 = c2 - c1;
        o = OASHL;
        goto rewrite2;
    }
    return;

rewrite0: /* get rid of both shifts */
if(debug['<'])prtree(n, "rewrite0");
    *n = *n->left;
    n->left = n->left->left;
    n->right->vconst = c3;
    return;
rewrite1: /* get rid of lower shift */
if(debug['<'])prtree(n, "rewrite1");
    n->left->left = n->left->left->left;
    n->left->right->vconst = c3;
    n->right->vconst = c1;
    n->op = o;
    return;
rewrite2: /* get rid of upper shift */
if(debug['<'])prtree(n, "rewrite2");
    *n = *n->left;
    n->right->vconst = c3;
    n->left->right->vconst = c1;
    n->left->op = o;
}
```

⟨*global* typechlv 442a⟩≡ (443b)

```
char typechlv[NTYPE];
```

⟨*global* typeil 442b⟩≡ (443b)

```
char typeil[NTYPE];
```

⟨*global* typeilinit 442c⟩≡ (443b)

```
int typeilinit[] =
{
    TINT, TUINT, TLONG, TULONG, -1,
};
```

⟨tinit() *initialise typexxx type sets* 443a⟩+≡ (34b) ◁118g

```
for(ip=typeilinit; *ip>=0; ip++) {
    urk("typeil", nelem(typeil), *ip);
    typeil[*ip] = 1;
}
```

⟨cc/sub.c 443b⟩≡

```
#include "cc.h"

int simpleg(long);
long dotoffset(Type*, Type*, Node*);

extern char tab[NTYPE][NTYPE];

⟨function new 17d⟩

⟨function new1 18b⟩

⟨function prtree 413b⟩

⟨function prtree1 413c⟩

⟨function typ 19h⟩

⟨function copytyp 99f⟩

⟨function garbt 99e⟩

⟨function simpleg 100a⟩

⟨function simplec 99a⟩

⟨function simplet 97c⟩

⟨function stcompat 119c⟩

⟨function tcompat 119a⟩

⟨function makedot 194b⟩

⟨function dotsearch 137d⟩

⟨function dotoffset 362⟩

⟨function allfloat 382e⟩

⟨function constas 154b⟩

⟨function typeext1 382d⟩

⟨function typeext 360e⟩

⟨function nocast 202a⟩

⟨function nilcast 246a⟩

⟨function arith 127d⟩

⟨function simplifyshift 440⟩
```

⟨*function* side 242c⟩

⟨*function* vconst 142b⟩

⟨*function* log2 427f⟩

⟨*function* vlog 256b⟩

⟨*function* topbit 427a⟩

⟨*function* relcon 245c⟩

⟨*function* relindex 187d⟩

⟨*function* invert 81a⟩

⟨*function* bitno 427b⟩

⟨*function* typebitor 96a⟩

⟨*function* diag 422a⟩

⟨*function* warn 113a⟩

⟨*function* fatal 422b⟩

⟨*global* thash1 335a⟩
⟨*global* thash2 335b⟩
⟨*global* thash3 335c⟩
⟨*global* thash 335d⟩
⟨*global* thashinit 335e⟩

⟨*global* bnames 226c⟩
⟨*global* bnamesinit 226d⟩

⟨*global* tnames 404a⟩
⟨*global* tnamesinit 404b⟩

⟨*global* gnames 412a⟩
⟨*global* gnamesinit 412b⟩

⟨*global* qnames 411c⟩
⟨*global* qnamesinit 411d⟩
⟨*global* cnames 411a⟩
⟨*global* cnamesinit 411b⟩

⟨*global* onames 401d⟩
⟨*global* onamesinit 401e⟩

⟨*global* comrel 208a⟩
⟨*global* invrel 208b⟩
⟨*global* logrel 187c⟩

⟨*global* typei 118a⟩
⟨*global* typeiinit 118b⟩
⟨*global* typeu 186e⟩
⟨*global* typeuinit 186f⟩

⟨*global* typesuv 222c⟩
⟨*global* typesuvinit 222d⟩

```
// not used on ARM
⟨global typechlv 442a⟩
⟨global typeil 442b⟩
⟨global typeilinit 442c⟩

⟨global typeilp 172e⟩
⟨global typeilpinit 172f⟩

⟨global typechl 144f⟩
⟨global typechlinit 145a⟩

⟨global typechlp 173a⟩
⟨global typechlpinit 173b⟩

⟨global typechlpfd 297a⟩
⟨global typechlpfdinit 297b⟩

⟨global typev 229a⟩
⟨global typevinit 229b⟩

⟨global typefd 118e⟩
⟨global typefdinit 118f⟩

⟨global typeaf 122d⟩
⟨global typeafinit 122e⟩

⟨global typesu 118c⟩
⟨global typesuinit 118d⟩

⟨global tasign 133c⟩
⟨global tasigninit 133d⟩

⟨global tasadd 135a⟩
⟨global tasaddinit 135b⟩

⟨global tcast 141b⟩
⟨global tcastinit 141c⟩

⟨global tadd 120c⟩
⟨global taddinit 120d⟩

⟨global tsub 124e⟩
⟨global tsubinit 124f⟩

⟨global tmul 125b⟩
⟨global tmulinit 125c⟩

⟨global tand 127a⟩
⟨global tandinit 127b⟩

⟨global trel 131b⟩
⟨global trelinit 131c⟩

⟨global tfunct 138c⟩

⟨global tindir 136d⟩

⟨global tdot 137b⟩
```

⟨*global* tnot 132b⟩

⟨*global* targ 139a⟩

⟨*global* tab 128c⟩

⟨*function* urk 34d⟩

⟨*function* tinit 34b⟩

⟨*function* deadhead 150b⟩

⟨*function* deadheads 151a⟩

⟨*function* mixedasop 134f⟩

D.1.8 cc/bits.c

⟨cc/bits.c 446a⟩≡

```
#include "cc.h"
```

⟨*function* bor 290e⟩

⟨*function* bany 290f⟩

⟨*function* beq 291a⟩

⟨*function* bnum 291b⟩

⟨*function* blsh 291c⟩

⟨*function* bset 291d⟩

D.1.9 cc/lex.c

⟨cc/lex.c 446b⟩≡

```
#include "cc.h"
#include "y.tab.h"

int compile(char*, char**, int);
void syminit(Sym*);
int mpatov(char*, vlong*);
long getr(void);
long escchar(long, int, int);
void cinit(void);

int Oconv(Fmt*);
int Lconv(Fmt*);
int Tconv(Fmt*);
int FNconv(Fmt*);
int Qconv(Fmt*);
int VBconv(Fmt*);

void setinclude(char*);


#ifndef CPP
```

⟨*constant* CPP 396c⟩

```
#endif

/*
 * known debug flags
 * -a  acid declaration output
 * -A  !B
 * -B  non ANSI
 * -d  print declarations
 * -D name  define
 * -F  format specification check
 * -i  print initialization
 * -I path  include
 * -l  generate little-endian code
 * -L  print every NAME symbol
 * -M  constant multiplication
 * -m  print add/sub/mul trees
 * -n  print acid to file (%.c=%.acid) (with -a or -aa)
 * -o file  output file
 * -p  use standard cpp ANSI preprocessor (not on windows)
 * -r  print registerization
 * -s  print structure offsets (with -a or -aa)
 * -S  print assembly
 * -t  print type trees
 * -V  enable void* conversion warnings
 * -v  verbose printing
 * -w  print warnings
 * -X  abort on error
 * -.  Inhibit search for includes in source directory
 */
```

⟨*function* main 30d⟩

⟨*function* compile 32e⟩

⟨*function* pushio 62b⟩

⟨*function* newio 36h⟩

⟨*function* newfile 37e⟩

⟨*function* slookup 22f⟩

⟨*function* lookup 23a⟩

⟨*function* syminit 24f⟩

⟨*constant* EOF 38d⟩
⟨*constant* IGN 40d⟩
⟨*constant* ESC 54b⟩
⟨*function* GETC 38c⟩

⟨*enum* numxxx 46c⟩

⟨*function* yylex 40a⟩

⟨*function* mpatov 47c⟩

⟨*function* getc 58d⟩

⟨*function* getr 53a⟩

⟨*function* getnsc 57c⟩

⟨*function* unget 58b⟩

⟨*function* escchar 52a⟩

⟨*global* itab 42a⟩

⟨*function* cinit 35a⟩

⟨*function* filbuf 38e⟩

⟨*function* Oconv 401c⟩

⟨*struct* Atab 423d⟩

⟨*function* Lconv 423e⟩

⟨*function* Tconv 403⟩

⟨*function* FNconv 405a⟩

⟨*function* Qconv 405b⟩

⟨*function* VBconv 405c⟩

⟨*function* setinclude 59d⟩

D.1.10 cc/acid.c

⟨cc/acid.c 448⟩≡

```
#include "cc.h"
```

⟨*global* kwd 340b⟩

⟨*function* amap 340c⟩

⟨*function* acidsue 339c⟩

⟨*function* acidfun 340a⟩

⟨*global* acidchar 341c⟩
⟨*global* acidcinit 342a⟩

⟨*function* acidinit 342b⟩

⟨*function* acidmember 342c⟩

⟨*function* acidtype 340d⟩

⟨*function* acidvar 338e⟩

D.1.11 cc/pickle.c

⟨cc/pickle.c 449a⟩≡
```
#include "cc.h"

⟨global kwd (cc/pickle.c) 344b⟩
⟨global picklestr 344c⟩

⟨function pmap 344d⟩

⟨function picklesue 345a⟩

⟨global picklechar 345b⟩
⟨global picklecinit 345c⟩

⟨function pickleinit 345d⟩

⟨function picklemember 345e⟩

⟨function pickletype 347⟩
```

D.1.12 cc/com64.c

⟨cc/com64.c 449b⟩≡
```
#include "cc.h"

⟨constant FNX 171c⟩

⟨global nodaddv ??⟩
⟨global nodsubv ??⟩
⟨global nodmulv ??⟩
⟨global noddivv ??⟩
⟨global noddivvu ??⟩
⟨global nodmodv ??⟩
⟨global nodmodvu ??⟩
⟨global nodlshv ??⟩
⟨global nodrshav ??⟩
⟨global nodrshlv ??⟩
⟨global nodandv ??⟩
⟨global nodorv ??⟩
⟨global nodxorv ??⟩
⟨global nodnegv ??⟩
⟨global nodcomv ??⟩

⟨global nodtestv ??⟩
⟨global nodeqv ??⟩
⟨global nodnev ??⟩
⟨global nodlev ??⟩
⟨global nodltv ??⟩
⟨global nodgev ??⟩
⟨global nodgtv ??⟩
⟨global nodhiv ??⟩
⟨global nodhsv ??⟩
⟨global nodlov ??⟩
⟨global nodlsv ??⟩

⟨global nodf2v ??⟩
⟨global nodd2v ??⟩
```

⟨*global* nodp2v ??⟩
⟨*global* nodsi2v ??⟩
⟨*global* nodui2v ??⟩
⟨*global* nodsl2v ??⟩
⟨*global* nodul2v ??⟩
⟨*global* nodsh2v ??⟩
⟨*global* noduh2v ??⟩
⟨*global* nodsc2v ??⟩
⟨*global* noduc2v ??⟩

⟨*global* nodv2f ??⟩
⟨*global* nodv2d ??⟩
⟨*global* nodv2ui ??⟩
⟨*global* nodv2si ??⟩
⟨*global* nodv2ul ??⟩
⟨*global* nodv2sl ??⟩
⟨*global* nodv2uh ??⟩
⟨*global* nodv2sh ??⟩
⟨*global* nodv2uc ??⟩
⟨*global* nodv2sc ??⟩

⟨*global* nodvpp ??⟩
⟨*global* nodppv ??⟩
⟨*global* nodvmm ??⟩
⟨*global* nodmmv ??⟩

⟨*global* nodvasop ??⟩

⟨*global* etconv 388a⟩
⟨*global* initetconv 388b⟩

⟨*function* fvn 389b⟩

⟨*function* com64init 387d⟩

⟨*function* com64 389d⟩

⟨*function* bool64 396b⟩

⟨*function* convvtox 47b⟩

D.1.13 cc/dcl.c

⟨cc/dcl.c 450⟩≡

```
#include "cc.h"

Sym* mkstatic(Sym*);
Type* tcopy(Type*);
Node* doinit(Sym*, Type*, long, Node*);
Node* init1(Sym*, Type*, long, int);
Node* newlist(Node*, Node*);
void walkparam(Node*, int);
Type* fnproto(Node*);
Type* fnproto1(Node*);
int anyproto(Node*);
void dbgdecl(Sym*);
Decl* push(void);
Decl* push1(Sym*);
int rsametype(Type*, Type*, int, int);
```

```
Type* paramconv(Type*, int);
void adecl(int, Type*, Sym*);
void pdecl(int, Type*, Sym*);
void tmerge(Type*, Sym*);
```

⟨*function* dodecl 78d⟩

⟨*function* mkstatic 351a⟩

⟨*function* tcopy 147b⟩

⟨*function* doinit 93d⟩

⟨*function* peekinit 218a⟩

⟨*function* nextinit 218b⟩

⟨*function* isstruct 217b⟩

⟨*function* init1 213f⟩

⟨*function* newlist 217a⟩

⟨*function* sualign 227d⟩

⟨*function* round 427e⟩

⟨*function* ofnproto 357d⟩

⟨*constant* ANSIPROTO 357e⟩
⟨*constant* OLDPROTO 357a⟩

⟨*function* argmark 109a⟩

⟨*function* walkparam 109b⟩

⟨*function* markdcl 26a⟩

⟨*function* revertdcl 26d⟩

⟨*function* fnproto 107c⟩

⟨*function* anyproto 358a⟩

⟨*function* fnproto1 107d⟩

⟨*function* dbgdecl 416c⟩

⟨*function* push 25e⟩

⟨*function* push1 25f⟩

⟨*function* sametype 115d⟩

⟨*function* rsametype 115e⟩

```
typedef struct Typetab Typetab;
```

⟨*struct* Typetab 336a⟩

⟨*function* sigind 336b⟩

⟨*function* signat 334c⟩

⟨*function* signature 334b⟩

⟨*function* sign 334a⟩

⟨*function* snap 117a⟩

⟨*function* dotag 101a⟩

⟨*function* dcllabel 83h⟩

⟨*function* paramconv 146e⟩

⟨*function* adecl 111b⟩

⟨*function* pdecl 110c⟩

⟨*function* xdecl 79g⟩

⟨*function* tmerge 117d⟩

⟨*function* edecl 104a⟩

⟨*function* maxtype 145h⟩

⟨*function* doenum 105d⟩

⟨*function* symadjust 220⟩

⟨*function* contig 219b⟩

D.1.14 cc/funct.c

⟨cc/funct.c 452a⟩≡

```
#include "cc.h"

typedef struct Ftab Ftab;
typedef struct Gtab Gtab;

⟨struct Ftab 369a⟩
⟨struct Gtab 369b⟩

⟨global gtabinit 369c⟩
⟨global ftabinit 369d⟩

⟨function isfunct 371⟩
⟨function dclfunct 367f⟩
```

D.1.15 cc/scon.c

⟨cc/scon.c 452b⟩≡

```
#include "cc.h"

void acom1(vlong, Node*);
void acom2(Node*, Type*);
```

```
int acomcmp1(const void*, const void*);
int acomcmp2(const void*, const void*);
int addo(Node*);

⟨function acast 275a⟩

⟨function evconst 250d⟩

⟨function acom 272g⟩

⟨function acomcmp1 275b⟩

⟨function acomcmp2 275c⟩

⟨function acom2 276⟩

⟨function acom1 274⟩

⟨function addo 273⟩
```

D.1.16 cc/dpchk.c

```
⟨cc/dpchk.c 453⟩≡
  #include "cc.h"
  #include "y.tab.h"

  ⟨enum fxxx 375b⟩

  typedef struct Tprot Tprot;
  ⟨struct Tprot 375c⟩

  typedef struct Tname Tname;
  ⟨struct Tname 375d⟩

  ⟨global indchar 375e⟩
  ⟨global flagbits 375f⟩
  ⟨global fmtbuf 375g⟩
  ⟨global lastadj 375h⟩
  ⟨global lastverb 375i⟩
  ⟨global nstar 375j⟩
  ⟨global tprot 375k⟩
  ⟨global tname 376a⟩

  ⟨function argflag 376b⟩

  ⟨function getflag 376c⟩

  ⟨function arginit 377a⟩

  ⟨function nextarg 377b⟩

  ⟨function checkargs 377c⟩

  ⟨function dpcheck 379b⟩


  ⟨function pragpack 398c⟩

  ⟨function newname 381a⟩
```

⟨*function* newprot 381b⟩

⟨*function* pragvararg 379d⟩

⟨*function* pragfpround 436c⟩

⟨*function* pragprofile 348e⟩

⟨*function* pragincomplete 336d⟩

D.1.17 cc/com.c

⟨cc/com.c 454⟩≡

```
#include "cc.h"

int tcomd(Node*);
int tcomx(Node*);
int tlvalue(Node*);

typedef struct Com Com;
typedef struct Big Big;
```

⟨*struct* Com 254c⟩

```
int compar(Node*, int);
static void comma(Node*);
static Node* commas(Com*, Node*);
```

⟨*function* complex 168b⟩

⟨*enum* _anon_ (cc/com.c) 121b⟩

⟨*function* tcom 121a⟩

⟨*function* tcomo 121c⟩

⟨*function* tcoma 138d⟩

⟨*function* tcomd 137c⟩

⟨*function* tcomx 363d⟩

⟨*function* tlvalue 133b⟩

⟨*function* comargs 256a⟩

⟨*function* commas 255⟩

⟨*function* comma 254d⟩

⟨*function* ccom 240a⟩

⟨*global* cmps 247d⟩

⟨*struct* Big 246b⟩
⟨*function* cmp 247a⟩
⟨*function* add 247b⟩

⟨*function* big 246c⟩

⟨*function* compar 247e⟩

D.2 cc2/

D.2.1 cc2/pswt.c

⟨cc2/pswt.c 455a⟩≡
```
#include "gc.h"
```

⟨*function* swcmp 230f⟩

⟨*function* doswit 230g⟩

⟨*function* casf 232a⟩

⟨*function* outlstring 183b⟩

⟨*function* nullwarn 152d⟩

⟨*function* ieeedtod 384a⟩

D.2.2 cc2/pgen.c

⟨cc2/pgen.c 455b⟩≡
```
#include "gc.h"

void gen(Node*);
void usedset(Node*, int);
int  bcomplex(Node*, Node*);
```

⟨*function* codgen 156d⟩

⟨*function* supgen 152a⟩

⟨*function* uncomma 166b⟩

⟨*function* gen 158i⟩

⟨*function* usedset 148i⟩

⟨*function* bcomplex 160a⟩

D.3 5c/

D.3.1 5c/enam.c

⟨5c/enam.c 455c⟩≡
⟨*global* anames*(arm)* 418⟩

D.3.2 5c/gc.h

⟨5c/gc.h 456⟩≡

```
#include "../cc/cc.h"

#include <common.out.h>
#include <5.out.h>

/*
 * 5c/arm
 * Arm
 */

⟨constant SZ_CHAR(arm) 224b⟩
⟨constant SZ_SHORT(arm) 224c⟩
⟨constant SZ_INT(arm) 224d⟩
⟨constant SZ_LONG(arm) 224e⟩
⟨constant SZ_IND(arm) 224f⟩
⟨constant SZ_FLOAT(arm) 224g⟩
⟨constant SZ_VLONG(arm) 224h⟩
⟨constant SZ_DOUBLE(arm) 224i⟩
⟨constant FNX(arm) 171b⟩
⟨constant BTRUE(arm) 205a⟩

typedef struct Adr  Adr;
typedef struct Prog Prog;
typedef struct Case Case;
typedef struct C1  C1;
typedef struct Multab Multab;
typedef struct Hintab Hintab;
typedef struct Var  Var;
typedef struct Reg  Reg;
typedef struct Rgn  Rgn;


⟨struct Adr(arm) 29a⟩
⟨constant A 29b⟩

⟨constant INDEXED(arm) 169a⟩
⟨struct Prog(arm) 28d⟩
⟨constant P 155g⟩

⟨struct Case 229d⟩
⟨constant C 230a⟩

⟨struct C1 230b⟩

⟨struct Multab(arm) 261a⟩

⟨struct Hintab(arm) 262b⟩

⟨struct Var 292d⟩

⟨struct Reg 292e⟩
⟨constant R 292f⟩

⟨constant NRGN(arm) 293e⟩
⟨struct Rgn 293h⟩

extern long breakpc;
```

```
extern long nbreak;
extern Case* cases;
extern Node constnode;
extern Node fconstnode;
extern long continpc;
extern long curarg;
extern long cursafe;
extern Prog* firstp;
extern Prog* lastp;
extern long maxargsafe;
extern int mnstring;
extern Multab multab[20];
extern int hintabsize;
extern Node* nodrat;
extern Node* nodret;
extern Node* nodsafe;
extern long nrathole;
extern long nstring;
extern Prog* p;
extern long pc;
extern Node regnode;
extern char string[NSNAME];
extern Sym* symrathole;
extern Node znode;
extern Prog zprog;
extern char reg[NREG+NFREG];
extern long exregoffset;
extern long exfregoffset;
extern int suppress;
```

⟨*function* LOAD*(arm)* 293c⟩
⟨*function* STORE*(arm)* 293d⟩

⟨*function* bset*(arm)* 291e⟩

⟨*constant* CLOAD*(arm)* 301a⟩
⟨*constant* CREF*(arm)* 301b⟩
⟨*constant* LOOP*(arm)* 299a⟩

```
extern Rgn region[NRGN];
extern Rgn* rgp;
extern int nregion;
extern int nvar;

extern Bits externs;
extern Bits params;
extern Bits consts;
extern Bits addrs;

extern long regbits;
extern long exregbits;

extern int change;

extern Reg* firstr;
extern Reg* lastr;
extern Reg zreg;
extern Reg* freer;
extern Var var[NVAR];
extern long* idom;
```

```
extern Reg** rpo2r;
extern long maxnr;

extern char* anames[];
extern Hintab hintab[];

/*
 * sgen.c
 */
void codgen(Node*, Node*);
void gen(Node*);
void noretval(int);
void usedset(Node*, int);
void xcom(Node*);
int bcomplex(Node*, Node*);

/*
 * cgen.c
 */
void cgen(Node*, Node*);
void cgenrel(Node*, Node*, int);
void reglcgen(Node*, Node*, Node*);
void lcgen(Node*, Node*);
void bcgen(Node*, int);
void boolgen(Node*, int, Node*);
void sugen(Node*, Node*, long);
void layout(Node*, Node*, int, int, Node*);

/*
 * txt.c
 */
void ginit(void);
void gclean(void);
void nextpc(void);
void gargs(Node*, Node*, Node*);
void garg1(Node*, Node*, Node*, int, Node**);
Node* nodconst(long);
Node* nod32const(vlong);
Node* nodfconst(double);
void nodreg(Node*, Node*, int);
void regret(Node*, Node*);
int  tmpreg(void);
void regalloc(Node*, Node*, Node*);
void regfree(Node*);
void regialloc(Node*, Node*, Node*);
void regsalloc(Node*, Node*);
void regaalloc1(Node*, Node*);
void regaalloc(Node*, Node*);
void regind(Node*, Node*);
void gprep(Node*, Node*);
void raddr(Node*, Prog*);
void naddr(Node*, Adr*);
void gmovm(Node*, Node*, int);
void gmove(Node*, Node*);
void gmover(Node*, Node*);
void gins(int a, Node*, Node*);
void gopcode(int, Node*, Node*, Node*);
int  samaddr(Node*, Node*);
void gbranch(int);
void patch(Prog*, long);
```

```
int  sconst(Node*);
int  sval(long);
void gpseudo(int, Sym*, Node*);

/*
 * swt.c
 */
int  swcmp(const void*, const void*);
void doswit(Node*);
void swit1(C1*, int, long, Node*);
void swit2(C1*, int, long, Node*, Node*);
void casf(void);
void bitload(Node*, Node*, Node*, Node*, Node*);
void bitstore(Node*, Node*, Node*, Node*, Node*);
long outstring(char*, long);
int  mulcon(Node*, Node*);
Multab* mulcon0(long);
void nullwarn(Node*, Node*);
void gextern(Sym*, Node*, long, long);
void outcode(void);
void ieeedtod(Ieee*, double);

/*
 * list
 */
void listinit(void);
int Pconv(Fmt*);
int Aconv(Fmt*);
int Dconv(Fmt*);
int Sconv(Fmt*);
int Nconv(Fmt*);
int Bconv(Fmt*);
int Rconv(Fmt*);

/*
 * reg.c
 */
Reg* rega(void);
int  rcmp(const void*, const void*);
void regopt(Prog*);
void addmove(Reg*, int, int, int);
Bits mkvar(Adr*, int);
void prop(Reg*, Bits, Bits);
void loopit(Reg*, long);
void synch(Reg*, Bits);
ulong allreg(ulong, Rgn*);
void paint1(Reg*, int);
ulong paint2(Reg*, int);
void paint3(Reg*, int, long, int);
void addreg(Adr*, int);

/*
 * peep.c
 */
void peep(void);
void excise(Reg*);
Reg* uniqp(Reg*);
Reg* uniqs(Reg*);
int regtyp(Adr*);
int regzer(Adr*);
```

```
int anyvar(Adr*);
int subprop(Reg*);
int copyprop(Reg*);
int shiftprop(Reg*);
void constprop(Adr*, Adr*, Reg*);
int copy1(Adr*, Adr*, Reg*, int);
int copyu(Prog*, Adr*, Adr*);

int copyas(Adr*, Adr*);
int copyau(Adr*, Adr*);
int copyau1(Prog*, Adr*);
int copysub(Adr*, Adr*, Adr*, int);
int copysub1(Prog*, Adr*, Adr*, int);

long RtoB(int);
long FtoB(int);
int  BtoR(long);
int  BtoF(long);

void predicate(void);
int isbranch(Prog *);
int predicable(Prog *p);
int modifiescpsr(Prog *p);

#pragma varargck type "A" int
#pragma varargck type "B" Bits
#pragma varargck type "D" Adr*
#pragma varargck type "N" Adr*
#pragma varargck type "R" Adr*
#pragma varargck type "P" Prog*
#pragma varargck type "S" char*
```

D.3.3 5c/machcap.c

⟨5c/machcap.c 460a⟩≡

```
#include "../cc/cc.h"

⟨function machcap(arm) 126c⟩
```

D.3.4 5c/globals2.c

⟨5c/globals2.c 460b⟩≡

```
#include "gc.h"

⟨global breakpc 161c⟩
⟨global nbreak 165d⟩
⟨global cases 230c⟩
⟨global constnode 157a⟩
⟨global fconstnode 383g⟩
⟨global continpc 161d⟩

⟨global curarg 210b⟩
⟨global cursafe 210e⟩

⟨global firstp 155e⟩
⟨global lastp 156a⟩

⟨global maxargsafe 210c⟩
```

⟨*global* mnstring 181k⟩
⟨*global* multab*(arm)* 261b⟩
⟨*global* hintabsize*(arm)* 272b⟩
⟨*global* nodrat 195b⟩
⟨*global* nodret 222g⟩
⟨*global* nodsafe 211b⟩
⟨*global* nrathole 195g⟩
⟨*global* nstring 181j⟩
⟨*global* p 155d⟩
⟨*global* pc 155a⟩
⟨*global* regnode 167f⟩
⟨*global* string 182c⟩
⟨*global* symrathole 195c⟩
⟨*global* znode 18c⟩
⟨*global* zprog 29e⟩
⟨*global* reg*(arm)* 166d⟩
⟨*global* exregoffset 365e⟩
⟨*global* exfregoffset 365f⟩
⟨*global* suppress 151d⟩

⟨*global* region 293f⟩
⟨*global* rgp 293g⟩
⟨*global* nregion 293i⟩

⟨*global* externs 281c⟩
⟨*global* params 281d⟩
⟨*global* consts 281e⟩
⟨*global* addrs 281f⟩

⟨*global* regbits 281g⟩
⟨*global* exregbits 281h⟩

⟨*global* change 281i⟩

⟨*global* firstr 281j⟩
⟨*global* lastr 281k⟩
⟨*global* zreg 281l⟩
⟨*global* freer 281m⟩
⟨*global* var 292c⟩
⟨*global* nvar 292b⟩

⟨*global* idom 299c⟩
⟨*global* rpo2r 299d⟩
⟨*global* maxnr 299e⟩

D.3.5 5c/list.c

⟨5c/list.c 461⟩≡

```
#include "gc.h"
```

⟨*function* listinit*(arm)* 406b⟩

⟨*function* Bconv 406c⟩

⟨*global* extra*(arm)* 407a⟩

⟨*function* Pconv*(arm)* 406d⟩

⟨*function* Aconv 407b⟩

⟨*function* Dconv*(arm)* 408⟩

⟨*function* Rconv*(arm)* 409a⟩

⟨*function* Sconv*(arm)* 409b⟩

⟨*function* Nconv*(arm)* 410⟩

D.3.6 5c/txt.c

⟨5c/txt.c 462⟩≡

```
#include "gc.h"
```

⟨*global* resvreg*(arm)* 166f⟩

⟨*function* ginit*(arm)* 35d⟩

⟨*function* gclean*(arm)* 34a⟩

⟨*function* nextpc 155c⟩

⟨*function* gargs 210g⟩

⟨*function* garg1*(arm)* 210h⟩

⟨*function* nodconst 157c⟩

⟨*function* nod32const*(arm)* 223e⟩

⟨*function* nodfconst 383i⟩

⟨*function* nodreg*(arm)* 167e⟩

⟨*function* regret*(arm)* 166c⟩

⟨*function* tmpreg*(arm)* 200b⟩

⟨*function* regalloc*(arm)* 178b⟩

⟨*function* regialloc 190d⟩

⟨*function* regfree*(arm)* 167h⟩

⟨*function* regsalloc 211d⟩

⟨*function* regaalloc1*(arm)* 280f⟩

⟨*function* regaalloc*(arm)* 211a⟩

⟨*function* regind 190e⟩

⟨*function* raddr*(arm)* 384c⟩

⟨*function* naddr*(arm)* 176b⟩

⟨*function* gmovm*(arm)* 200a⟩

⟨*function* gmove*(arm)* 171d⟩

⟨*function* gmover*(arm)* 201b⟩

⟨*function* gins*(arm)* 176a⟩

⟨*function* gopcode*(arm)* 184c⟩

⟨*function* samaddr 279b⟩

⟨*function* gbranch*(arm)* 157e⟩

⟨*function* patch 158a⟩

⟨*function* gpseudo*(arm)* 157d⟩

⟨*function* sconst*(arm)* 192b⟩

⟨*function* sval*(arm)* 297c⟩

⟨*function* exreg*(arm)* 366a⟩

⟨*global* ewidth 224a⟩

⟨*global* ncast 202b⟩

D.3.7 5c/peep.c

⟨5c/peep.c 463⟩≡

```
#include "gc.h"

int xtramodes(Reg*, Adr*);
```

⟨*function* peep*(arm)* 306b⟩

⟨*function* excise*(arm)* 309a⟩

⟨*function* uniqp 309b⟩

⟨*function* uniqs 310a⟩

⟨*function* regtyp*(arm)* 310b⟩

⟨*function* subprop*(arm)* 310c⟩

⟨*function* copyprop 312⟩

⟨*function* copy1 313⟩

⟨*function* constprop*(arm)* 314⟩

⟨*function* FAIL*(arm)* 315a⟩
⟨*function* shiftprop*(arm)* 315b⟩

⟨*function* findpre*(arm)* 317⟩

⟨*function* findinc*(arm)* 318a⟩

⟨*function* nochange*(arm)* 318b⟩

⟨*function* `findu1`*(arm)* 319a⟩

⟨*function* `finduse`*(arm)* 319b⟩

⟨*function* `xtramodes`*(arm)* 319c⟩

⟨*function* `copyu`*(arm)* 320⟩

⟨*function* `a2type`*(arm)* 324⟩

⟨*function* `copyas`*(arm)* 325a⟩

⟨*function* `copyau`*(arm)* 325b⟩

⟨*function* `copyau1`*(arm)* 326a⟩

⟨*function* `copysub`*(arm)* 326b⟩

⟨*function* `copysub1`*(arm)* 326c⟩

⟨*global* `predinfo`*(arm)* 327a⟩

⟨*struct* `Joininfo` 328b⟩

⟨*enum* `_anon_`*(arm)* 327b⟩

⟨*enum* `_anon_ (5c/peep.c)`*(arm)* 327c⟩

⟨*function* `isbranch`*(arm)* 327d⟩

⟨*function* `predicable`*(arm)* 327e⟩

⟨*function* `modifiescpsr`*(arm)* 328a⟩

⟨*function* `joinsplit`*(arm)* 328c⟩

⟨*function* `successor`*(arm)* 329a⟩

⟨*function* `applypred`*(arm)* 329b⟩

⟨*function* `predicate`*(arm)* 330a⟩

D.3.8 5c/sgen.c

⟨`5c/sgen.c` 464a⟩≡

```
#include "gc.h"
```

⟨*function* `noretval`*(arm)* 282f⟩

⟨*function* `xcom`*(arm)* 168c⟩

D.3.9 5c/swt.c

⟨`5c/swt.c` 464b⟩≡

```
#include "gc.h"
```

⟨*function* `swit1`*(arm)* 232b⟩

⟨*function* swit2*(arm)* 232c⟩

⟨*function* bitload*(arm)* 355d⟩

⟨*function* bitstore*(arm)* 356a⟩

⟨*function* outstring*(arm)* 182d⟩

⟨*function* mulcon*(arm)* 259d⟩

⟨*function* gextern*(arm)* 221d⟩

```
void zname(Biobuf*, Sym*, int);
char* zaddr(char*, Adr*, int);
void zwrite(Biobuf*, Prog*, int, int);
void outhist(Biobuf*);
```

⟨*function* zwrite*(arm)* 236a⟩

⟨*struct* Htab 235a⟩

⟨*function* outcode*(arm)* 235c⟩

⟨*function* outhist*(arm)* 238b⟩

⟨*function* zname*(arm)* 238a⟩

⟨*function* zaddr*(arm)* 236b⟩

⟨*function* align*(arm)* 225d⟩

⟨*function* maxround 226a⟩

D.3.10 5c/mul.c

⟨5c/mul.c 465⟩≡

```
#include "gc.h"

/*
 * code sequences for multiply by constant.
 * [a-l][0-3]
 * lsl $(A-'a'),r0,r1
 * [+][0-7]
 * add r0,r1,r2
 * [-][0-7]
 * sub r0,r1,r2
 */
```

⟨*global* maxmulops*(arm)* 261c⟩
⟨*global* multabp*(arm)* 261d⟩
⟨*global* mulval*(arm)* 261e⟩
⟨*global* mulcp*(arm)* 261f⟩
⟨*global* valmax*(arm)* 261g⟩
⟨*global* shmax*(arm)* 261h⟩

```
static int docode(char *hp, char *cp, int r0, int r1);
static int gen1(int len);
static int gen2(int len, long r1);
```

```
static int gen3(int len, long r0, long r1, int flag);
```
⟨*enum* _anon_ (5c/mul.c)*(arm)* 262a⟩

⟨*function* mulcon0*(arm)* 262c⟩

⟨*function* docode*(arm)* 263⟩

⟨*function* gen1*(arm)* 265a⟩

⟨*function* gen2*(arm)* 265b⟩

⟨*function* gen3*(arm)* 266⟩

⟨*global* hintab*(arm)* 268⟩
⟨*global hintabsize (5c/mul.c)(arm)* 272a⟩

D.3.11 5c/reg.c

⟨5c/reg.c 466⟩≡
```
#include "gc.h"

void addsplits(void);
```

⟨*function* rega 293a⟩

⟨*function* rcmp 293b⟩

⟨*function* regopt*(arm)* 281b⟩

⟨*function* addsplits*(arm)* 294a⟩

⟨*function* addmove*(arm)* 294b⟩

⟨*function* mkvar*(arm)* 295⟩

⟨*function* prop*(arm)* 297d⟩

⟨*function* postorder 330b⟩

⟨*function* rpolca 298a⟩

⟨*function* doms 298b⟩

⟨*function* loophead 298c⟩

⟨*function* loopmark 299b⟩

⟨*function* loopit 299f⟩

⟨*function* synch 300a⟩

⟨*function* allreg*(arm)* 300b⟩

⟨*function* paint1*(arm)* 301c⟩

⟨*function* paint2*(arm)* 303a⟩

⟨*function* paint3 303b⟩

⟨*function* `addreg`*(arm)* 305a⟩

⟨*function* `RtoB`*(arm)* 305b⟩

⟨*function* `BtoR`*(arm)* 305c⟩

⟨*function* `FtoB`*(arm)* 305d⟩

⟨*function* `BtoF`*(arm)* 306a⟩

D.3.12 5c/cgen.c

⟨`5c/cgen.c` 467⟩≡

```
#include "gc.h"
```

⟨*function* `cgen`*(arm)* 179e⟩

⟨*function* `cgenrel`*(arm)* 179f⟩

⟨*function* `reglcgen`*(arm)* 191e⟩

⟨*function* `reglpcgen`*(arm)* 200c⟩

⟨*function* `lcgen`*(arm)* 189b⟩

⟨*function* `bcgen` 207⟩

⟨*function* `boolgen`*(arm)* 205d⟩

⟨*function* `sugen`*(arm)* 196⟩

Glossary

```
SUE = Structure/Union/Enum
SU  = Structure/Union
```

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. For an introduction see http://en.wikipedia.org/wiki/Literate_Program. cited page(s) 13

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 13